

agent_framework Package

Packages

 Expand table

a2a
ag_ui
anthropic
azure
chatkit
declarative
devui
lab
mem0
microsoft
ollama
openai
redis

Modules

 Expand table

exceptions
observability

Classes

 Expand table

AIFunction	A tool that wraps a Python function to make it callable by AI models.
------------	---

	This class wraps a Python function to make it callable by AI models with automatic parameter validation and JSON schema generation. Initialize the AIFunction.
AgentExecutor	built-in executor that wraps an agent for handling messages. AgentExecutor adapts its behavior based on the workflow execution mode: - run_stream(): Emits incremental AgentRunUpdateEvent events as the agent produces tokens - run(): Emits a single AgentRunEvent containing the complete response The executor automatically detects the mode via WorkflowContext.is_streaming(). Initialize the executor with a unique identifier.
AgentExecutorRequest	A request to an agent executor.
AgentExecutorResponse	A response from an agent executor.
AgentInputRequest	Request for human input before an agent runs in high-level builder workflows. Emitted via RequestInfoEvent when a workflow pauses before an agent executes. The response is injected into the conversation as a user message to steer the agent's behavior. This is the standard request type used by <code>.with_request_info()</code> on SequentialBuilder, ConcurrentBuilder, GroupChatBuilder, and HandoffBuilder.
AgentMiddleware	Abstract base class for agent middleware that can intercept agent invocations. Agent middleware allows you to intercept and modify agent invocations before and after execution. You can inspect messages, modify context, override results, or terminate execution early.  Note AgentMiddleware is an abstract base class. You must subclass it and implement the <code>process()</code> method to create custom agent middleware.
AgentProtocol	A protocol for an agent that can be invoked.

This protocol defines the interface that all agents must implement, including properties for identification and methods for execution. Note Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible. This allows you to create completely custom agents without using any Agent Framework base classes.	
AgentRunContext	Context object for agent middleware invocations. This context is passed through the agent middleware pipeline and contains all information about the agent invocation. Initialize the AgentRunContext.
AgentRunEvent	Event triggered when an agent run is completed. Initialize the agent run event.
AgentRunResponse	Represents the response to an Agent run request. Provides one or more response messages and metadata about the response. A typical response will contain a single message, but may contain multiple messages in scenarios involving function calls, RAG retrievals, or complex logic. Initialize an AgentRunResponse.
AgentRunResponseUpdate	Represents a single streaming response chunk from an Agent. Initialize an AgentRunResponseUpdate.
AgentRunUpdateEvent	Event triggered when an agent is streaming messages. Initialize the agent streaming event.
AgentThread	The Agent thread class, this can represent both a locally managed thread or a thread managed by the service. An <code>AgentThread</code> maintains the conversation state and message history for an agent interaction. It can either use a service-managed thread (via <code>service_thread_id</code>) or a local message store (via <code>message_store</code>), but not both.

Initialize an AgentThread, do not use this method manually, always use: `agent.get_new_thread()`.

⚠ **Note**

Either `service_thread_id` or `message_store` may be set, but not both.

AggregateContextProvider

A ContextProvider that contains multiple context providers.

It delegates events to multiple context providers and aggregates responses from those events before returning. This allows you to combine multiple context providers into a single provider.

⚠ **Note**

An AggregateContextProvider is created automatically when you pass a single context

provider or a sequence of context providers to the agent constructor.

Initialize the AggregateContextProvider with context providers.

BaseAgent

Base class for all Agent Framework agents.

This class provides core functionality for agent implementations, including context providers, middleware support, and thread management.

⚠ **Note**

BaseAgent cannot be instantiated directly as it doesn't implement the

`run()`, `run_stream()`, and other methods required by AgentProtocol.

Use a concrete implementation like ChatAgent or create a subclass.

Initialize a BaseAgent instance.

BaseAnnotation

Base class for all AI Annotation types.

	Initialize BaseAnnotation.
BaseChatClient	Base class for chat clients. This abstract base class provides core functionality for chat client implementations, including middleware support, message preparation, and tool normalization. ⚠ Note BaseChatClient cannot be instantiated directly as it's an abstract base class. Subclasses must implement <code>_inner_get_response()</code> and <code>_inner_get_streaming_response()</code>. Initialize a BaseChatClient instance.
BaseContent	Represents content used by AI services. Initialize BaseContent.
Case	Runtime wrapper combining a switch-case predicate with its target. Each <i>Case</i> couples a boolean predicate with the executor that should handle the message when the predicate evaluates to <i>True</i>. The runtime keeps this lightweight container separate from the serialisable <i>SwitchCaseEdgeGroupCase</i> so that execution can operate with live callables without polluting persisted state.
ChatAgent	A Chat Client Agent. This is the primary agent implementation that uses a chat client to interact with language models. It supports tools, context providers, middleware, and both streaming and non-streaming responses. Initialize a ChatAgent instance. ⚠ Note The set of parameters from <code>frequency_penalty</code> to <code>request_kwargs</code> are used to call the chat client. They can also be passed to both <code>run</code> methods. When both are set, the ones passed to the <code>run</code> methods take precedence.

ChatClientProtocol	A protocol for a chat client that can generate responses. This protocol defines the interface that all chat clients must implement, including methods for generating both streaming and non-streaming responses.  Note Protocols use structural subtyping (duck typing). Classes don't need to explicitly inherit from this protocol to be considered compatible.
ChatContext	Context object for chat middleware invocations. This context is passed through the chat middleware pipeline and contains all information about the chat request. Initialize the ChatContext.
ChatMessage	Represents a chat message. Initialize ChatMessage.
ChatMessageStore	An in-memory implementation of ChatMessageStoreProtocol that stores messages in a list. This implementation provides a simple, list-based storage for chat messages with support for serialization and deserialization. It implements all the required methods of the <code>ChatMessageStoreProtocol</code> protocol. The store maintains messages in memory and provides methods to serialize and deserialize the state for persistence purposes. Create a ChatMessageStore for use in a thread.
ChatMessageStoreProtocol	Defines methods for storing and retrieving chat messages associated with a specific thread. Implementations of this protocol are responsible for managing the storage of chat messages, including handling large volumes of data by truncating or summarizing messages as necessary.
ChatMiddleware	Abstract base class for chat middleware that can intercept chat client requests. Chat middleware allows you to intercept and modify chat client requests before and after execution. You can modify messages, add system prompts, log requests, or override chat responses.

ⓘ **Note**

ChatMiddleware is an abstract base class. You must subclass it and implement the `process()` method to create custom chat middleware.

ChatOptions

Common request settings for AI services.

Initialize ChatOptions.

ChatResponse

Represents the response to a chat request.

Initializes a ChatResponse with the provided parameters.

ChatResponseUpdate

Represents a single streaming response chunk from a *ChatClient*.

Initializes a ChatResponseUpdate with the provided parameters.

CheckpointStorage

Protocol for checkpoint storage backends.

CitationAnnotation

Represents a citation annotation.

Initialize CitationAnnotation.

ConcurrentBuilder

High-level builder for concurrent agent workflows.

- *participants([...])* accepts a list of AgentProtocol (recommended) or Executor.
- *register_participants([...])* accepts a list of factories for AgentProtocol (recommended) or Executor factories
- *build()* wires: dispatcher -> fan-out -> participants -> fan-in -> aggregator.
- *with_aggregator(...)* overrides the default aggregator with an Executor or callback.
- *register_aggregator(...)* accepts a factory for an Executor as custom aggregator.

Usage:

Python

```
from agent_framework import ConcurrentBuilder

# Minimal: use default aggregator (returns list[ChatMessage])
workflow =
```

```

ConcurrentBuilder().participants([agent1, agent2,
agent3]).build()

# With agent factories
workflow =
ConcurrentBuilder().register_participants([create_age
nt1, create_agent2, create_agent3]).build()

# Custom aggregator via callback (sync or async).
The callback receives
# list[AgentExecutorResponse] and its return value
becomes the workflow's output.
def summarize(results:
list[AgentExecutorResponse]) -> str:
    return " |
".join(r.agent_run_response.messages[-1].text for r
in results)

workflow =
ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_aggregator(summarize).build()

# Custom aggregator via a factory
class MyAggregator(Executor):
    @handler
    async def aggregate(self, results:
list[AgentExecutorResponse], ctx:
WorkflowContext[Never, str]) -> None:
        await ctx.yield_output(" |
".join(r.agent_run_response.messages[-1].text for r
in results))

workflow = (
    ConcurrentBuilder()
        .register_participants([create_agent1,
create_agent2, create_agent3])
        .register_aggregator(lambda:
MyAggregator(id="my_aggregator"))
        .build()
)

# Enable checkpoint persistence so runs can resume
workflow =
ConcurrentBuilder().participants([agent1, agent2,
agent3]).with_checkpointing(storage).build()

# Enable request info before aggregation
workflow =
ConcurrentBuilder().participants([agent1,
agent2]).with_request_info().build()

```


Context

A class containing any context that should be provided to the AI model as supplied by a ContextProvider.

Each ContextProvider has the ability to provide its own context for each invocation. The Context class contains the additional context supplied by the ContextProvider. This context will be combined with context supplied by other providers before being passed to the AI model. This context is per invocation, and will not be stored as part of the chat history.

Create a new Context object.

ContextProvider

Base class for all context providers.

A context provider is a component that can be used to enhance the AI's context management. It can listen to changes in the conversation and provide additional context to the AI model just before invocation.

ⓘ Note

ContextProvider is an abstract base class. You must subclass it and implement

the invoking() method to create a custom context provider. Ideally, you should

also implement the invoked() and thread_created() methods to track conversation

state, but these are optional.

DataContent

Represents binary data content with an associated media type (also known as a MIME type).

ⓘ Important

This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Initializes a DataContent instance.

ⓘ Important

This is for binary data that is represented as a data URI, not for online resources.

Use UriContent for online resources.

Default

Runtime representation of the default branch in a switch-case group.

The default branch is invoked only when no other case predicates match. In practice it is guaranteed to exist so that routing never produces an empty target.

Edge

Model a directed, optionally-conditional hand-off between two executors.

Each *Edge* captures the minimal metadata required to move a message from one executor to another inside the workflow graph. It optionally embeds a boolean predicate that decides if the edge should be taken at runtime. By serialising the edge down to primitives we can reconstruct the topology of a workflow irrespective of the original Python process.

Initialize a fully-specified edge between two workflow executors.

EdgeDuplicationError

Exception raised when duplicate edges are detected in the workflow.

ErrorContent

Represents an error.

Remarks: Typically used for non-fatal errors, where something went wrong as part of the operation, but the operation was still able to continue.

Initializes an ErrorContent instance.

Executor

Base class for all workflow executors that process messages and perform computations.

Overview

Executors are the fundamental building blocks of workflows, representing individual processing units that receive messages, perform operations, and produce outputs. Each executor is uniquely identified and can handle specific message types through decorated handler methods.

Type System

Executors have a rich type system that defines their capabilities:

Input Types

The types of messages an executor can process, discovered from handler method signatures:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_string(self, message: str,
ctx: WorkflowContext) -> None:
    # This executor can handle 'str' input
types
```

Access via the *input_types* property.

Output Types

The types of messages an executor can send to other executors via *ctx.send_message()*:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_data(self, message: str, ctx:
WorkflowContext[int | bool]) -> None:
    # This executor can send 'int' or 'bool'
messages
```

Access via the *output_types* property.

Workflow Output Types

The types of data an executor can emit as workflow-level outputs via *ctx.yield_output()*:

Python

```
class MyExecutor(Executor):
    @handler
    async def process(self, message: str, ctx:
WorkflowContext[int, str]) -> None:
    # Can send 'int' messages AND yield 'str'
workflow outputs
```

Access via the *workflow_output_types* property.

Handler Discovery

Executors discover their capabilities through decorated methods:

@handler Decorator

Marks methods that process incoming messages:

Python

```
class MyExecutor(Executor):
    @handler
    async def handle_text(self, message: str, ctx:
WorkflowContext[str]) -> None:
        await ctx.send_message(message.upper())
```

Sub-workflow Request Interception

Use @handler methods to intercept sub-workflow requests:

Python

```
class ParentExecutor(Executor):
    @handler
    async def handle_subworkflow_request(
        self,
        request: SubWorkflowRequestMessage,
        ctx:
WorkflowContext[SubWorkflowResponseMessage],
    ) -> None:
        if self.is_allowed(request.domain):
            response =
request.create_response(data=True)
            await ctx.send_message(response,
target_id=request.executor_id)
        else:
            await
ctx.request_info(request.source_event,
response_type=request.source_event.response_type)
```

Context Types

Handler methods receive different WorkflowContext variants based on their type annotations:

WorkflowContext (no type parameters)

For handlers that only perform side effects without sending messages or yielding outputs:

Python

```
class LoggingExecutor(Executor):
    @handler
    async def log_message(self, msg: str, ctx:
WorkflowContext) -> None:
        print(f"Received: {msg}") # Only logging,
no outputs
```

WorkflowContext[T_Out]

Enables sending messages of type T_Out via `ctx.send_message()`:

Python

```
class ProcessorExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx:
WorkflowContext[int]) -> None:
        await ctx.send_message(42) # Can send int
messages
```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```
class DualOutputExecutor(Executor):
    @handler
    async def handler(self, msg: str, ctx:
WorkflowContext[int, str]) -> None:
        await ctx.send_message(42) # Send int
message
        await ctx.yield_output("done") # Yield
str workflow output
```

Function Executors

Simple functions can be converted to executors using the `@executor` decorator:

Python

```
@executor
async def process_text(text: str, ctx:
WorkflowContext[str]) -> None:
    await ctx.send_message(text.upper())

# Or with custom ID:
@executor(id="text_processor")
def sync_process(text: str, ctx:
WorkflowContext[str]) -> None:
    ctx.send_message(text.lower()) # Sync
functions run in thread pool
```

Sub-workflow Composition

Executors can contain sub-workflows using `WorkflowExecutor`. Sub-workflows can make requests that parent workflows can intercept. See `WorkflowExecutor` documentation for details on workflow composition patterns and request/response handling.

State Management

Executors can contain states that persist across workflow runs and checkpoints. Override the `on_checkpoint_save` and `on_checkpoint_restore` methods to implement custom state serialization and restoration logic.

Implementation Notes

- Do not call `execute()` directly - it's invoked by the workflow engine
- Do not override `execute()` - define handlers using decorators instead
- Each executor must have at least one `@handler` method
- Handler method signatures are validated at initialization time

Initialize the executor with a unique identifier.

ExecutorCompletedEvent

Event triggered when an executor handler is completed.

Initialize the executor event with an executor ID and optional data.

ExecutorEvent

Base class for executor events.

	Initialize the executor event with an executor ID and optional data.
ExecutorFailedEvent	Event triggered when an executor handler raises an error.
ExecutorInvokedEvent	Event triggered when an executor handler is invoked. Initialize the executor event with an executor ID and optional data.
FanInEdgeGroup	Represent a converging set of edges that feed a single downstream executor. Fan-in groups are typically used when multiple upstream stages independently produce messages that should all arrive at the same downstream processor. Build a fan-in mapping that merges several sources into one target.
FanOutEdgeGroup	Represent a broadcast-style edge group with optional selection logic. A fan-out forwards a message produced by a single source executor to one or more downstream executors. At runtime we may further narrow the targets by executing a <i>selection_func</i> that inspects the payload and returns the subset of ids that should receive the message. Create a fan-out mapping from a single source to many targets.
FileCheckpointStorage	File-based checkpoint storage for persistence. Initialize the file storage.
FinishReason	Represents the reason a chat response completed. Initialize FinishReason with a value.
FunctionApprovalRequestContent	Represents a request for user approval of a function call. Initializes a FunctionApprovalRequestContent instance.
FunctionApprovalResponseContent	Represents a response for user approval of a function call. Initializes a FunctionApprovalResponseContent instance.
FunctionCallContent	Represents a function call request. Initializes a FunctionCallContent instance.
FunctionExecutor	Executor that wraps a user-defined function. This executor allows users to define simple functions (both sync and async) and use them as workflow executors without needing to create full executor classes. Synchronous functions are executed in a thread pool using <code>asyncio.to_thread()</code> to avoid blocking the event loop.

	Initialize the FunctionExecutor with a user-defined function.
FunctionInvocationConfiguration	Configuration for function invocation in chat clients. This class is created automatically on every chat client that supports function invocation. This means that for most cases you can just alter the attributes on the instance, rather than creating a new one. Initialize FunctionInvocationConfiguration.
FunctionInvocationContext	Context object for function middleware invocations. This context is passed through the function middleware pipeline and contains all information about the function invocation. Initialize the FunctionInvocationContext.
FunctionMiddleware	Abstract base class for function middleware that can intercept function invocations. Function middleware allows you to intercept and modify function/tool invocations before and after execution. You can validate arguments, cache results, log invocations, or override function execution. Note FunctionMiddleware is an abstract base class. You must subclass it and implement the <code>process()</code> method to create custom function middleware.
FunctionResultContent	Represents the result of a function call. Initializes a FunctionResultContent instance.
GraphConnectivityError	Exception raised when graph connectivity issues are detected.
GroupChatBuilder	High-level builder for manager-directed group chat workflows with dynamic orchestration. GroupChat coordinates multi-agent conversations using a manager that selects which participant speaks next. The manager can be a simple Python function (set_select_speakers_func) or an agent-based selector via set_manager. These two approaches are mutually exclusive. Core Workflow: 1. Define participants: list of agents (uses their <code>.name</code>) or dict mapping names to agents

2. Configure speaker selection: `set_select_speakers_func` OR `set_manager` (not both)
3. Optional: set round limits, checkpointing, termination conditions
4. Build and run the workflow

Speaker Selection Patterns:

Pattern 1: Simple function-based selection (recommended)

Python

```
from agent_framework import GroupChatBuilder,
GroupChatStateSnapshot

def select_next_speaker(state:
GroupChatStateSnapshot) -> str | None:
    # state contains: task, participants,
    conversation, history, round_index
    if state["round_index"] >= 5:
        return None # Finish
    last_speaker = state["history"][-1].speaker if
state["history"] else None
    if last_speaker == "researcher":
        return "writer"
    return "researcher"

workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher_agent,
writer_agent]) # Uses agent.name
    .build()
)
```

Pattern 2: LLM-based selection

Python

```
from agent_framework import ChatAgent
from agent_framework.azure import
AzureOpenAIChatClient

manager_agent =
AzureOpenAIChatClient().create_agent(
    instructions="Coordinate the conversation and
pick the next speaker.",
    name="Coordinator",
    temperature=0.3,
```

```

        seed=42,
        max_tokens=500,
    )

    workflow = (
        GroupChatBuilder()
        .set_manager(manager_agent,
display_name="Coordinator")
        .participants([researcher, writer]) # Or use
dict: researcher=r, writer=w
        .with_max_rounds(10)
        .build()
    )

```

Pattern 3: Request info for mid-conversation feedback

Python

```

from agent_framework import GroupChatBuilder

# Pause before all participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer])
    .with_request_info()
    .build()
)

# Pause only before specific participants
workflow = (
    GroupChatBuilder()
    .set_select_speakers_func(select_next_speaker)
    .participants([researcher, writer, editor])
    .with_request_info(agents=[editor]) # Only
pause before editor responds
    .build()
)

```

Participant Specification:

Two ways to specify participants:

- List form: *[agent1, agent2]* - uses *agent.name* attribute for participant names
- Dict form: *{name1: agent1, name2: agent2}* - explicit name control
- Keyword form: *participants(name1=agent1, name2=agent2)* - explicit name control

State Snapshot Structure:

The GroupChatStateSnapshot passed to `set_select_speakers_func` contains:

- *task*: ChatMessage - Original user task
- *participants*: dict[str, str] - Mapping of participant names to descriptions
- *conversation*: tuple[ChatMessage, ...] - Full conversation history
- *history*: tuple[GroupChatTurn, ...] - Turn-by-turn record with speaker attribution
- *round_index*: int - Number of manager selection rounds so far
- *pending_agent*: str | None - Name of agent currently processing (if any)

Important Constraints:

- Cannot combine `set_select_speakers_func` and `set_manager`
- Participant names must be unique
- When using list form, agents must have a non-empty *name* attribute

Initialize the GroupChatBuilder.

GroupChatDirective

Instruction emitted by a group chat manager implementation.

HandoffBuilder

Fluent builder for conversational handoff workflows with coordinator and specialist agents.

The handoff pattern enables a coordinator agent to route requests to specialist agents. Interaction mode controls whether the workflow requests user input after each agent response or completes autonomously once agents finish responding. A termination condition determines when the workflow should stop requesting input and complete.

Routing Patterns:

Single-Tier (Default): Only the coordinator can hand off to specialists. By default, after any specialist responds, control returns to the user for more input. This creates a cyclical flow: user -> coordinator -> [optional specialist] -> user -> coordinator -> ... Use `with_interaction_mode("autonomous")` to skip requesting additional user input and yield the final conversation when an agent responds without delegating.

Multi-Tier (Advanced): Specialists can hand off to other specialists using `.add_handoff()`. This provides more flexibility for complex workflows but is less controllable than the single-tier pattern. Users lose real-time visibility into intermediate steps during specialist-to-specialist handoffs (though the full conversation history including all handoffs is preserved and can be inspected afterward).

Key Features:

- **Automatic handoff detection:** The coordinator invokes a handoff tool whose arguments (for example `{"handoff_to": "shipping_agent"}`) identify the specialist to receive control.
- **Auto-generated tools:** By default the builder synthesizes *handoff_to_<agent>* tools for the coordinator, so you don't manually define placeholder functions.
- **Full conversation history:** The entire conversation (including any *ChatMessage.additional_properties*) is preserved and passed to each agent.
- **Termination control:** By default, terminates after 10 user messages. Override with *.with_termination_condition(lambda conv: ...)* for custom logic (e.g., detect "goodbye").
- **Interaction modes:** Choose *human_in_loop* (default) to prompt users between agent turns, or *autonomous* to continue routing back to agents without prompting for user input until a handoff occurs or a termination/turn limit is reached (default autonomous turn limit: 50).
- **Checkpointing:** Optional persistence for resumable workflows.

Usage (Single-Tier):

Python

```
from agent_framework import HandoffBuilder
from agent_framework.openai import
OpenAIChatClient

chat_client = OpenAIChatClient()

# Create coordinator and specialist agents
coordinator = chat_client.create_agent(
    instructions=(
        "You are a frontline support agent. Assess
the user's issue and decide "
        "whether to hand off to 'refund_agent' or
'shipping_agent'. When delegation is "
        "required, call the matching handoff tool
(for example `handoff_to_refund_agent`)."
    ),
    name="coordinator_agent",
)

refund = chat_client.create_agent(
    instructions="You handle refund requests. Ask
for order details and process refunds.",
    name="refund_agent",
)

shipping = chat_client.create_agent(
```

```

        instructions="You resolve shipping issues.
Track packages and update delivery status.",
        name="shipping_agent",
    )

    # Build the handoff workflow - default single-tier
    routing
    workflow = (
        HandoffBuilder(
            name="customer_support",
            participants=[coordinator, refund,
shipping],
        )
        .set_coordinator(coordinator)
        .build()
    )

    # Run the workflow
    events = await workflow.run_stream("My package
hasn't arrived yet")
    async for event in events:
        if isinstance(event, RequestInfoEvent):
            # Request user input
            user_response = input("You: ")
            await
workflow.send_response(event.data.request_id,
user_response)

```

Multi-Tier Routing with `.add_handoff()`:

Python

```

    # Enable specialist-to-specialist handoffs with
    fluent API
    workflow = (
        HandoffBuilder(participants=[coordinator,
replacement, delivery, billing])
        .set_coordinator(coordinator)
        .add_handoff(coordinator, [replacement,
delivery, billing]) # Coordinator routes to all
        .add_handoff(replacement, [delivery, billing])
    # Replacement delegates to delivery/billing
        .add_handoff(delivery, billing) # Delivery
escalates to billing
        .build()
    )

    # Flow: User → Coordinator → Replacement →
    Delivery → Back to User
    # (Replacement hands off to Delivery without
    returning to user)

```

Use Participant Factories for State Isolation:

Custom Termination Condition:

Python

```
# Terminate when user says goodbye or after 5 exchanges
workflow = (
    HandoffBuilder(participants=[coordinator,
refund, shipping])
    .set_coordinator(coordinator)
    .with_termination_condition(
        lambda conv: (
            sum(1 for msg in conv if
msg.role.value == "user") >= 5
            or any("goodbye" in msg.text.lower()
for msg in conv[-2:]))
        )
    .build()
)
```

Checkpointing:

Python

```
from agent_framework import
InMemoryCheckpointStorage

storage = InMemoryCheckpointStorage()
workflow = (
    HandoffBuilder(participants=[coordinator,
refund, shipping])
    .set_coordinator(coordinator)
    .with_checkpointing(storage)
    .build()
)
```

Initialize a HandoffBuilder for creating conversational handoff workflows.

The builder starts in an unconfigured state and requires you to call:

1. `.participants([...])` - Register agents
2. or `.participant_factories({...})` - Register agent/executor factories
3. `.set_coordinator(...)` - Designate which agent receives initial user input
4. `.build()` - Construct the final Workflow

Optional configuration methods allow you to customize context management, termination logic, and persistence.

ⓘ **Note**

Participants must have stable names/ids because the workflow maps the handoff tool arguments to these identifiers. Agent names should match the strings emitted by the coordinator's handoff tool (e.g., a tool that outputs {"handoff_to": "billing"} requires an agent named billing).

HandoffUserInputRequest	Request message emitted when the workflow needs fresh user input. Note: The conversation field is intentionally excluded from checkpoint serialization to prevent duplication. The conversation is preserved in the coordinator's state and will be reconstructed on restore. See issue #2667.
HostedCodeInterpreterTool	Represents a hosted tool that can be specified to an AI service to enable it to execute generated code. This tool does not implement code interpretation itself. It serves as a marker to inform a service that it is allowed to execute generated code if the service is capable of doing so. Initialize the HostedCodeInterpreterTool.
HostedFileContent	Represents a hosted file content. Initializes a HostedFileContent instance.
HostedFileSearchTool	Represents a file search tool that can be specified to an AI service to enable it to perform file searches. Initialize a FileSearchTool.
HostedMCPSpecificApproval	Represents the specific mode for a hosted tool. When using this mode, the user must specify which tools always or never require approval. This is represented as a dictionary with two optional keys:
HostedMCPTool	Represents a MCP tool that is managed and executed by the service.

	Create a hosted MCP tool.
HostedVectorStoreContent	Represents a hosted vector store content. Initializes a HostedVectorStoreContent instance.
HostedWebSearchTool	Represents a web search tool that can be specified to an AI service to enable it to perform web searches. Initialize a HostedWebSearchTool.
InMemoryCheckpointStorage	In-memory checkpoint storage for testing and development. Initialize the memory storage.
InProcRunnerContext	In-process execution context for local execution and optional checkpointing. Initialize the in-process execution context.
MCPStdioTool	MCP tool for connecting to stdio-based MCP servers. This class connects to MCP servers that communicate via standard input/output, typically used for local processes. Initialize the MCP stdio tool.  Note The arguments are used to create a StdioServerParameters object, which is then used to create a stdio client. See <code>mcp.client.stdio.stdio_client</code> and <code>mcp.client.stdio.stdio_server_parameters</code> for more details.
MCPStreamableHTTPTool	MCP tool for connecting to HTTP-based MCP servers. This class connects to MCP servers that communicate via streamable HTTP/SSE. Initialize the MCP streamable HTTP tool.  Note The arguments are used to create a streamable HTTP client.

See `mcp.client.streamable_http.streamablehttp_client` for more details.

Any extra arguments passed to the constructor will be passed to the

streamable HTTP client constructor.

MCPWebsocketTool

MCP tool for connecting to WebSocket-based MCP servers.

This class connects to MCP servers that communicate via WebSocket.

Initialize the MCP WebSocket tool.

ⓘ Note

The arguments are used to create a WebSocket client.

See `mcp.client.websocket.websocket_client` for more details.

Any extra arguments passed to the constructor will be passed to the

WebSocket client constructor.

MagenticBuilder

Fluent builder for creating Magentic One multi-agent orchestration workflows.

Magentic One workflows use an LLM-powered manager to coordinate multiple agents through dynamic task planning, progress tracking, and adaptive replanning. The manager creates plans, selects agents, monitors progress, and determines when to replan or complete.

The builder provides a fluent API for configuring participants, the manager, optional plan review, checkpointing, and event callbacks.

Human-in-the-loop Support: Magentic provides specialized HITL mechanisms via:

- `.with_plan_review()` - Review and approve/revise plans before execution
- `.with_human_input_on_stall()` - Intervene when workflow stalls
- Tool approval via *FunctionApprovalRequestContent* - Approve individual tool calls

These emit *MagenticHumanInterventionRequest* events that provide structured decision options (APPROVE, REVISE, CONTINUE, REPLAN, GUIDANCE) appropriate for Magentic's planning-based orchestration.

Usage:

Python

```
from agent_framework import MagenticBuilder,
StandardMagenticManager
from azure.ai.projects.aio import AIProjectClient

# Create manager with LLM client
project_client =
AIProjectClient.from_connection_string(...)
chat_client =
project_client.inference.get_chat_completions_client(
)

# Build Magentic workflow with agents
workflow = (
    MagenticBuilder()
        .participants(researcher=research_agent,
writer=writing_agent, coder=coding_agent)

.with_standard_manager(chat_client=chat_client,
max_round_count=20, max_stall_count=3)
    .with_plan_review(enable=True)
    .with_checkpointing(checkpoint_storage)
    .build()
)

# Execute workflow
async for message in workflow.run("Research and
write article about AI agents"):
    print(message.text)
```

With custom manager:

Python

```
# Create custom manager subclass
class MyCustomManager(MagenticManagerBase):
    async def plan(self, context: MagenticContext)
-> ChatMessage:
    # Custom planning logic
    ...

manager = MyCustomManager()
workflow =
MagenticBuilder().participants(agent1=agent1,
agent2=agent2).with_standard_manager(manager).build()
```

MagenticContext	Context for the Magentic manager.
MagenticManagerBase	Base class for the Magentic One manager.
ManagerDirectiveModel	Pydantic model for structured manager directive output. Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
ManagerSelectionRequest	Request sent to manager agent for next speaker selection. This dataclass packages the full conversation state and task context for the manager agent to analyze and make a speaker selection decision.
ManagerSelectionResponse	Response from manager agent with speaker selection decision. The manager agent must produce this structure (or compatible dict/JSON) to communicate its decision back to the orchestrator. Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
Message	A class representing a message in the workflow.
OrchestrationState	Unified state container for orchestrator checkpointing. This dataclass standardizes checkpoint serialization across all three group chat patterns while allowing pattern-specific extensions via metadata. Common attributes cover shared orchestration concerns (task, conversation, round tracking). Pattern-specific state goes in the metadata dict.
RequestInfoEvent	Event triggered when a workflow executor requests external information. Initialize the request info event.
RequestInfoInterceptor	Internal executor that pauses workflow for human input before agent runs.

	This executor is inserted into the workflow graph by builders when <code>.with_request_info()</code> is called. It intercepts <code>AgentExecutorRequest</code> messages BEFORE the agent runs and pauses the workflow via <code>ctx.request_info()</code> with an <code>AgentInputRequest</code>. When a response is received, the response handler injects the input as a user message into the conversation and forwards the request to the agent. The optional <code>agent_filter</code> parameter allows limiting which agents trigger the pause. If the target agent's ID is not in the filter set, the request is forwarded without pausing. Initialize the request info interceptor executor.
Role	Describes the intended purpose of a message within a chat interaction. Properties: SYSTEM: The role that instructs or sets the behavior of the AI system. USER: The role that provides user input for chat interactions. ASSISTANT: The role that provides responses to system-instructed, user-prompted input. TOOL: The role that provides additional information and references in response to tool use requests. Initialize Role with a value.
Runner	A class to run a workflow in Pregel supersteps. Initialize the runner with edges, shared state, and context.
RunnerContext	Protocol for the execution context used by the runner. A single context that supports messaging, events, and optional checkpointing. If checkpoint storage is not configured, checkpoint methods may raise.
SequentialBuilder	High-level builder for sequential agent/executor workflows with shared context. • <code>participants([...])</code> accepts a list of <code>AgentProtocol</code> (recommended) or <code>Executor</code> instances • <code>register_participants([...])</code> accepts a list of factories for <code>AgentProtocol</code> (recommended) or <code>Executor</code> factories • Executors must define a handler that consumes <code>list[ChatMessage]</code> and sends out a <code>list[ChatMessage]</code> • The workflow wires participants in order, passing a <code>list[ChatMessage]</code> down the chain • Agents append their assistant messages to the conversation

- Custom executors can transform/summarize and return a list[ChatMessage]
- The final output is the conversation produced by the last participant

Usage:

Python

```
from agent_framework import SequentialBuilder

# With agent instances
workflow =
SequentialBuilder().participants([agent1, agent2,
summarizer_exec]).build()

# With agent factories
workflow = (

SequentialBuilder().register_participants([create_age
nt1, create_agent2, create_summarizer_exec]).build()
)

# Enable checkpoint persistence
workflow =
SequentialBuilder().participants([agent1,
agent2]).with_checkpointing(storage).build()

# Enable request info for mid-workflow feedback
(pauses before each agent)
workflow =
SequentialBuilder().participants([agent1,
agent2]).with_request_info().build()

# Enable request info only for specific agents
workflow = (
    SequentialBuilder()
    .participants([agent1, agent2, agent3])
    .with_request_info(agents=[agent2]) # Only
pause before agent2
    .build()
)
```

SharedState

A class to manage shared state in a workflow.

SharedState provides thread-safe access to workflow state data that needs to be shared across executors during workflow execution.

Reserved Keys: The following keys are reserved for internal framework use and should not be modified by user code:

- `_executor_state`: Stores executor state for checkpointing (managed by Runner)

⚠ Warning

Do not use keys starting with underscore (`_`) as they may be reserved for

internal framework operations.

Initialize the shared state.

SingleEdgeGroup

Convenience wrapper for a solitary edge, keeping the group API uniform.

Create a one-to-one edge group between two executors.

StandardMagenticManager

Standard Magentic manager that performs real LLM calls via a ChatAgent.

The manager constructs prompts that mirror the original Magentic One orchestration:

- Facts gathering
- Plan creation
- Progress ledger in JSON
- Facts update and plan update on reset
- Final answer synthesis

Initialize the Standard Magentic Manager.

SubWorkflowRequestMessage

Message sent from a sub-workflow to an executor in the parent workflow to request information.

This message wraps a `RequestInfoEvent` emitted by the executor in the sub-workflow.

SubWorkflowResponseMessage

Message sent from a parent workflow to a sub-workflow via `WorkflowExecutor` to provide requested information.

This message wraps the response data along with the original `RequestInfoEvent` emitted by the sub-workflow executor.

SuperStepCompletedEvent

Event triggered when a superstep ends.

Initialize the superstep event.

SuperStepStartedEvent

Event triggered when a superstep starts.

Initialize the superstep event.

SwitchCaseEdgeGroup	Fan-out variant that mimics a traditional switch/case control flow. Each case inspects the message payload and decides whether it should handle the message. Exactly one case-or the default branch-returns a target at runtime, preserving single-dispatch semantics. Configure a switch/case routing structure for a single source executor.
SwitchCaseEdgeGroupCase	Persistable description of a single conditional branch in a switch-case. Unlike the runtime <i>Case</i> object this serialisable variant stores only the target identifier and a descriptive name for the predicate. When the underlying callable is unavailable during deserialisation we substitute a proxy placeholder that fails loudly, ensuring the missing dependency is immediately visible. Record the routing metadata for a conditional case branch.
SwitchCaseEdgeGroupDefault	Persistable descriptor for the fallback branch of a switch-case group. The default branch is guaranteed to exist and is invoked when every other case predicate fails to match the payload. Point the default branch toward the given executor identifier.
TextContent	Represents text content in a chat. Initializes a TextContent instance.
TextReasoningContent	Represents text reasoning content in a chat. Remarks: This class and <i>TextContent</i> are superficially similar, but distinct. Initializes a TextReasoningContent instance.
TextSpanRegion	Represents a region of text that has been annotated. Initialize TextSpanRegion.
ToolMode	Defines if and how tools are used in a chat request. Initialize ToolMode.
ToolProtocol	Represents a generic tool. This protocol defines the interface that all tools must implement to be compatible with the agent framework. It is implemented by various tool classes such as HostedMCPTool, HostedWebSearchTool, and AIFunction's. A AIFunction is usually created by the <i>ai_function</i> decorator.

Messages between executors are delivered at the end of each superstep and are not visible in the event stream. Only workflow-level events (outputs, custom events) and status events are observable to callers.

Input/Output Types

Workflow types are discovered at runtime by inspecting:

- Input types: From the start executor's input types
- Output types: Union of all executors' workflow output types
Access these via the `input_types` and `output_types` properties.

Execution Methods

The workflow provides two primary execution APIs, each supporting multiple scenarios:

- `run()`: Execute to completion, returns `WorkflowRunResult` with all events
- `run_stream()`: Returns async generator yielding events as they occur

Both methods support:

- Initial workflow runs: Provide *message* parameter
- Checkpoint restoration: Provide *checkpoint_id* (and optionally *checkpoint_storage*)
- HIL continuation: Provide *responses* to continue after `RequestInfoExecutor` requests
- Runtime checkpointing: Provide *checkpoint_storage* to enable/override checkpointing for this run

State Management

Workflow instances contain states and states are preserved across calls to `run` and `run_stream`. To execute multiple independent runs, create separate Workflow instances via `WorkflowBuilder`.

External Input Requests

Executors within a workflow can request external input using `ctx.request_info()`:

1. Executor calls `ctx.request_info()` to request input
2. Executor implements `response_handler()` to process the response
3. Requests are emitted as `RequestInfoEvent` instances in the event stream

4. Workflow enters `IDLE_WITH_PENDING_REQUESTS` state
5. Caller handles requests and provides responses via the `send_responses` or `send_responses_streaming` methods
6. Responses are routed to the requesting executors and response handlers are invoked

Checkpointing

Checkpointing can be configured at build time or runtime:

Build-time (via `WorkflowBuilder`): `workflow = WorkflowBuilder().with_checkpointing(storage).build()`

Runtime (via `run/run_stream` parameters): `result = await workflow.run(message, checkpoint_storage=runtime_storage)`

When enabled, checkpoints are created at the end of each superstep, capturing:

- Executor states
- Messages in transit
- Shared state Workflows can be paused and resumed across process restarts using checkpoint storage.

Composition

Workflows can be nested using `WorkflowExecutor`, which wraps a child workflow as an executor. The nested workflow's input/output types become part of the `WorkflowExecutor`'s types. When invoked, the `WorkflowExecutor` runs the nested workflow to completion and processes its outputs.

Initialize the workflow with a list of edges.

`WorkflowAgent`

An *Agent* subclass that wraps a workflow and exposes it as an agent.

Initialize the `WorkflowAgent`.

`WorkflowBuilder`

A builder class for constructing workflows.

This class provides a fluent API for defining workflow graphs by connecting executors with edges and configuring execution parameters. Call `build` to create an immutable `Workflow` instance.

Initialize the `WorkflowBuilder` with an empty list of edges and no starting executor.

`WorkflowCheckpoint`

Represents a complete checkpoint of workflow state.

Checkpoints capture the full execution state of a workflow at a specific point, enabling workflows to be paused and resumed.

⚠ Note

The `shared_state` dict may contain reserved keys managed by the framework.

See `SharedState` class documentation for details on reserved keys.

WorkflowCheckpointSummary

Human-readable summary of a workflow checkpoint.

WorkflowContext

Execution context that enables executors to interact with workflows and other executors.

Overview

`WorkflowContext` provides a controlled interface for executors to send messages, yield outputs, manage state, and interact with the broader workflow ecosystem. It enforces type safety through generic parameters while preventing direct access to internal runtime components.

Type Parameters

The context is parameterized to enforce type safety for different operations:

WorkflowContext (no parameters)

For executors that only perform side effects without sending messages or yielding outputs:

Python

```
async def log_handler(message: str, ctx:
WorkflowContext) -> None:
    print(f"Received: {message}") # Only side
effects
```

WorkflowContext[T_Out]

Enables sending messages of type `T_Out` to other executors:

Python

```

    async def processor(message: str, ctx:
WorkflowContext[int]) -> None:
        result = len(message)
        await ctx.send_message(result) # Send int to
downstream executors

```

WorkflowContext[T_Out, T_W_Out]

Enables both sending messages (T_Out) and yielding workflow outputs (T_W_Out):

Python

```

    async def dual_output(message: str, ctx:
WorkflowContext[int, str]) -> None:
        await ctx.send_message(42) # Send int message
        await ctx.yield_output("complete") # Yield
str workflow output

```

Union Types

Multiple types can be specified using union notation:

Python

```

    async def flexible(message: str, ctx:
WorkflowContext[int | str, bool | dict]) -> None:
        await ctx.send_message("text") # or send 42
        await ctx.yield_output(True) # or yield
{"status": "done"}

```

Initialize the executor context with the given workflow context.

WorkflowErrorDetails	Structured error information to surface in error events/results.
WorkflowEvent	Base class for workflow events. Initialize the workflow event with optional data.
WorkflowExecutor	An executor that wraps a workflow to enable hierarchical workflow composition.

Overview

WorkflowExecutor makes a workflow behave as a single executor within a parent workflow, enabling nested workflow architectures. It handles the complete lifecycle of sub-workflow execution including event processing, output forwarding, and request/response coordination between parent and child workflows.

Execution Model

When invoked, WorkflowExecutor:

1. Starts the wrapped workflow with the input message
2. Runs the sub-workflow to completion or until it needs external input
3. Processes the sub-workflow's complete event stream after execution
4. Forwards outputs to the parent workflow as messages
5. Handles external requests by routing them to the parent workflow
6. Accumulates responses and resumes sub-workflow execution

Event Stream Processing

WorkflowExecutor processes events after sub-workflow completion:

Output Forwarding

All outputs from the sub-workflow are automatically forwarded to the parent:

When *allow_direct_output* is False (default):

Python

```
# An executor in the sub-workflow yields outputs
await ctx.yield_output("sub-workflow result")

# WorkflowExecutor forwards to parent via
ctx.send_message()
# Parent receives the output as a regular message
```

When *allow_direct_output* is True:

Request/Response Coordination

When sub-workflows need external information:

Python

```
# An executor in the sub-workflow makes request
request = MyDataRequest(query="user info")

# WorkflowExecutor captures RequestInfoEvent and
wraps it in a SubWorkflowRequestMessage
# then send it to the receiving executor in parent
workflow. The executor in parent workflow
# can handle the request locally or forward it to
an external source.
# The WorkflowExecutor tracks the pending request,
and implements a response handler.
# When the response is received, it executes the
response handler to accumulate responses
# and resume the sub-workflow when all expected
responses are received.
# The response handler expects a
SubWorkflowResponseMessage wrapping the response
data.
```

State Management

WorkflowExecutor maintains execution state across request/response cycles:

- Tracks pending requests by request_id
- Accumulates responses until all expected responses are received
- Resumes sub-workflow execution with complete response batch
- Handles concurrent executions and multiple pending requests

Type System Integration

WorkflowExecutor inherits its type signature from the wrapped workflow:

Input Types

Matches the wrapped workflow's start executor input types:

Python

```
# If sub-workflow accepts str, WorkflowExecutor
accepts str
workflow_executor = WorkflowExecutor(my_workflow,
id="wrapper")
assert workflow_executor.input_types ==
my_workflow.input_types
```

Output Types

Combines sub-workflow outputs with request coordination types:

Python

```
# Includes all sub-workflow output types
# Plus SubWorkflowRequestMessage if sub-workflow
# can make requests
output_types = workflow.output_types +
[SubWorkflowRequestMessage] # if applicable
```

Error Handling

WorkflowExecutor propagates sub-workflow failures:

- Captures WorkflowFailedEvent from sub-workflow
- Converts to WorkflowErrorEvent in parent context
- Provides detailed error information including sub-workflow ID

Concurrent Execution Support

WorkflowExecutor fully supports multiple concurrent sub-workflow executions:

Per-Execution State Isolation

Each sub-workflow invocation creates an isolated ExecutionContext:

Python

```
# Multiple concurrent invocations are supported
workflow_executor = WorkflowExecutor(my_workflow,
id="concurrent_executor")

# Each invocation gets its own execution context
# Execution 1: processes input_1 independently
# Execution 2: processes input_2 independently
# No state interference between executions
```

Request/Response Coordination

Responses are correctly routed to the originating execution:

- Each execution tracks its own pending requests and expected responses
- Request-to-execution mapping ensures responses reach the correct sub-workflow
- Response accumulation is isolated per execution
- Automatic cleanup when execution completes

Memory Management

- Unlimited concurrent executions supported
- Each execution has unique UUID-based identification
- Cleanup of completed execution contexts
- Thread-safe state management for concurrent access

Important Considerations

Shared Workflow Instance: All concurrent executions use the same underlying workflow instance. For proper isolation, ensure that the wrapped workflow and its executors are stateless.

Python

```
# Avoid: Stateful executor with instance variables
class StatefulExecutor(Executor):
    def __init__(self):
        super().__init__(id="stateful")
        self.data = [] # This will be shared
                        # across concurrent executions!
```

Integration with Parent Workflows

Parent workflows can intercept sub-workflow requests:

Implementation Notes

- Sub-workflows run to completion before processing their results
- Event processing is atomic - all outputs are forwarded before requests
- Response accumulation ensures sub-workflows receive complete response batches
- Execution state is maintained for proper resumption after external requests
- Concurrent executions are fully isolated and do not interfere with each other

Initialize the WorkflowExecutor.

WorkflowFailedEvent	Built-in lifecycle event emitted when a workflow run terminates with an error.
WorkflowOutputEvent	Event triggered when a workflow executor yields output. Initialize the workflow output event.
WorkflowRunResult	Container for events generated during non-streaming workflow execution. Overview Represents the complete execution results of a workflow run, containing all events generated from start to idle state. Workflows produce outputs incrementally through <code>ctx.yield_output()</code> calls during execution. Event Structure Maintains separation between data-plane and control-plane events: • Data-plane events: Executor invocations, completions, outputs, and requests (in main list) • Control-plane events: Status timeline accessible via <code>status_timeline()</code> method Key Methods • <code>get_outputs()</code>: Extract all workflow outputs from the execution • <code>get_request_info_events()</code>: Retrieve external input requests made during execution • <code>get_final_state()</code>: Get the final workflow state (IDLE, IDLE_WITH_PENDING_REQUESTS, etc.) • <code>status_timeline()</code>: Access the complete status event history
WorkflowStartedEvent	Built-in lifecycle event emitted when a workflow run begins. Initialize the workflow event with optional data.
WorkflowStatusEvent	Built-in lifecycle event emitted for workflow run state transitions. Initialize the workflow status event with a new state and optional data.
WorkflowValidationError	Base exception for workflow validation errors.
WorkflowViz	A class for visualizing workflows using graphviz and Mermaid. Initialize the WorkflowViz with a workflow.

Enums

[Expand table](#)

MagneticHumanInterventionDecision	Decision options for human intervention responses.
MagneticHumanInterventionKind	The kind of human intervention being requested.
ValidationTypeEnum	Enumeration of workflow validation types.
WorkflowEventSource	Identifies whether a workflow event came from the framework or an executor. Use <i>FRAMEWORK</i> for events emitted by built-in orchestration paths—even when the code that raises them lives in runner-related modules—and <i>EXECUTOR</i> for events surfaced by developer-provided executor implementations.
WorkflowRunState	Run-level state of a workflow execution. Semantics: • STARTED: Run has been initiated and the workflow context has been created. This is an initial state before any meaningful work is performed. In this codebase we emit a dedicated <i>WorkflowStartedEvent</i> for telemetry, and typically advance the status directly to <i>IN_PROGRESS</i>. Consumers may still rely on <i>STARTED</i> for state machines that need an explicit pre-work phase. • IN_PROGRESS: The workflow is actively executing (e.g., the initial message has been delivered to the start executor or a superstep is running). This status is emitted at the beginning of a run and can be followed by other statuses as the run progresses. • IN_PROGRESS_PENDING_REQUESTS: Active execution while one or more request-for-information operations are outstanding. New work may still be scheduled while requests are in flight. • IDLE: The workflow is quiescent with no outstanding requests and no more work to do. This is the normal terminal state for workflows that have finished executing, potentially having produced outputs along the way. • IDLE_WITH_PENDING_REQUESTS: The workflow is paused awaiting external input (e.g., emitted a <i>RequestInfoEvent</i>). This is a non-terminal state; the workflow can resume when responses are supplied. • FAILED: Terminal state indicating an error surfaced. Accompanied by a <i>WorkflowFailedEvent</i> with structured error details.

- CANCELLED: Terminal state indicating the run was cancelled by a caller or orchestrator. Not currently emitted by default runner paths but included for integrators/orchestrators that support cancellation.

Functions

agent_middleware

Decorator to mark a function as agent middleware.

This decorator explicitly identifies a function as agent middleware, which processes AgentRunContext objects.

Python

```
agent_middleware(func: Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]) -> Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]
```

Parameters

 Expand table

Name	Description
func Required*	<code>Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]</code> The middleware function to mark as agent middleware.

Returns

 Expand table

Type	Description
<code>Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]</code>	The same function with agent middleware marker.

Examples

Python

```

from agent_framework import agent_middleware, AgentRunContext, ChatAgent

@agent_middleware
async def logging_middleware(context: AgentRunContext, next):
    print(f"Before: {context.agent.name}")
    await next(context)
    print(f"After: {context.result}")

# Use with an agent
agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)

```

ai_function

Decorate a function to turn it into a AIFunction that can be passed to models and executed automatically.

This decorator creates a Pydantic model from the function's signature, which will be used to validate the arguments passed to the function and to generate the JSON schema for the function's parameters.

To add descriptions to parameters, use the `Annotated` type from `typing` with a string description as the second argument. You can also use Pydantic's `Field` class for more advanced configuration.

ⓘ Note

When `approval_mode` is set to "always_require", the function will not be executed until explicit approval is given, this only applies to the auto-invocation flow.

It is also important to note that if the model returns multiple function calls, some that require approval

and others that do not, it will ask approval for all of them.

Python

```

ai_function(func: Callable[[...], ReturnT | Awaitable[ReturnT]] | None = None, *,
name: str | None = None, description: str | None = None, approval_mode:
Literal['always_require', 'never_require'] | None = None, max_invocations: int |
None = None, max_invocation_exceptions: int | None = None, additional_properties:

```

```
dict[str, Any] | None = None) -> AIFunction[Any, ReturnT] |  
Callable[[Callable[[...], ReturnT | Awaitable[ReturnT]]], AIFunction[Any,  
ReturnT]]
```

Parameters

 Expand table

Name	Description
func	Callable [[...], <xref:agent_framework_tools.ReturnT> Awaitable <xref:agent_framework_tools.ReturnT>]] None The function to decorate. Default value: None
name Required*	str None
description Required*	str None
approval_mode Required*	Literal ['always_require', 'never_require'] None
max_invocations Required*	int None
max_invocation_exceptions Required*	int None
additional_properties Required*	dict [str , Any] None

Keyword-Only Parameters

 Expand table

Name	Description
name	The name of the function. If not provided, the function's <code>__name__</code> attribute will be used. Default value: None
description	A description of the function. If not provided, the function's docstring will be used. Default value: None
approval_mode	Whether or not approval is required to run this tool. Default is that approval is not needed.

Name	Description
	Default value: None
<code>max_invocations</code>	The maximum number of times this function can be invoked. If None, there is no limit, should be at least 1. Default value: None
<code>max_invocation_exceptions</code>	The maximum number of exceptions allowed during invocations. If None, there is no limit, should be at least 1. Default value: None
<code>additional_properties</code>	Additional properties to set on the function. Default value: None

Returns

 Expand table

Type	Description
<code>AIFunction[Any, <xref:agent_framework_tools.ReturnT>] Callable[[Callable[...], <xref:agent_framework_tools.ReturnT> Awaitable[<xref:agent_framework_tools.ReturnT>]]], AIFunction[Any, <xref:agent_framework_tools.ReturnT>]</code>	

Examples

Python

```
from agent_framework import ai_function
from typing import Annotated

@ai_function
def ai_function_example(
    arg1: Annotated[str, "The first argument"],
    arg2: Annotated[int, "The second argument"],
) -> str:
    # An example function that takes two arguments and returns a string.
    return f"arg1: {arg1}, arg2: {arg2}"

# the same function but with approval required to run
@ai_function(approval_mode="always_require")
def ai_function_example(
    arg1: Annotated[str, "The first argument"],
    arg2: Annotated[int, "The second argument"],
) -> str:
```

```

# An example function that takes two arguments and returns a string.
return f"arg1: {arg1}, arg2: {arg2}"

# With custom name and description
@ai_function(name="custom_weather", description="Custom weather function")
def another_weather_func(location: str) -> str:
    return f"Weather in {location}"

# Async functions are also supported
@ai_function
async def async_get_weather(location: str) -> str:
    '''Get weather asynchronously.'''
    # Simulate async operation
    return f"Weather in {location}"

```

chat_middleware

Decorator to mark a function as chat middleware.

This decorator explicitly identifies a function as chat middleware, which processes ChatContext objects.

Python

```

chat_middleware(func: Callable[[ChatContext, Callable[[ChatContext],
Awaitable[None]]], Awaitable[None]]) -> Callable[[ChatContext,
Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]

```

Parameters

[Expand table](#)

Name	Description
func Required*	Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] The middleware function to mark as chat middleware.

Returns

[Expand table](#)

Type	Description
Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]	The same function with chat middleware marker.

Examples

Python

```
from agent_framework import chat_middleware, ChatContext, ChatAgent

@chat_middleware
async def logging_middleware(context: ChatContext, next):
    print(f"Messages: {len(context.messages)}")
    await next(context)
    print(f"Response: {context.result}")

# Use with an agent
agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)
```

create_edge_runner

Factory function to create the appropriate edge runner for an edge group.

Python

```
create_edge_runner(edge_group: EdgeGroup, executors: dict[str, Executor]) ->
EdgeRunner
```

Parameters

[Expand table](#)

Name	Description
edge_group Required*	<xref:agent_framework_workflows_edge.EdgeGroup> The edge group to create a runner for.
executors Required*	dict[str, Executor] Map of executor IDs to executor instances.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_workflows_edge_runner.EdgeRunner>	The appropriate EdgeRunner instance.

executor

Decorator that converts a standalone function into a FunctionExecutor instance.

The @executor decorator is designed for **standalone module-level functions only**. For class-based executors, use the Executor base class with @handler on instance methods.

Supports both synchronous and asynchronous functions. Synchronous functions are executed in a thread pool to avoid blocking the event loop.

Important

Use @executor for standalone functions (module-level or local functions)

Do NOT use @executor with [staticmethod](#) or [classmethod](#)

For class-based executors, subclass Executor and use @handler on instance methods

Usage:

Python

```
# Standalone async function (RECOMMENDED):
@executor(id="upper_case")
async def to_upper(text: str, ctx: WorkflowContext[str]):
    await ctx.send_message(text.upper())

# Standalone sync function (runs in thread pool):
@executor
def process_data(data: str):
    return data.upper()

# For class-based executors, use @handler instead:
class MyExecutor(Executor):
    def __init__(self):
        super().__init__(id="my_executor")

    @handler
    async def process(self, data: str, ctx: WorkflowContext[str]):
        await ctx.send_message(data.upper())
```

Python

```
executor(func: Callable[[...], Any] | None = None, *, id: str | None = None) ->
Callable[[Callable[[...], Any]], FunctionExecutor] | FunctionExecutor
```

Parameters

 Expand table

Name	Description
func	Callable[[...], Any] None The function to decorate (when used without parentheses) Default value: None
id Required*	str None Optional custom ID for the executor. If None, uses the function name.

Keyword-Only Parameters

 Expand table

Name	Description
id	Default value: None

Returns

 Expand table

Type	Description
Callable[[Callable[[...], Any], FunctionExecutor]] FunctionExecutor	A FunctionExecutor instance that can be wired into a Workflow.

Exceptions

 Expand table

Type	Description
ValueError	If used with staticmethod or classmethod (unsupported pattern)

function_middleware

Decorator to mark a function as function middleware.

This decorator explicitly identifies a function as function middleware, which processes `FunctionInvocationContext` objects.

Python

```
function_middleware(func: Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]) ->
Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext],
Awaitable[None]]], Awaitable[None]]
```

Parameters

[Expand table](#)

Name	Description
func Required*	<code>Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]</code> The middleware function to mark as function middleware.

Returns

[Expand table](#)

Type	Description
<code>Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]</code>	The same function with function middleware marker.

Examples

Python

```
from agent_framework import function_middleware, FunctionInvocationContext, ChatAgent
```

```
@function_middleware
async def logging_middleware(context: FunctionInvocationContext, next):
    print(f"Calling: {context.function.name}")
    await next(context)
    print(f"Result: {context.result}")
```

```
# Use with an agent
```

```
agent = ChatAgent(chat_client=client, name="assistant",
middleware=logging_middleware)
```

get_checkpoint_summary

Python

```
get_checkpoint_summary(checkpoint: WorkflowCheckpoint) ->
WorkflowCheckpointSummary
```

Parameters

[↗](#) Expand table

Name	Description
checkpoint Required*	WorkflowCheckpoint

Returns

[↗](#) Expand table

Type	Description
WorkflowCheckpointSummary	

get_logger

Get a logger with the specified name, defaulting to 'agent_framework'.

Python

```
get_logger(name: str = 'agent_framework') -> Logger
```

Parameters

[↗](#) Expand table

Name	Description
name	str The name of the logger. Defaults to 'agent_framework'. Default value: "agent_framework"

Returns

[Expand table](#)

Type	Description
Logger	The configured logger instance.

handler

Decorator to register a handler for an executor.

Python

```
handler(func: Callable[[ExecutorT, Any, ContextT], Awaitable[Any]]) ->
Callable[[ExecutorT, Any, ContextT], Awaitable[Any]]
```

Parameters

[Expand table](#)

Name	Description
func Required*	Callable [[<xref:agent_framework._workflows._executor.ExecutorT> , Any , <xref:agent_framework._workflows._executor.ContextT>], Awaitable [Any]] The function to decorate. Can be None when used without parameters.

Returns

[Expand table](#)

Type	Description
Callable [[<xref:agent_framework._workflows._executor.ExecutorT> , Any , <xref:agent_framework._workflows._executor.ContextT>], Awaitable [Any]]	The decorated function with handler metadata.

Examples

```
@handler async def handle_string(self, message: str, ctx: WorkflowContext[str]) -> None:
```

...

```
@handler async def handle_data(self, message: dict, ctx: WorkflowContext[str | int]) -> None:
```

...

prepare_function_call_results

Prepare the values of the function call results.

Python

```
prepare_function_call_results(content: TextContent | DataContent |
    TextReasoningContent | UriContent | FunctionCallContent | FunctionResultContent |
    ErrorContent | UsageContent | HostedFileContent | HostedVectorStoreContent |
    FunctionApprovalRequestContent | FunctionApprovalResponseContent | Any |
    list[TextContent | DataContent | TextReasoningContent | UriContent |
    FunctionCallContent | FunctionResultContent | ErrorContent | UsageContent |
    HostedFileContent | HostedVectorStoreContent | FunctionApprovalRequestContent |
    FunctionApprovalResponseContent | Any]) -> str
```

Parameters

 Expand table

Name	Description
content Required*	TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent Any list[TextContent DataContent TextReasoningContent UriContent FunctionCallContent FunctionResultContent ErrorContent UsageContent HostedFileContent HostedVectorStoreContent FunctionApprovalRequestContent FunctionApprovalResponseContent Any]

Returns

 Expand table

Type	Description
str	

prepend_agent_framework_to_user_agent

Prepend "agent-framework" to the User-Agent in the headers.

When user agent telemetry is disabled through the `AGENT_FRAMEWORK_USER_AGENT_DISABLED` environment variable, the User-Agent header will not include the agent-framework

information. It will be sent back as is, or as an empty dict when None is passed.

Python

```
prepend_agent_framework_to_user_agent(headers: dict[str, Any] | None = None) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
headers	<code>dict[str, Any] None</code> The existing headers dictionary. Default value: None

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	A new dict with "User-Agent" set to "agent-framework-python/{version}" if headers is None. The modified headers dictionary with "agent-framework-python/{version}" prepended to the User-Agent.

Examples

Python

```
from agent_framework import prepend_agent_framework_to_user_agent

# Add agent-framework to new headers
headers = prepend_agent_framework_to_user_agent()
print(headers["User-Agent"]) # "agent-framework-python/0.1.0"

# Prepend to existing headers
existing = {"User-Agent": "my-app/1.0"}
headers = prepend_agent_framework_to_user_agent(existing)
print(headers["User-Agent"]) # "agent-framework-python/0.1.0 my-app/1.0"
```

response_handler

Decorator to register a handler to handle responses for a request.

Python

```
response_handler(func: Callable[[ExecutorT, Any, Any, ContextT], Awaitable[None]])
-> Callable[[ExecutorT, Any, Any, ContextT], Awaitable[None]]
```

Parameters

[Expand table](#)

Name	Description
func Required*	Callable [[<xref:agent_framework_workflows_request_info_mixin.ExecutorT>, Any , Any , <xref:agent_framework_workflows_request_info_mixin.ContextT>], Awaitable [None]] The function to decorate.

Returns

[Expand table](#)

Type	Description
Callable [[<xref:agent_framework_workflows_request_info_mixin.ExecutorT>, Any , Any , <xref:agent_framework_workflows_request_info_mixin.ContextT>], Awaitable [None]]	The decorated function with handler metadata.

Examples

Python

```
@handler
async def run(self, message: int, context: WorkflowContext[str]) -> None:
    # Example of a handler that sends a request
    ...
    # Send a request with a `CustomRequest` payload and expect a `str`
    response.
    await context.request_info(CustomRequest(...), str)

@response_handler
async def handle_response(
    self,
    original_request: CustomRequest,
    response: str,
    context: WorkflowContext[str],
) -> None:
    # Example of a response handler for the above request
    ...
```



```

@response_handler
async def handle_response(
    self,
    original_request: CustomRequest,
    response: dict,
    context: WorkflowContext[int],
) -> None:
    # Example of a response handler for a request expecting a dict response
    ...

```

setup_logging

Setup the logging configuration for the agent framework.

Python

```
setup_logging() -> None
```

Returns

[Expand table](#)

Type	Description
None	

use_agent_middleware

Class decorator that adds middleware support to an agent class.

This decorator adds middleware functionality to any agent class. It wraps the `run()` and `run_stream()` methods to provide middleware execution.

The middleware execution can be terminated at any point by setting the `context.terminate` property to True. Once set, the pipeline will stop executing further middleware as soon as control returns to the pipeline.

ⓘ Note

This decorator is already applied to built-in agent classes. You only need to use it if you're creating custom agent implementations.

Python

```
use_agent_middleware(agent_class: type[TAgent]) -> type[TAgent]
```

Parameters

[Expand table](#)

Name	Description
agent_class Required*	<code>type[<xref:TAgent>]</code> The agent class to add middleware support to.

Returns

[Expand table](#)

Type	Description
<code>type[~<xref:TAgent>]</code>	The modified agent class with middleware support.

Examples

Python

```
from agent_framework import use_agent_middleware

@use_agent_middleware
class CustomAgent:
    async def run(self, messages, **kwargs):
        # Agent implementation
        pass

    async def run_stream(self, messages, **kwargs):
        # Streaming implementation
        pass
```

use_chat_middleware

Class decorator that adds middleware support to a chat client class.

This decorator adds middleware functionality to any chat client class. It wraps the `get_response()` and `get_streaming_response()` methods to provide middleware execution.

ⓘ Note

This decorator is already applied to built-in chat client classes. You only need to use it if you're creating custom chat client implementations.

Python

```
use_chat_middleware(chat_client_class: type[TChatClient]) -> type[TChatClient]
```

Parameters

[↗](#) Expand table

Name	Description
chat_client_class Required*	<code>type[<xref:TChatClient>]</code> The chat client class to add middleware support to.

Returns

[↗](#) Expand table

Type	Description
<code>type[~<xref:TChatClient>]</code>	The modified chat client class with middleware support.

Examples

Python

```
from agent_framework import use_chat_middleware

@use_chat_middleware
class CustomChatClient:
    async def get_response(self, messages, **kwargs):
        # Chat client implementation
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Streaming implementation
        pass
```

use_function_invocation

Class decorator that enables tool calling for a chat client.

This decorator wraps the `get_response` and `get_streaming_response` methods to automatically handle function calls from the model, execute them, and return the results back to the model for further processing.

Python

```
use_function_invocation(chat_client: type[TChatClient]) -> type[TChatClient]
```

Parameters

 Expand table

Name	Description
<code>chat_client</code> Required*	<code>type[<xref:TChatClient>]</code> The chat client class to decorate.

Returns

 Expand table

Type	Description
<code>type[~<xref:TChatClient>]</code>	The decorated chat client class with function invocation enabled.

Exceptions

 Expand table

Type	Description
<code>ChatClientInitializationError</code>	If the chat client does not have the required methods.

Examples

Python

```
from agent_framework import use_function_invocation, BaseChatClient
```

```

@use_function_invocation
class MyCustomClient(BaseChatClient):
    async def get_response(self, messages, **kwargs):
        # Implementation here
        pass

    async def get_streaming_response(self, messages, **kwargs):
        # Implementation here
        pass

# The client now automatically handles function calls
client = MyCustomClient()

```

validate_workflow_graph

Convenience function to validate a workflow graph.

Python

```
validate_workflow_graph(edge_groups: Sequence[EdgeGroup], executors: dict[str, Executor], start_executor: Executor) -> None
```

Parameters

 Expand table

Name	Description
edge_groups Required*	Sequence [<xref:agent_framework_workflows_edge.EdgeGroup>] list of edge groups in the workflow
executors Required*	dict [str, Executor] Map of executor IDs to executor instances
start_executor Required*	Executor The starting executor (can be instance or ID)

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
WorkflowValidationError	If any validation fails

a2a Package

Classes

 Expand table

A2AAgent Agent2Agent (A2A) protocol implementation.

Wraps an A2A Client to connect the Agent Framework with external A2A-compliant agents via HTTP/JSON-RPC. Converts framework ChatMessages to A2A Messages on send, and converts A2A responses (Messages/Tasks) back to framework types. Inherits BaseAgent capabilities while managing the underlying A2A protocol communication.

Can be initialized with a URL, AgentCard, or existing A2A Client instance.

Initialize the A2AAgent.

A2AAgent Class

Agent2Agent (A2A) protocol implementation.

Wraps an A2A Client to connect the Agent Framework with external A2A-compliant agents via HTTP/JSON-RPC. Converts framework ChatMessages to A2A Messages on send, and converts A2A responses (Messages/Tasks) back to framework types. Inherits BaseAgent capabilities while managing the underlying A2A protocol communication.

Can be initialized with a URL, AgentCard, or existing A2A Client instance.

Initialize the A2AAgent.

Constructor

Python
A2AAgent()

Keyword-Only Parameters

 Expand table

Name	Description
name	The name of the agent. Default value: None
id	The unique identifier for the agent, will be created automatically if not provided. Default value: None
description	A brief description of the agent's purpose. Default value: None
agent_card	The agent card for the agent. Default value: None
url	The URL for the A2A server. Default value: None
client	The A2A client for the agent. Default value: None
http_client	Optional httpx.AsyncClient to use.

Name	Description
	Default value: None
auth_interceptor	Optional authentication interceptor for secured endpoints. Default value: None
timeout	Request timeout configuration. Can be a float (applied to all timeout components), httpx.Timeout object (for full control), or None (uses 10.0s connect, 60.0s read, 10.0s write, 5.0s pool - optimized for A2A operations). Default value: None
kwargs	any additional properties, passed to BaseAgent.

Methods

 Expand table

<code>__init__</code>	Initialize the A2AAgent.
<code>__new__</code>	
<code>as_tool</code>	Create an AIFunction tool that wraps this agent.
<code>deserialize_thread</code>	Deserialize a thread from its serialized state.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_new_thread</code>	Return a new AgentThread instance that is compatible with the agent.
<code>run</code>	Get a response from the agent. This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.
<code>run_stream</code>	Run the agent as a stream. This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

<code>to_dict</code>	Convert the instance and any nested objects to a dictionary. This method performs deep serialization, automatically converting nested <code>SerializationProtocol</code> objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging. Fields marked in <code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code> are automatically excluded from the output, as are any private attributes (starting with '_').
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the A2AAgent.

Python

```
__init__(*, name: str | None = None, id: str | None = None, description: str | None = None, agent_card: AgentCard | None = None, url: str | None = None, client: Client | None = None, http_client: AsyncClient | None = None, auth_interceptor: AuthInterceptor | None = None, timeout: float | Timeout | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>name</code> Required*	<code>str</code> <code>None</code>
<code>id</code> Required*	<code>str</code> <code>None</code>
<code>description</code> Required*	<code>str</code> <code>None</code>
<code>agent_card</code> Required*	<code><xref:a2a.types.AgentCard></code> <code>None</code>
<code>url</code> Required*	<code>str</code> <code>None</code>

Name	Description
client Required*	<xref:a2a.client.client.Client> None
http_client Required*	<xref:httpx.AsyncClient> None
auth_interceptor Required*	<xref:a2a.client.auth.interceptor.AuthInterceptor> None
timeout Required*	float <xref:httpx.Timeout> None
kwargs Required*	Any

Keyword-Only Parameters

 [Expand table](#)

Name	Description
name	The name of the agent. Default value: None
id	The unique identifier for the agent, will be created automatically if not provided. Default value: None
description	A brief description of the agent's purpose. Default value: None
agent_card	The agent card for the agent. Default value: None
url	The URL for the A2A server. Default value: None
client	The A2A client for the agent. Default value: None
http_client	Optional httpx.AsyncClient to use. Default value: None
auth_interceptor	Optional authentication interceptor for secured endpoints. Default value: None
timeout	Request timeout configuration. Can be a float (applied to all timeout components), httpx.Timeout object (for full control), or None (uses 10.0s connect, 60.0s read, 10.0s write, 5.0s pool - optimized for A2A operations).

Name	Description
	Default value: None
kwargs	any additional properties, passed to BaseAgent.

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

as_tool

Create an AIFunction tool that wraps this agent.

Python
<code>as_tool(*, name: str None = None, description: str None = None, arg_name: str = 'task', arg_description: str None = None, stream_callback: Callable[[AgentRunResponseUpdate], None] Callable[[AgentRunResponseUpdate], Awaitable[None]] None = None) -> AIFunction[BaseModel, str]</code>

Parameters

[Expand table](#)

Name	Description
name Required*	str None
description Required*	str None
arg_name Required*	str

Name	Description
arg_description Required*	str None
stream_callback Required*	Callable [[AgentRunResponseUpdate], None] Callable [[AgentRunResponseUpdate], Awaitable [None]] None

Keyword-Only Parameters

[⌵](#) Expand table

Name	Description
name	The name for the tool. If None, uses the agent's name. Default value: None
description	The description for the tool. If None, uses the agent's description or empty string. Default value: None
arg_name	The name of the function argument (default: "task"). Default value: "task"
arg_description	The description for the function argument. If None, defaults to "Task for {tool_name}". Default value: None
stream_callback	Optional callback for streaming responses. If provided, uses run_stream. Default value: None

Returns

[⌵](#) Expand table

Type	Description
AIFunction [<xref:pydantic.main.BaseModel>, str]	An AIFunction that can be used as a tool by other agents.

Exceptions

[⌵](#) Expand table

Type	Description
TypeError	If the agent does not implement AgentProtocol.

Type	Description
ValueError	If the agent tool name cannot be determined.

Examples

Python

```
from agent_framework import ChatAgent

# Create an agent
agent = ChatAgent(chat_client=client, name="research-agent",
description="Performs research tasks")

# Convert the agent to a tool
research_tool = agent.as_tool()

# Use the tool with another agent
coordinator = ChatAgent(chat_client=client, name="coordinator",
tools=research_tool)
```

deserialize_thread

Deserialize a thread from its serialized state.

Python

```
async deserialize_thread(serialized_thread: Any, **kwargs: Any) -> AgentThread
```

Parameters

 Expand table

Name	Description
serialized_thread Required*	Any The serialized thread data.
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
kwargs	Additional keyword arguments.

Returns

[↗](#) Expand table

Type	Description
AgentThread	A new AgentThread instance restored from the serialized state.

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

[↗](#) Expand table

Name	Description
value Required*	

Parameters

[↗](#) Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.

Name	Description
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI
```



```

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework._serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_new_thread

Return a new AgentThread instance that is compatible with the agent.

Python
<code>get_new_thread(**kwargs: Any) -> AgentThread</code>

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments passed to AgentThread.

Returns

[Expand table](#)

Type	Description
AgentThread	A new AgentThread instance configured with the agent's context provider.

run

Get a response from the agent.

This method returns the final result of the agent's execution as a single AgentRunResponse object. The caller is blocked until the final result is available.

Python

```
async run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *, thread: AgentThread | None = None, **kwargs: Any) -> AgentRunResponse
```

Parameters

 Expand table

Name	Description
messages	<code>str ChatMessage list[str] list[ChatMessage] None</code> The message(s) to send to the agent. Default value: None
thread Required*	<code>AgentThread None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
<code>AgentRunResponse</code>	An agent response item.

run_stream

Run the agent as a stream.

This method will return the intermediate steps and final results of the agent's execution as a stream of AgentRunResponseUpdate objects to the caller.

Python

```
async run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] |  
None = None, *, thread: AgentThread | None = None, **kwargs: Any) ->  
AsyncIterable[AgentRunResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the agent. Default value: None
thread Required*	AgentThread None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	The conversation thread associated with the message(s). Default value: None
kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
AsyncIterable[AgentRunResponseUpdate]	

to_dict

Convert the instance and any nested objects to a dictionary.

This method performs deep serialization, automatically converting nested `SerializationProtocol` objects, lists, and dictionaries containing serializable objects. Non-serializable objects are skipped with debug logging.

Fields marked in `DEFAULT_EXCLUDE` and `INJECTABLE` are automatically excluded from the output, as are any private attributes (starting with `'_'`).

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str]</code> <code>None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization beyond the default exclusions (<code>DEFAULT_EXCLUDE</code> and <code>INJECTABLE</code>). Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. When <code>True</code> , <code>None</code> values are omitted from the dictionary. Defaults to <code>True</code> . Default value: <code>True</code>

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance including a 'type' field for type identification during deserialization (unless 'type' is excluded).

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Name	Description
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

display_name

Returns the display name of the agent.

This is the name if present, otherwise the id.

AGENT_PROVIDER_NAME

Python

```
AGENT_PROVIDER_NAME: Final[str] = 'A2A'
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

additional_properties

Python

```
additional_properties: dict[str, Any]
```

middleware

Python

```
middleware: list[Middleware] | None
```

ag_ui Package

Classes

 Expand table

AGUIChatClient

Chat client for communicating with AG-UI compliant servers.

This client implements the BaseChatClient interface and automatically handles:

- Thread ID management for conversation continuity
- State synchronization between client and server
- Server-Sent Events (SSE) streaming
- Event conversion to Agent Framework types

Important: Message History Management This client sends exactly the messages it receives to the server. It does NOT automatically maintain conversation history. The server must handle history via thread_id.

For stateless servers: Use ChatAgent wrapper which will send full message history on each request. However, even with ChatAgent, the server must echo back all context for the agent to maintain history across turns.

Important: Tool Handling (Hybrid Execution - matches .NET)

1. Client tool metadata sent to server - LLM knows about both client and server tools
2. Server has its own tools that execute server-side
3. When LLM calls a client tool, @use_function_invocation executes it locally
4. Both client and server tools work together (hybrid pattern)

The wrapping ChatAgent's @use_function_invocation handles client tool execution automatically when the server's LLM decides to call them.

Initialize the AG-UI chat client.

AGUIEventConverter

Converter for AG-UI events to Agent Framework types.

Handles conversion of AG-UI protocol events to ChatResponseUpdate objects while maintaining state, aggregating content, and tracking metadata.

Initialize the converter with fresh state.

AGUIHttpService	HTTP service for AG-UI protocol communication. Handles HTTP POST requests and Server-Sent Events (SSE) stream parsing for the AG-UI protocol. Initialize the HTTP service.
AgentFrameworkAgent	Wraps Agent Framework agents for AG-UI protocol compatibility. Translates between Agent Framework's AgentProtocol and AG-UI's event-based protocol. Uses orchestrators to handle different execution flows (standard execution, human-in-the-loop, etc.). Orchestrators are checked in order; the first matching orchestrator handles the request. Supports predictive state updates for agentic generative UI, with optional confirmation requirements configurable per use case. Initialize the AG-UI compatible agent wrapper.
ConfirmationStrategy	Strategy for generating confirmation messages during human-in-the-loop flows.
DefaultConfirmationStrategy	Generic confirmation messages suitable for most agents. This preserves the original behavior from v1.
DocumentWriterConfirmationStrategy	Domain-specific confirmation messages for document writing agents.
RecipeConfirmationStrategy	Domain-specific confirmation messages for recipe agents.
TaskPlannerConfirmationStrategy	Domain-specific confirmation messages for task planning agents.

Functions

add_agent_framework_fastapi_endpoint

Add an AG-UI endpoint to a FastAPI app.

Python

```
add_agent_framework_fastapi_endpoint(app: FastAPI, agent: AgentProtocol | AgentFrameworkAgent, path: str = '/', state_schema: Any | None = None, predict_state_config: dict[str, dict[str, str]] | None = None, allow_origins: list[str] | None = None, default_state: dict[str, Any] | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
app Required*	<xref:fastapi.applications.FastAPI> The FastAPI application
agent Required*	AgentProtocol AgentFrameworkAgent The agent to expose (can be raw AgentProtocol or wrapped)
path	str The endpoint path Default value: "/"
state_schema	Any None Optional state schema for shared state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	dict[str, dict[str, str]] None Optional predictive state update configuration. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
allow_origins	list[str] None CORS origins (not yet implemented) Default value: None
default_state	dict[str, Any] None Optional initial state to seed when the client does not provide state keys Default value: None

Returns

[Expand table](#)

Type	Description
None	

AGUIChatClient Class

Chat client for communicating with AG-UI compliant servers.

This client implements the BaseChatClient interface and automatically handles:

- Thread ID management for conversation continuity
- State synchronization between client and server
- Server-Sent Events (SSE) streaming
- Event conversion to Agent Framework types

Important: Message History Management This client sends exactly the messages it receives to the server. It does NOT automatically maintain conversation history. The server must handle history via `thread_id`.

For stateless servers: Use ChatAgent wrapper which will send full message history on each request. However, even with ChatAgent, the server must echo back all context for the agent to maintain history across turns.

Important: Tool Handling (Hybrid Execution - matches .NET)

1. Client tool metadata sent to server - LLM knows about both client and server tools
2. Server has its own tools that execute server-side
3. When LLM calls a client tool, `@use_function_invocation` executes it locally
4. Both client and server tools work together (hybrid pattern)

The wrapping ChatAgent's `@use_function_invocation` handles client tool execution automatically when the server's LLM decides to call them.

Initialize the AG-UI chat client.

Constructor

Python
<code>AGUIChatClient()</code>

Parameters

Name	Description
endpoint Required*	The AG-UI server endpoint URL (e.g., " http://localhost:8888/ )
http_client Required*	Optional httpx.AsyncClient instance. If None, one will be created.
timeout Required*	Request timeout in seconds (default: 60.0)
additional_properties Required*	Additional properties to store
**kwargs Required*	Additional arguments passed to BaseChatClient

Keyword-Only Parameters

 Expand table

Name	Description
endpoint Required*	
http_client	Default value: None
timeout	Default value: 60.0
additional_properties	Default value: None

Examples

Direct usage (server manages thread history):

Python

```
from agent_framework.ag_ui import AGUIChatClient

client = AGUIChatClient(endpoint="http://localhost:8888/")

# First message - thread ID auto-generated
response = await client.get_response("Hello!")
thread_id = response.additional_properties.get("thread_id")

# Second message - server retrieves history using thread_id
response2 = await client.get_response(
```

```

        "How are you?",
        metadata={"thread_id": thread_id}
    )

```

Recommended usage with ChatAgent (client manages history):

Python

```

from agent_framework import ChatAgent
from agent_framework.ag_ui import AGUIChatClient

client = AGUIChatClient(endpoint="http://localhost:8888/")
agent = ChatAgent(name="assistant", client=client)
thread = await agent.get_new_thread()

# ChatAgent automatically maintains history and sends full context
response = await agent.run("Hello!", thread=thread)
response2 = await agent.run("How are you?", thread=thread)

```

Streaming usage:

Python

```

async for update in client.get_streaming_response("Tell me a story"):
    if update.contents:
        for content in update.contents:
            if hasattr(content, "text"):
                print(content.text, end="", flush=True)

```

Context manager:

Python

```

async with AGUIChatClient(endpoint="http://localhost:8888/") as client:
    response = await client.get_response("Hello!")
    print(response.messages[0].text)

```

Methods

[Expand table](#)

<code>__init__</code>	Initialize the AG-UI chat client.
<code>__new__</code>	

<code>close</code>	Close the HTTP client.
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize the AG-UI chat client.

Python

```
__init__(*, endpoint: str, http_client: AsyncClient | None = None, timeout: float = 60.0, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
endpoint Required*	str The AG-UI server endpoint URL (e.g., " http://localhost:8888/ )
http_client Required*	<xref:httpx.AsyncClient> None Optional httpx.AsyncClient instance. If None, one will be created.
timeout Required*	float Request timeout in seconds (default: 60.0)
additional_properties Required*	dict[str, Any] None Additional properties to store
**kwargs Required*	Any Additional arguments passed to BaseChatClient

Keyword-Only Parameters

 Expand table

Name	Description
endpoint Required*	
http_client	Default value: None
timeout	Default value: 60.0
additional_properties	Default value: None

Returns

 Expand table

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

close

Close the HTTP client.

Python
<code>async close() -> None</code>

Returns

[🔗](#) Expand table

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python
<code>create_agent(*, id: str None = None, name: str None = None, description: str None = None, instructions: str None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] None = None, context_providers: ContextProvider list[ContextProvider] AggregateContextProvider None = None, middleware: AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext,</code>

```
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]],

Name	Description
	Awaitable[None] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p	float None

Name	Description
Required*	
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[🔗](#) Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

 Expand table

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
```

```

function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TCIass>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

[Expand table](#)

Name	Description
<code>messages</code>	<code>str ChatMessage list[str] list[ChatMessage]</code>

Name	Description
Required*	The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None

Name	Description
additional_properties Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: <code>None</code>
frequency_penalty	The frequency penalty to use. Default value: <code>None</code>
logit_bias	The logit bias to use. Default value: <code>None</code>
max_tokens	The maximum number of tokens to generate. Default value: <code>None</code>
metadata	Additional metadata to include in the request. Default value: <code>None</code>
model_id	The <code>model_id</code> to use for the agent. Default value: <code>None</code>
presence_penalty	The presence penalty to use. Default value: <code>None</code>
response_format	The format of the response. Default value: <code>None</code>
seed	The random seed to use. Default value: <code>None</code>
stop	The stop sequence(s) for the request. Default value: <code>None</code>
store	Whether to store the response. Default value: <code>None</code>
temperature	The sampling temperature to use. Default value: <code>None</code>

Name	Description
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None
<code>tools</code>	The tools to use for the request. Default value: None
<code>top_p</code>	The nucleus sampling probability to use. Default value: None
<code>user</code>	The user to associate with the request. Default value: None
<code>additional_properties</code>	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
<code>kwargs</code>	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
<code>ChatResponse</code>	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in `kwargs`) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |  
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,  
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None  
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,  
model_id: str | None = None, presence_penalty: float | None = None,  
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str  
| Sequence[str] | None = None, store: bool | None = None, temperature: float |  
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |  
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
```



```
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |  
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |  
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -  
> AsyncIterable[ChatResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None

Name	Description
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable [..., Any] MutableMapping [str, Any] Sequence [ToolProtocol Callable [..., Any] MutableMapping [str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python

```
service_url() -> str
```

Returns

[Expand table](#)

Type	Description
str	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) ->  
dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True.

Name	Description
	Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python
<pre>to_json(*, exclude: set[str] None = None, exclude_none: bool = True, **kwargs: Any) -> str</pre>

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'agui'
```

AGUIEventConverter Class

Converter for AG-UI events to Agent Framework types.

Handles conversion of AG-UI protocol events to ChatResponseUpdate objects while maintaining state, aggregating content, and tracking metadata.

Initialize the converter with fresh state.

Constructor

Python
<code>AGUIEventConverter()</code>

Methods

[Expand table](#)

<code>__init__</code>	Initialize the converter with fresh state.
<code>__new__</code>	
<code>convert_event</code>	Convert a single AG-UI event to ChatResponseUpdate.

`__init__`

Initialize the converter with fresh state.

Python
<code>__init__() -> None</code>

Returns

[Expand table](#)

Type	Description
<code>None</code>	

__new__

Python

```
__new__(**kwargs)
```

convert_event

Convert a single AG-UI event to ChatResponseUpdate.

Python

```
convert_event(event: dict[str, Any]) -> ChatResponseUpdate | None
```

Parameters

[Expand table](#)

Name	Description
event Required*	<code>dict[str, Any]</code> AG-UI event dictionary

Returns

[Expand table](#)

Type	Description
<code>ChatResponseUpdate None</code>	ChatResponseUpdate if event produces content, None otherwise

Examples

RUN_STARTED event:

Python

```
converter = AGUIEventConverter()
event = {"type": "RUN_STARTED", "threadId": "t1", "runId": "r1"}
update = converter.convert_event(event)
assert update.additional_properties["thread_id"] == "t1"
```

TEXT_MESSAGE_CONTENT event:

Python

```
event = {"type": "TEXT_MESSAGE_CONTENT", "messageId": "m1", "delta": "Hello"}
update = converter.convert_event(event)
assert update.contents[0].text == "Hello"
```

AGUIHttpService Class

HTTP service for AG-UI protocol communication.

Handles HTTP POST requests and Server-Sent Events (SSE) stream parsing for the AG-UI protocol.

Initialize the HTTP service.

Constructor

Python

```
AGUIHttpService()
```

Parameters

 Expand table

Name	Description
endpoint Required*	AG-UI server endpoint URL (e.g., " http://localhost:8888/ )
http_client	Optional httpx AsyncClient. If None, creates a new one. Default value: None
timeout	Request timeout in seconds (default: 60.0) Default value: 60.0

Examples

Basic usage:

Python

```
service = AGUIHttpService("http://localhost:8888/")
async for event in service.post_run(
    thread_id="thread_123",
    run_id="run_456",
    messages=[{"role": "user", "content": "Hello"}]
):
    print(event["type"])
```

With context manager:

Python

```
async with AGUIHttpService("http://localhost:8888/") as service:
    async for event in service.post_run(...):
        print(event)
```

Methods

 Expand table

<code>__init__</code>	Initialize the HTTP service.
<code>__new__</code>	
<code>close</code>	Close the HTTP client if owned by this service. Only closes the client if it was created by this service instance. If an external client was provided, it remains the caller's responsibility to close it.
<code>post_run</code>	Post a run request and stream AG-UI events.

`__init__`

Initialize the HTTP service.

Python

```
__init__(endpoint: str, http_client: AsyncClient | None = None, timeout: float = 60.0) -> None
```

Parameters

 Expand table

Name	Description
<code>endpoint</code> Required*	<code>str</code> AG-UI server endpoint URL (e.g., " http://localhost:8888/ )
<code>http_client</code>	<code><xref:htpx.AsyncClient></code> <code>None</code> Optional htpx AsyncClient. If None, creates a new one. Default value: None

Name	Description
timeout	<code>float</code> Request timeout in seconds (default: 60.0) Default value: 60.0

Returns

 Expand table

Type	Description
<code>None</code>	

__new__

Python
<code>__new__(**kwargs)</code>

close

Close the HTTP client if owned by this service.

Only closes the client if it was created by this service instance. If an external client was provided, it remains the caller's responsibility to close it.

Python
<code>async close() -> None</code>

Returns

 Expand table

Type	Description
<code>None</code>	

post_run

Post a run request and stream AG-UI events.

--

Python

```
async post_run(thread_id: str, run_id: str, messages: list[dict[str, Any]], state: dict[str, Any] | None = None, tools: list[dict[str, Any]] | None = None) -> AsyncIterable[dict[str, Any]]
```

Parameters

[Expand table](#)

Name	Description
thread_id Required*	str Thread identifier for conversation continuity
run_id Required*	str Unique run identifier
messages Required*	list[dict[str, Any]] List of messages in AG-UI format
state	dict[str, Any] None Optional state object to send to server Default value: None
tools	list[dict[str, Any]] None Optional list of tools available to the agent Default value: None

Returns

[Expand table](#)

Type	Description
AsyncIterable[dict[str, Any]]	

Exceptions

[Expand table](#)

Type	Description
[<xref:HTTPStatusError>](<xref:httpx.HTTPStatusError>)	If the HTTP request fails
ValueError	If SSE parsing encounters invalid data

Examples

Python

```
service = AGUIHttpService("http://localhost:8888/")
async for event in service.post_run(
    thread_id="thread_abc",
    run_id="run_123",
    messages=[{"role": "user", "content": "Hello"}],
    state={"user_context": {"name": "Alice"}}
):
    if event["type"] == "TEXT_MESSAGE_CONTENT":
        print(event["delta"])
```

AgentFrameworkAgent Class

Wraps Agent Framework agents for AG-UI protocol compatibility.

Translates between Agent Framework's AgentProtocol and AG-UI's event-based protocol. Uses orchestrators to handle different execution flows (standard execution, human-in-the-loop, etc.). Orchestrators are checked in order; the first matching orchestrator handles the request.

Supports predictive state updates for agentic generative UI, with optional confirmation requirements configurable per use case.

Initialize the AG-UI compatible agent wrapper.

Constructor

Python
<code>AgentFrameworkAgent()</code>

Parameters

 Expand table

Name	Description
agent Required*	The Agent Framework agent to wrap
name	Optional name for the agent Default value: None
description	Optional description Default value: None
state_schema	Optional state schema for state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	Configuration for predictive state updates. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
require_confirmation	Whether predictive updates require confirmation. Set to False for agentic generative UI that updates automatically.

Name	Description
	Default value: True
orchestrators	Custom orchestrators (auto-configured if None). Orchestrators are checked in order; first match handles the request. Default value: None
confirmation_strategy	Strategy for generating confirmation messages. Defaults to DefaultConfirmationStrategy if None. Default value: None

Methods

 Expand table

<code>__init__</code>	Initialize the AG-UI compatible agent wrapper.
<code>__new__</code>	
<code>run_agent</code>	Run the agent and yield AG-UI events. This is the ONLY public method - much simpler than the original 376-line implementation. All orchestration logic has been extracted into dedicated Orchestrator classes. The method creates an ExecutionContext with all needed data, then finds the first orchestrator that can handle the request and delegates to it.

`__init__`

Initialize the AG-UI compatible agent wrapper.

Python

```
__init__(agent: AgentProtocol, name: str | None = None, description: str | None = None, state_schema: Any | None = None, predict_state_config: dict[str, dict[str, str]] | None = None, require_confirmation: bool = True, orchestrators: list[Orchestrator] | None = None, confirmation_strategy: ConfirmationStrategy | None = None)
```

Parameters

 Expand table

Name	Description
agent	<code>AgentProtocol</code>

Name	Description
Required*	The Agent Framework agent to wrap
name	str None Optional name for the agent Default value: None
description	str None Optional description Default value: None
state_schema	Any None Optional state schema for state management; accepts dict or Pydantic model/class Default value: None
predict_state_config	dict[str, dict[str, str]] None Configuration for predictive state updates. Format: {"state_key": {"tool": "tool_name", "tool_argument": "arg_name"}} Default value: None
require_confirmation	bool Whether predictive updates require confirmation. Set to False for agentic generative UI that updates automatically. Default value: True
orchestrators	list[<xref:agent_framework_ag_ui_orchestrators.Orchestrator>] None Custom orchestrators (auto-configured if None). Orchestrators are checked in order; first match handles the request. Default value: None
confirmation_strategy	ConfirmationStrategy None Strategy for generating confirmation messages. Defaults to DefaultConfirmationStrategy if None. Default value: None

__new__

Python

```
__new__(**kwargs)
```

run_agent

Run the agent and yield AG-UI events.

This is the ONLY public method - much simpler than the original 376-line implementation. All orchestration logic has been extracted into dedicated Orchestrator classes.

The method creates an ExecutionContext with all needed data, then finds the first orchestrator that can handle the request and delegates to it.

Python

```
async run_agent(input_data: dict[str, Any]) -> AsyncGenerator[BaseEvent, None]
```

Parameters

[Expand table](#)

Name	Description
input_data Required*	<code>dict[str, Any]</code> The AG-UI run input containing messages, state, etc.

Returns

[Expand table](#)

Type	Description
<code>AsyncGenerator</code> [<xref:ag_ui.core.events.BaseEvent> , <code>None</code>]	

Exceptions

[Expand table](#)

Type	Description
<code>RuntimeError</code>	If no orchestrator matches (should never happen if DefaultOrchestrator is last in the chain)

ConfirmationStrategy Class

Strategy for generating confirmation messages during human-in-the-loop flows.

Constructor

Python
<code>ConfirmationStrategy()</code>

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>on_approval_accepted</code>	Generate message when user approves function execution.
<code>on_approval_rejected</code>	Generate message when user rejects function execution.
<code>on_state_confirmed</code>	Generate message when user confirms predictive state changes.
<code>on_state_rejected</code>	Generate message when user rejects predictive state changes.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_approval_accepted`

Generate message when user approves function execution.

Python

```
abstract on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code> List of approved steps with 'description', 'status', etc.

Returns

[Expand table](#)

Type	Description
<code>str</code>	Message to display to user

on_approval_rejected

Generate message when user rejects function execution.

Python

```
abstract on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code> List of rejected steps

Returns

[Expand table](#)

Type	Description
<code>str</code>	Message to display to user

on_state_confirmed

Generate message when user confirms predictive state changes.

Python
<code>abstract on_state_confirmed() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	Message to display to user

on_state_rejected

Generate message when user rejects predictive state changes.

Python
<code>abstract on_state_rejected() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	Message to display to user

DefaultConfirmationStrategy Class

Generic confirmation messages suitable for most agents.

This preserves the original behavior from v1.

Constructor

Python
<code>DefaultConfirmationStrategy()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>on_approval_accepted</code>	Generate generic approval message with step list.
<code>on_approval_rejected</code>	Generate generic rejection message.
<code>on_state_confirmed</code>	Generate generic state confirmation message.
<code>on_state_rejected</code>	Generate generic state rejection message.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_approval_accepted`

Generate generic approval message with step list.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

 Expand table

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

 Expand table

Type	Description
<code>str</code>	

on_approval_rejected

Generate generic rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

 Expand table

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

 Expand table

Type	Description
<code>str</code>	

on_state_confirmed

Generate generic state confirmation message.

Python
<code>on_state_confirmed() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_rejected

Generate generic state rejection message.

Python
<code>on_state_rejected() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

DocumentWriterConfirmationStrategy Class

Domain-specific confirmation messages for document writing agents.

Constructor

Python
<code>DocumentWriterConfirmationStrategy()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>on_approval_accepted</code>	Generate document-specific approval message.
<code>on_approval_rejected</code>	Generate document-specific rejection message.
<code>on_state_confirmed</code>	Generate document-specific state confirmation message.
<code>on_state_rejected</code>	Generate document-specific state rejection message.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_approval_accepted`

Generate document-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_approval_rejected

Generate document-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_confirmed

Generate document-specific state confirmation message.

Python
<code>on_state_confirmed() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_rejected

Generate document-specific state rejection message.

Python
<code>on_state_rejected() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

RecipeConfirmationStrategy Class

Domain-specific confirmation messages for recipe agents.

Constructor

Python
<code>RecipeConfirmationStrategy()</code>

Methods

[Expand table](#)

<code>__init__</code>	
<code>__new__</code>	
<code>on_approval_accepted</code>	Generate recipe-specific approval message.
<code>on_approval_rejected</code>	Generate recipe-specific rejection message.
<code>on_state_confirmed</code>	Generate recipe-specific state confirmation message.
<code>on_state_rejected</code>	Generate recipe-specific state rejection message.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_approval_accepted`

Generate recipe-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_approval_rejected

Generate recipe-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_confirmed

Generate recipe-specific state confirmation message.

Python
<code>on_state_confirmed() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_rejected

Generate recipe-specific state rejection message.

Python
<code>on_state_rejected() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

TaskPlannerConfirmationStrategy Class

Domain-specific confirmation messages for task planning agents.

Constructor

Python
<code>TaskPlannerConfirmationStrategy()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>on_approval_accepted</code>	Generate task-specific approval message.
<code>on_approval_rejected</code>	Generate task-specific rejection message.
<code>on_state_confirmed</code>	Task planners typically don't use state confirmation.
<code>on_state_rejected</code>	Task planners typically don't use state confirmation.

`__init__`

Python
<code>__init__()</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_approval_accepted`

Generate task-specific approval message.

Python

```
on_approval_accepted(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_approval_rejected

Generate task-specific rejection message.

Python

```
on_approval_rejected(steps: list[dict[str, Any]]) -> str
```

Parameters

[Expand table](#)

Name	Description
steps Required*	<code>list[dict[str, Any]]</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_confirmed

Task planners typically don't use state confirmation.

Python
<code>on_state_confirmed() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

on_state_rejected

Task planners typically don't use state confirmation.

Python
<code>on_state_rejected() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

anthropic Package

Classes

 Expand table

AnthropicClient	Anthropic Chat client. Initialize an Anthropic Agent client.
---------------------------------	---

AnthropicClient Class

Anthropic Chat client.

Initialize an Anthropic Agent client.

Constructor

Python
<pre>AnthropicClient()</pre>

Keyword-Only Parameters

 Expand table

Name	Description
<code>api_key</code>	The Anthropic API key to use for authentication. Default value: None
<code>model_id</code>	The ID of the model to use. Default value: None
<code>anthropic_client</code>	An existing Anthropic client to use. If not provided, one will be created. This can be used to further configure the client before passing it in. For instance if you need to set a different <code>base_url</code> for testing or private deployments. Default value: None
<code>additional_beta_flags</code>	Additional beta flags to enable on the client. Default flags are: "mcp-client-2025-04-04", "code-execution-2025-08-25". Default value: None
<code>env_file_path</code>	Path to environment file for loading settings. Default value: None
<code>env_file_encoding</code>	Encoding of the environment file. Default value: None
<code>kwargs</code>	Additional keyword arguments passed to the parent class.

Examples

Python

```
from agent_framework.anthropic import AnthropicClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set ANTHROPIC_API_KEY=your_anthropic_api_key
# ANTHROPIC_CHAT_MODEL_ID=claude-sonnet-4-5-20250929

# Or passing parameters directly
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    api_key="your_anthropic_api_key",
)

# Or loading from a .env file
client = AnthropicClient(env_file_path="path/to/.env")

# Or passing in an existing client
from anthropic import AsyncAnthropic

anthropic_client = AsyncAnthropic(
    api_key="your_anthropic_api_key", base_url="https://custom-anthropic-
endpoint.com"
)
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    anthropic_client=anthropic_client,
)
```

Methods

 Expand table

<code>__init__</code>	Initialize an Anthropic Agent client.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string.

	This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the service URL for the chat client.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Anthropic Agent client.

Python

```
__init__(*, api_key: str | None = None, model_id: str | None = None, anthropic_client: AsyncAnthropic | None = None, additional_beta_flags: list[str] | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
<code>api_key</code> Required*	<code>str</code> <code>None</code>

Name	Description
model_id Required*	str None
anthropic_client Required*	<xref:anthropic.AsyncAnthropic> None
additional_beta_flags Required*	list[str] None
env_file_path Required*	str None
env_file_encoding Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
api_key	The Anthropic API key to use for authentication. Default value: None
model_id	The ID of the model to use. Default value: None
anthropic_client	An existing Anthropic client to use. If not provided, one will be created. This can be used to further configure the client before passing it in. For instance if you need to set a different base_url for testing or private deployments. Default value: None
additional_beta_flags	Additional beta flags to enable on the client. Default flags are: "mcp-client-2025-04-04", "code-execution-2025-08-25". Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Returns

Type	Description
None	

Examples

Python

```
from agent_framework.anthropic import AnthropicClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set ANTHROPIC_API_KEY=your_anthropic_api_key
# ANTHROPIC_CHAT_MODEL_ID=claude-sonnet-4-5-20250929

# Or passing parameters directly
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    api_key="your_anthropic_api_key",
)

# Or loading from a .env file
client = AnthropicClient(env_file_path="path/to/.env")

# Or passing in an existing client
from anthropic import AsyncAnthropic

anthropic_client = AsyncAnthropic(
    api_key="your_anthropic_api_key", base_url="https://custom-anthropic-
endpoint.com"
)
client = AnthropicClient(
    model_id="claude-sonnet-4-5-20250929",
    anthropic_client=anthropic_client,
)
```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None

Name	Description
chat_message_store_factory Required*	Callable [[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list [ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable [[AgentRunContext , Callable [[AgentRunContext , Awaitable [None]]], Awaitable [None]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]] ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]] list [AgentMiddleware Callable [[AgentRunContext , Callable [[AgentRunContext , Awaitable [None]]], Awaitable [None]] FunctionMiddleware Callable [[FunctionInvocationContext , Callable [[FunctionInvocationContext , Awaitable [None]]], Awaitable [None]] ChatMiddleware Callable [[ChatContext , Callable [[ChatContext , Awaitable [None]]], Awaitable [None]]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict [str int , float] None
max_tokens Required*	int None
metadata Required*	dict [str , Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type [<xref:pydantic.main.BaseModel>] None
seed Required*	int None

Name	Description
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>
user Required*	<code>str</code> <code>None</code>
additional_chat_options Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: <code>None</code>
name	The name of the agent. Default value: <code>None</code>
description	A brief description of the agent's purpose. Default value: <code>None</code>
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: <code>None</code>

Name	Description
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request.

Name	Description
	Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
<code>from_dict()</code>

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any] None</code>

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: - Simple injection: <code>{"<type>": {"<parameter>": value}}</code> - Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code>

Name	Description
	Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
```

```
dependencies=dependencies)
# Now ready to make API calls with the injected client
```

Function Injection for Tools - AIFunction runtime dependency:

Python

```
from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
```

```

my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str

Name	Description
	The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None

Name	Description
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>
user Required*	<code>str</code> <code>None</code>
additional_properties Required*	<code>dict[str, Any]</code> <code>None</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: <code>None</code>
frequency_penalty	The frequency penalty to use. Default value: <code>None</code>
logit_bias	The logit bias to use. Default value: <code>None</code>
max_tokens	The maximum number of tokens to generate. Default value: <code>None</code>
metadata	Additional metadata to include in the request. Default value: <code>None</code>

Name	Description
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id	<code>str None</code>

Name	Description
Required*	
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use.

Name	Description
	Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the service URL for the chat client.

Python
<code>service_url() -> str</code>

Returns

 Expand table

Type	Description
<code>str</code>	The service URL for the chat client, or None if not set.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python
<code>to_dict(*, exclude: set[str] None = None, exclude_none: bool = True) -> dict[str, Any]</code>

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>

Name	Description
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
<code>exclude</code> Required*	<code>set[str]</code> <code>None</code>
<code>exclude_none</code> Required*	<code>bool</code>
<code>kwargs</code> Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Additional field names to exclude from serialization. Default value: <code>None</code>
<code>exclude_none</code>	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python
<code>DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}</code>

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'anthropic'
```

azure Package

Classes

 Expand table

AgentCallbackContext	Context supplied to callback invocations.
AgentFunctionApp	Main application class for creating durable agent function apps using Durable Entities. This class uses Durable Entities pattern for agent execution, providing: • Stateful agent conversations • Conversation history management • Signal-based operation invocation • Better state management than orchestrations

Example:

Python

```
from agent_framework.azure import AgentFunctionApp, AzureOpenAIChatClient

# Create agents with unique names
weather_agent = AzureOpenAIChatClient(...).create_agent(
    name="WeatherAgent",
    instructions="You are a helpful weather agent.",
    tools=[get_weather],
)

math_agent = AzureOpenAIChatClient(...).create_agent(
    name="MathAgent",
    instructions="You are a helpful math assistant.",
    tools=[calculate],
)

# Option 1: Pass list of agents during initialization
app = AgentFunctionApp(agents=[weather_agent, math_agent])

# Option 2: Add agents after initialization
```

```

app = AgentFunctionApp()
app.add_agent(weather_agent)
app.add_agent(math_agent)

@app.orchestration_trigger(context_name="context")
def my_orchestration(context):
    writer = app.get_agent(context, "WeatherAgent")
    thread = writer.get_new_thread()
    forecast_task = writer.run("What's the
forecast?", thread=thread)
    forecast = yield forecast_task
    return forecast

```

This creates:

- HTTP trigger endpoint for each agent's requests (if enabled)
- Durable entity for each agent's state management and execution
- Full access to all Azure Functions capabilities

agents

Dictionary of agent name to AgentProtocol instance

enable_health_check

Whether health check endpoint is enabled

Type: `bool`

enable_http_endpoints

Whether HTTP endpoints are created for agents

Type: `bool`

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

Type: `bool`

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

Initialize the AgentFunctionApp.

param agents: List of agent instances to register.

param http_auth_level: HTTP authentication level (default: `func.AuthLevel.FUNCTION`).

param enable_health_check: Enable the built-in health check endpoint (default: `True`).

param enable_http_endpoints: Enable HTTP endpoints for agents (default: `True`).

param enable_mcp_tool_trigger: Enable MCP tool triggers for agents (default: `False`). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients.

param max_poll_retries: Maximum polling attempts when waiting for a response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.

param poll_interval_seconds: Delay in seconds between polling attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.

param default_callback: Optional callback invoked for agents without specific callbacks.

note: If no agents are provided, they can be added later using [add_agent](#).

`init(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel = func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints: bool = True, max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES, poll_interval_seconds: float = DEFAULT_POLL_INTERVAL_SECONDS, enable_mcp_tool_trigger: bool = False, default_callback: AgentResponseCallbackProtocol | None = None)`

Initialize the AgentFunctionApp.

Parameters: * **agents** ([list\[AgentProtocol\]](#) *| *None) – List of agent instances to register.

```
* **http_auth_level**
(<xref:azure.functions.decorators.core.AuthLevel>) -
HTTP authentication level (default:
`func.AuthLevel.FUNCTION`).

* **enable_health_check** (<xref:bool>) - Enable the
built-in health check endpoint (default: `True`).

* **enable_http_endpoints** (<xref:bool>) - Enable
HTTP endpoints for agents (default: `True`).

* **enable_mcp_tool_trigger** (<xref:bool>) - Enable
MCP tool triggers for agents (default: `False`). When
enabled, agents will be exposed as MCP tools that can
be invoked by MCP-compatible clients.
```

```

    * max_poll_retries (<xref:int>) - Maximum polling
    attempts when waiting for a response. Defaults to
    `DEFAULT_MAX_POLL_RETRIES`.

    * poll_interval_seconds (<xref:float>) - Delay in
    seconds between polling attempts. Defaults to
    `DEFAULT_POLL_INTERVAL_SECONDS`.

    * default_callback
    (<xref:agent_framework_azurefunctions._callbacks.AgentR
    esponseCallbackProtocol>* | *<xref:None>*) - Optional
    callback invoked for agents without specific callbacks.

```

Note: If no agents are provided, they can be added later using [add_agent](#).

new(**kwargs)

activity_trigger(input_name: str, activity: str | None = None)

Register an Activity Function.

Parameters: * **input_name** ([str](#)) – Parameter name of the Activity input.

```

    * activity
    (<xref:agent_framework.azure.Optional[str]>*) - Name of
    Activity Function. The value is None by default, in
    which case the name of the method is used.

```

add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> [None](#)

Add an agent to the function app after initialization.

Parameters: * **agent** ([AgentProtocol](#)) – The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.

```

    * callback
    (<xref:agent_framework_azurefunctions._callbacks.AgentR
    esponseCallbackProtocol>* | *<xref:None>*) - Optional
    callback invoked during agent execution

    * enable_http_endpoint (<xref:bool>* | *
    <xref:None>*) - Optional flag to enable/disable HTTP
    endpoint for this agent. The app level

```

enable_http_endpoints setting will override this setting.

* **enable_mcp_tool_trigger** (<xref:bool>* | * <xref:None>) - Optional flag to enable/disable MCP tool trigger for this agent. The app level enable_mcp_tool_trigger setting will override this setting.

Raises: [ValueError](#) – If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

Return type: [None](#)

property agents: [dict](#)[[str](#), [AgentProtocol](#)]

Returns dict of agent names to agent instances.

Returns: Dictionary mapping agent names to their AgentProtocol instances.

property app_script_file: [str](#)

Name of function app script file in which all the functions are defined.

Script file defined here is for placeholder purpose, please refer to worker defined script file path as the single point of truth.

Returns: Script file name.

assistant_create_output(arg_name: [str](#), data_type: [DataType](#) | [str](#) | [None](#) = [None](#), **kwargs) -> [Callable](#)[[...](#), [Any](#)]

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | * <xref:str>* | * <xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for

specifying extra binding fields to include in the `*function.json*`.

Returns: Decorator function.

Return type: [Callable](#)[\[\[...\], Any\]](#)

```
assistant_post_input(arg_name: str, id: str, user_message: str,
ai_connection_name: str | None = "", chat_model: str | OpenAIModels |
None = OpenAIModels.DefaultChatModel,
chat_storage_connection_setting: str | None = 'AzureWebJobsStorage',
collection_name: str | None = 'ChatState', temperature: str | None = '0.5',
top_p: str | None = None, max_tokens: str | None = '100',
is_reasoning_model: bool | None = False, data_type: DataType | str | None
= None, **kwargs) -> Callable\[\[...\], Any\]
```

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **id** (<xref:str>) - The ID of the assistant to
update.

* **user_message** (<xref:str>) - The message entered
by the user for the assistant to respond to.

* **ai_connection_name** (<xref:str>* | *<xref:None>*)
-

    The name of the configuration section for AI
    service
    connectivity.

* **Azure OpenAI**: If specified, looks for
"Endpoint" and/or "Key" in the section.

* If not specified or missing, falls back to the
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*
environment variables.

* **Managed Identity**: Required for user-assigned
identity auth.

* **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY*
environment variable.

* **model** - *Deprecated.* Use *chat_model* instead.
This parameter is unused and will be removed in future
versions.
```

chat_model (`<xref:str>` | `*`
`<xref:azure.functions.decorators.openai.OpenAIModels>`
`| *<xref:None>`) - The deployment or model name of the
 OpenAI Chat Completion API. Default is `"gpt-3.5-turbo"`.

chat_storage_connection_setting (`<xref:str>` | `*`
`<xref:None>`) - The config section name for assistant
 chat table storage. Default is `"AzureWebJobsStorage"`.

collection_name (`<xref:str>` | `*<xref:None>`) -
 The collection or table name used for assistant chat
 storage. Default is `"ChatState"`.

temperature (`<xref:str>` | `*<xref:None>`) -
 Sampling temperature, between 0 and 2. Higher values
 (e.g., 0.8) increase randomness; lower values (e.g.,
 0.2) make output more deterministic.

top_p (`<xref:str>` | `*<xref:None>`) -
 Alternative to temperature sampling. Uses nucleus
 sampling to consider only the tokens comprising the top
`*top_p*` probability mass. Recommended to use either
 this or `*temperature*`.

max_tokens (`<xref:str>` | `*<xref:None>`) -
 Maximum tokens to generate. Total tokens (prompt +
 output) must not exceed the model's context length.

is_reasoning_model (`<xref:bool>` | `*`
`<xref:None>`) - Whether the chat model is a reasoning
 model. If true, `*max_tokens*` and `*temperature*` are not
 supported.

data_type
 (`<xref:azure.functions.decorators.core.DataType>` | `*`
`<xref:str>` | `*<xref:None>`) - Defines how the Functions
 runtime should treat the parameter value.

kwargs - Additional keyword arguments for
 specifying extra binding fields in `*function.json*`.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`assistant_query_input(arg_name: str, id: str, timestamp_utc: str,
 chat_storage_connection_setting: str | None = 'AzureWebJobsStorage',
 collection_name: str | None = 'ChatState', data_type: DataType | str | None
 = None, **kwargs) -> Callable[..., Any]`

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Parameters: * **arg_name** (**str**) – The name of the binding parameter in the function code.

```
* **timestamp_utc** (<xref:str>) - The earliest
timestamp (in UTC) for the messages to retrieve from
the chat history. Must be in ISO 8601 format, e.g.,
*"2023-08-01T00:00:00Z"*.

* **chat_storage_connection_setting** (<xref:str>* |
*<xref:None>) - The name of the configuration section
containing the connection settings for assistant chat
storage. Defaults to *"AzureWebJobsStorage"*.

* **collection_name** (<xref:str>* | *<xref:None>) -
The name of the table or collection used for assistant
chat storage. Defaults to *"ChatState"*.

* **id** (<xref:str>) - The unique identifier of the
assistant whose history is being queried.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
assistant_skill_trigger(arg_name: str, function_description: str,
function_name: str | None = None, parameter_description_json: str | None
= None, data_type: DataType | str | None = None, **kwargs: Any) ->
Callable[..., Any]
```

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when.

Ref: <https://platform.openai.com/docs/guides/function-calling> [↗](#)

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the

extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **function_description** (<xref:str>*) – A
description of the assistant function, which is
provided to the model.

* **function_name** (<xref:str>* | *<xref:None>*) –
The name of the assistant function, which is passed to
the language model.

* **parameter_description_json** (<xref:str>* | *
<xref:None>*) – A JSON-formatted description of the
function parameters, provided to the model. If omitted,
the description is autogenerated.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>*)
– Additional keyword arguments for specifying binding
fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

property `auth_level`: [AuthLevel](#)

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns: Authorization level of the function app.

`blob_input(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *blob_input* decorator adds [<xref:agent_framework.azure.BlobInput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents the blob in the function code.

```
* **path** (<xref:str>) – The path to the blob.

* **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should
treat the parameter value.

* **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *blob_output* decorator adds `<xref:agent_framework.azure.BlobOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents the blob in the function code.

```
* **path** (<xref:str>) - The path to the blob.

* **connection** (<xref:str>) - The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

* **kwargs** - Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The `blob_change_trigger` decorator adds `<xref:agent_framework.azure.BlobTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building `<xref:agent_framework.azure.Function>` object used in worker function indexing model. This is equivalent to defining `BlobTrigger` in the `function.json` which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.InputStream>` object in function code.

```
* **path** (<xref:str>) - The path to the blob.

* **connection** (<xref:str>) - The name of an app
setting or setting collection that specifies how to
connect to Azure Blobs.

* **source**
(<xref:azure.functions.decorators.core.BlobSource>* | *
<xref:None>) - Sets the source of the triggering event.
Use "EventGrid" for an Event Grid-based blob trigger,
which provides much lower latency. The default is
```

"LogsAndContainerScan", which uses the standard polling mechanism to detect changes in the container.

* **data_type** (`<xref:azure.functions.decorators.core.DataType>` | `<xref:None>`) – Defines how Functions runtime should treat the parameter value.

* **kwargs** – Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`cosmos_db_input(arg_name: str, connection: str, database_name: str, container_name: str, partition_key: str | None = None, id: str | None = None, sql_query: str | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The `cosmos_db_input` decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining `CosmosDBInput` in the `function.json`, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> [↗](#) Additional details: <https://aka.ms/cosmosdb-v4-update> [↗](#)

Parameters: * **arg_name** (`str`) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` input object in the function code.

* **connection** (`<xref:str>`) – The name of an app setting or setting container that specifies how to connect to the Azure Cosmos DB account being monitored.

* **database_name** (`<xref:str>`) – The name of the Cosmos DB database that contains the document.

* **container_name** (`<xref:str>`) – The name of the

container that holds the document.

* **partition_key** (<xref:str>* | *<xref:None>) - The partition key value for the document lookup.

* **id** (<xref:str>* | *<xref:None>) - The ID of the document to retrieve.

* **sql_query** (<xref:str>* | *<xref:None>) - An Azure Cosmos DB SQL query used to retrieve multiple documents.

* **preferred_locations** (<xref:str>* | *<xref:None>) - (Optional) Preferred geo-replicated regions. Values should be comma-separated, e.g., "East US,South Central US,North Europe".

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional binding fields to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, id: str | None = None, sql_query: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The `cosmos_db_input_v3` decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining `CosmosDBInput` in the `function.json`, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` input object in the function code.

- * **database_name** (<xref:str>) - The name of the Azure Cosmos DB database containing the target collection.
- * **collection_name** (<xref:str>) - The name of the collection that contains the document(s).
- * **connection_string_setting** (<xref:str>) - The name of the app setting that contains the Azure Cosmos DB connection string.
- * **id** (<xref:str>* | *<xref:None>) - The ID of a single document to retrieve.
- * **sql_query** (<xref:str>* | *<xref:None>) - An Azure Cosmos DB SQL query used to retrieve multiple documents.
- * **partition_key** (<xref:str>* | *<xref:None>) - Specifies the partition key value for the lookup.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
- * **kwargs** - Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
cosmos_db_output(arg_name: str, connection: str, database_name: str,
container_name: str, create_if_not_exists: bool | None = None,
partition_key: str | None = None, container_throughput: int | None =
None, preferred_locations: str | None = None, data_type: DataType | str |
None = None, **kwargs) -> Callable[..., Any]
```

The *cosmos_db_output* decorator adds <xref:agent_framework.azure.CosmosDBOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> [↗]

Parameters: * **arg_name** (**str**) – The name of the variable that represents the Cosmos DB output object in the function code.

```
* **connection** (<xref:str>) - The name of an app
setting or setting collection that specifies how to
connect to the Azure Cosmos DB account.

* **database_name** (<xref:str>) - The name of the
Azure Cosmos DB database containing the target
container.

* **container_name** (<xref:str>) - The name of the
container to which documents are written.

* **create_if_not_exists** (<xref:bool>* | *
<xref:None>) - A boolean indicating whether the
container should be created automatically if it does
not exist.

* **partition_key** (<xref:str>* | *<xref:None>) -
When *create_if_not_exists* is *True*, this defines the
partition key path for the created container.

* **container_throughput** (<xref:int>* | *
<xref:None>) - When *create_if_not_exists* is *True*,
this defines the throughput (Request Units) for the
created container.

* **preferred_locations** (<xref:str>* | *
<xref:None>) - Preferred geographic regions for geo-
replicated database accounts in Azure Cosmos DB. When
set along with *use_multiple_write_locations*, it
enables multi-region writes.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

```
cosmos_db_output_v3(arg_name: str, database_name: str,
collection_name: str, connection_string_setting: str, create_if_not_exists:
bool | None = None, partition_key: str | None = None,
collection_throughput: int | None = None, use_multiple_write_locations:
bool | None = None, preferred_locations: str | None = None, data_type:
DataType | str | None = None, **kwargs) -> Callable[..., Any]
```

The *cosmos_db_output_v3* decorator adds

<xref:agent_framework.azure.CosmosDBOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗]

Parameters: * **arg_name** (**str**) – The name of the variable that represents the Cosmos DB output object in the function code.

```
* database_name (<xref:str>) – The name of the
Azure Cosmos DB database containing the monitored
collection.
```

```
* collection_name (<xref:str>) – The name of the
collection to which documents are written.
```

```
* connection_string_setting (<xref:str>) – The
name of an app setting or setting collection that
specifies how to connect to the Azure Cosmos DB
account.
```

```
* create_if_not_exists (<xref:bool>* | *
<xref:None>) – A boolean indicating whether the
collection should be created automatically if it does
not exist.
```

```
* partition_key (<xref:str>* | *<xref:None>) –
When create_if_not_exists is True, this defines the
partition key path for the created collection.
```

```
* collection_throughput (<xref:int>* | *
<xref:None>) – When create_if_not_exists is True,
this defines the throughput (Request Units) for the
created collection.
```

```
* use_multiple_write_locations (<xref:bool>* | *
```

<xref:None>) – When set to **True** and used in conjunction with **preferred_locations**, this enables multi-region writes in the Azure Cosmos DB service.

** **preferred_locations*** (<xref:str>* | * <xref:None>) – Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB.

** **data_type***
(<xref:azure.functions.decorators.core.DataType>* | * <xref:str>* | * <xref:None>) – Defines how the Functions runtime should treat the parameter value.

** **kwargs*** – Additional keyword arguments for specifying binding fields to include in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`cosmos_db_trigger(arg_name: str, connection: str, database_name: str, container_name: str, lease_connection: str | None = None, lease_database_name: str | None = None, lease_container_name: str | None = None, create_lease_container_if_not_exists: bool | None = None, leases_container_throughput: int | None = None, lease_container_prefix: str | None = None, feed_poll_delay: int | None = None, lease_acquire_interval: int | None = None, lease_expiration_interval: int | None = None, lease_renew_interval: int | None = None, max_items_per_invocation: int | None = None, start_from_beginning: time | None = None, start_from_time: time | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The *cosmos_db_trigger* decorator adds

<xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.

* **connection** (`<xref:str>`) - The name of an app setting or setting collection that specifies how to connect to the monitored Azure Cosmos DB account.

* **database_name** (`<xref:str>`) - The name of the Azure Cosmos DB database containing the monitored collection.

* **container_name** (`<xref:str>`) - The name of the container being monitored.

* **lease_connection** (`<xref:str>`* | *`<xref:None>`) - (Optional) The name of an app setting or setting collection that specifies how to connect to the Cosmos DB account that holds the lease container.

* **lease_database_name** (`<xref:str>`* | *`<xref:None>`) - The name of the database that holds the collection used to store leases.

* **lease_container_name** (`<xref:str>`* | *`<xref:None>`) - (Optional) The name of the container used to store leases. If not set, the default value "leases" is used.

* **create_lease_container_if_not_exists** (`<xref:bool>`* | *`<xref:None>`) - (Optional) If *True*, the leases container is created automatically if it does not exist. Defaults to *False*. Note: When using Azure AD identities, container creation is not allowed, and the function will not start if this is *True*.

* **leases_container_throughput** (`<xref:int>`* | *`<xref:None>`) - (Optional) The number of Request Units (RUs) to assign when the leases container is created. This is used only when `create_lease_container_if_not_exists` is *True*. It is automatically set when configured through the Azure Portal.

* **lease_container_prefix** (`<xref:str>`* | *`<xref:None>`) - (Optional) A prefix added to leases created in the lease container for this function. Use this to allow multiple functions to share the same lease container with different prefixes.

* **feed_poll_delay** (`<xref:int>`* | *`<xref:None>`) - The delay (in milliseconds) between polling a partition for new changes after draining current changes.

* **lease_acquire_interval** (`<xref:int>`* | *`<xref:None>`) - The interval (in milliseconds) to

trigger a task to check if partitions are evenly distributed across host instances.

*** **lease_expiration_interval**** (<xref:int>* | * <xref:None>) - The interval (in milliseconds) for which a lease is held for a partition.

*** **lease_renew_interval**** (<xref:int>* | * <xref:None>) - The interval (in milliseconds) to renew all leases for partitions currently held by an instance.

*** **max_items_per_invocation**** (<xref:int>* | * <xref:None>) - Maximum number of items received per function call.

*** **start_from_beginning**** (<xref:datetime.time>* | * <xref:None>) - If **True**, the trigger starts reading changes from the beginning of the collection's change history instead of the current time.

*** **start_from_time**** (<xref:datetime.time>* | * <xref:None>) - (Optional) The date and time from which to begin reading the change feed. Use ISO 8601 format with a UTC designator, e.g., **2021-02-16T14:19:29Z**. Only used to set the initial trigger state; has no effect once the trigger has a lease state.

*** **preferred_locations**** (<xref:str>* | * <xref:None>) - Preferred locations (regions) for geo-replicated Cosmos DB accounts.

*** **data_type**** (<xref:azure.functions.decorators.core.DataType>* | * <xref:str>* | * <xref:None>) - Defines how the Functions runtime should treat the parameter value.

*** **kwargs**** (<xref:agent_framework.exceptions.Any>) - Additional keyword arguments for specifying binding fields to include in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, lease_collection_name: str | None = None, lease_connection_string_setting: str | None = None, lease_database_name: str | None = None, create_lease_collection_if_not_exists: bool | None = None, leases_collection_throughput: int | None = None, lease_collection_prefix: str | None = None, checkpoint_interval: int | None = None,

```
checkpoint_document_count: int | None = None, feed_poll_delay: int |
None = None, lease_renew_interval: int | None = None,
lease_acquire_interval: int | None = None, lease_expiration_interval: int |
None = None, max_items_per_invocation: int | None = None,
start_from_beginning: bool | None = None, preferred_locations: str |
None = None, data_type: DataType | str | None = None, **kwargs: Any) -
> Callable[[...], Any]
```

The *cosmos_db_trigger_v3* decorator adds

<xref:agent_framework.azure.CosmosDBTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model. This decorator only works with extension
bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which
enables the function to be triggered when Cosmos DB data changes. All
optional fields will be assigned default values by the function host when
they are parsed.

See: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗]

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents
the <xref:agent_framework.azure.DocumentList> object in the function
code.

```
* **database_name** (<xref:str>) – The name of the
Azure Cosmos DB database containing the monitored
collection.
```

```
* **collection_name** (<xref:str>) – The name of the
collection being monitored.
```

```
* **connection_string_setting** (<xref:str>) – The
name of an app setting or setting collection specifying
how to connect to the monitored Azure Cosmos DB
account.
```

```
* **lease_collection_name** (<xref:str>* | *
<xref:None>) – The name of the collection used to store
leases.
```

```
* **lease_connection_string_setting** (<xref:str>* |
*<xref:None>) – The name of an app setting or setting
collection that specifies how to connect to the Azure
Cosmos DB account that holds the lease collection.
```

```
* **lease_database_name** (<xref:str>* | *
<xref:None>) – The name of the database that holds the
lease collection.
```

*** **create_lease_collection_if_not_exists****
([<xref:bool>](#)* | *[<xref:None>](#)) - If **True**, the lease collection is automatically created when it does not already exist.

*** **leases_collection_throughput**** ([<xref:int>](#)* | *[<xref:None>](#)) - The number of Request Units (RUs) assigned when the lease collection is created.

*** **lease_collection_prefix**** ([<xref:str>](#)* | *[<xref:None>](#)) - A prefix added to leases created in the lease collection for this Function.

*** **checkpoint_interval**** ([<xref:int>](#)* | *[<xref:None>](#)) - The interval (in milliseconds) between lease checkpoints. The default behavior is to checkpoint after every function call.

*** **checkpoint_document_count**** ([<xref:int>](#)* | *[<xref:None>](#)) - Number of documents processed between lease checkpoints. Default is after every function call.

*** **feed_poll_delay**** ([<xref:int>](#)* | *[<xref:None>](#)) - The delay (in milliseconds) between polling a partition for new changes after all current changes are drained.

*** **lease_renew_interval**** ([<xref:int>](#)* | *[<xref:None>](#)) - The interval (in milliseconds) to renew leases for partitions currently held by an instance.

*** **lease_acquire_interval**** ([<xref:int>](#)* | *[<xref:None>](#)) - The interval (in milliseconds) to trigger a task that checks whether partitions are evenly distributed among host instances.

*** **lease_expiration_interval**** ([<xref:int>](#)* | *[<xref:None>](#)) - The interval (in milliseconds) for which a lease is held for a partition.

*** **max_items_per_invocation**** ([<xref:int>](#)* | *[<xref:None>](#)) - Maximum number of items received per function call.

*** **start_from_beginning**** ([<xref:bool>](#)* | *[<xref:None>](#)) - If **True**, the trigger starts reading changes from the beginning of the collection's change history rather than from the current time.

*** **preferred_locations**** ([<xref:str>](#)* | *[<xref:None>](#)) - Preferred locations (regions) for geo-replicated database accounts in Azure Cosmos DB.

*** **data_type****


```
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>) - Defines how the Functions  
runtime should treat the parameter value.
```

```
* **kwargs** (<xref:agent_framework.exceptions.Any>)  
- Additional keyword arguments for specifying binding  
fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

dapr_binding_output(arg_name: str, binding_name: str, operation: str,
dapr_address: str | None = None, data_type: DataType | str | None =
None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *dapr_binding_output* decorator adds
<xref:agent_framework.azure.DaprBindingOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*,
which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host
when parsed.

Ref: <https://aka.ms/azure-function-dapr-binding-output-binding> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents
the Dapr state output object in the function code.

```
* **binding_name** (<xref:str>) - The configured name  
of the binding.
```

```
* **operation** (<xref:str>) - The configured  
operation.
```

```
* **dapr_address** (<xref:str>* | *<xref:None>) - The  
Dapr address. Optional; defaults to [http://localhost]  
(http://localhost):{daprHttpPort}.
```

```
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>) - Defines how the Functions  
runtime should treat the parameter value.
```

```
* **kwargs** - Additional keyword arguments for  
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]`

The *dapr_binding_trigger* decorator adds `<xref:agent_framework.azure.DaprBindingTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-binding> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the trigger input in the function code.

```
* **binding_name** (<xref:str>) – The name of the
Dapr trigger. If not specified, the function name is
used as the trigger name.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>)
– Keyword arguments for specifying additional binding
fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The *dapr_invoke_output* decorator adds `<xref:agent_framework.azure.DaprInvokeOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-invoke-output-binding> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents the Dapr state output object in the function code.

```
* app_id (<xref:str>) – The Dapr app ID to invoke.
```

```
* method_name (<xref:str>) – The method name of the app to invoke.
```

```
* http_verb (<xref:str>) – The HTTP verb to use for the invocation (e.g., GET, POST).
```

```
* dapr_address (<xref:str>* | *<xref:None>) – The Dapr address. Optional; defaults to [http://localhost] (http://localhost):{daprHttpPort}.
```

```
* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

```
* kwargs – Additional keyword arguments for specifying binding fields in the function.json.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str,
dapr_address: str | None = None, data_type: DataType | str | None =
None, **kwargs) -> Callable[..., Any]
```

The *dapr_publish_output* decorator adds `<xref:agent_framework.azure.DaprPublishOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-publish-output-binding> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state output object in the function code.

```
* **pub_sub_name** (<xref:str>) – The pub/sub
component name to publish to.

* **topic** (<xref:str>) – The name of the topic to
publish to.

* **dapr_address** (<xref:str>* | *<xref:None>) – The
Dapr address. Optional; defaults to [http://localhost]
(http://localhost):{daprHttpPort}.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *dapr_secret_input* decorator adds [<xref:agent_framework.azure.DaprSecretInput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-secret-input-binding> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state input object in the function code.

- * **secret_store_name** (<xref:str>) - The name of the secret store from which to retrieve the secret.
- * **key** (<xref:str>) - The key identifying the name of the secret to retrieve.
- * **metadata** (<xref:str>) - Metadata properties in the form "key1=value1&key2=value2".
- * **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional; Defaults to [http://localhost] (http://localhost):{daprHttpPort}.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
- * **kwargs** - Additional keyword arguments for specifying binding fields in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type: [DataType](#) | str | None = None, **kwargs: [Any](#)) -> [Callable](#)[..., [Any](#)]

The *dapr_service_invocation_trigger* decorator adds <xref:agent_framework.azure.DaprServiceInvocationTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-service-invocation> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the service invocation input in the function code.

- * **method_name** (<xref:str>) - The name of the method on a remote Dapr App. If not specified, the name of the function is used as the method name.

```
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) - Defines how the Functions  
runtime should treat the parameter value.  
  
* **kwargs** (<xref:agent_framework.exceptions.Any>)  
- Keyword arguments for specifying additional binding  
fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str |
None = None, data_type: [DataType](#) | str | None = None, **kwargs) ->
[Callable](#)[[...](#), [Any](#)]

The *dapr_state_input* decorator adds
<xref:agent_framework.azure.DaprStateInput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which
enables the function to read state from the underlying state store
component.

All optional fields will be assigned default values by the function host
when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-input-binding> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents
the Dapr state input object in the function code.

```
* **state_store** (<xref:str>) - The state store  
containing the state.  
  
* **key** (<xref:str>) - The name of the key to  
retrieve.  
  
* **dapr_address** (<xref:str>* | *<xref:None>*) - The  
Dapr address. Optional; Defaults to [http://localhost]  
(http://localhost):{daprHttpPort}.  
  
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) - Defines how the Functions  
runtime should treat the parameter value.
```

* ****kwargs**** – Additional keyword arguments for specifying binding fields in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[[...], [Any](#)]

The *dapr_state_output* decorator adds [DaprStateOutput](#) to the [FunctionBuilder](#) object for building a [Function](#) object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-output-binding> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state output object in the function code.

* ****state_store**** ([str](#)) – The state store containing the state for the specified keys.

* ****key**** ([str](#)) – The name of the key used to store the state.

* ****dapr_address**** ([str](#)* | *[None](#)>) – The Dapr address. Optional; defaults to [http://localhost] (http://localhost):{daprHttpPort}.

* ****data_type**** ([azure.functions.decorators.core.DataType](#)* | *[str](#)* | *[None](#)>) – Defines how the Functions runtime should treat the parameter value.

* ****kwargs**** – Additional keyword arguments for specifying binding fields in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> [Callable](#)[..., [Any](#)]

The *dapr_topic_trigger* decorator adds [<xref:agent_framework.azure.DaprTopicTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-topic>

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the trigger input in the function code.

```
* **pub_sub_name** (<xref:str>) - The name of the
Dapr pub/sub component.

* **topic** (<xref:str>) - The topic name. If
unspecified, the function name is used.

* **route** (<xref:str>* | *<xref:None>) - The route
for the trigger. If unspecified, the topic name is
used.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>)
- Additional keyword arguments for specifying extra
binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

durable_client_input(client_name: str, task_hub: str | None = None, connection_name: str | None = None)

Register a Durable-client Function.

Parameters: * **client_name** ([str](#)) – Parameter name of durable client.

*** **task_hub****
(<xref:agent_framework.azure.Optional[str]>) – Used in scenarios where multiple function apps share the same storage account but need to be isolated from each other. If not specified, the default value from host.json is used. This value must match the value used by the target orchestrator functions.

*** **connection_name****
(<xref:agent_framework.azure.Optional[str]>) – The name of an app setting that contains a storage account connection string. The storage account represented by this connection string must be the same one used by the target orchestrator functions. If not specified, the default storage account connection string for the function app is used.

```
durable_openai_agent_orchestrator(_func=None, *,  
model_provider=None, model_retry_options: RetryOptions | None =  
RetryOptions(first_retry_interval_in_milliseconds=2000,  
max_number_of_attempts=5))
```

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Parameters: * **model_provider**

(<xref:agent_framework.azure.Optional[ModelProvider]>) – Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.

*** **model_retry_options****
(<xref:azure.durable_functions.models.RetryOptions.RetryOptions>* | *<xref:None>) –

```
embeddings_input(arg_name: str, input: str, input_type: InputType,  
ai_connection_name: str | None = "", embeddings_model: str |  
OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel,  
max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128,  
data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: <https://platform.openai.com/docs/guides/embeddings> 

Parameters: * **arg_name** (**str**) – The name of the binding parameter in the function code.

* **input** (<xref:str>) - The input source containing the data to generate embeddings for.

* **input_type** (<xref:azure.functions.decorators.openai.InputType>) - The type of the input (e.g., string, list, file reference).

* **ai_connection_name** (<xref:str>* | *<xref:None>) -

The name of the configuration section for AI service connectivity settings.

* **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in this section.

* If not specified or not found, falls back to the environment variables `*AZURE_OPENAI_ENDPOINT*` and `*AZURE_OPENAI_KEY*`.

* For user-assigned managed identity authentication, this is required.

* **OpenAI (non-Azure)**: Set the `*OPENAI_API_KEY*` environment variable.

* **model** - *(Deprecated)* Use `*embeddings_model*` instead. This parameter is unused and will be removed in future versions.

* **embeddings_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name for OpenAI Embeddings. Default is `"text-embedding-ada-002"`.

* **max_chunk_length** (<xref:int>* | *<xref:None>) - The maximum number of characters into which the input should be chunked. Default is `*8 * 1024*`.

* **max_overlap** (<xref:int>* | *<xref:None>) - The maximum number of characters to overlap between input chunks. Default is `*128*`.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Optional. Defines how the Functions runtime should interpret the parameter value.

* **kwargs** - Additional keyword arguments to include in the `*function.json*` binding configuration.

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`embeddings_store_output(arg_name: str, input: str, input_type: InputType, store_connection_name: str, collection: str, ai_connection_name: str | None = "", embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- Azure AI Search
- Azure Data Explorer
- Azure Cosmos DB using MongoDB

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **input** (<xref:str>) – The input data for which
embeddings should be generated.

* **input_type**
(<xref:azure.functions.decorators.openai.InputType>) –
The type of the input (e.g., string, list).

* **store_connection_name** (<xref:str>) – The name
of an app setting or environment variable containing
the vector store connection value.

* **collection** (<xref:str>) – The collection or
table to search within.

* **ai_connection_name** (<xref:str>* | *<xref:None>)
–

The name of the configuration section for AI
service
connectivity settings.

* For Azure OpenAI: If specified, looks for
"Endpoint" and/or "Key" in this configuration section.

* If not specified, or if the section is missing,
falls back to environment variables:
```

`*AZURE_OPENAI_ENDPOINT*` and `*AZURE_OPENAI_KEY*`.

* For user-assigned managed identity, this property is required.

* For OpenAI (non-Azure), set the `*OPENAI_API_KEY*` environment variable.

* **`**model**`** - Deprecated. Use `*embeddings_model*` instead. This parameter is unused and will be removed in future versions.

* **`**embeddings_model**`** (`<xref:str>*` | `*<xref:azure.functions.decorators.openai.OpenAIModels>*` | `*<xref:None>*`) - The deployment or model name for OpenAI Embeddings. The default is "text-embedding-ada-002".

* **`**max_chunk_length**`** (`<xref:int>*` | `*<xref:None>*`) - The maximum number of characters to chunk the input into.

* **`**max_overlap**`** (`<xref:int>*` | `*<xref:None>*`) - The maximum number of characters to overlap between chunks.

* **`**data_type**`**
(`<xref:azure.functions.decorators.core.DataType>*` | `*<xref:str>*` | `*<xref:None>*`) - Optional. Defines how the Functions runtime should treat the parameter value. Defaults to None.

* **`**kwargs**`** - Additional keyword arguments to include in the `*function.json*` binding.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

enable_health_check: [bool](#)

enable_http_endpoints: [bool](#)

enable_mcp_tool_trigger: [bool](#)

entity_trigger(context_name: str, entity_name: str | None = None)

Register an Entity Function.

Parameters: * **context_name** ([str](#)) - Parameter name of the Entity input.

* **`**entity_name**`**
(`<xref:agent_framework.azure.Optional[str]>*`) - Name of

Entity Function. The value is None by default, in which case the name of the method is used.

```
event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None,
topic_key_setting: str | None = None, connection: str | None = None,
data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

The *event_grid_output* decorator adds

<xref:agent_framework.azure.EventGridOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model.

This is equivalent to defining the output binding in the *function.json*,
which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host
when parsed.

Ref: <https://aka.ms/eventgridtrigger> ↗

Parameters: * **arg_name** (**str**) – The variable name used in the function
code that represents the event.

```
* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) – Defines how the Functions
runtime should treat the parameter value.

* **topic_endpoint_uri** (<xref:str>* | *<xref:None>*)
– The name of an app setting that contains the URI for
the custom topic.

* **topic_key_setting** (<xref:str>* | *<xref:None>*)
– The name of an app setting that contains an access
key for the custom topic.

* **connection** (<xref:str>* | *<xref:None>*) – The
value of the common prefix for the setting that
contains the topic endpoint URI.
```

Returns: Decorator function.

Return type: [Callable\[\[...\], Any\]](#)

```
event_grid_trigger(arg_name: str, data_type: DataType | str | None =
None, **kwargs) -> Callable[[...], Any]
```

The *event_grid_trigger* decorator adds

<xref:agent_framework.azure.EventGridTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function
indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which
enables the function to be triggered in response to an event sent to an
Event Grid topic.

All optional fields will be given default values by the function host when
they are parsed.

Ref: <https://aka.ms/eventgridtrigger> ↗

Parameters: * **arg_name** ([str](#)) – The variable name used in the function
code for the parameter that receives the event data.

```
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) – Defines how the Functions  
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
event_hub_message_trigger(arg_name: str, connection: str,  
event_hub_name: str, data_type: DataType | str | None = None, cardinality:  
Cardinality | str | None = None, consumer_group: str | None = None,  
**kwargs: Any) -> Callable[..., Any]
```

The *event_hub_message_trigger* decorator adds
<xref:agent_framework.azure.EventHubTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function
indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*,
which enables the function to be triggered when new message(s) are sent
to the Event Hub.

All optional fields will be given default values by the function host when
they are parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents
the <xref:agent_framework.azure.EventHubEvent> object in the function
code.

* **connection** (`<xref:str>`) – The name of an app setting or setting collection that specifies how to connect to Event Hubs.

* **event_hub_name** (`<xref:str>`) – The name of the Event Hub.

* **data_type**
 (`<xref:azure.functions.decorators.core.DataType>`* | *`<xref:str>`* | *`<xref:None>`) – Defines how the Functions runtime should treat the parameter value.

* **cardinality**
 (`<xref:azure.functions.decorators.core.Cardinality>`* | *`<xref:str>`* | *`<xref:None>`) – Set to "many" to enable batching.

* **consumer_group** (`<xref:str>`* | *`<xref:None>`) – Optional. The consumer group used to subscribe to events in the hub.

* **kwargs** (`<xref:agent_framework.exceptions.Any>`) – Additional keyword arguments to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *event_hub_output* decorator adds `<xref:agent_framework.azure.EventHubOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> ↗

Parameters: * **arg_name** (`str`) – The name of the variable representing the Event Hub output object in the function code.

* **connection** ([<xref:str>](#)) – The name of an app setting or setting collection that specifies how to connect to Event Hub.

* **event_hub_name** ([<xref:str>](#)) – The name of the Event Hub to send messages to.

* **data_type**
([<xref:azure.functions.decorators.core.DataType>](#)* | *
[<xref:str>](#)* | *[<xref:None>](#)) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[..., Any]`

Optional: Sets name of the [<xref:agent_framework.azure.Function>](#) object. If not set, it will default to the name of the method name.

Parameters: * **name** ([str](#)) – Name of the function.

* **setting_extra_fields** ([<xref:typing.Dict>](#)*[*
[<xref:str>](#)*, *[<xref:agent_framework.exceptions.Any>](#)*]
**| *[<xref:None>](#)) – Keyword arguments for specifying

Return type: [Callable](#)[[...](#), [Any](#)]

additional setting fields :return: Decorator function.

`generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *generic_input_binding* decorator adds [<xref:agent_framework.azure.GenericInputBinding>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> ↗

Parameters: * **arg_name** ([str](#)) – The name of the input parameter in the function code.

```
* **type** (<xref:str>) – The type of the binding.  
  
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) – Defines how the Functions  
runtime should treat the parameter value.  
  
* **kwargs** – Additional keyword arguments for  
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[\[...\]](#), [Any](#)]

`generic_output_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[\[...\], Any]`

The *generic_output_binding* decorator adds `<xref:agent_framework.azure.GenericOutputBinding>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> ↗

Parameters: * **arg_name** ([str](#)) – The name of the output parameter in the function code.

```
* **type** (<xref:str>) – The type of the binding.  
  
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) – Defines how the Functions
```

```
runtime should treat the parameter value.
```

```
* **kwargs** - Additional keyword arguments for  
specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`generic_trigger(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *generic_trigger* decorator adds `<xref:agent_framework.azure.GenericTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-custom> ↗

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **type** (<xref:str>) - The type of the binding.  
  
* **data_type**  
(<xref:azure.functions.decorators.core.DataType>* | *  
<xref:str>* | *<xref:None>*) - Defines how the Functions  
runtime should treat the parameter value.  
  
* **kwargs** - Keyword arguments for specifying  
additional binding fields to include in the binding  
JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`get_agent(context: Any, agent_name: str) ->
<xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>`

Return a `DurableAIAgent` proxy for a registered agent.

Parameters: * **context** ([Any](#)) – Durable Functions orchestration context invoking the agent.

* **agent_name** (<xref:str>) – Name of the agent registered on this app.

Raises: [ValueError](#) – If the requested agent has not been registered.

Returns: DurableAIAgent wrapper bound to the orchestration context.

Return type:

<xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>

get_functions() -> [List\[Function\]](#)

Get the function objects in the function app.

Returns: A list of <xref:agent_framework.azure.Function> objects defined in the app.

Return type: [List\[Function\]](#)

http_type(http_type: str) -> [Callable\[\[...\], Any\]](#)

Set http type of the <xref:agent_framework.azure.Function> object.

Parameters: **http_type** ([str](#)) – Http type of the function.

Returns: Decorator function.

Return type: [Callable\[\[...\], Any\]](#)

kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000, batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms: int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647, authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol: BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType | str | None = None, **kwargs) -> [Callable\[\[...\], Any\]](#)

The *kafka_output* decorator adds

<xref:agent_framework.azure.KafkaOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker
function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*,
which enables the function to write events to a Kafka topic. All optional
fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/kafkaoutput> ↗

Parameters: * **arg_name** ([str](#)) – The variable name used in the function
code that represents the output event.

* **topic** (<xref:str>) – The Kafka topic to which
messages are published.

* **broker_list** (<xref:str>) – The list of Kafka
brokers to which the producer connects.

* **avro_schema** (<xref:str>* | *<xref:None>) –
Optional. Avro schema to generate a generic record.

* **username** (<xref:str>* | *<xref:None>) – SASL
username for use with the PLAIN and SASL-SCRAM
mechanisms. Equivalent to `'sasl.username'` in
`librdkafka`.

* **password** (<xref:str>* | *<xref:None>) – SASL
password for use with the PLAIN and SASL-SCRAM
mechanisms. Equivalent to `'sasl.password'` in
`librdkafka`.

* **ssl_key_location** (<xref:str>* | *<xref:None>) –
Path to the client's private key (PEM) for
authentication. Equivalent to `'ssl.key.location'` in
`librdkafka`.

* **ssl_ca_location** (<xref:str>* | *<xref:None>) –
Path to the CA certificate for verifying the broker's
certificate. Equivalent to `'ssl.ca.location'` in
`librdkafka`.

* **ssl_certificate_location** (<xref:str>* | *
<xref:None>) – Path to the client's certificate file.
Equivalent to `'ssl.certificate.location'` in
`librdkafka`.

* **ssl_key_password** (<xref:str>* | *<xref:None>) –
Password for the client's SSL key. Equivalent to
`'ssl.key.password'` in `librdkafka`.

****schema_registry_url**** (<xref:str>* | *
<xref:None>) - URL of the Avro Schema Registry.

****schema_registry_username**** (<xref:str>* | *
<xref:None>) - Username for accessing the Schema Registry.

****schema_registry_password**** (<xref:str>* | *
<xref:None>) - Password for accessing the Schema Registry.

****o_auth_bearer_method****
(<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *<xref:None>) - OAuth bearer method to use, e.g., `'default'` or `'oidc'`. Equivalent to `'sasl.oauthbearer.method'` in `librdkafka`.

****o_auth_bearer_client_id**** (<xref:str>* | *
<xref:None>) - Used with `*oidc*` method. Equivalent to `'sasl.oauthbearer.client.id'` in `librdkafka`.

****o_auth_bearer_client_secret**** (<xref:str>* | *
<xref:None>) - Used with `*oidc*` method. Equivalent to `'sasl.oauthbearer.client.secret'` in `librdkafka`.

****o_auth_bearer_scope**** (<xref:str>* | *
<xref:None>) - Scope of the access request sent to the broker. Equivalent to `'sasl.oauthbearer.scope'` in `librdkafka`.

****o_auth_bearer_token_endpoint_url**** (<xref:str>* | *
<xref:None>) - OAuth token endpoint. Equivalent to `'sasl.oauthbearer.token.endpoint.url'` in `librdkafka`.

****o_auth_bearer_extensions**** (<xref:str>* | *
<xref:None>) - Additional metadata sent to the broker. Comma-separated key=value pairs, e.g., `"supportFeatureX=true,organizationId=sales-emea"`. Equivalent to `'sasl.oauthbearer.extensions'` in `librdkafka`.

****max_message_bytes**** (<xref:int>) - Maximum size (in bytes) of a transmitted message. Default is 1MB.

****batch_size**** (<xref:int>) - Maximum number of messages batched in one `MessageSet`. Default is 10,000.

****enable_idempotence**** (<xref:bool>) - If `*True*`, ensures messages are delivered exactly once and in order. Default is `*False*`.

****message_timeout_ms**** (<xref:int>) - Local timeout for message delivery. Default is 300000 ms.

* **request_timeout_ms** (<xref:int>) - Timeout for producer request acknowledgment. Default is 5000 ms.

* **max_retries** (<xref:int>) - Maximum number of retry attempts for failed messages. Default is 2147483647. Retrying may cause reordering unless `*enable_idempotence=True*`.

* **authentication_mode**
(<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode> | <xref:str> | <xref:None>) - SASL mechanism used for authentication. Allowed values: `*Gssapi*`, `*Plain*`, `*ScramSha256*`, `*ScramSha512*`. Default is `*Plain*`. Equivalent to `*'saslmecanism'*` in `librdkafka`.

* **protocol**
(<xref:azure.functions.decorators.kafka.BrokerProtocol> | <xref:str> | <xref:None>) - Security protocol used for broker communication. Default is plaintext. Equivalent to `*'security.protocol'*` in `librdkafka`.

* **linger_ms** (<xref:int>) - Time to wait between sending batches of messages to allow for better throughput via batching.

* **data_type**
(<xref:azure.functions.decorators.core.DataType> | <xref:str> | <xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include in the `*function.json*`.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`kafka_trigger`(arg_name: str, topic: str, broker_list: str, event_hub_connection_string: str | None = None, consumer_group: str | None = None, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, authentication_mode:

BrokerAuthenticationMode | str | None = 'NotSet', protocol:
BrokerProtocol | str | None = 'NotSet', cardinality: Cardinality | str | None
= 'One', lag_threshold: int = 1000, data_type: DataType | str | None =
None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *kafka_trigger* decorator adds

<xref:agent_framework.azure.KafkaTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function
indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which
enables the function to be triggered in response to an event sent to a
Kafka topic.

All optional fields will be given default values by the function host when
parsed.

Ref: <https://aka.ms/kafkatrigger>

Parameters: * **arg_name** (*str*) – The variable name used in the function
code for the parameter that receives the Kafka event data.

* **topic** (<xref:str>) – The topic monitored by the
trigger.

* **broker_list** (<xref:str>) – The list of Kafka
brokers monitored by the trigger.

* **event_hub_connection_string** (<xref:str>* | *
<xref:None>) – The name of an app setting that contains
the connection string for Event Hub (when using Kafka
protocol headers with Azure Event Hubs).

* **consumer_group** (<xref:str>* | *<xref:None>) –
Kafka consumer group used by the trigger.

* **avro_schema** (<xref:str>* | *<xref:None>) – Used
only if a generic Avro record should be generated.

* **username** (<xref:str>* | *<xref:None>) – SASL
username for use with the PLAIN or SASL-SCRAM
mechanisms. Equivalent to 'sasl.username' in
librdkafka. Default is empty string.

* **password** (<xref:str>* | *<xref:None>) – SASL
password for use with the PLAIN or SASL-SCRAM
mechanisms. Equivalent to 'sasl.password' in
librdkafka. Default is empty string.

* **ssl_key_location** (<xref:str>* | *<xref:None>) –

Path to the client's private key (PEM) used for authentication. Equivalent to 'ssl.key.location' in librdkafka.

* **ssl_ca_location** (<xref:str>* | *<xref:None>) - Path to the CA certificate file for verifying the broker's certificate. Equivalent to 'ssl.ca.location' in librdkafka.

* **ssl_certificate_location** (<xref:str>* | *<xref:None>) - Path to the client's certificate. Equivalent to 'ssl.certificate.location' in librdkafka.

* **ssl_key_password** (<xref:str>* | *<xref:None>) - Password for the client's certificate. Equivalent to 'ssl.key.password' in librdkafka.

* **schema_registry_url** (<xref:str>* | *<xref:None>) - URL of the Avro Schema Registry.

* **schema_registry_username** (<xref:str>* | *<xref:None>) - Username for the Schema Registry.

* **schema_registry_password** (<xref:str>* | *<xref:None>) - Password for the Schema Registry.

* **o_auth_bearer_method** (<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *<xref:None>) - Either 'default' or 'oidc'. Equivalent to 'sasl.oauthbearer.method' in librdkafka.

* **o_auth_bearer_client_id** (<xref:str>* | *<xref:None>) - Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.id' in librdkafka.

* **o_auth_bearer_client_secret** (<xref:str>* | *<xref:None>) - Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.client.secret' in librdkafka.

* **o_auth_bearer_scope** (<xref:str>* | *<xref:None>) - Specify only if *o_auth_bearer_method* is 'oidc'. Used to specify access scope. Equivalent to 'sasl.oauthbearer.scope'.

* **o_auth_bearer_token_endpoint_url** (<xref:str>* | *<xref:None>) - Specify only if *o_auth_bearer_method* is 'oidc'. Equivalent to 'sasl.oauthbearer.token.endpoint.url' in librdkafka.

* **o_auth_bearer_extensions** (<xref:str>* | *<xref:None>) - Additional information for the broker, in the form of a comma-separated list of key=value

pairs (e.g., "orgId=abc,flag=true"). Equivalent to 'sasl.oauthbearer.extensions' in librdkafka.

*** **authentication_mode****
(<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode>* | *<xref:str>* | *<xref:None>*) - SASL mechanism to use. Allowed: Gssapi, Plain, ScramSha256, ScramSha512. Default: Plain. Equivalent to 'sasl.mechanism'.

*** **protocol****
(<xref:azure.functions.decorators.kafka.BrokerProtocol>* | *<xref:str>* | *<xref:None>*) - Security protocol used to communicate with brokers. Default: plaintext. Equivalent to 'security.protocol'.

*** **lag_threshold**** (<xref:int>) - Max number of unprocessed messages per worker instance. Used in scaling logic to estimate needed worker instances. Default is 1000.

*** **data_type****
(<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>*) - Defines how Functions runtime should treat the parameter value.

*** **kwargs**** - Additional keyword arguments for extra binding fields in the binding JSON.

*** **cardinality****
(<xref:azure.functions.decorators.core.Cardinality>* | *<xref:str>* | *<xref:None>*) -

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None, tool_properties: str | None = None, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *mcp_tool_trigger* decorator adds <xref:agent_framework.azure.MCPToolTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/remote-mcp-functions-python> ↗

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **tool_name** (<xref:str>) – The logical tool name
exposed to the host.

* **description** (<xref:str>* | *<xref:None>) –
Optional human-readable description of the tool.

* **tool_properties** (<xref:str>* | *<xref:None>) –
JSON-serialized tool properties/parameters list.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** – Keyword arguments for specifying
additional binding fields to include in the binding
JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`mysql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *mysql_input* decorator adds `<xref:agent_framework.azure.MySqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySQLInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a `<xref:agent_framework.azure.MySqlRowList>` input object in the function code.

* **command_text** (<xref:str>) - The SQL query string or the name of the stored procedure to execute.

* **connection_string_setting** (<xref:str>) - The name of the app setting that contains the connection string to the MySQL database.

* **command_type** (<xref:str>* | *<xref:None>) - The type of command being executed. Accepts **"Text"** for SQL queries or **"StoredProcedure"** for stored procedures.

* **parameters** (<xref:str>* | *<xref:None>) - One or more parameter values passed to the SQL command. Should be provided as a comma-separated string in the format: **"@param1=value1,@param2=value2"**.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments used to define fields in the **function.json** binding.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`mysql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *mysql_output* decorator adds `<xref:agent_framework.azure.MySqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySQLOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the MySQL output object in the function code.

* **command_text** (<xref:str>) – The SQL command or the name of the stored procedure executed by the binding.

* **connection_string_setting** (<xref:str>) – The name of the app setting that contains the connection string for the target MySQL database.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments used to define fields in the *function.json* binding.

Returns: Decorator function.

Return type: [Callable](#)[...], [Any](#)

`mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[...], Any`

The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> object in the function code.

* **table_name** (<xref:str>) – The name of the MySQL table monitored by the trigger.

* **connection_string_setting** (<xref:str>) – The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.

* **leases_table_name** (<xref:str>* | *<xref:None>)

```
- The name of the table used to store leases. If not
specified, the default table name is
*Leases_{FunctionId}_{TableId}*.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) - Defines how the Functions runtime should
treat the parameter value.

* **kwargs** - Additional keyword arguments for
specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

orchestration_trigger(context_name: str, orchestration: str | None = None)

Register an Orchestrator Function.

Parameters: * **context_name** ([str](#)) – Parameter name of the DurableOrchestrationContext object.

```
* **orchestration**
(<xref:agent_framework.azure.Optional[str]>) - Name of
Orchestrator Function. The value is None by default, in
which case the name of the method is used.
```

queue_output(arg_name: str, queue_name: str, connection: str, data_type: [DataType](#) | None = None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *queue_output* decorator adds

<xref:agent_framework.azure.QueueOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function
indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the storage queue output object in the function code.

* **queue_name** (<xref:str>) – The name of the queue to which messages will be written.

* **connection** (<xref:str>) – The name of an app setting or setting collection that specifies how to connect to Azure Queues.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | * <xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: [DataType](#) | None = None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the <xref:agent_framework.azure.QueueMessage> object in the function code.

* **queue_name** (<xref:str>) – The name of the queue to monitor.

* **connection** (<xref:str>) – The name of an app setting or setting collection that specifies how to connect to Azure Queues.

* **data_type**

([<xref:azure.functions.decorators.core.DataType>](#)* | *
[<xref:None>](#)) – Defines how the Functions runtime should treat the parameter value.

* ****kwargs** – Additional keyword arguments to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`register_blueprint(function_container: DecoratorApi) -> None`

Register a list of functions in the function app.

Parameters: **function_container** ([DecoratorApi](#)) – Instance extending [<xref:agent_framework.azure.DecoratorApi>](#), which contains a list of functions to be registered.

Raises: [TypeError](#) – If the provided container is an instance of [<xref:agent_framework.azure.FunctionRegister>](#).

Return type: [None](#)

`register_functions(function_container: DecoratorApi) -> None`

Register a list of functions in the function app.

Parameters: **function_container** ([DecoratorApi](#)) – Instance extending [<xref:agent_framework.azure.DecoratorApi>](#), which contains a list of functions to be registered.

Raises: [TypeError](#) – If the provided container is an instance of [<xref:agent_framework.azure.FunctionRegister>](#).

Return type: [None](#)

`retry(strategy: str, max_retry_count: str, delay_interval: str | None = None, minimum_interval: str | None = None, maximum_interval: str | None = None, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[..., Any]`

The *retry* decorator adds a [<xref:agent_framework.azure.RetryPolicy>](#) to the function settings object.

Used for building a [<xref:agent_framework.azure.Function>](#) object in the worker function indexing model. This is equivalent to defining a *RetryPolicy* in *function.json*, which enables the function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries ↗

Parameters: * **strategy** ([str](#)) – The retry strategy to use.

```
* max_retry_count (<xref:str>) – The maximum
number of retry attempts.

* delay_interval (<xref:str>* | *<xref:None>) –
The delay interval between retry attempts.

* minimum_interval (<xref:str>* | *<xref:None>) –
The minimum delay interval between retry attempts.

* maximum_interval (<xref:str>* | *<xref:None>) –
The maximum delay interval between retry attempts.

* setting_extra_fields (<xref:typing.Dict>*[*
<xref:str>*, *<xref:agent_framework.exceptions.Any>*]
**| *<xref:None>) – Keyword arguments for specifying
additional setting fields.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

```
route(route: str | None = None, trigger_arg_name: str = 'req',
binding_arg_name: str = '$return', methods: Iterable[str] |
Iterable[HttpMethod] | None = None, auth_level: AuthLevel | str | None =
None, trigger_extra_fields: Dict[str, Any] | None = None,
binding_extra_fields: Dict[str, Any] | None = None) -> Callable[..., Any]
```

The *route* decorator adds [<xref:agent_framework.azure.HttpTrigger>](#) and [<xref:agent_framework.azure.HttpOutput>](#) bindings to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-http> [↗](#)

Parameters: * **route** ([str](#)* | *[None](#)) – Route for the HTTP endpoint. If *None*, it defaults to the function name (if present) or the user-defined Python function name.


```

    * **trigger_arg_name** (<xref:str>) - Argument name
    for <xref:agent_framework.azure.HttpRequest>. Defaults
    to 'req'.

    * **binding_arg_name** (<xref:str>) - Argument name
    for <xref:agent_framework.azure.HttpResponse>. Defaults
    to '$return'.

    * **methods** (<xref:typing.Iterable>[*<xref:str>]*
    **| *<xref:typing.Iterable>[*[HttpMethod]
    (azure.functions.decorators.http.HttpMethod)*] **| *
    <xref:None>) - A tuple of the HTTP methods to which the
    function responds.

    * **auth_level**
    (<xref:azure.functions.decorators.core.AuthLevel>* | *
    <xref:str>* | *<xref:None>) - Determines what keys, if
    any, need to be present on the request in order to
    invoke the function.

    * **trigger_extra_fields** (<xref:typing.Dict>[*
    <xref:str>*, *<xref:agent_framework.exceptions.Any>]*
    **| *<xref:None>) - Additional fields to include in the
    trigger JSON. For example: >>> data_type='STRING' #
    results in 'dataType': 'STRING' in the trigger JSON

    * **binding_extra_fields** (<xref:typing.Dict>[*
    <xref:str>*, *<xref:agent_framework.exceptions.Any>]*
    **| *<xref:None>) - Additional fields to include in the
    binding JSON. For example: >>> data_type='STRING' #
    results in 'dataType': 'STRING' in the binding JSON

```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```

schedule(arg_name: str, schedule: str, run_on_startup: bool | None =
None, use_monitor: bool | None = None, data_type: DataType | str | None
= None, **kwargs: Any) -> Callable[..., Any]

```

The *schedule* or *timer* decorator adds

<xref:agent_framework.azure.TimerTrigger> to the
 <xref:agent_framework.azure.FunctionBuilder> object for building a
 <xref:agent_framework.azure.Function> used in the worker function
 indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which
 enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when
 parsed.

Ref: <https://aka.ms/azure-function-binding-timer> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.TimerRequest>` object in function code.

```
* **schedule** (<xref:str>) – A string representing a
CRON expression used to schedule the function
execution.

* **run_on_startup** (<xref:bool>* | *<xref:None>) –
If True, the function is invoked when the runtime
starts.

* **use_monitor** (<xref:bool>* | *<xref:None>) – Set
to True or False to indicate whether the schedule
should be monitored.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`semantic_search_input(arg_name: str, search_connection_name: str, collection: str, query: str | None = None, ai_connection_name: str | None = "", embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None = semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: <https://platform.openai.com/docs/guides/embeddings> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

* **search_connection_name** (<xref:str>) – The name of the app setting or environment variable that contains the vector search connection value.

* **collection** (<xref:str>) – The name of the collection or table to search or store.

* **query** (<xref:str>* | *<xref:None>) – The semantic query text used for searching the database.

* **ai_connection_name** (<xref:str>* | *<xref:None>)
–

The name of the configuration section for AI service connectivity settings.

* For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section.

* If not specified or the section is missing, defaults to the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.

* For user-assigned managed identity authentication, this field is required.

* For OpenAI (non-Azure), set the *OPENAI_API_KEY* environment variable.

* **embeddings_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The deployment or model name for OpenAI Embeddings. Defaults to "text-embedding-ada-002".

* **chat_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) – The deployment or model name for the OpenAI Chat Completion API. Defaults to "gpt-3.5-turbo".

* **system_prompt** (<xref:str>* | *<xref:None>) – Optional. The system prompt provided to the large language model.

* **max_knowledge_count** (<xref:int>* | *<xref:None>) – Optional. The number of knowledge items to inject into the system prompt. Default is 1.

* **temperature** (`<xref:str>`* | *`<xref:None>`*) - The sampling temperature to use (range: 0 to 2). Higher values like 0.8 yield more random output; lower values like 0.2 make output more focused.

* **top_p** (`<xref:str>`* | *`<xref:None>`*) - Alternative to temperature sampling (nucleus sampling). For example, 0.1 considers only the top 10% of tokens by probability mass. Use this or *temperature*.

* **max_tokens** (`<xref:str>`* | *`<xref:None>`*) - The maximum number of tokens to generate in the completion. The sum of prompt tokens and *max_tokens* must not exceed the model's context length (usually 2048 or 4096 for newer models).

* **is_reasoning_model** (`<xref:bool>`* | *`<xref:None>`*) - Indicates whether the chat model is a reasoning model. If True, *max_tokens* and *temperature* are not supported.

* **data_type** (`<xref:azure.functions.decorators.core.DataType>`* | *`<xref:str>`* | *`<xref:None>`*) - Optional. Defines how the Functions runtime should interpret the parameter value. Default is None.

* **kwargs** - Additional keyword arguments for specifying extra fields in the *function.json* binding.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`service_bus_queue_output(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[..., Any]`

The *service_bus_queue_output* decorator adds `<xref:agent_framework.azure.ServiceBusQueueOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Service Bus queue output object in the function code.

```
* **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

* **queue_name** (<xref:str>) – The name of the queue
to which messages will be sent.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) – Defines how the Functions
runtime should treat the parameter value.

* **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>*) – The access rights
required for the connection string.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The `on_service_bus_queue_change` decorator adds `<xref:agent_framework.azure.ServiceBusQueueTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a `ServiceBusQueueTrigger` in the `function.json`, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.ServiceBusMessage>` object in the function code.

```

    * **connection** (<xref:str>) - The name of an app
    setting or setting collection that specifies how to
    connect to Service Bus.

    * **queue_name** (<xref:str>) - The name of the queue
    to monitor.

    * **data_type**
    (<xref:azure.functions.decorators.core.DataType>* | *
    <xref:str>* | *<xref:None>*) - Defines how the Functions
    runtime should treat the parameter value.

    * **access_rights**
    (<xref:azure.functions.decorators.core.AccessRights>* |
    *<xref:str>* | *<xref:None>*) - Access rights for the
    connection string.

    * **is_sessions_enabled** (<xref:bool>* | *
    <xref:None>*) - Set to True if connecting to a session-
    aware queue or subscription.

    * **cardinality**
    (<xref:azure.functions.decorators.core.Cardinality>* |
    *<xref:str>* | *<xref:None>*) - Set to "many" to enable
    batching.

    * **kwargs** (<xref:agent_framework.exceptions.Any>)
    -

```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`service_bus_topic_output(arg_name: str, connection: str, topic_name: str, subscription_name: str | None = None, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[..., Any]`

The *service_bus_topic_output* decorator adds `<xref:agent_framework.azure.ServiceBusTopicOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *ServiceBusTopicOutput* in the *function.json*, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Service Bus topic output object in the function code.

```
* **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

* **topic_name** (<xref:str>) – The name of the topic
to which messages will be sent.

* **subscription_name** (<xref:str>* | *<xref:None>)
– The name of the subscription (optional for output).

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value. Defaults to
DataType.UNDEFINED.

* **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>) – The access rights
required for the connection string.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str,
subscription_name: str, data_type: DataType | str | None = None,
access_rights: AccessRights | str | None = None, is_sessions_enabled: bool
| None = None, cardinality: Cardinality | str | None = None, **kwargs: Any)
-> Callable[..., Any]
```

The *on_service_bus_topic_change* decorator adds [<xref:agent_framework.azure.ServiceBusTopicTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.ServiceBusMessage>` object in the function code.

```
* **connection** (<xref:str>) – The name of an app
setting or setting collection that specifies how to
connect to Service Bus.

* **topic_name** (<xref:str>) – The name of the topic
to monitor.

* **subscription_name** (<xref:str>) – The name of
the subscription to monitor.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) – Defines how the Functions
runtime should treat the parameter value.

* **access_rights**
(<xref:azure.functions.decorators.core.AccessRights>* |
*<xref:str>* | *<xref:None>*) – Access rights for the
connection string.

* **is_sessions_enabled** (<xref:bool>* | *
<xref:None>*) – Set to True if connecting to a session-
aware queue or subscription.

* **cardinality**
(<xref:azure.functions.decorators.core.Cardinality>* |
*<xref:str>* | *<xref:None>*) – Set to "many" to enable
batching.

* **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: [Callable](#)[[\[...\]](#), [Any](#)]

`sql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[\[...\], Any]`

The `sql_input` decorator adds `<xref:agent_framework.azure.SqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining `SqlInput` in the `function.json`, which enables the

function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a `<xref:agent_framework.azure.SqlRowList>` input object in the function code.

```
* **command_text** (<xref:str>) – The Transact-SQL query or the name of the stored procedure executed by the binding.

* **connection_string_setting** (<xref:str>) – The name of the app setting that contains the connection string to the target SQL database.

* **command_type** (<xref:str>* | *<xref:None>) – The command type. Use "Text" for a raw SQL query and "StoredProcedure" for a stored procedure.

* **parameters** (<xref:str>* | *<xref:None>) – A comma-separated string of parameter assignments to pass to the SQL command. Format:
*@param1=value1,@param2=value2*

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields to include in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[\[...\]](#), [Any](#)]

`sql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[\[...\], Any]`

The `sql_output` decorator adds `<xref:agent_framework.azure.SqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining `SqlOutput` in the `function.json`, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the SQL output object in the function code.

```
* **command_text** (<xref:str>) – The Transact-SQL query or the name of the stored procedure executed by the binding.

* **connection_string_setting** (<xref:str>) – The name of the app setting that contains the connection string to the target SQL database.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`sql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The `sql_trigger` decorator adds [<xref:agent_framework.azure.SqlTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining `SqlTrigger` in the `function.json`, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a [<xref:agent_framework.azure.SqlRowList>](#) object in the function code.

```
* **table_name** (<xref:str>) – The name of the SQL table monitored by the trigger.
```

```

    * connection_string_setting (<xref:str>) - The
    name of an app setting that contains the connection
    string for the database against which the query or
    stored procedure is executed.

    * leases_table_name (<xref:str>* | *<xref:None>)
    - The name of the table used to store leases. If not
    specified, the default name is
    *Leases_{FunctionId}_{TableId}*.

    * data_type
    (<xref:azure.functions.decorators.core.DataType>* | *
    <xref:None>) - Defines how the Functions runtime should
    treat the parameter value.

    * kwargs - Additional keyword arguments for
    specifying binding fields to include in the
    *function.json*.

```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```

table_input(arg_name: str, connection: str, table_name: str, row_key: str |
None = None, partition_key: str | None = None, take: int | None = None,
filter: str | None = None, data_type: DataType | str | None = None) ->
Callable[..., Any]

```

The *table_input* decorator adds [<xref:agent_framework.azure.TableInput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the table or entity in the function code.

```

    * connection (<xref:str>) - The name of an app
    setting or setting collection that specifies how to
    connect to the table service.

    * table_name (<xref:str>) - The name of the

```

table.

* ****row_key**** (<xref:str>* | *<xref:None>) – The row key of the table entity to read.

* ****partition_key**** (<xref:str>* | *<xref:None>) – The partition key of the table entity to read.

* ****take**** (<xref:int>* | *<xref:None>) – The maximum number of entities to return.

* ****filter**** (<xref:str>* | *<xref:None>) – An OData filter expression to apply when retrieving entities.

* ****data_type****
(<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

table_output(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, data_type: [DataType](#) | str | None = None) -> [Callable](#)[[...], [Any](#)]

The *table_output* decorator adds <xref:agent_framework.azure.TableOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the table or entity in the function code.

* ****connection**** (<xref:str>) – The name of an app setting or setting collection that specifies how to connect to the table service.

* ****table_name**** (<xref:str>) – The name of the table.

```
* **row_key** (<xref:str>* | *<xref:None>) - The row
key of the table entity to read.

* **partition_key** (<xref:str>* | *<xref:None>) -
The partition key of the table entity to read.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) - Defines how the Functions
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

```
text_completion_input(arg_name: str, prompt: str, ai_connection_name: str
| None = "", chat_model: str | OpenAIModels | None =
OpenAIModels.DefaultChatModel, temperature: str | None = '0.5', top_p:
str | None = None, max_tokens: str | None = '100', is_reasoning_model:
bool | None = False, data_type: DataType | str | None = None, **kwargs) -
> Callable[..., Any]
```

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: <https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions> [↗](#)

The examples typically define an HTTP-triggered function with a hardcoded prompt such as "*who is {name}?*", where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **prompt** (<xref:str>) - The text prompt to
generate completions for.

* **ai_connection_name** (<xref:str>* | *<xref:None>)
-

The name of the configuration section that defines
AI service connectivity settings.

* For **Azure OpenAI**: If specified, it looks for
**"Endpoint"** and/or

**"Key"** in this section. If not specified or
```

missing, it falls back to the environment variables `*AZURE_OPENAI_ENDPOINT*` and `*AZURE_OPENAI_KEY*`.

- For **user-assigned managed identity**, this setting is required.
- For **non-Azure OpenAI**, ensure the `*OPENAI_API_KEY*` environment variable is set.

*** **model**** - **(Deprecated)** Use `*chat_model*` instead. This parameter is unused and will be removed in future versions.

*** **chat_model**** (`<xref:str>*` | `* <xref:azure.functions.decorators.openai.OpenAIModels>*` | `* <xref:None>*`) - The deployment or model name to use for the Chat Completions API. Default is `"gpt-3.5-turbo"`.

*** **temperature**** (`<xref:str>*` | `* <xref:None>*`) - Sampling temperature (0-2). Higher values (e.g., 0.8) produce more random output; lower values (e.g., 0.2) make output more focused and deterministic.

*** **top_p**** (`<xref:str>*` | `* <xref:None>*`) - Controls nucleus sampling. For example, `*top_p=0.1*` considers only tokens in the top 10% cumulative probability. Recommended to use either `*temperature*` or `*top_p*`, but not both.

*** **max_tokens**** (`<xref:str>*` | `* <xref:None>*`) - The maximum number of tokens to generate. The sum of prompt tokens and `*max_tokens*` must not exceed the model's context length (usually 2048 or 4096 for newer models).

*** **is_reasoning_model**** (`<xref:bool>*` | `* <xref:None>*`) - If `*True*`, indicates that the configured chat model is a reasoning model. In this case, `*max_tokens*` and `*temperature*` are not supported.

*** **data_type**** (`<xref:azure.functions.decorators.core.DataType>*` | `* <xref:str>*` | `* <xref:None>*`) - Defines how the Functions runtime should treat the parameter value.

*** **kwargs**** - Additional keyword arguments for specifying binding fields to include in the `*function.json*`.

Returns: Decorator function.

Return type: [Callable](#)[\[\[...\], Any\]](#)

`timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]`

The *schedule* or *timer* decorator adds

<xref:agent_framework.azure.TimerTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function
indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which
enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when
parsed.

Ref: <https://aka.ms/azure-function-binding-timer> 

Parameters: * **arg_name** (**str**) – The name of the variable that represents
the <xref:agent_framework.azure.TimerRequest> object in function code.

```
* **schedule** (<xref:str>) – A string representing a
CRON expression used to schedule the function
execution.

* **run_on_startup** (<xref:bool>* | *<xref:None>) –
If True, the function is invoked when the runtime
starts.

* **use_monitor** (<xref:bool>* | *<xref:None>) – Set
to True or False to indicate whether the schedule
should be monitored.

* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>) – Defines how the Functions
runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>)
–
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`validate_function_names(functions: List[Function])`

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings.
If multiple functions share the same name, indexing will fail.

Parameters: **functions** ([List/Function](#)) – A list of `<xref:agent_framework.azure.Function>` objects to validate.

Raises: [ValueError](#) – If any function name is duplicated.

`warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable`

The `warm_up` decorator adds `<xref:agent_framework.azure.WarmUpTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a `WarmUpTrigger` in the `function.json`, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-warmup> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.TimerRequest>` object in the function code.

```
* **data_type**
(<xref:azure.functions.decorators.core.DataType>* | *
<xref:str>* | *<xref:None>*) - Defines how the Functions
runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)

AgentResponseCallbackProtocol	Protocol describing the callbacks invoked during agent execution.
AzureAIAgentClient	Azure AI Agent Chat client. Initialize an Azure AI Agent client.
AzureAIClient	Azure AI Agent client. Initialize an Azure AI Agent client.
AzureAISearchContextProvider	Azure AI Search Context Provider with hybrid search and semantic ranking. This provider retrieves relevant documents from Azure AI Search to provide context to the AI agent. It supports two modes:

- **agentic**: Recommended for most scenarios. Uses Knowledge Bases for query planning and multi-hop reasoning. Slightly slower with more token consumption, but provides more accurate results (up to 36% improvement in response relevance).
- **semantic** (default): Fast hybrid search combining vector and keyword search with semantic reranking. Best for simple queries where speed is critical.

Initialize Azure AI Search Context Provider.

AzureAISearchSettings

Settings for Azure AI Search Context Provider with auto-loading from environment.

The settings are first loaded from environment variables with the prefix '>AZURE_SEARCH_<'. If the environment variables are not found, the settings can be loaded from a .env file.

Initialize the settings class.

JSON

```
{
  "title": "AzureAISearchSettings",
  "description": "Settings for Azure AI Search
Context Provider with auto-loading from
environment.\n\nThe settings are first loaded from
environment variables with the prefix
'AZURE_SEARCH_'.\nIf the environment variables are not
found, the settings can be loaded from a .env
file.\n\nKeyword Args:\n    endpoint: Azure AI Search
endpoint URL.\n    Can be set via environment
variable AZURE_SEARCH_ENDPOINT.\n    index_name: Name
of the search index.\n    Can be set via
environment variable AZURE_SEARCH_INDEX_NAME.\n
knowledge_base_name: Name of an existing Knowledge Base
(for agentic mode).\n    Can be set via environment
variable AZURE_SEARCH_KNOWLEDGE_BASE_NAME.\n
api_key: API key for authentication (optional, use
managed identity if not provided).\n    Can be set
via environment variable AZURE_SEARCH_API_KEY.\n
env_file_path: If provided, the .env settings are read
from this file path location.\n    env_file_encoding:
The encoding of the .env file, defaults to 'utf-
8'.\n\nExamples:\n    .. code-block:: python\n\nfrom agent_framework_aisearch import
AzureAISearchSettings\n\n    # Using environment
variables\n    # Set
AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.
et\n    # Set AZURE_SEARCH_INDEX_NAME=my-index\n
settings = AzureAISearchSettings()\n\n    # Or
passing parameters directly\n    settings =
AzureAISearchSettings(\n
```

```
endpoint=\\https://mysearch.search.windows.net\\,\\n
index_name=\\my-index\\,\\n          )\\n\\n          # Or
loading from a .env file\\n          settings =
AzureAISearchSettings(env_file_path=\\path/to/.env\\"),
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Env File Path"
    },
    "env_file_encoding": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": "utf-8",
      "title": "Env File Encoding"
    },
    "endpoint": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Endpoint"
    },
    "index_name": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Index Name"
    },
    "knowledge_base_name": {
```

```

        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Knowledge Base Name"
    },
    "api_key": {
        "anyOf": [
            {
                "format": "password",
                "type": "string",
                "writeOnly": true
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Api Key"
    }
}

```

AzureAISettings

Azure AI Project settings.

The settings are first loaded from environment variables with the prefix '>AZURE_AI_<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON

```

{
    "title": "AzureAISettings",
    "description": "Azure AI Project settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_AI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the settings are missing.\n\nKeyword Args:\n    project_endpoint: The Azure AI Project endpoint URL.\n                    Can be set via environment

```

```

variable AZURE_AI_PROJECT_ENDPOINT.\n
model_deployment_name: The name of the model deployment
to use.\n          Can be set via environment variable
AZURE_AI_MODEL_DEPLOYMENT_NAME.\n    env_file_path: If
provided, the .env settings are read from this file
path location.\n    env_file_encoding: The encoding of
the .env file, defaults to 'utf-8'.\n\nExamples:\n
.. code-block:: python\n\n    from
agent_framework.azure import AzureAISettings\n\n
# Using environment variables\n          # Set
AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com\n          # Set
AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4\n          settings
= AzureAISettings()\n\n    # Or passing parameters
directly\n          settings = AzureAISettings(\n
project_endpoint="https://your-
project.cognitiveservices.azure.com",
model_deployment_name="gpt-4"\n          )\n\n    #
Or loading from a .env file\n          settings =
AzureAISettings(env_file_path="path/to/.env"),
    "type": "object",
    "properties": {
        "env_file_path": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Env File Path"
        },
        "env_file_encoding": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
        },
        "project_endpoint": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],

```

```

        "default": null,
        "title": "Project Endpoint"
    },
    "model_deployment_name": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Model Deployment Name"
    }
}

```

AzureOpenAIAssistantsClient

Azure OpenAI Assistants client.

Initialize an Azure OpenAI Assistants client.

AzureOpenAIChatClient

Azure OpenAI Chat completion class.

Initialize an Azure OpenAI Chat completion client.

AzureOpenAIResponsesClient

Azure Responses completion class.

Initialize an Azure OpenAI Responses client.

AzureOpenAISettings

AzureOpenAI model settings.

The settings are first loaded from environment variables with the prefix '>AZURE_OPENAI_<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON

```

{
  "title": "AzureOpenAISettings",
  "description": "AzureOpenAI model settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_OPENAI_'.\n\nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the

```

settings are missing.\n\nKeyword Args:\n endpoint:
The endpoint of the Azure deployment. This value\n can be found in the Keys & Endpoint section when
examining\n your resource from the Azure portal,
the endpoint should end in openai.azure.com.\n If both base_url and endpoint are supplied, base_url
will be used.\n Can be set via environment
variable AZURE_OPENAI_ENDPOINT.\n chat_deployment_name: The name of the Azure Chat
deployment. This value\n will correspond to the
custom name you chose for your deployment\n when
you deployed a model. This value can be found under\n Resource Management > Deployments in the Azure portal
or, alternatively,\n under Management >
Deployments in Azure AI Foundry.\n Can be set
via environment variable
AZURE_OPENAI_CHAT_DEPLOYMENT_NAME.\n responses_deployment_name: The name of the Azure
Responses deployment. This value\n will
correspond to the custom name you chose for your
deployment\n when you deployed a model. This
value can be found under\n Resource Management >
Deployments in the Azure portal or, alternatively,\n under Management > Deployments in Azure AI Foundry.\n Can be set via environment variable
AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME.\n api_key:
The API key for the Azure deployment. This value can
be\n found in the Keys & Endpoint section when
examining your resource in\n the Azure portal.
You can use either KEY1 or KEY2.\n Can be set
via environment variable AZURE_OPENAI_API_KEY.\n api_version: The API version to use. The default value
is `default\_api\_version`.\n Can be set via
environment variable AZURE_OPENAI_API_VERSION.\n base_url: The url of the Azure deployment. This value\n can be found in the Keys & Endpoint section when
examining\n your resource from the Azure portal,
the base_url consists of the endpoint,\n followed by /openai/deployments/{deployment_name}/,\n use endpoint if you only want to supply the endpoint.\n Can be set via environment variable
AZURE_OPENAI_BASE_URL.\n token_endpoint: The token
endpoint to use to retrieve the authentication token.\n The default value is `default\_token\_endpoint`.\n Can be set via environment variable
AZURE_OPENAI_TOKEN_ENDPOINT.\n default_api_version:
The default API version to use if not specified.\n The default value is `"2024-10-21"`.\n default_token_endpoint: The default token endpoint to
use if not specified.\n The default value is
`"https://cognitiveservices.azure.com/.default"`.\n env_file_path: The path to the .env file to load
settings from.\n env_file_encoding: The encoding of
the .env file, defaults to 'utf-8'.\n\nExamples:\n .. code-block:: python\n from

```

agent_framework.azure import AzureOpenAISettings\n\n
# Using environment variables\n          # Set
AZURE_OPENAI_ENDPOINT=https://your-
endpoint.openai.azure.com\n          # Set
AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4\n          # Set
AZURE_OPENAI_API_KEY=your-key\n          settings =
AzureOpenAISettings()\n\n          # Or passing
parameters directly\n          settings =
AzureOpenAISettings(\n
endpoint=\"https://your-endpoint.openai.azure.com\",
chat_deployment_name=\"gpt-4\", api_key=\"your-key\"\n
)\n\n          # Or loading from a .env file\n
settings =
AzureOpenAISettings(env_file_path=\"path/to/.env\"),
    "type": "object",
    "properties": {
        "env_file_path": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Env File Path"
        },
        "env_file_encoding": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": "utf-8",
            "title": "Env File Encoding"
        },
        "chat_deployment_name": {
            "anyOf": [
                {
                    "type": "string"
                },
                {
                    "type": "null"
                }
            ],
            "default": null,
            "title": "Chat Deployment Name"
        },
        "responses_deployment_name": {
            "anyOf": [
                {

```

```
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Responses Deployment Name"
  },
  "endpoint": {
    "anyOf": [
      {
        "format": "uri",
        "maxLength": 2083,
        "minLength": 1,
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Endpoint"
  },
  "base_url": {
    "anyOf": [
      {
        "format": "uri",
        "maxLength": 2083,
        "minLength": 1,
        "type": "string"
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Base Url"
  },
  "api_key": {
    "anyOf": [
      {
        "format": "password",
        "type": "string",
        "writeOnly": true
      },
      {
        "type": "null"
      }
    ],
    "default": null,
    "title": "Api Key"
  },
  "api_version": {
    "anyOf": [
```



```

        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Api Version"
},
"token_endpoint": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Token Endpoint"
},
"default_api_version": {
    "default": "2024-10-21",
    "title": "Default Api Version",
    "type": "string"
},
"default_token_endpoint": {
    "default":
"https://cognitiveservices.azure.com/.default",
    "title": "Default Token Endpoint",
    "type": "string"
}
}
}

```

DurableAIAgent

A durable agent implementation that uses entity methods to interact with agent entities.

This class implements AgentProtocol and provides methods to work with Azure Durable Functions orchestrations, which use generators and yield instead of async/await.

Key methods:

- `get_new_thread()`: Create a new conversation thread
- `run()`: Execute the agent and return a Task for yielding in orchestrations

Note: The `run()` method is NOT async. It returns a Task directly that must be yielded in orchestrations to wait for the entity call to complete.

Example usage in orchestration: `writer = app.get_agent(context, "WriterAgent")` `thread = writer.get_new_thread()` # NOT yielded - returns

immediately

```
response = yield writer.run( # Yielded - waits for entity call
message="Write a haiku about coding", thread=thread
)
```

Initialize the DurableAI Agent.

Functions

get_entra_auth_token

Retrieve a Microsoft Entra Auth Token for a given token endpoint.

The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None.

Python

```
get_entra_auth_token(credential: TokenCredential, token_endpoint: str, **kwargs:
Any) -> str | None
```

Parameters

[Expand table](#)

Name	Description
credential Required*	<xref:TokenCredential> The Azure credential to use for authentication.
token_endpoint Required*	str The token endpoint to use to retrieve the authentication token.
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
**kwargs	Additional keyword arguments to pass to the token retrieval method.

Returns

 Expand table

Type	Description
str None	The Azure token or None if the token could not be retrieved.

AgentCallbackContext Class

Context supplied to callback invocations.

Constructor

Python
<code>AgentCallbackContext()</code>

Parameters

 Expand table

Name	Description
<code>agent_name</code> Required*	
<code>correlation_id</code> Required*	
<code>thread_id</code>	Default value: None
<code>request_message</code>	Default value: None

Methods

 Expand table

<code>__init__</code>
<code>__new__</code>

`__init__`

Python
<code>__init__(agent_name: str, correlation_id: str, thread_id: str None = None, request_message: str None = None) -> None</code>

Parameters

[Expand table](#)

Name	Description
agent_name Required*	str
correlation_id Required*	str
thread_id	str None Default value: None
request_message	str None Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

Attributes

agent_name

Python
<code>agent_name: str</code>

correlation_id

Python
<code>correlation_id: str</code>

request_message

Python

```
request_message: str | None = None
```

thread_id

Python

```
thread_id: str | None = None
```

AgentFunctionApp Class

Main application class for creating durable agent function apps using Durable Entities.

This class uses Durable Entities pattern for agent execution, providing:

- Stateful agent conversations
- Conversation history management
- Signal-based operation invocation
- Better state management than orchestrations

Example:

Python

```
from agent_framework.azure import AgentFunctionApp, AzureOpenAIChatClient

# Create agents with unique names
weather_agent = AzureOpenAIChatClient(...).create_agent(
    name="WeatherAgent",
    instructions="You are a helpful weather agent.",
    tools=[get_weather],
)

math_agent = AzureOpenAIChatClient(...).create_agent(
    name="MathAgent",
    instructions="You are a helpful math assistant.",
    tools=[calculate],
)

# Option 1: Pass list of agents during initialization
app = AgentFunctionApp(agents=[weather_agent, math_agent])

# Option 2: Add agents after initialization
app = AgentFunctionApp()
app.add_agent(weather_agent)
app.add_agent(math_agent)

@app.orchestration_trigger(context_name="context")
def my_orchestration(context):
    writer = app.get_agent(context, "WeatherAgent")
    thread = writer.get_new_thread()
    forecast_task = writer.run("What's the forecast?", thread=thread)
    forecast = yield forecast_task
    return forecast
```

This creates:

- HTTP trigger endpoint for each agent's requests (if enabled)
- Durable entity for each agent's state management and execution
- Full access to all Azure Functions capabilities

agents

Dictionary of agent name to AgentProtocol instance

enable_health_check

Whether health check endpoint is enabled

Type: `bool`

enable_http_endpoints

Whether HTTP endpoints are created for agents

Type: `bool`

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

Type: `bool`

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

Initialize the AgentFunctionApp.

param agents: List of agent instances to register.

param http_auth_level: HTTP authentication level (default: `func.AuthLevel.FUNCTION`).

param enable_health_check: Enable the built-in health check endpoint (default: `True`).

param enable_http_endpoints: Enable HTTP endpoints for agents (default: `True`).

param enable_mcp_tool_trigger: Enable MCP tool triggers for agents (default: `False`). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients.

param max_poll_retries: Maximum polling attempts when waiting for a response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.

param poll_interval_seconds: Delay in seconds between polling attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.

param default_callback: Optional callback invoked for agents without specific callbacks.

note: If no agents are provided, they can be added later using [add_agent](#).

```
init(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel =  
func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints: bool = True,  
max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES, poll_interval_seconds: float =  
DEFAULT_POLL_INTERVAL_SECONDS, enable_mcp_tool_trigger: bool = False, default_callback:  
AgentResponseCallbackProtocol | None = None)
```

Initialize the AgentFunctionApp.

Parameters: * **agents** ([list\[AgentProtocol\]](#)*) **| *[None](#)) – List of agent instances to register.

```
* **http_auth_level** (<xref:azure.functions.decorators.core.AuthLevel>) - HTTP  
authentication level (default: `func.AuthLevel.FUNCTION`).  
  
* **enable_health_check** (<xref:bool>) - Enable the built-in health check endpoint  
(default: `True`).  
  
* **enable_http_endpoints** (<xref:bool>) - Enable HTTP endpoints for agents  
(default: `True`).  
  
* **enable_mcp_tool_trigger** (<xref:bool>) - Enable MCP tool triggers for agents  
(default: `False`). When enabled, agents will be exposed as MCP tools that can be  
invoked by MCP-compatible clients.  
  
* **max_poll_retries** (<xref:int>) - Maximum polling attempts when waiting for a  
response. Defaults to `DEFAULT_MAX_POLL_RETRIES`.  
  
* **poll_interval_seconds** (<xref:float>) - Delay in seconds between polling  
attempts. Defaults to `DEFAULT_POLL_INTERVAL_SECONDS`.  
  
* **default_callback**  
(<xref:agent_framework_azurefunctions._callbacks.AgentResponseCallbackProtocol>* | *  
<xref:None>*) - Optional callback invoked for agents without specific callbacks.
```

Note: If no agents are provided, they can be added later using [add_agent](#).

new(**kwargs)

activity_trigger(input_name: str, activity: str | None = None)

Register an Activity Function.

Parameters: * **input_name** ([str](#)) – Parameter name of the Activity input.

```
* activity (<xref:agent\_framework.azure.Optional\[str\]>) – Name of Activity Function. The value is None by default, in which case the name of the method is used.
```

add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> [None](#)

Add an agent to the function app after initialization.

Parameters: * **agent** ([AgentProtocol](#)) – The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.

```
* callback (<xref:agent\_framework\_azurefunctions.\_callbacks.AgentResponseCallbackProtocol>* | *<xref:None>) – Optional callback invoked during agent execution

* enable_http_endpoint (<xref:bool>* | *<xref:None>) – Optional flag to enable/disable HTTP endpoint for this agent. The app level enable_http_endpoints setting will override this setting.

* enable_mcp_tool_trigger (<xref:bool>* | *<xref:None>) – Optional flag to enable/disable MCP tool trigger for this agent. The app level enable_mcp_tool_trigger setting will override this setting.
```

Raises: [ValueError](#) – If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

Return type: [None](#)

property agents: [dict\[str, AgentProtocol\]](#)

Returns dict of agent names to agent instances.

Returns: Dictionary mapping agent names to their AgentProtocol instances.

property app_script_file: [str](#)

Name of function app script file in which all the functions are defined.

Script file defined here is for placeholder purpose, please refer to worker defined script file path as the single point of truth.

Returns: Script file name.

assistant_create_output(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[[...], [Any](#)]

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

* **data_type** ([Azure.Functions.decorators.core.DataType](#) | [str](#) | [None](#)) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying extra binding fields to include in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

assistant_post_input(arg_name: str, id: str, user_message: str, ai_connection_name: str | None = "", chat_model: str | [OpenAIModels](#) | None = [OpenAIModels.DefaultChatModel](#), chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[[...], [Any](#)]

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

* **id** (<xref:str>) - The ID of the assistant to update.

* **user_message** (<xref:str>) - The message entered by the user for the assistant to respond to.

* **ai_connection_name** (<xref:str>* | *<xref:None>) -

The name of the configuration section for AI service connectivity.

* **Azure OpenAI**: If specified, looks for "Endpoint" and/or "Key" in the section.

* If not specified or missing, falls back to the **AZURE_OPENAI_ENDPOINT** and **AZURE_OPENAI_KEY** environment variables.

* **Managed Identity**: Required for user-assigned identity auth.

* **OpenAI (non-Azure)**: Set the **OPENAI_API_KEY** environment variable.

* **model** - *Deprecated.* Use **chat_model** instead. This parameter is unused and will be removed in future versions.

* **chat_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name of the OpenAI Chat Completion API. Default is **"gpt-3.5-turbo"**.

* **chat_storage_connection_setting** (<xref:str>* | *<xref:None>) - The config section name for assistant chat table storage. Default is **"AzureWebJobsStorage"**.

* **collection_name** (<xref:str>* | *<xref:None>) - The collection or table name used for assistant chat storage. Default is **"ChatState"**.

* **temperature** (<xref:str>* | *<xref:None>) - Sampling temperature, between 0 and 2. Higher values (e.g., 0.8) increase randomness; lower values (e.g., 0.2) make output more deterministic.

* **top_p** (<xref:str>* | *<xref:None>) - Alternative to temperature sampling. Uses nucleus sampling to consider only the tokens comprising the top **top_p** probability mass. Recommended to use either this or **temperature**.

* **max_tokens** (<xref:str>* | *<xref:None>) - Maximum tokens to generate. Total tokens (prompt + output) must not exceed the model's context length.

* **is_reasoning_model** (<xref:bool>* | *<xref:None>) - Whether the chat model is a reasoning model. If true, **max_tokens** and **temperature** are not supported.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying extra binding fields in **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`assistant_query_input(arg_name: str, id: str, timestamp_utc: str, chat_storage_connection_setting: str | None = 'AzureWebJobsStorage', collection_name: str | None = 'ChatState', data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **timestamp_utc** (<xref:str>) - The earliest timestamp (in UTC) for the messages
to retrieve from the chat history. Must be in ISO 8601 format, e.g., "2023-08-
01T00:00:00Z".

* **chat_storage_connection_setting** (<xref:str>* | *<xref:None>) - The name of the
configuration section containing the connection settings for assistant chat storage.
Defaults to "AzureWebJobsStorage".

* **collection_name** (<xref:str>* | *<xref:None>) - The name of the table or
collection used for assistant chat storage. Defaults to "ChatState".

* **id** (<xref:str>) - The unique identifier of the assistant whose history is
being queried.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`assistant_skill_trigger(arg_name: str, function_description: str, function_name: str | None = None, parameter_description_json: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]`

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions

to invoke and when.

Ref: <https://platform.openai.com/docs/guides/function-calling> 

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **function_description** (<xref:str>) – A description of the assistant function,
which is provided to the model.

* **function_name** (<xref:str>* | *<xref:None>) – The name of the assistant
function, which is passed to the language model.

* **parameter_description_json** (<xref:str>* | *<xref:None>) – A JSON-formatted
description of the function parameters, provided to the model. If omitted, the
description is autogenerated.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments
for specifying binding fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

property auth_level: [AuthLevel](#)

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns: Authorization level of the function app.

blob_input(arg_name: str, path: str, connection: str, data_type: [DataType](#) | None = None, **kwargs) -> [Callable](#)[[...](#), [Any](#)]

The *blob_input* decorator adds `<xref:agent_framework.azure.BlobInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

Parameters: * **arg_name** (`str`) – The name of the variable that represents the blob in the function code.

```
* path (<xref:str>) – The path to the blob.

* connection (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

* kwargs – Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: `Callable[[...], Any]`

```
blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None = None,
**kwargs) -> Callable[[...], Any]
```

The *blob_output* decorator adds `<xref:agent_framework.azure.BlobOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the blob in the function code.

```
* path (<xref:str>) – The path to the blob.

* connection (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

* kwargs – Keyword arguments for specifying additional binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The `blob_change_trigger` decorator adds [<xref:agent_framework.azure.BlobTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building [<xref:agent_framework.azure.Function>](#) object used in worker function indexing model. This is equivalent to defining `BlobTrigger` in the `function.json` which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the [<xref:agent_framework.azure.InputStream>](#) object in function code.

```
* path (<xref:str>) – The path to the blob.

* connection (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Azure Blobs.

* source (<xref:azure.functions.decorators.core.BlobSource>* | *<xref:None>) –
Sets the source of the triggering event. Use "EventGrid" for an Event Grid-based blob
trigger, which provides much lower latency. The default is "LogsAndContainerScan",
which uses the standard polling mechanism to detect changes in the container.
```



```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>*) -  
Defines how Functions runtime should treat the parameter value.  
  
* **kwargs** - Keyword arguments for specifying additional binding fields to include  
in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`cosmos_db_input(arg_name: str, connection: str, database_name: str, container_name: str,
partition_key: str | None = None, id: str | None = None, sql_query: str | None = None,
preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) ->
Callable[[...], Any]`

The `cosmos_db_input` decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining `CosmosDBInput` in the `function.json`, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> ↗ Additional details:
<https://aka.ms/cosmosdb-v4-update> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` input object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting container that  
specifies how to connect to the Azure Cosmos DB account being monitored.  
  
* **database_name** (<xref:str>) - The name of the Cosmos DB database that contains  
the document.  
  
* **container_name** (<xref:str>) - The name of the container that holds the  
document.  
  
* **partition_key** (<xref:str>* | *<xref:None>*) - The partition key value for the  
document lookup.  
  
* **id** (<xref:str>* | *<xref:None>*) - The ID of the document to retrieve.
```

* **sql_query** (<xref:str>* | *<xref:None>) - An Azure Cosmos DB SQL query used to retrieve multiple documents.

* **preferred_locations** (<xref:str>* | *<xref:None>) - (Optional) Preferred geo-replicated regions. Values should be comma-separated, e.g., "East US,South Central US,North Europe".

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional binding fields to include in the binding JSON.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, id: str | None = None, sql_query: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The `cosmos_db_input_v3` decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining `CosmosDBInput` in the `function.json`, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` input object in the function code.

* **database_name** (<xref:str>) - The name of the Azure Cosmos DB database containing the target collection.

* **collection_name** (<xref:str>) - The name of the collection that contains the document(s).

* **connection_string_setting** (<xref:str>) - The name of the app setting that contains the Azure Cosmos DB connection string.

* **id** (<xref:str>* | *<xref:None>) - The ID of a single document to retrieve.

```
* **sql_query** (<xref:str>* | *<xref:None>) - An Azure Cosmos DB SQL query used to retrieve multiple documents.

* **partition_key** (<xref:str>* | *<xref:None>) - Specifies the partition key value for the lookup.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include in the function.json.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`cosmos_db_output(arg_name: str, connection: str, database_name: str, container_name: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, container_throughput: int | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The `cosmos_db_output` decorator adds [<xref:agent_framework.azure.CosmosDBOutput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining `CosmosDBOutput` in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Cosmos DB output object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.

* **database_name** (<xref:str>) - The name of the Azure Cosmos DB database containing the target container.

* **container_name** (<xref:str>) - The name of the container to which documents are written.

* **create_if_not_exists** (<xref:bool>* | *<xref:None>) - A boolean indicating
```

whether the container should be created automatically if it does not exist.

* **partition_key** (`<xref:str>` | `*<xref:None>`) – When `*create_if_not_exists*` is `*True*`, this defines the partition key path for the created container.

* **container_throughput** (`<xref:int>` | `*<xref:None>`) – When `*create_if_not_exists*` is `*True*`, this defines the throughput (Request Units) for the created container.

* **preferred_locations** (`<xref:str>` | `*<xref:None>`) – Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB. When set along with `*use_multiple_write_locations*`, it enables multi-region writes.

* **data_type** (`<xref:azure.functions.decorators.core.DataType>` | `*<xref:str>` | `*<xref:None>`) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields to include in the `*function.json*`.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`cosmos_db_output_v3(arg_name: str, database_name: str, collection_name: str, connection_string_setting: str, create_if_not_exists: bool | None = None, partition_key: str | None = None, collection_throughput: int | None = None, use_multiple_write_locations: bool | None = None, preferred_locations: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The `cosmos_db_output_v3` decorator adds `<xref:agent_framework.azure.CosmosDBOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining `CosmosDBOutput` in the `function.json`, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗](#)

Parameters: * **arg_name** (`str`) – The name of the variable that represents the Cosmos DB output object in the function code.

* **database_name** (`<xref:str>`) – The name of the Azure Cosmos DB database containing the monitored collection.

- * **collection_name** (<xref:str>) - The name of the collection to which documents are written.
- * **connection_string_setting** (<xref:str>) - The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.
- * **create_if_not_exists** (<xref:bool>* | *<xref:None>) - A boolean indicating whether the collection should be created automatically if it does not exist.
- * **partition_key** (<xref:str>* | *<xref:None>) - When *create_if_not_exists* is *True*, this defines the partition key path for the created collection.
- * **collection_throughput** (<xref:int>* | *<xref:None>) - When *create_if_not_exists* is *True*, this defines the throughput (Request Units) for the created collection.
- * **use_multiple_write_locations** (<xref:bool>* | *<xref:None>) - When set to *True* and used in conjunction with *preferred_locations*, this enables multi-region writes in the Azure Cosmos DB service.
- * **preferred_locations** (<xref:str>* | *<xref:None>) - Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
- * **kwargs** - Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
cosmos_db_trigger(arg_name: str, connection: str, database_name: str, container_name: str,
lease_connection: str | None = None, lease_database_name: str | None = None,
lease_container_name: str | None = None, create_lease_container_if_not_exists: bool | None =
None, leases_container_throughput: int | None = None, lease_container_prefix: str | None = None,
feed_poll_delay: int | None = None, lease_acquire_interval: int | None = None,
lease_expiration_interval: int | None = None, lease_renew_interval: int | None = None,
max_items_per_invocation: int | None = None, start_from_beginning: time | None = None,
start_from_time: time | None = None, preferred_locations: str | None = None, data_type:
DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]
```

The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: <https://aka.ms/cosmosdb-v4-update> Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4>

Parameters: * **arg_name** (*str*) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to the monitored Azure Cosmos DB account.

* **database_name** (<xref:str>) - The name of the Azure Cosmos DB database
containing the monitored collection.

* **container_name** (<xref:str>) - The name of the container being monitored.

* **lease_connection** (<xref:str>* | *<xref:None>) - (Optional) The name of an app
setting or setting collection that specifies how to connect to the Cosmos DB account
that holds the lease container.

* **lease_database_name** (<xref:str>* | *<xref:None>) - The name of the database
that holds the collection used to store leases.

* **lease_container_name** (<xref:str>* | *<xref:None>) - (Optional) The name of the
container used to store leases. If not set, the default value "leases" is used.

* **create_lease_container_if_not_exists** (<xref:bool>* | *<xref:None>) -
(Optional) If *True*, the leases container is created automatically if it does not
exist. Defaults to *False*. Note: When using Azure AD identities, container creation
is not allowed, and the function will not start if this is *True*.

* **leases_container_throughput** (<xref:int>* | *<xref:None>) - (Optional) The
number of Request Units (RUs) to assign when the leases container is created. This is
used only when *create_lease_container_if_not_exists* is *True*. It is automatically
set when configured through the Azure Portal.

* **lease_container_prefix** (<xref:str>* | *<xref:None>) - (Optional) A prefix
added to leases created in the lease container for this function. Use this to allow
multiple functions to share the same lease container with different prefixes.

* **feed_poll_delay** (<xref:int>* | *<xref:None>) - The delay (in milliseconds)
between polling a partition for new changes after draining current changes.

* **lease_acquire_interval** (<xref:int>* | *<xref:None>) - The interval (in
milliseconds) to trigger a task to check if partitions are evenly distributed across
host instances.
```

* **lease_expiration_interval** (<xref:int>* | *<xref:None>) - The interval (in milliseconds) for which a lease is held for a partition.

* **lease_renew_interval** (<xref:int>* | *<xref:None>) - The interval (in milliseconds) to renew all leases for partitions currently held by an instance.

* **max_items_per_invocation** (<xref:int>* | *<xref:None>) - Maximum number of items received per function call.

* **start_from_beginning** (<xref:datetime.time>* | *<xref:None>) - If **True**, the trigger starts reading changes from the beginning of the collection's change history instead of the current time.

* **start_from_time** (<xref:datetime.time>* | *<xref:None>) - (Optional) The date and time from which to begin reading the change feed. Use ISO 8601 format with a UTC designator, e.g., **2021-02-16T14:19:29Z**. Only used to set the initial trigger state; has no effect once the trigger has a lease state.

* **preferred_locations** (<xref:str>* | *<xref:None>) - Preferred locations (regions) for geo-replicated Cosmos DB accounts.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>) - Additional keyword arguments for specifying binding fields to include in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, lease_collection_name: str | None = None,
lease_connection_string_setting: str | None = None, lease_database_name: str | None = None,
create_lease_collection_if_not_exists: bool | None = None, leases_collection_throughput: int |
None = None, lease_collection_prefix: str | None = None, checkpoint_interval: int | None = None,
checkpoint_document_count: int | None = None, feed_poll_delay: int | None = None,
lease_renew_interval: int | None = None, lease_acquire_interval: int | None = None,
lease_expiration_interval: int | None = None, max_items_per_invocation: int | None = None,
start_from_beginning: bool | None = None, preferred_locations: str | None = None, data_type:
DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]
```

The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: <https://aka.ms/cosmosdb-v4-update> [↗](#) Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗](#)

Parameters: * **arg_name** (**str**) – The name of the variable that represents the `<xref:agent_framework.azure.DocumentList>` object in the function code.

```
* **database_name** (<xref:str>) – The name of the Azure Cosmos DB database
containing the monitored collection.

* **collection_name** (<xref:str>) – The name of the collection being monitored.

* **connection_string_setting** (<xref:str>) – The name of an app setting or setting
collection specifying how to connect to the monitored Azure Cosmos DB account.

* **lease_collection_name** (<xref:str>* | *<xref:None>) – The name of the
collection used to store leases.

* **lease_connection_string_setting** (<xref:str>* | *<xref:None>) – The name of an
app setting or setting collection that specifies how to connect to the Azure Cosmos DB
account that holds the lease collection.

* **lease_database_name** (<xref:str>* | *<xref:None>) – The name of the database
that holds the lease collection.

* **create_lease_collection_if_not_exists** (<xref:bool>* | *<xref:None>) – If
*True*, the lease collection is automatically created when it does not already exist.

* **leases_collection_throughput** (<xref:int>* | *<xref:None>) – The number of
Request Units (RUs) assigned when the lease collection is created.

* **lease_collection_prefix** (<xref:str>* | *<xref:None>) – A prefix added to
leases created in the lease collection for this Function.

* **checkpoint_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) between lease checkpoints. The default behavior is to checkpoint after
every function call.

* **checkpoint_document_count** (<xref:int>* | *<xref:None>) – Number of documents
processed between lease checkpoints. Default is after every function call.

* **feed_poll_delay** (<xref:int>* | *<xref:None>) – The delay (in milliseconds)
between polling a partition for new changes after all current changes are drained.

* **lease_renew_interval** (<xref:int>* | *<xref:None>) – The interval (in
milliseconds) to renew leases for partitions currently held by an instance.
```


- * **lease_acquire_interval** (<xref:int>* | *<xref:None>) - The interval (in milliseconds) to trigger a task that checks whether partitions are evenly distributed among host instances.
- * **lease_expiration_interval** (<xref:int>* | *<xref:None>) - The interval (in milliseconds) for which a lease is held for a partition.
- * **max_items_per_invocation** (<xref:int>* | *<xref:None>) - Maximum number of items received per function call.
- * **start_from_beginning** (<xref:bool>* | *<xref:None>) - If *True*, the trigger starts reading changes from the beginning of the collection's change history rather than from the current time.
- * **preferred_locations** (<xref:str>* | *<xref:None>) - Preferred locations (regions) for geo-replicated database accounts in Azure Cosmos DB.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
- * **kwargs** (<xref:agent_framework.exceptions.Any>) - Additional keyword arguments for specifying binding fields in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_binding_output(arg_name: str, binding_name: str, operation: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-binding-output-binding> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state output object in the function code.

- * **binding_name** (<xref:str>) - The configured name of the binding.

```
* **operation** (<xref:str>) - The configured operation.

* **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The *dapr_binding_trigger* decorator adds `<xref:agent_framework.azure.DaprBindingTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-binding> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the trigger input in the function code.

```
* **binding_name** (<xref:str>) - The name of the Dapr trigger. If not specified,
the function name is used as the trigger name.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>) - Keyword arguments for
specifying additional binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The `dapr_invoke_output` decorator adds `<xref:agent_framework.azure.DaprInvokeOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `DaprInvokeOutput` in the `function.json`, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-invoke-output-binding> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state output object in the function code.

```
* **app_id** (<xref:str>) - The Dapr app ID to invoke.

* **method_name** (<xref:str>) - The method name of the app to invoke.

* **http_verb** (<xref:str>) - The HTTP verb to use for the invocation (e.g., GET, POST).

* **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional; defaults to [http://localhost](http://localhost):{daprHttpPort}.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable\[\[...\], Any\]](#)

`dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The `dapr_publish_output` decorator adds `<xref:agent_framework.azure.DaprPublishOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-publish-output-binding> ↗

Parameters: * **arg_name** (**str**) – The name of the variable that represents the Dapr state output object in the function code.

```
* **pub_sub_name** (<xref:str>) - The pub/sub component name to publish to.

* **topic** (<xref:str>) - The name of the topic to publish to.

* **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str, dapr_address: str |
None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]
```

The *dapr_secret_input* decorator adds `<xref:agent_framework.azure.DaprSecretInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-secret-input-binding> ↗

Parameters: * **arg_name** (**str**) – The name of the variable that represents the Dapr state input object in the function code.

- * **secret_store_name** (<xref:str>) - The name of the secret store from which to retrieve the secret.
- * **key** (<xref:str>) - The key identifying the name of the secret to retrieve.
- * **metadata** (<xref:str>) - Metadata properties in the form "key1=value1&key2=value2".
- * **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional; Defaults to http://localhost:{daprHttpPort}.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
- * **kwargs** - Additional keyword arguments for specifying binding fields in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The *dapr_service_invocation_trigger* decorator adds `<xref:agent_framework.azure.DaprServiceInvocationTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-service-invocation> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the service invocation input in the function code.

- * **method_name** (<xref:str>) - The name of the method on a remote Dapr App. If not specified, the name of the function is used as the method name.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.

```
* **kwargs** (<xref:agent_framework.exceptions.Any>) – Keyword arguments for specifying additional binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

```
dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

The *dapr_state_input* decorator adds <xref:agent_framework.azure.DaprStateInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-input-binding> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Dapr state input object in the function code.

```
* **state_store** (<xref:str>) – The state store containing the state.

* **key** (<xref:str>) – The name of the key to retrieve.

* **dapr_address** (<xref:str>* | *<xref:None>) – The Dapr address. Optional; Defaults to [http://localhost](http://localhost):{daprHttpPort}.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **kwargs** – Additional keyword arguments for specifying binding fields in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

```
dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

The *dapr_state_output* decorator adds `<xref:agent_framework.azure.DaprStateOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-output-binding> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents the Dapr state output object in the function code.

```
* **state_store** (<xref:str>) - The state store containing the state for the
specified keys.

* **key** (<xref:str>) - The name of the key used to store the state.

* **dapr_address** (<xref:str>* | *<xref:None>) - The Dapr address. Optional;
defaults to [http://localhost](http://localhost):{daprHttpPort}.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The *dapr_topic_trigger* decorator adds `<xref:agent_framework.azure.DaprTopicTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-topic> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the trigger input in the function code.

```
* pub_sub_name (<xref:str>) – The name of the Dapr pub/sub component.

* topic (<xref:str>) – The topic name. If unspecified, the function name is used.

* route (<xref:str>* | *<xref:None>) – The route for the trigger. If unspecified, the topic name is used.

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* kwargs (<xref:agent_framework.exceptions.Any>) – Additional keyword arguments for specifying extra binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`durable_client_input(client_name: str, task_hub: str | None = None, connection_name: str | None = None)`

Register a Durable-client Function.

Parameters: * **client_name** ([str](#)) – Parameter name of durable client.

```
* task_hub (<xref:agent_framework.azure.Optional[str]>) – Used in scenarios where multiple function apps share the same storage account but need to be isolated from each other. If not specified, the default value from host.json is used. This value must match the value used by the target orchestrator functions.

* connection_name (<xref:agent_framework.azure.Optional[str]>) – The name of an app setting that contains a storage account connection string. The storage account represented by this connection string must be the same one used by the target orchestrator functions. If not specified, the default storage account connection string for the function app is used.
```

`durable_openai_agent_orchestrator(_func=None, *, model_provider=None, model_retry_options: RetryOptions | None = RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5))`

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Parameters: * **model_provider** (<xref:agent_framework.azure.Optional[ModelProvider]>) – Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.

```
* **model_retry_options**  
(<xref:azure.durable_functions.models.RetryOptions.RetryOptions>* | *<xref:None>*) –
```

embeddings_input(arg_name: str, input: str, input_type: InputType, ai_connection_name: str | None = "", embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: <https://platform.openai.com/docs/guides/embeddings> ↗

Parameters: * **arg_name** (str) – The name of the binding parameter in the function code.

```
* **input** (<xref:str>) – The input source containing the data to generate  
embeddings for.  
  
* **input_type** (<xref:azure.functions.decorators.openai.InputType>) – The type of  
the input (e.g., string, list, file reference).  
  
* **ai_connection_name** (<xref:str>* | *<xref:None>*) –  
  
The name of the configuration section for AI service  
connectivity settings.  
  
* **Azure OpenAI**:: If specified, looks for "Endpoint" and/or "Key" in this  
section.  
  
* If not specified or not found, falls back to the environment variables  
*AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.  
  
* For user-assigned managed identity authentication, this is required.  
  
* **OpenAI (non-Azure)**: Set the *OPENAI_API_KEY* environment variable.  
  
* **model** – *(Deprecated)* Use *embeddings_model* instead. This parameter is  
unused and will be removed in future versions.  
  
* **embeddings_model** (<xref:str>* | *
```

```
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name for OpenAI Embeddings. Default is "text-embedding-ada-002".

* max_chunk_length (<xref:int>* | *<xref:None>) - The maximum number of characters into which the input should be chunked. Default is 8 * 1024.

* max_overlap (<xref:int>* | *<xref:None>) - The maximum number of characters to overlap between input chunks. Default is 128.

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Optional. Defines how the Functions runtime should interpret the parameter value.

* kwargs - Additional keyword arguments to include in the function.json binding configuration.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

```
embeddings_store_output(arg_name: str, input: str, input_type: InputType,
store_connection_name: str, collection: str, ai_connection_name: str | None = "",
embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel,
max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type: DataType | str
| None = None, **kwargs) -> Callable[[...], Any]
```

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- [Azure AI Search](#)
- [Azure Data Explorer](#)
- [Azure Cosmos DB using MongoDB](#)

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* input (<xref:str>) - The input data for which embeddings should be generated.

* input_type (<xref:azure.functions.decorators.openai.InputType>) - The type of the input (e.g., string, list).

* store_connection_name (<xref:str>) - The name of an app setting or environment variable containing the vector store connection value.
```

- * **collection** (<xref:str>) - The collection or table to search within.
- * **ai_connection_name** (<xref:str>* | *<xref:None>) -

The name of the configuration section for AI service connectivity settings.

* For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section.

* If not specified, or if the section is missing, falls back to environment variables: `*AZURE_OPENAI_ENDPOINT*` and `*AZURE_OPENAI_KEY*`.

* For user-assigned managed identity, this property is required.

* For OpenAI (non-Azure), set the `*OPENAI_API_KEY*` environment variable.
- * **model** - Deprecated. Use `*embeddings_model*` instead. This parameter is unused and will be removed in future versions.
- * **embeddings_model** (<xref:str>* | *<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name for OpenAI Embeddings. The default is "text-embedding-ada-002".
- * **max_chunk_length** (<xref:int>* | *<xref:None>) - The maximum number of characters to chunk the input into.
- * **max_overlap** (<xref:int>* | *<xref:None>) - The maximum number of characters to overlap between chunks.
- * **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Optional. Defines how the Functions runtime should treat the parameter value. Defaults to None.
- * **kwargs** - Additional keyword arguments to include in the `*function.json*` binding.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

enable_health_check: [bool](#)

enable_http_endpoints: [bool](#)

enable_mcp_tool_trigger: [bool](#)

entity_trigger(context_name: str, entity_name: str | None = None)

Register an Entity Function.

Parameters: * **context_name** ([str](#)) – Parameter name of the Entity input.

```
* **entity_name** (<xref:agent_framework.azure.Optional[str]>) – Name of Entity Function. The value is None by default, in which case the name of the method is used.
```

`event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None, topic_key_setting: str | None = None, connection: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The `event_grid_output` decorator adds `<xref:agent_framework.azure.EventGridOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/eventgridtrigger> [↗](#)

Parameters: * **arg_name** ([str](#)) – The variable name used in the function code that represents the event.

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) – Defines how the Functions runtime should treat the parameter value.
```

```
* **topic_endpoint_uri** (<xref:str>* | *<xref:None>) – The name of an app setting that contains the URI for the custom topic.
```

```
* **topic_key_setting** (<xref:str>* | *<xref:None>) – The name of an app setting that contains an access key for the custom topic.
```

```
* **connection** (<xref:str>* | *<xref:None>) – The value of the common prefix for the setting that contains the topic endpoint URI.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`event_grid_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *event_grid_trigger* decorator adds `<xref:agent_framework.azure.EventGridTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/eventgridtrigger> ↗

Parameters: * **arg_name** (*str*) – The variable name used in the function code for the parameter that receives the event data.

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *  
<xref:None>*) - Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
event_hub_message_trigger(arg_name: str, connection: str, event_hub_name: str, data_type:  
DataType | str | None = None, cardinality: Cardinality | str | None = None, consumer_group: str |  
None = None, **kwargs: Any) -> Callable[..., Any]
```

The *event_hub_message_trigger* decorator adds `<xref:agent_framework.azure.EventHubTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents the `<xref:agent_framework.azure.EventHubEvent>` object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Event Hubs.

* **event_hub_name** (<xref:str>) - The name of the Event Hub.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) - Set to "many" to enable batching.

* **consumer_group** (<xref:str>* | *<xref:None>) - Optional. The consumer group
used to subscribe to events in the hub.

* **kwargs** (<xref:agent_framework.exceptions.Any>) - Additional keyword arguments
to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *event_hub_output* decorator adds `<xref:agent_framework.azure.EventHubOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> 

Parameters: * **arg_name** ([str](#)) – The name of the variable representing the Event Hub output object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Event Hub.

* **event_hub_name** (<xref:str>) - The name of the Event Hub to send messages to.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

```
* **kwargs** - Additional keyword arguments for specifying binding fields to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[...], [Any](#)

function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) -> [Callable](#)[...], [Any](#)

Optional: Sets name of the `<xref:agent_framework.azure.Function>` object. If not set, it will default to the name of the method name.

Parameters: * **name** ([str](#)) – Name of the function.

```
* **setting_extra_fields** (<xref:typing.Dict>[*<xref:str>*, *  
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) - Keyword arguments for  
specifying
```

Return type: [Callable](#)[...], [Any](#)

additional setting fields :return: Decorator function.

generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs)
-> [Callable](#)[...], [Any](#)

The *generic_input_binding* decorator adds `<xref:agent_framework.azure.GenericInputBinding>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the input parameter in the function code.

```
* **type** (<xref:str>) - The type of the binding.  
  
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *  
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

```
* **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`generic_output_binding(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The *generic_output_binding* decorator adds `<xref:agent_framework.azure.GenericOutputBinding>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the output parameter in the function code.

```
* **type** (<xref:str>) - The type of the binding.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields in the
*function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`generic_trigger(arg_name: str, type: str, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The *generic_trigger* decorator adds `<xref:agent_framework.azure.GenericTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-custom> 

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **type** (<xref:str>) – The type of the binding.  
  
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *  
<xref:None>) – Defines how the Functions runtime should treat the parameter value.  
  
* **kwargs** – Keyword arguments for specifying additional binding fields to include  
in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`get_agent(context: Any, agent_name: str) ->`

`<xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>`

Return a DurableAIAgent proxy for a registered agent.

Parameters: * **context** ([Any](#)) – Durable Functions orchestration context invoking the agent.

```
* **agent_name** (<xref:str>) – Name of the agent registered on this app.
```

Raises: [ValueError](#) – If the requested agent has not been registered.

Returns: DurableAIAgent wrapper bound to the orchestration context.

Return type: `<xref:agent_framework_azurefunctions._orchestration.DurableAIAgent>`

`get_functions() -> List[Function]`

Get the function objects in the function app.

Returns: A list of `<xref:agent_framework.azure.Function>` objects defined in the app.

Return type: [List\[Function\]](#)

`http_type(http_type: str) -> Callable[..., Any]`

Set http type of the `<xref:agent_framework.azure.Function>` object.

Parameters: **http_type** ([str](#)) – Http type of the function.

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None = None, o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000, batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms: int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647, authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol: BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

The *kafka_output* decorator adds `<xref:agent_framework.azure.KafkaOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/kafkaoutput> [↗](#)

Parameters: * **arg_name** ([str](#)) – The variable name used in the function code that represents the output event.

* **topic** (`<xref:str>`) – The Kafka topic to which messages are published.

* **broker_list** (<xref:str>) - The list of Kafka brokers to which the producer connects.

* **avro_schema** (<xref:str>* | *<xref:None>) - Optional. Avro schema to generate a generic record.

* **username** (<xref:str>* | *<xref:None>) - SASL username for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to `'sasl.username'` in `librdkafka`.

* **password** (<xref:str>* | *<xref:None>) - SASL password for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to `'sasl.password'` in `librdkafka`.

* **ssl_key_location** (<xref:str>* | *<xref:None>) - Path to the client's private key (PEM) for authentication. Equivalent to `'ssl.key.location'` in `librdkafka`.

* **ssl_ca_location** (<xref:str>* | *<xref:None>) - Path to the CA certificate for verifying the broker's certificate. Equivalent to `'ssl.ca.location'` in `librdkafka`.

* **ssl_certificate_location** (<xref:str>* | *<xref:None>) - Path to the client's certificate file. Equivalent to `'ssl.certificate.location'` in `librdkafka`.

* **ssl_key_password** (<xref:str>* | *<xref:None>) - Password for the client's SSL key. Equivalent to `'ssl.key.password'` in `librdkafka`.

* **schema_registry_url** (<xref:str>* | *<xref:None>) - URL of the Avro Schema Registry.

* **schema_registry_username** (<xref:str>* | *<xref:None>) - Username for accessing the Schema Registry.

* **schema_registry_password** (<xref:str>* | *<xref:None>) - Password for accessing the Schema Registry.

* **o_auth_bearer_method**
(<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *<xref:None>) - OAuth bearer method to use, e.g., `'default'` or `'oidc'`. Equivalent to `'sasl.oauthbearer.method'` in `librdkafka`.

* **o_auth_bearer_client_id** (<xref:str>* | *<xref:None>) - Used with `'oidc'` method. Equivalent to `'sasl.oauthbearer.client.id'` in `librdkafka`.

* **o_auth_bearer_client_secret** (<xref:str>* | *<xref:None>) - Used with `'oidc'` method. Equivalent to `'sasl.oauthbearer.client.secret'` in `librdkafka`.

* **o_auth_bearer_scope** (<xref:str>* | *<xref:None>) - Scope of the access request sent to the broker. Equivalent to `'sasl.oauthbearer.scope'` in `librdkafka`.

* **o_auth_bearer_token_endpoint_url** (<xref:str>* | *<xref:None>) - OAuth token endpoint. Equivalent to `'sasl.oauthbearer.token.endpoint.url'` in `librdkafka`.

* **o_auth_bearer_extensions** (<xref:str>* | *<xref:None>) - Additional metadata sent to the broker. Comma-separated key=value pairs, e.g., `"supportFeatureX=true,organizationId=sales-emea"`. Equivalent to `'sasl.oauthbearer.extensions'` in `librdkafka`.

* **max_message_bytes** (<xref:int>) - Maximum size (in bytes) of a transmitted message. Default is 1MB.

* **batch_size** (<xref:int>) - Maximum number of messages batched in one MessageSet. Default is 10,000.

* **enable_idempotence** (<xref:bool>) - If **True**, ensures messages are delivered exactly once and in order. Default is **False**.

* **message_timeout_ms** (<xref:int>) - Local timeout for message delivery. Default is 300000 ms.

* **request_timeout_ms** (<xref:int>) - Timeout for producer request acknowledgment. Default is 5000 ms.

* **max_retries** (<xref:int>) - Maximum number of retry attempts for failed messages. Default is 2147483647. Retrying may cause reordering unless **enable_idempotence=True**.

* **authentication_mode** (<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode>* | *<xref:str>* | *<xref:None>*) - SASL mechanism used for authentication. Allowed values: **Gssapi**, **Plain**, **ScramSha256**, **ScramSha512**. Default is **Plain**. Equivalent to **'sasl.mechanism'** in *librdkafka*.

* **protocol** (<xref:azure.functions.decorators.kafka.BrokerProtocol>* | *<xref:str>* | *<xref:None>*) - Security protocol used for broker communication. Default is plaintext. Equivalent to **'security.protocol'** in *librdkafka*.

* **linger_ms** (<xref:int>) - Time to wait between sending batches of messages to allow for better throughput via batching.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>*) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include in the **function.json**.

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

kafka_trigger(arg_name: str, topic: str, broker_list: str, event_hub_connection_string: str | None = None, consumer_group: str | None = None, avro_schema: str | None = None, username: str | None = None, password: str | None = None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None, ssl_certificate_location: str | None = None, ssl_key_password: str | None = None, schema_registry_url: str | None = None, schema_registry_username: str | None = None, schema_registry_password: str | None = None, o_auth_bearer_method: OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None, o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None = None, o_auth_bearer_token_endpoint_url: str | None =

None, o_auth_bearer_extensions: str | None = None, authentication_mode: BrokerAuthenticationMode | str | None = 'NotSet', protocol: BrokerProtocol | str | None = 'NotSet', cardinality: Cardinality | str | None = 'One', lag_threshold: int = 1000, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *kafka_trigger* decorator adds `<xref:agent_framework.azure.KafkaTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/kafkatrigger> [↗](#)

Parameters: * **arg_name** ([str](#)) – The variable name used in the function code for the parameter that receives the Kafka event data.

```
* **topic** (<xref:str>) - The topic monitored by the trigger.

* **broker_list** (<xref:str>) - The list of Kafka brokers monitored by the trigger.

* **event_hub_connection_string** (<xref:str>* | *<xref:None>) - The name of an app
setting that contains the connection string for Event Hub (when using Kafka protocol
headers with Azure Event Hubs).

* **consumer_group** (<xref:str>* | *<xref:None>) - Kafka consumer group used by the
trigger.

* **avro_schema** (<xref:str>* | *<xref:None>) - Used only if a generic Avro record
should be generated.

* **username** (<xref:str>* | *<xref:None>) - SASL username for use with the PLAIN
or SASL-SCRAM mechanisms. Equivalent to 'sasl.username' in librdkafka. Default is
empty string.

* **password** (<xref:str>* | *<xref:None>) - SASL password for use with the PLAIN
or SASL-SCRAM mechanisms. Equivalent to 'sasl.password' in librdkafka. Default is
empty string.

* **ssl_key_location** (<xref:str>* | *<xref:None>) - Path to the client's private
key (PEM) used for authentication. Equivalent to 'ssl.key.location' in librdkafka.

* **ssl_ca_location** (<xref:str>* | *<xref:None>) - Path to the CA certificate file
for verifying the broker's certificate. Equivalent to 'ssl.ca.location' in librdkafka.
```

* **ssl_certificate_location** (<xref:str>* | *<xref:None>) - Path to the client's certificate. Equivalent to 'ssl.certificate.location' in librdkafka.

* **ssl_key_password** (<xref:str>* | *<xref:None>) - Password for the client's certificate. Equivalent to 'ssl.key.password' in librdkafka.

* **schema_registry_url** (<xref:str>* | *<xref:None>) - URL of the Avro Schema Registry.

* **schema_registry_username** (<xref:str>* | *<xref:None>) - Username for the Schema Registry.

* **schema_registry_password** (<xref:str>* | *<xref:None>) - Password for the Schema Registry.

* **o_auth_bearer_method**
(<xref:azure.functions.decorators.kafka.OAuthBearerMethod>* | *<xref:str>* | *<xref:None>) - Either 'default' or 'oidc'. Equivalent to 'sasl.oauthbearer.method' in librdkafka.

* **o_auth_bearer_client_id** (<xref:str>* | *<xref:None>) - Specify only if **o_auth_bearer_method** is 'oidc'. Equivalent to 'sasl.oauthbearer.client.id' in librdkafka.

* **o_auth_bearer_client_secret** (<xref:str>* | *<xref:None>) - Specify only if **o_auth_bearer_method** is 'oidc'. Equivalent to 'sasl.oauthbearer.client.secret' in librdkafka.

* **o_auth_bearer_scope** (<xref:str>* | *<xref:None>) - Specify only if **o_auth_bearer_method** is 'oidc'. Used to specify access scope. Equivalent to 'sasl.oauthbearer.scope'.

* **o_auth_bearer_token_endpoint_url** (<xref:str>* | *<xref:None>) - Specify only if **o_auth_bearer_method** is 'oidc'. Equivalent to 'sasl.oauthbearer.token.endpoint.url' in librdkafka.

* **o_auth_bearer_extensions** (<xref:str>* | *<xref:None>) - Additional information for the broker, in the form of a comma-separated list of key=value pairs (e.g., "orgId=abc,flag=true"). Equivalent to 'sasl.oauthbearer.extensions' in librdkafka.

* **authentication_mode**
(<xref:azure.functions.decorators.kafka.BrokerAuthenticationMode>* | *<xref:str>* | *<xref:None>) - SASL mechanism to use. Allowed: Gssapi, Plain, ScramSha256, ScramSha512. Default: Plain. Equivalent to 'sasl.mechanism'.

* **protocol** (<xref:azure.functions.decorators.kafka.BrokerProtocol>* | *<xref:str>* | *<xref:None>) - Security protocol used to communicate with brokers. Default: plaintext. Equivalent to 'security.protocol'.

* **lag_threshold** (<xref:int>) - Max number of unprocessed messages per worker instance. Used in scaling logic to estimate needed worker instances. Default is 1000.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>) - Defines how Functions runtime should treat the parameter value.

```
* **kwargs** - Additional keyword arguments for extra binding fields in the binding JSON.
```

```
* **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *  
<xref:str>* | *<xref:None>) -
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None, tool_properties: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]`

The `mcp_tool_trigger` decorator adds `<xref:agent_framework.azure.MCPToolTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `MCPToolTrigger` in the `function.json`, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/remote-mcp-functions-python> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the trigger parameter in the function code.

```
* **tool_name** (<xref:str>) - The logical tool name exposed to the host.  
  
* **description** (<xref:str>* | *<xref:None>) - Optional human-readable description  
of the tool.  
  
* **tool_properties** (<xref:str>* | *<xref:None>) - JSON-serialized tool  
properties/parameters list.  
  
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *  
<xref:None>) - Defines how the Functions runtime should treat the parameter value.  
  
* **kwargs** - Keyword arguments for specifying additional binding fields to include  
in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`mysql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str | None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]`

The `mysql_input` decorator adds `<xref:agent_framework.azure.MySqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining `MySqlInput` in the `function.json`, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> 

Parameters: * **arg_name** (`str`) – The name of the variable that represents a `<xref:agent_framework.azure.MySqlRowList>` input object in the function code.

```
* **command_text** (<xref:str>) - The SQL query string or the name of the stored
procedure to execute.

* **connection_string_setting** (<xref:str>) - The name of the app setting that
contains the connection string to the MySQL database.

* **command_type** (<xref:str>* | *<xref:None>) - The type of command being
executed. Accepts "Text" for SQL queries or "StoredProcedure" for stored
procedures.

* **parameters** (<xref:str>* | *<xref:None>) - One or more parameter values passed
to the SQL command. Should be provided as a comma-separated string in the format:
"@param1=value1,@param2=value2".

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments used to define fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: `Callable[[...], Any]`

`mysql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]`

The *mysql_output* decorator adds `<xref:agent_framework.azure.MySqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> 

Parameters: * **arg_name** (*str*) – The name of the variable that represents the MySQL output object in the function code.

```
* command_text (<xref:str>) – The SQL command or the name of the stored
procedure executed by the binding.

* connection_string_setting (<xref:str>) – The name of the app setting that
contains the connection string for the target MySQL database.

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) –
Defines how the Functions runtime should treat the parameter value.

* kwargs – Additional keyword arguments used to define fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *mysql_trigger* decorator adds `<xref:agent_framework.azure.MySqlTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a [<xref:agent_framework.azure.MySqlRowList>](#) object in the function code.

```
* table_name (<xref:str>) - The name of the MySQL table monitored by the trigger.
```

```
* connection_string_setting (<xref:str>) - The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.
```

```
* leases_table_name (<xref:str>* | *<xref:None>) - The name of the table used to store leases. If not specified, the default table name is *Leases_{FunctionId}_{TableId}*.
```

```
* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

```
* kwargs - Additional keyword arguments for specifying binding fields to include in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[[\[...\]](#), [Any](#)]

`orchestration_trigger(context_name: str, orchestration: str | None = None)`

Register an Orchestrator Function.

Parameters: * **context_name** ([str](#)) – Parameter name of the `DurableOrchestrationContext` object.

```
* orchestration (<xref:agent_framework.azure.Optional[str]>) - Name of Orchestrator Function. The value is None by default, in which case the name of the method is used.
```

`queue_output(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[\[...\], Any]`

The `queue_output` decorator adds [<xref:agent_framework.azure.QueueOutput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) used in the worker function indexing model.

This is equivalent to defining `QueueOutput` in the `function.json`, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the storage queue output object in the function code.

```
* **queue_name** (<xref:str>) - The name of the queue to which messages will be
written.

* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Azure Queues.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include
in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *queue_trigger* decorator adds `<xref:agent_framework.azure.QueueTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.QueueMessage>` object in the function code.

```
* **queue_name** (<xref:str>) - The name of the queue to monitor.
```

```
* **connection** (<xref:str>) - The name of an app setting or setting collection that specifies how to connect to Azure Queues.
```

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

```
* **kwargs** - Additional keyword arguments to include in the binding JSON.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`register_blueprint(function_container: DecoratorApi) -> None`

Register a list of functions in the function app.

Parameters: **function_container** ([DecoratorApi](#)) – Instance extending `<xref:agent_framework.azure.DecoratorApi>`, which contains a list of functions to be registered.

Raises: [TypeError](#) – If the provided container is an instance of `<xref:agent_framework.azure.FunctionRegister>`.

Return type: [None](#)

`register_functions(function_container: DecoratorApi) -> None`

Register a list of functions in the function app.

Parameters: **function_container** ([DecoratorApi](#)) – Instance extending `<xref:agent_framework.azure.DecoratorApi>`, which contains a list of functions to be registered.

Raises: [TypeError](#) – If the provided container is an instance of `<xref:agent_framework.azure.FunctionRegister>`.

Return type: [None](#)

`retry(strategy: str, max_retry_count: str, delay_interval: str | None = None, minimum_interval: str | None = None, maximum_interval: str | None = None, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[..., Any]`

The *retry* decorator adds a `<xref:agent_framework.azure.RetryPolicy>` to the function settings object.

Used for building a `<xref:agent_framework.azure.Function>` object in the worker function indexing model. This is equivalent to defining a `RetryPolicy` in *function.json*, which enables the

function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries ↗

Parameters: * **strategy** ([str](#)) – The retry strategy to use.

```
* **max_retry_count** (<xref:str>) - The maximum number of retry attempts.

* **delay_interval** (<xref:str>* | *<xref:None>) - The delay interval between retry
attempts.

* **minimum_interval** (<xref:str>* | *<xref:None>) - The minimum delay interval
between retry attempts.

* **maximum_interval** (<xref:str>* | *<xref:None>) - The maximum delay interval
between retry attempts.

* **setting_extra_fields** (<xref:typing.Dict>[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>) - Keyword arguments for
specifying additional setting fields.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

```
route(route: str | None = None, trigger_arg_name: str = 'req', binding_arg_name: str = '$return',
methods: Iterable[str] | Iterable[HttpMethod] | None = None, auth_level: AuthLevel | str | None =
None, trigger_extra_fields: Dict[str, Any] | None = None, binding_extra_fields: Dict[str, Any] | None
= None) -> Callable[..., Any]
```

The *route* decorator adds `<xref:agent_framework.azure.HttpTrigger>` and `<xref:agent_framework.azure.HttpOutput>` bindings to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-http> ↗

Parameters: * **route** ([str](#)* | [*None](#)) – Route for the HTTP endpoint. If None, it defaults to the function name (if present) or the user-defined Python function name.

```
* **trigger_arg_name** (<xref:str>) – Argument name for
<xref:agent_framework.azure.HttpRequest>. Defaults to *'req'*.

* **binding_arg_name** (<xref:str>) – Argument name for
<xref:agent_framework.azure.HttpResponse>. Defaults to *'$return'*.

* **methods** (<xref:typing.Iterable>[*<xref:str>*] **| *<xref:typing.Iterable>[*
[HttpMethod](azure.functions.decorators.http.HttpMethod)*] **| *<xref:None>*) – A tuple
of the HTTP methods to which the function responds.

* **auth_level** (<xref:azure.functions.decorators.core.AuthLevel>* | *<xref:str>* |
*<xref:None>*) – Determines what keys, if any, need to be present on the request in
order to invoke the function.

* **trigger_extra_fields** (<xref:typing.Dict>[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>*) – Additional fields to
include in the trigger JSON. For example: >>> data_type='STRING' # results in
'dataType': 'STRING' in the trigger JSON

* **binding_extra_fields** (<xref:typing.Dict>[*<xref:str>*, *
<xref:agent_framework.exceptions.Any>*] **| *<xref:None>*) – Additional fields to
include in the binding JSON. For example: >>> data_type='STRING' # results in
'dataType': 'STRING' in the binding JSON
```

Returns: Decorator function.

Return type: [Callable](#)[[\[...\]](#), [Any](#)]

`schedule(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[\[...\], Any]`

The *schedule* or *timer* decorator adds `<xref:agent_framework.azure.TimerTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-timer> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.TimerRequest>` object in function code.

```
* **schedule** (<xref:str>) - A string representing a CRON expression used to
schedule the function execution.

* **run_on_startup** (<xref:bool>* | *<xref:None>) - If True, the function is
invoked when the runtime starts.

* **use_monitor** (<xref:bool>* | *<xref:None>) - Set to True or False to indicate
whether the schedule should be monitored.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>) -
```

Returns: Decorator function.

Return type: [Callable](#)[[...](#), [Any](#)]

`semantic_search_input(arg_name: str, search_connection_name: str, collection: str, query: str | None = None, ai_connection_name: str | None = "", embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None = semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]`

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: <https://platform.openai.com/docs/guides/embeddings> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **search_connection_name** (<xref:str>) - The name of the app setting or
environment variable that contains the vector search connection value.
```

* **collection** (<xref:str>) - The name of the collection or table to search or store.

* **query** (<xref:str>* | *<xref:None>) - The semantic query text used for searching the database.

* **ai_connection_name** (<xref:str>* | *<xref:None>) -

The name of the configuration section for AI service connectivity settings.

* For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section.

* If not specified or the section is missing, defaults to the environment variables `*AZURE_OPENAI_ENDPOINT*` and `*AZURE_OPENAI_KEY*`.

* For user-assigned managed identity authentication, this field is required.

* For OpenAI (non-Azure), set the `*OPENAI_API_KEY*` environment variable.

* **embeddings_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name for OpenAI Embeddings. Defaults to "text-embedding-ada-002".

* **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The deployment or model name for the OpenAI Chat Completion API. Defaults to "gpt-3.5-turbo".

* **system_prompt** (<xref:str>* | *<xref:None>) - Optional. The system prompt provided to the large language model.

* **max_knowledge_count** (<xref:int>* | *<xref:None>) - Optional. The number of knowledge items to inject into the system prompt. Default is 1.

* **temperature** (<xref:str>* | *<xref:None>) - The sampling temperature to use (range: 0 to 2). Higher values like 0.8 yield more random output; lower values like 0.2 make output more focused.

* **top_p** (<xref:str>* | *<xref:None>) - Alternative to temperature sampling (nucleus sampling). For example, 0.1 considers only the top 10% of tokens by probability mass. Use this or `*temperature*`.

* **max_tokens** (<xref:str>* | *<xref:None>) - The maximum number of tokens to generate in the completion. The sum of prompt tokens and `*max_tokens*` must not exceed the model's context length (usually 2048 or 4096 for newer models).

* **is_reasoning_model** (<xref:bool>* | *<xref:None>) - Indicates whether the chat model is a reasoning model. If True, `*max_tokens*` and `*temperature*` are not supported.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Optional. Defines how the Functions runtime should interpret the parameter value. Default is None.


```
* **kwargs** - Additional keyword arguments for specifying extra fields in the
*function.json* binding.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

```
service_bus_queue_output(arg_name: str, connection: str, queue_name: str, data_type: DataType |
str | None = None, access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...],
Any]
```

The *service_bus_queue_output* decorator adds
<xref:agent_framework.azure.ServiceBusQueueOutput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Service Bus queue output object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Service Bus.
```

```
* **queue_name** (<xref:str>) - The name of the queue to which messages will be
sent.
```

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

```
* **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) - The access rights required for the connection string.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...], Any]`

The `on_service_bus_queue_change` decorator adds

<xref:agent_framework.azure.ServiceBusQueueTrigger> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

Parameters: * **arg_name** (`str`) – The name of the variable that represents the
<xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
* **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

* **queue_name** (<xref:str>) – The name of the queue to monitor.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.

* **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – Access rights for the connection string.

* **is_sessions_enabled** (<xref:bool>* | *<xref:None>) – Set to True if connecting
to a session-aware queue or subscription.

* **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) – Set to "many" to enable batching.

* **kwargs** (<xref:agent_framework.exceptions.Any>) –
```

Returns: Decorator function.

Return type: `Callable[[...], Any]`

`service_bus_topic_output(arg_name: str, connection: str, topic_name: str, subscription_name: str | None = None, data_type: DataType | str | None = None, access_rights: AccessRights | str | None =`

None, **kwargs) -> [Callable](#)[..., [Any](#)]

The `service_bus_topic_output` decorator adds

`<xref:agent_framework.azure.ServiceBusTopicOutput>` to the
`<xref:agent_framework.azure.FunctionBuilder>` object for building a
`<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining `ServiceBusTopicOutput` in the `function.json`, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the Service Bus topic output object in the function code.

```
* **connection** (<xref:str>) – The name of an app setting or setting collection
that specifies how to connect to Service Bus.

* **topic_name** (<xref:str>) – The name of the topic to which messages will be
sent.

* **subscription_name** (<xref:str>* | *<xref:None>) – The name of the subscription
(optional for output).

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) – Defines how the Functions runtime should treat the parameter value.
Defaults to DataType.UNDEFINED.

* **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) – The access rights required for the connection string.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str, subscription_name: str, data_type: DataType | str | None = None, access_rights: AccessRights | str | None = None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[..., Any]`

The `on_service_bus_topic_change` decorator adds

`<xref:agent_framework.azure.ServiceBusTopicTrigger>` to the

<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the
<xref:agent_framework.azure.ServiceBusMessage> object in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to Service Bus.

* **topic_name** (<xref:str>) - The name of the topic to monitor.

* **subscription_name** (<xref:str>) - The name of the subscription to monitor.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.

* **access_rights** (<xref:azure.functions.decorators.core.AccessRights>* | *
<xref:str>* | *<xref:None>) - Access rights for the connection string.

* **is_sessions_enabled** (<xref:bool>* | *<xref:None>) - Set to True if connecting
to a session-aware queue or subscription.

* **cardinality** (<xref:azure.functions.decorators.core.Cardinality>* | *
<xref:str>* | *<xref:None>) - Set to "many" to enable batching.

* **kwargs** (<xref:agent_framework.exceptions.Any>) -
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`sql_input(arg_name: str, command_text: str, connection_string_setting: str, command_type: str |
None = 'Text', parameters: str | None = None, data_type: DataType | None = None, **kwargs) ->
Callable[..., Any]`

The *sql_input* decorator adds <xref:agent_framework.azure.SqlInput> to the
<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Parameters: * **arg_name** (*str*) – The name of the variable that represents a `<xref:agent_framework.azure.SqlRowList>` input object in the function code.

```
* command_text (<xref:str>) - The Transact-SQL query or the name of the stored
procedure executed by the binding.

* connection_string_setting (<xref:str>) - The name of the app setting that
contains the connection string to the target SQL database.

* command_type (<xref:str>* | *<xref:None>) - The command type. Use "Text" for
a raw SQL query and "StoredProcedure" for a stored procedure.

* parameters (<xref:str>* | *<xref:None>) - A comma-separated string of
parameter assignments to pass to the SQL command. Format:
*@param1=value1,@param2=value2*

* data_type (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

* kwargs - Additional keyword arguments for specifying binding fields to include
in the function.json.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`sql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *sql_output* decorator adds `<xref:agent_framework.azure.SqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the SQL output object in the function code.

```
* **command_text** (<xref:str>) - The Transact-SQL query or the name of the stored
procedure executed by the binding.

* **connection_string_setting** (<xref:str>) - The name of the app setting that
contains the connection string to the target SQL database.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>) -
Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include
in the *function.json*.
```

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`sql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str | None = None, data_type: DataType | None = None, **kwargs) -> Callable[..., Any]`

The *sql_trigger* decorator adds [<xref:agent_framework.azure.SqlTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents a [<xref:agent_framework.azure.SqlRowList>](#) object in the function code.

```
* **table_name** (<xref:str>) - The name of the SQL table monitored by the trigger.

* **connection_string_setting** (<xref:str>) - The name of an app setting that
contains the connection string for the database against which the query or stored
procedure is executed.

* **leases_table_name** (<xref:str>* | *<xref:None>) - The name of the table used to
store leases. If not specified, the default name is *Leases_{FunctionId}_{TableId}*.
```

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:None>*) -  
Defines how the Functions runtime should treat the parameter value.  
  
* **kwargs** - Additional keyword arguments for specifying binding fields to include  
in the function.json.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`table_input(arg_name: str, connection: str, table_name: str, row_key: str | None = None,
partition_key: str | None = None, take: int | None = None, filter: str | None = None, data_type:
DataType | str | None = None) -> Callable[[...], Any]`

The *table_input* decorator adds `<xref:agent_framework.azure.TableInput>` to the
`<xref:agent_framework.azure.FunctionBuilder>` object for building a
`<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read
from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the table or entity in the
function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection  
that specifies how to connect to the table service.  
  
* **table_name** (<xref:str>) - The name of the table.  
  
* **row_key** (<xref:str>* | *<xref:None>*) - The row key of the table entity to  
read.  
  
* **partition_key** (<xref:str>* | *<xref:None>*) - The partition key of the table  
entity to read.  
  
* **take** (<xref:int>* | *<xref:None>*) - The maximum number of entities to return.  
  
* **filter** (<xref:str>* | *<xref:None>*) - An OData filter expression to apply when  
retrieving entities.
```

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`table_output(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]`

The `table_output` decorator adds `<xref:agent_framework.azure.TableOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `TableOutput` in the `function.json`, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> ↗

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the table or entity in the function code.

```
* **connection** (<xref:str>) - The name of an app setting or setting collection
that specifies how to connect to the table service.

* **table_name** (<xref:str>) - The name of the table.

* **row_key** (<xref:str>* | *<xref:None>) - The row key of the table entity to
read.

* **partition_key** (<xref:str>* | *<xref:None>) - The partition key of the table
entity to read.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>) - Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)[[...], [Any](#)]

`text_completion_input(arg_name: str, prompt: str, ai_connection_name: str | None = "", chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel, temperature: str |`

None = '0.5', top_p: str | None = None, max_tokens: str | None = '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None = None, **kwargs) -> [Callable](#)[..., [Any](#)]

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: <https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions> ↗

The examples typically define an HTTP-triggered function with a hardcoded prompt such as "*who is {name}?*", where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Parameters: * **arg_name** ([str](#)) – The name of the binding parameter in the function code.

```
* **prompt** (<xref:str>) - The text prompt to generate completions for.

* **ai_connection_name** (<xref:str>* | *<xref:None>) -

    The name of the configuration section that defines
    AI service connectivity settings.

    * For **Azure OpenAI**: If specified, it looks for "Endpoint" and/or
    "Key" in this section. If not specified or missing, it falls back to
    the environment variables *AZURE_OPENAI_ENDPOINT* and *AZURE_OPENAI_KEY*.
    - For **user-assigned managed identity**, this setting is required.
    - For **non-Azure OpenAI**, ensure the *OPENAI_API_KEY* environment
    variable is set.

* **model** - *(Deprecated)* Use *chat_model* instead. This parameter is unused and
will be removed in future versions.

* **chat_model** (<xref:str>* | *
<xref:azure.functions.decorators.openai.OpenAIModels>* | *<xref:None>) - The
deployment or model name to use for the Chat Completions API. Default is "gpt-3.5-
turbo".

* **temperature** (<xref:str>* | *<xref:None>) - Sampling temperature (0-2). Higher
values (e.g., 0.8) produce more random output; lower values (e.g., 0.2) make output
more focused and deterministic.

* **top_p** (<xref:str>* | *<xref:None>) - Controls nucleus sampling. For example,
*top_p=0.1* considers only tokens in the top 10% cumulative probability. Recommended
to use either *temperature* or *top_p*, but not both.

* **max_tokens** (<xref:str>* | *<xref:None>) - The maximum number of tokens to
generate. The sum of prompt tokens and *max_tokens* must not exceed the model's
```

context length (usually 2048 or 4096 for newer models).

* **is_reasoning_model** (<xref:bool>* | *<xref:None>*) - If *True*, indicates that the configured chat model is a reasoning model. In this case, *max_tokens* and *temperature* are not supported.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>*) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** - Additional keyword arguments for specifying binding fields to include in the *function.json*.

Returns: Decorator function.

Return type: [Callable](#)[[...], Any]

`timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None, use_monitor: bool | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]`

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-timer> 

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the <xref:agent_framework.azure.TimerRequest> object in function code.

* **schedule** (<xref:str>*) - A string representing a CRON expression used to schedule the function execution.

* **run_on_startup** (<xref:bool>* | *<xref:None>*) - If *True*, the function is invoked when the runtime starts.

* **use_monitor** (<xref:bool>* | *<xref:None>*) - Set to *True* or *False* to indicate whether the schedule should be monitored.

* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *<xref:None>*) - Defines how the Functions runtime should treat the parameter value.

* **kwargs** (<xref:agent_framework.exceptions.Any>*) -

Returns: Decorator function.

Return type: [Callable](#)[..., [Any](#)]

`validate_function_names(functions: List[Function])`

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.

Parameters: **functions** ([List\[Function\]](#)) – A list of `<xref:agent_framework.azure.Function>` objects to validate.

Raises: [ValueError](#) – If any function name is duplicated.

`warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs) -> Callable`

The *warm_up* decorator adds `<xref:agent_framework.azure.WarmUpTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *WarmUpTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-warmup> [↗](#)

Parameters: * **arg_name** ([str](#)) – The name of the variable that represents the `<xref:agent_framework.azure.TimerRequest>` object in the function code.

```
* **data_type** (<xref:azure.functions.decorators.core.DataType>* | *<xref:str>* | *
<xref:None>*) – Defines how the Functions runtime should treat the parameter value.
```

Returns: Decorator function.

Return type: [Callable](#)

Constructor

Python

AgentFunctionApp()

Parameters

[Expand table](#)

Name	Description
agents	Default value: None
http_auth_level	Default value: FUNCTION
enable_health_check	Default value: True
enable_http_endpoints	Default value: True
max_poll_retries	Default value: 30
poll_interval_seconds	Default value: 1.0
enable_mcp_tool_trigger	Default value: False
default_callback	Default value: None

Methods

[Expand table](#)

__init__	Initialize the AgentFunctionApp.
__new__	
activity_trigger	Register an Activity Function.
add_agent	Add an agent to the function app after initialization.
assistant_create_output	The <i>assistantCreate</i> output binding creates a new assistant with a specified system prompt.
assistant_post_input	The <i>assistantPost</i> output binding sends a message to the assistant and saves the response in its internal state.
assistant_query_input	The <i>assistantQuery</i> input binding retrieves assistant chat history and passes it to the function. This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

assistant_skill_trigger

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when.

Ref: <https://platform.openai.com/docs/guides/function-calling> ↗

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

blob_input

The *blob_input* decorator adds `<xref:agent_framework.azure.BlobInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

blob_output

The *blob_output* decorator adds `<xref:agent_framework.azure.BlobOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

blob_trigger

The *blob_change_trigger* decorator adds `<xref:agent_framework.azure.BlobTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building `<xref:agent_framework.azure.Function>` object used in worker function indexing model. This is equivalent to defining *BlobTrigger* in the *function.json* which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

cosmos_db_input

The *cosmos_db_input* decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> ↗ Additional details: <https://aka.ms/cosmosdb-v4-update> ↗

cosmos_db_input_v3

The *cosmos_db_input_v3* decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> ↗ Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> ↗

cosmos_db_output

The *cosmos_db_output* decorator adds `<xref:agent_framework.azure.CosmosDBOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> ↗ Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> ↗

cosmos_db_output_v3

The *cosmos_db_output_v3* decorator adds `<xref:agent_framework.azure.CosmosDBOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗]

cosmos_db_trigger

The *cosmos_db_trigger* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> [↗]

cosmos_db_trigger_v3

The *cosmos_db_trigger_v3* decorator adds <xref:agent_framework.azure.CosmosDBTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: <https://aka.ms/cosmosdb-v4-update> [↗] Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> [↗]

dapr_binding_output

The *dapr_binding_output* decorator adds <xref:agent_framework.azure.DaprBindingOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-binding-output-binding> ↗

dapr_binding_trigger

The *dapr_binding_trigger* decorator adds `<xref:agent_framework.azure.DaprBindingTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-binding> ↗

dapr_invoke_output

The *dapr_invoke_output* decorator adds `<xref:agent_framework.azure.DaprInvokeOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprInvokeOutput* in the *function.json*, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-invoke-output-binding> ↗

dapr_publish_output

The *dapr_publish_output* decorator adds `<xref:agent_framework.azure.DaprPublishOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-publish-output-binding> ↗

dapr_secret_input

The *dapr_secret_input* decorator adds `<xref:agent_framework.azure.DaprSecretInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret

store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-secret-input-binding> ↗

dapr_service_invocation_trigger

The *dapr_service_invocation_trigger* decorator adds `<xref:agent_framework.azure.DaprServiceInvocationTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-service-invocation> ↗

dapr_state_input

The *dapr_state_input* decorator adds `<xref:agent_framework.azure.DaprStateInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprStateInput* in the *function.json*, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-input-binding> ↗

dapr_state_output

The *dapr_state_output* decorator adds `<xref:agent_framework.azure.DaprStateOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-output-binding> ↗

dapr_topic_trigger

The *dapr_topic_trigger* decorator adds `<xref:agent_framework.azure.DaprTopicTrigger>` to the

	<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model. This is equivalent to defining <i>DaprTopicTrigger</i> in the <i>function.json</i>, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system. All optional fields will be assigned default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-dapr-trigger-topic ↗
durable_client_input	Register a Durable-client Function.
durable_openai_agent_orchestrator	Decorate Azure Durable Functions orchestrators that use OpenAI Agents.
embeddings_input	The embeddings input decorator generates embeddings used to measure the relatedness of text strings. Ref: https://platform.openai.com/docs/guides/embeddings ↗
embeddings_store_output	Add an embeddings input binding to the function. The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions: • Azure AI Search • Azure Data Explorer • Azure Cosmos DB using MongoDB
entity_trigger	Register an Entity Function.
event_grid_output	The <i>event_grid_output</i> decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model. This is equivalent to defining the output binding in the <i>function.json</i>, which enables the function to write events to a custom topic. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/eventgridtrigger ↗
event_grid_trigger	The <i>event_grid_trigger</i> decorator adds <xref:agent_framework.azure.EventGridTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/eventgridtrigger> ↗

event_hub_message_trigger

The *event_hub_message_trigger* decorator adds <xref:agent_framework.azure.EventHubTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> ↗

event_hub_output

The *event_hub_output* decorator adds <xref:agent_framework.azure.EventHubOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> ↗

function_name

Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name.

additional setting fields :return: Decorator function.

generic_input_binding

The *generic_input_binding* decorator adds <xref:agent_framework.azure.GenericInputBinding> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

	This is equivalent to defining a generic input binding in the <i>function.json</i>, which enables the function to read data from a custom-defined input source. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-custom ↗
generic_output_binding	The <i>generic_output_binding</i> decorator adds <code><xref:agent_framework.azure.GenericOutputBinding></code> to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> object used in the worker function indexing model. This is equivalent to defining a generic output binding in the <i>function.json</i>, which enables the function to write data to a custom-defined output source. All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-custom ↗
generic_trigger	The <i>generic_trigger</i> decorator adds <code><xref:agent_framework.azure.GenericTrigger></code> to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> used in the worker function indexing model. This is equivalent to defining a generic trigger in the <i>function.json</i>, which triggers the function to execute when generic trigger events are received by the host. All optional fields will be given default values by the function host when they are parsed. Ref: https://aka.ms/azure-function-binding-custom ↗
get_agent	Return a DurableAI Agent proxy for a registered agent.
get_functions	Get the function objects in the function app.
http_type	Set http type of the <code><xref:agent_framework.azure.Function></code> object.
kafka_output	The <i>kafka_output</i> decorator adds <code><xref:agent_framework.azure.KafkaOutput></code> to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> object used in the worker function indexing model. This is equivalent to defining a Kafka output binding in <i>function.json</i>, which enables the function to write events to a Kafka topic. All

optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/kafkaoutput> ↗

kafka_trigger

The *kafka_trigger* decorator adds `<xref:agent_framework.azure.KafkaTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/kafkatrigger> ↗

mcp_tool_trigger

The *mcp_tool_trigger* decorator adds `<xref:agent_framework.azure.MCPToolTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/remote-mcp-functions-python> ↗

mysql_input

The *mysql_input* decorator adds `<xref:agent_framework.azure.MySqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> ↗

mysql_output

The *mysql_output* decorator adds `<xref:agent_framework.azure.MySqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> ↗

mysql_trigger

The *mysql_trigger* decorator adds <xref:agent_framework.azure.MySqlTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> ↗

orchestration_trigger

Register an Orchestrator Function.

queue_output

The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> ↗

queue_trigger

The *queue_trigger* decorator adds <xref:agent_framework.azure.QueueTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

	All optional fields will be assigned default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-queue ↗
register_blueprint	Register a list of functions in the function app.
register_functions	Register a list of functions in the function app.
retry	The <i>retry</i> decorator adds a <code><xref:agent_framework.azure.RetryPolicy></code> to the function settings object. Used for building a <code><xref:agent_framework.azure.Function></code> object in the worker function indexing model. This is equivalent to defining a <code>RetryPolicy</code> in <i>function.json</i>, which enables the function to automatically retry on failure. All optional fields are assigned default values by the function host during parsing. Ref: https://aka.ms/azure_functions_retries ↗
route	The <i>route</i> decorator adds <code><xref:agent_framework.azure.HttpTrigger></code> and <code><xref:agent_framework.azure.HttpOutput></code> bindings to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> used in the worker function indexing model. This is equivalent to defining both <i>HttpTrigger</i> and <i>HttpOutput</i> bindings in the <i>function.json</i>, which enables your function to be triggered when HTTP requests hit the specified route. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-http ↗
schedule	The <i>schedule</i> or <i>timer</i> decorator adds <code><xref:agent_framework.azure.TimerTrigger></code> to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> used in the worker function indexing model. This is equivalent to defining a <i>TimerTrigger</i> in the <i>function.json</i>, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-timer ↗
semantic_search_input	Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: <https://platform.openai.com/docs/guides/embeddings> ↗

`service_bus_queue_output`

The `service_bus_queue_output` decorator adds `<xref:agent_framework.azure.ServiceBusQueueOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining `ServiceBusQueueOutput` in the `function.json`, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

`service_bus_queue_trigger`

The `on_service_bus_queue_change` decorator adds `<xref:agent_framework.azure.ServiceBusQueueTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a `ServiceBusQueueTrigger` in the `function.json`, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

`service_bus_topic_output`

The `service_bus_topic_output` decorator adds `<xref:agent_framework.azure.ServiceBusTopicOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining `ServiceBusTopicOutput` in the `function.json`, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

service_bus_topic_trigger

The *on_service_bus_topic_change* decorator adds `<xref:agent_framework.azure.ServiceBusTopicTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *ServiceBusTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

sql_input

The *sql_input* decorator adds `<xref:agent_framework.azure.SqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

sql_output

The *sql_output* decorator adds `<xref:agent_framework.azure.SqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

sql_trigger

The *sql_trigger* decorator adds `<xref:agent_framework.azure.SqlTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All

optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

table_input

The *table_input* decorator adds `<xref:agent_framework.azure.TableInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *TableInput* in the *function.json*, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> ↗

table_output

The *table_output* decorator adds `<xref:agent_framework.azure.TableOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *TableOutput* in the *function.json*, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> ↗

text_completion_input

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: <https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions> ↗

The examples typically define an HTTP-triggered function with a hardcoded prompt such as "*who is {name}?*", where *{name}* is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

timer_trigger

The *schedule* or *timer* decorator adds `<xref:agent_framework.azure.TimerTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

	This is equivalent to defining a <i>TimerTrigger</i> in the <i>function.json</i>, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-timer ↗
validate_function_names	Validate that all functions have unique names within the app. The <i>functions_bindings</i> dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.
warm_up_trigger	The <i>warm_up</i> decorator adds <code><xref:agent_framework.azure.WarmUpTrigger></code> to the <code><xref:agent_framework.azure.FunctionBuilder></code> object for building a <code><xref:agent_framework.azure.Function></code> used in the worker function indexing model. This is equivalent to defining a <i>WarmUpTrigger</i> in the <i>function.json</i>, which enables your function to be triggered on the specified schedule. All optional fields will be given default values by the function host when parsed. Ref: https://aka.ms/azure-function-binding-warmup ↗

`__init__`

Initialize the AgentFunctionApp.

Python

```
__init__(agents: list[AgentProtocol] | None = None, http_auth_level: AuthLevel = func.AuthLevel.FUNCTION, enable_health_check: bool = True, enable_http_endpoints: bool = True, max_poll_retries: int = DEFAULT_MAX_POLL_RETRIES, poll_interval_seconds: float = DEFAULT_POLL_INTERVAL_SECONDS, enable_mcp_tool_trigger: bool = False, default_callback: AgentResponseCallbackProtocol | None = None)
```

Parameters

 Expand table

Name	Description
agents	list[AgentProtocol] None List of agent instances to register. Default value: None

Name	Description
<code>http_auth_level</code>	AuthLevel HTTP authentication level (default: <code>func.AuthLevel.FUNCTION</code>). Default value: <code>func.AuthLevel.FUNCTION</code>
<code>enable_health_check</code>	bool Enable the built-in health check endpoint (default: <code>True</code>). Default value: <code>True</code>
<code>enable_http_endpoints</code>	bool Enable HTTP endpoints for agents (default: <code>True</code>). Default value: <code>True</code>
<code>enable_mcp_tool_trigger</code>	bool Enable MCP tool triggers for agents (default: <code>False</code>). When enabled, agents will be exposed as MCP tools that can be invoked by MCP-compatible clients. Default value: <code>False</code>
<code>max_poll_retries</code>	int Maximum polling attempts when waiting for a response. Defaults to <code>DEFAULT_MAX_POLL_RETRIES</code> . Default value: <code>DEFAULT_MAX_POLL_RETRIES</code>
<code>poll_interval_seconds</code>	float Delay in seconds between polling attempts. Defaults to <code>DEFAULT_POLL_INTERVAL_SECONDS</code> . Default value: <code>DEFAULT_POLL_INTERVAL_SECONDS</code>
<code>default_callback</code>	AgentResponseCallbackProtocol None Optional callback invoked for agents without specific callbacks. Default value: <code>None</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

activity_trigger

Register an Activity Function.

Python
<code>activity_trigger(input_name: str, activity: str None = None)</code>

Parameters

 Expand table

Name	Description
input_name Required*	str Parameter name of the Activity input.
activity	<xref:Optional> [str] Name of Activity Function. The value is None by default, in which case the name of the method is used. Default value: None

add_agent

Add an agent to the function app after initialization.

Python

```
add_agent(agent: AgentProtocol, callback: AgentResponseCallbackProtocol | None = None, enable_http_endpoint: bool | None = None, enable_mcp_tool_trigger: bool | None = None) -> None
```

Parameters

 Expand table

Name	Description
agent Required*	AgentProtocol The Microsoft Agent Framework agent instance (must implement AgentProtocol) The agent must have a 'name' attribute.
callback	AgentResponseCallbackProtocol None Optional callback invoked during agent execution Default value: None
enable_http_endpoint	bool None Optional flag to enable/disable HTTP endpoint for this agent. The app level enable_http_endpoints setting will override this setting. Default value: None
enable_mcp_tool_trigger	bool None Optional flag to enable/disable MCP tool trigger for this agent. The app level enable_mcp_tool_trigger setting will override this setting. Default value: None

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
ValueError	If the agent doesn't have a 'name' attribute or if an agent with the same name is already registered

assistant_create_output

The *assistantCreate* output binding creates a new assistant with a specified system prompt.

Python

```
assistant_create_output(arg_name: str, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying extra binding fields to include in the <i>function.json</i> .

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

assistant_post_input

The *assistantPost* output binding sends a message to the assistant and saves the response in its internal state.

Python

```
assistant_post_input(arg_name: str, id: str, user_message: str,
ai_connection_name: str | None = '', chat_model: str | OpenAIModels | None =
OpenAIModels.DefaultChatModel, chat_storage_connection_setting: str | None =
'AzureWebJobsStorage', collection_name: str | None = 'ChatState', temperature: str
| None = '0.5', top_p: str | None = None, max_tokens: str | None = '100',
is_reasoning_model: bool | None = False, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the binding parameter in the function code.
id Required*	<code>str</code> The ID of the assistant to update.
user_message Required*	<code>str</code> The message entered by the user for the assistant to respond to.
ai_connection_name	<code>str</code> <code>None</code> The name of the configuration section for AI service connectivity. Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in the section. - If not specified or missing, falls back to the <code>AZURE_OPENAI_ENDPOINT</code> and <code>AZURE_OPENAI_KEY</code> environment variables. Managed Identity: Required for user-assigned identity auth. OpenAI (non-Azure): Set the <code>OPENAI_API_KEY</code> environment variable.

Name	Description
	Default value: ""
model Required*	<i>Deprecated.</i> Use <i>chat_model</i> instead. This parameter is unused and will be removed in future versions.
chat_model	str OpenAIModels None The deployment or model name of the OpenAI Chat Completion API. Default is " <i>gpt-3.5-turbo</i> ". Default value: <code>OpenAIModels.DefaultChatModel</code>
chat_storage_connection_setting	str None The config section name for assistant chat table storage. Default is " <i>AzureWebJobsStorage</i> ". Default value: " <i>AzureWebJobsStorage</i> "
collection_name	str None The collection or table name used for assistant chat storage. Default is " <i>ChatState</i> ". Default value: " <i>ChatState</i> "
temperature	str None Sampling temperature, between 0 and 2. Higher values (e.g., 0.8) increase randomness; lower values (e.g., 0.2) make output more deterministic. Default value: "0.5"
top_p	str None Alternative to temperature sampling. Uses nucleus sampling to consider only the tokens comprising the top <i>top_p</i> probability mass. Recommended to use either this or <i>temperature</i> . Default value: None
max_tokens	str None Maximum tokens to generate. Total tokens (prompt + output) must not exceed the model's context length. Default value: "100"
is_reasoning_model	bool None Whether the chat model is a reasoning model. If true, <i>max_tokens</i> and <i>temperature</i> are not supported. Default value: False
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying extra binding fields in <i>function.json</i> .

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

assistant_query_input

The *assistantQuery* input binding retrieves assistant chat history and passes it to the function.

This is typically used to provide the function access to previous messages in a conversation, enabling more context-aware responses.

Python

```
assistant_query_input(arg_name: str, id: str, timestamp_utc: str,
chat_storage_connection_setting: str | None = 'AzureWebJobsStorage',
collection_name: str | None = 'ChatState', data_type: DataType | str | None =
None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the binding parameter in the function code.
timestamp_utc Required*	<code>str</code> The earliest timestamp (in UTC) for the messages to retrieve from the chat history. Must be in ISO 8601 format, e.g., "2023-08-01T00:00:00Z".
chat_storage_connection_setting	<code>str</code> <code>None</code> The name of the configuration section containing the connection settings for assistant chat storage. Defaults to "AzureWebJobsStorage". Default value: "AzureWebJobsStorage"
collection_name	<code>str</code> <code>None</code> The name of the table or collection used for assistant chat storage. Defaults to "ChatState". Default value: "ChatState"

Name	Description
id Required*	<code>str</code> The unique identifier of the assistant whose history is being queried.
data_type	<code>DataType str None</code> Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

assistant_skill_trigger

Assistants build on top of chat functionality by supporting custom skills defined as functions. This internally uses OpenAI's function calling capabilities in GPT models to determine which functions to invoke and when.

Ref: <https://platform.openai.com/docs/guides/function-calling>

You can define functions to be triggered by assistants using the *assistantSkillTrigger* trigger binding. These functions are invoked by the extension when an assistant signals it would like to invoke a function in response to a user prompt.

The function name, its description (provided via the trigger), and the parameter descriptions are all used as hints by the language model to determine when and how to invoke an assistant function.

Python

```
assistant_skill_trigger(arg_name: str, function_description: str, function_name: str | None = None, parameter_description_json: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
function_description Required*	str A description of the assistant function, which is provided to the model.
function_name	str None The name of the assistant function, which is passed to the language model. Default value: None
parameter_description_json	str None A JSON-formatted description of the function parameters, provided to the model. If omitted, the description is autogenerated. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

blob_input

The *blob_input* decorator adds `<xref:agent_framework.azure.BlobInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobInput* in the *function.json*, which enables the function to read from Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob>

Python

```
blob_input(arg_name: str, path: str, connection: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the blob in the function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

blob_output

The *blob_output* decorator adds `<xref:agent_framework.azure.BlobOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *BlobOutput* in the *function.json*, which enables the function to write message(s) to Azure Storage blobs.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> 

Python

```
blob_output(arg_name: str, path: str, connection: str, data_type: DataType | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the blob in the function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

blob_trigger

The blob_change_trigger decorator adds `<xref:agent_framework.azure.BlobTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building `<xref:agent_framework.azure.Function>` object used in worker function indexing model. This

is equivalent to defining BlobTrigger in the function.json which enables the function to be triggered when new message(s) are sent to storage blobs.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-storage-blob> ↗

Python

```
blob_trigger(arg_name: str, path: str, connection: str, source: BlobSource | None = None, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.InputStream> object in function code.
path Required*	str The path to the blob.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Blobs.
source	BlobSource None Sets the source of the triggering event. Use "EventGrid" for an Event Grid-based blob trigger, which provides much lower latency. The default is "LogsAndContainerScan", which uses the standard polling mechanism to detect changes in the container. Default value: None
data_type	DataType None Defines how Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

cosmos_db_input

The `cosmos_db_input` decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle version 4.x and above. It is equivalent to defining `CosmosDBInput` in the `function.json`, which enables the function to read from Azure Cosmos DB.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> Additional details:
<https://aka.ms/cosmosdb-v4-update>

Python

```
cosmos_db_input(arg_name: str, connection: str, database_name: str,
container_name: str, partition_key: str | None = None, id: str | None = None,
sql_query: str | None = None, preferred_locations: str | None = None, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the <code><xref:agent_framework.azure.DocumentList></code> input object in the function code.
connection Required*	<code>str</code> The name of an app setting or setting container that specifies how to connect to the Azure Cosmos DB account being monitored.
database_name Required*	<code>str</code> The name of the Cosmos DB database that contains the document.
container_name	<code>str</code>

Name	Description
Required*	The name of the container that holds the document.
partition_key	str None The partition key value for the document lookup. Default value: None
id	str None The ID of the document to retrieve. Default value: None
sql_query	str None An Azure Cosmos DB SQL query used to retrieve multiple documents. Default value: None
preferred_locations	str None (Optional) Preferred geo-replicated regions. Values should be comma-separated, e.g., "East US,South Central US,North Europe". Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional binding fields to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [... , Any]	Decorator function.

cosmos_db_input_v3

The *cosmos_db_input_v3* decorator adds `<xref:agent_framework.azure.CosmosDBInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining *CosmosDBInput* in the *function.json*, which enables the function to read from Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update> ↗ Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> ↗

Python

```
cosmos_db_input_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, id: str | None = None, sql_query: str | None =
None, partition_key: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> input object in the function code.
database_name Required*	str The name of the Azure Cosmos DB database containing the target collection.
collection_name Required*	str The name of the collection that contains the document(s).
connection_string_setting Required*	str The name of the app setting that contains the Azure Cosmos DB connection string.
id	str None The ID of a single document to retrieve. Default value: None
sql_query	str None An Azure Cosmos DB SQL query used to retrieve multiple documents. Default value: None
partition_key	str None Specifies the partition key value for the lookup. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs	

Name	Description
Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

cosmos_db_output

The *cosmos_db_output* decorator adds `<xref:agent_framework.azure.CosmosDBOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBOutput* in the *function.json*, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update>  Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> 

Python

```
cosmos_db_output(arg_name: str, connection: str, database_name: str,
container_name: str, create_if_not_exists: bool | None = None, partition_key: str
| None = None, container_throughput: int | None = None, preferred_locations: str |
None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...],
Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the Cosmos DB output object in the function code.
connection	str

Name	Description
Required*	The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.
database_name Required*	str The name of the Azure Cosmos DB database containing the target container.
container_name Required*	str The name of the container to which documents are written.
create_if_not_exists	bool None A boolean indicating whether the container should be created automatically if it does not exist. Default value: None
partition_key	str None When <i>create_if_not_exists</i> is <i>True</i> , this defines the partition key path for the created container. Default value: None
container_throughput	int None When <i>create_if_not_exists</i> is <i>True</i> , this defines the throughput (Request Units) for the created container. Default value: None
preferred_locations	str None Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB. When set along with <i>use_multiple_write_locations</i> , it enables multi-region writes. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[🔗](#) Expand table

Type	Description
Callable [[...], Any]	Decorator function.

cosmos_db_output_v3

The `cosmos_db_output_v3` decorator adds `<xref:agent_framework.azure.CosmosDBOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 2.x or 3.x. It is equivalent to defining `CosmosDBOutput` in the `function.json`, which enables the function to write to Azure Cosmos DB. All optional fields are assigned default values by the function host when parsed.

For additional details, see: <https://aka.ms/cosmosdb-v4-update>  Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> 

Python

```
cosmos_db_output_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, create_if_not_exists: bool | None = None,
partition_key: str | None = None, collection_throughput: int | None = None,
use_multiple_write_locations: bool | None = None, preferred_locations: str | None
= None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the Cosmos DB output object in the function code.
database_name Required*	<code>str</code> The name of the Azure Cosmos DB database containing the monitored collection.
collection_name Required*	<code>str</code> The name of the collection to which documents are written.
connection_string_setting Required*	<code>str</code> The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account.
create_if_not_exists	<code>bool None</code> A boolean indicating whether the collection should be created automatically if it does not exist. Default value: None
partition_key	<code>str None</code>

Name	Description
	When <i>create_if_not_exists</i> is <i>True</i> , this defines the partition key path for the created collection. Default value: None
collection_throughput	int None When <i>create_if_not_exists</i> is <i>True</i> , this defines the throughput (Request Units) for the created collection. Default value: None
use_multiple_write_locations	bool None When set to <i>True</i> and used in conjunction with <i>preferred_locations</i> , this enables multi-region writes in the Azure Cosmos DB service. Default value: None
preferred_locations	str None Preferred geographic regions for geo-replicated database accounts in Azure Cosmos DB. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

cosmos_db_trigger

The *cosmos_db_trigger* decorator adds `<xref:agent_framework.azure.CosmosDBTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields are assigned default values by the function host when parsed.

See: <https://aka.ms/cosmosdb-v4-update>  Ref: <https://aka.ms/azure-function-binding-cosmosdb-v4> 

Python

```
cosmos_db_trigger(arg_name: str, connection: str, database_name: str,
container_name: str, lease_connection: str | None = None, lease_database_name: str
| None = None, lease_container_name: str | None = None,
create_lease_container_if_not_exists: bool | None = None,
leases_container_throughput: int | None = None, lease_container_prefix: str | None
= None, feed_poll_delay: int | None = None, lease_acquire_interval: int | None =
None, lease_expiration_interval: int | None = None, lease_renew_interval: int |
None = None, max_items_per_invocation: int | None = None, start_from_beginning:
time | None = None, start_from_time: time | None = None, preferred_locations: str
| None = None, data_type: DataType | str | None = None, **kwargs: Any) ->
Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to the monitored Azure Cosmos DB account.
database_name Required*	str The name of the Azure Cosmos DB database containing the monitored collection.
container_name Required*	str The name of the container being monitored.
lease_connection	str None (Optional) The name of an app setting or setting collection that specifies how to connect to the Cosmos DB account that holds the lease container. Default value: None
lease_database_name	str None The name of the database that holds the collection used to store leases.

Name	Description
	Default value: None
<code>lease_container_name</code>	str None (Optional) The name of the container used to store leases. If not set, the default value "leases" is used. Default value: None
<code>create_lease_container_if_not_exists</code>	bool None (Optional) If <i>True</i>, the leases container is created automatically if it does not exist. Defaults to <i>False</i>. Note: When using Azure AD identities, container creation is not allowed, and the function will not start if this is <i>True</i>. Default value: None
<code>leases_container_throughput</code>	int None (Optional) The number of Request Units (RUs) to assign when the leases container is created. This is used only when <i>create_lease_container_if_not_exists</i> is <i>True</i>. It is automatically set when configured through the Azure Portal. Default value: None
<code>lease_container_prefix</code>	str None (Optional) A prefix added to leases created in the lease container for this function. Use this to allow multiple functions to share the same lease container with different prefixes. Default value: None
<code>feed_poll_delay</code>	int None The delay (in milliseconds) between polling a partition for new changes after draining current changes. Default value: None
<code>lease_acquire_interval</code>	int None The interval (in milliseconds) to trigger a task to check if partitions are evenly distributed across host instances. Default value: None
<code>lease_expiration_interval</code>	int None The interval (in milliseconds) for which a lease is held for a partition. Default value: None
<code>lease_renew_interval</code>	int None The interval (in milliseconds) to renew all leases for partitions currently held by an instance. Default value: None

Name	Description
<code>max_items_per_invocation</code>	int None Maximum number of items received per function call. Default value: None
<code>start_from_beginning</code>	time None If <i>True</i> , the trigger starts reading changes from the beginning of the collection's change history instead of the current time. Default value: None
<code>start_from_time</code>	time None (Optional) The date and time from which to begin reading the change feed. Use ISO 8601 format with a UTC designator, e.g., <i>2021-02-16T14:19:29Z</i> . Only used to set the initial trigger state; has no effect once the trigger has a lease state. Default value: None
<code>preferred_locations</code>	str None Preferred locations (regions) for geo-replicated Cosmos DB accounts. Default value: None
<code>data_type</code>	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
<code>kwargs</code> Required*	Any Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...] , Any]	Decorator function.

cosmos_db_trigger_v3

The *cosmos_db_trigger_v3* decorator adds [<xref:agent_framework.azure.CosmosDBTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model. This decorator only works with extension bundle 2.x or 3.x.

This is equivalent to defining *CosmosDBTrigger* in the *function.json*, which enables the function to be triggered when Cosmos DB data changes. All optional fields will be assigned default values by the function host when they are parsed.

See: <https://aka.ms/cosmosdb-v4-update>  Ref: <https://aka.ms/azure-function-binding-cosmosdb-v2> 

Python

```
cosmos_db_trigger_v3(arg_name: str, database_name: str, collection_name: str,
connection_string_setting: str, lease_collection_name: str | None = None,
lease_connection_string_setting: str | None = None, lease_database_name: str |
None = None, create_lease_collection_if_not_exists: bool | None = None,
leases_collection_throughput: int | None = None, lease_collection_prefix: str |
None = None, checkpoint_interval: int | None = None, checkpoint_document_count:
int | None = None, feed_poll_delay: int | None = None, lease_renew_interval: int |
None = None, lease_acquire_interval: int | None = None, lease_expiration_interval:
int | None = None, max_items_per_invocation: int | None = None,
start_from_beginning: bool | None = None, preferred_locations: str | None = None,
data_type: DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.DocumentList> object in the function code.
database_name Required*	str The name of the Azure Cosmos DB database containing the monitored collection.
collection_name Required*	str The name of the collection being monitored.
connection_string_setting Required*	str The name of an app setting or setting collection specifying how to connect to the monitored Azure Cosmos DB account.
lease_collection_name	str None

Name	Description
	The name of the collection used to store leases. Default value: None
<code>lease_connection_string_setting</code>	<code>str</code> <code>None</code> The name of an app setting or setting collection that specifies how to connect to the Azure Cosmos DB account that holds the lease collection. Default value: None
<code>lease_database_name</code>	<code>str</code> <code>None</code> The name of the database that holds the lease collection. Default value: None
<code>create_lease_collection_if_not_exists</code>	<code>bool</code> <code>None</code> If <i>True</i> , the lease collection is automatically created when it does not already exist. Default value: None
<code>leases_collection_throughput</code>	<code>int</code> <code>None</code> The number of Request Units (RUs) assigned when the lease collection is created. Default value: None
<code>lease_collection_prefix</code>	<code>str</code> <code>None</code> A prefix added to leases created in the lease collection for this Function. Default value: None
<code>checkpoint_interval</code>	<code>int</code> <code>None</code> The interval (in milliseconds) between lease checkpoints. The default behavior is to checkpoint after every function call. Default value: None
<code>checkpoint_document_count</code>	<code>int</code> <code>None</code> Number of documents processed between lease checkpoints. Default is after every function call. Default value: None
<code>feed_poll_delay</code>	<code>int</code> <code>None</code> The delay (in milliseconds) between polling a partition for new changes after all current changes are drained. Default value: None
<code>lease_renew_interval</code>	<code>int</code> <code>None</code> The interval (in milliseconds) to renew leases for partitions currently held by an instance. Default value: None

Name	Description
<code>lease_acquire_interval</code>	int None The interval (in milliseconds) to trigger a task that checks whether partitions are evenly distributed among host instances. Default value: None
<code>lease_expiration_interval</code>	int None The interval (in milliseconds) for which a lease is held for a partition. Default value: None
<code>max_items_per_invocation</code>	int None Maximum number of items received per function call. Default value: None
<code>start_from_beginning</code>	bool None If <i>True</i> , the trigger starts reading changes from the beginning of the collection's change history rather than from the current time. Default value: None
<code>preferred_locations</code>	str None Preferred locations (regions) for geo-replicated database accounts in Azure Cosmos DB. Default value: None
<code>data_type</code>	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
<code>kwargs</code> Required*	Any Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

dapr_binding_output

The *dapr_binding_output* decorator adds `<xref:agent_framework.azure.DaprBindingOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprBindingOutput* in the *function.json*, which enables the function to send a value to a Dapr output binding.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-binding-output-binding> 

Python

```
dapr_binding_output(arg_name: str, binding_name: str, operation: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the Dapr state output object in the function code.
binding_name Required*	<code>str</code> The configured name of the binding.
operation Required*	<code>str</code> The configured operation.
dapr_address	<code>str</code> <code>None</code> The Dapr address. Optional; defaults to http://localhost  <code>{daprHttpPort}</code> . Default value: <code>None</code>
data_type	<code>DataType</code> <code>str</code> <code>None</code> Defines how the Functions runtime should treat the parameter value. Default value: <code>None</code>
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

dapr_binding_trigger

The *dapr_binding_trigger* decorator adds `<xref:agent_framework.azure.DaprBindingTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprBindingTrigger* in the *function.json*, which enables the function to be triggered by a Dapr input binding.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-binding> [↗](#)

Python

```
dapr_binding_trigger(arg_name: str, binding_name: str, data_type: DataType | str |  
None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the trigger input in the function code.
binding_name Required*	str The name of the Dapr trigger. If not specified, the function name is used as the trigger name.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

dapr_invoke_output

The `dapr_invoke_output` decorator adds `<xref:agent_framework.azure.DaprInvokeOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `DaprInvokeOutput` in the `function.json`, which enables the function to invoke another Dapr app.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-invoke-output-binding>

Python

```
dapr_invoke_output(arg_name: str, app_id: str, method_name: str, http_verb: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
app_id Required*	str The Dapr app ID to invoke.
method_name Required*	str The method name of the app to invoke.
http_verb Required*	str The HTTP verb to use for the invocation (e.g., GET, POST).
dapr_address	str None The Dapr address. Optional; defaults to http://localhost : <code>{daprHttpPort}</code> .

Name	Description
	Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [..., Any]	Decorator function.

dapr_publish_output

The *dapr_publish_output* decorator adds `<xref:agent_framework.azure.DaprPublishOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprPublishOutput* in the *function.json*, which enables the function to publish to a topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-publish-output-binding> [↗](#)

Python

```
dapr_publish_output(arg_name: str, pub_sub_name: str, topic: str, dapr_address: str | None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[..., Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
pub_sub_name Required*	str The pub/sub component name to publish to.
topic Required*	str The name of the topic to publish to.
dapr_address	str None The Dapr address. Optional; defaults to http://localhost ↗ :{daprHttpPort}. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

[↗](#) Expand table

Type	Description
Callable [..., Any]	Decorator function.

dapr_secret_input

The *dapr_secret_input* decorator adds [<xref:agent_framework.azure.DaprSecretInput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining *DaprSecretInput* in the *function.json*, which enables the function to read secrets from the underlying secret store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-secret-input-binding> [↗](#)

Python


```
dapr_secret_input(arg_name: str, secret_store_name: str, key: str, metadata: str,
dapr_address: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state input object in the function code.
secret_store_name Required*	str The name of the secret store from which to retrieve the secret.
key Required*	str The key identifying the name of the secret to retrieve.
metadata Required*	str Metadata properties in the form "key1=value1&key2=value2".
dapr_address	str None The Dapr address. Optional; Defaults to http://localhost ↗ :{daprHttpPort}. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

dapr_service_invocation_trigger

The *dapr_service_invocation_trigger* decorator adds
<xref:agent_framework.azure.DaprServiceInvocationTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a
<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *DaprServiceInvocationTrigger* in the *function.json*, which enables the function to be triggered when a service invocation occurs through Dapr.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-service-invocation> 

Python

```
dapr_service_invocation_trigger(arg_name: str, method_name: str, data_type:
    DataType | str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the service invocation input in the function code.
method_name Required*	str The name of the method on a remote Dapr App. If not specified, the name of the function is used as the method name.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

dapr_state_input

The `dapr_state_input` decorator adds `<xref:agent_framework.azure.DaprStateInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `DaprStateInput` in the `function.json`, which enables the function to read state from the underlying state store component.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-input-binding> 

Python

```
dapr_state_input(arg_name: str, state_store: str, key: str, dapr_address: str |  
None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...],  
Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the Dapr state input object in the function code.
state_store Required*	<code>str</code> The state store containing the state.
key Required*	<code>str</code> The name of the key to retrieve.
dapr_address	<code>str</code> <code>None</code> The Dapr address. Optional; Defaults to http://localhost  :{daprHttpPort}. Default value: <code>None</code>
data_type	<code>DataType</code> <code>str</code> <code>None</code> Defines how the Functions runtime should treat the parameter value. Default value: <code>None</code>
kwargs Required*	Additional keyword arguments for specifying binding fields in the <code>function.json</code> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

dapr_state_output

The *dapr_state_output* decorator adds `<xref:agent_framework.azure.DaprStateOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *DaprStateOutput* in the *function.json*, which enables the function to write to the Dapr state store.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-dapr-state-output-binding> [↗](#)

Python

```
dapr_state_output(arg_name: str, state_store: str, key: str, dapr_address: str |  
None = None, data_type: DataType | str | None = None, **kwargs) -> Callable[[...],  
Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the Dapr state output object in the function code.
state_store Required*	str The state store containing the state for the specified keys.
key Required*	str The name of the key used to store the state.
dapr_address	str None The Dapr address. Optional; defaults to http://localhost ↗ :{daprHttpPort}. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value.

Name	Description
	Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

 Expand table

Type	Description
Callable [..., Any]	Decorator function.

dapr_topic_trigger

The *dapr_topic_trigger* decorator adds `<xref:agent_framework.azure.DaprTopicTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *DaprTopicTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Dapr pub/sub system.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-dapr-trigger-topic> 

Python

```
dapr_topic_trigger(arg_name: str, pub_sub_name: str, topic: str, route: str | None = None, data_type: DataType | str | None = None, **kwargs: Any) -> Callable[..., Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the trigger input in the function code.
pub_sub_name Required*	str The name of the Dapr pub/sub component.

Name	Description
topic Required*	str The topic name. If unspecified, the function name is used.
route	str None The route for the trigger. If unspecified, the topic name is used. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any Additional keyword arguments for specifying extra binding fields to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

durable_client_input

Register a Durable-client Function.

Python

```
durable_client_input(client_name: str, task_hub: str | None = None,
connection_name: str | None = None)
```

Parameters

[Expand table](#)

Name	Description
client_name Required*	str Parameter name of durable client.
task_hub	<xref:Optional> str Used in scenarios where multiple function apps share the same storage account but need to be isolated from each other. If not specified, the default value from

Name	Description
	host.json is used. This value must match the value used by the target orchestrator functions. Default value: None
connection_name	<xref:Optional> [str] The name of an app setting that contains a storage account connection string. The storage account represented by this connection string must be the same one used by the target orchestrator functions. If not specified, the default storage account connection string for the function app is used. Default value: None

durable_openai_agent_orchestrator

Decorate Azure Durable Functions orchestrators that use OpenAI Agents.

Python

```
durable_openai_agent_orchestrator(_func=None, *, model_provider=None,
model_retry_options: RetryOptions | None =
RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5))
```

Parameters

 Expand table

Name	Description
model_provider Required*	<xref:Optional>[<xref:ModelProvider>] Use a non-default ModelProvider instead of the default OpenAIProvider, such as when testing.
model_retry_options Required*	RetryOptions None
_func	Default value: None

Keyword-Only Parameters

 Expand table

Name	Description
model_provider	Default value: None
model_retry_options	

Name	Description
	Default value: RetryOptions(first_retry_interval_in_milliseconds=2000, max_number_of_attempts=5)

embeddings_input

The embeddings input decorator generates embeddings used to measure the relatedness of text strings.

Ref: <https://platform.openai.com/docs/guides/embeddings>

Python

```

embeddings_input(arg_name: str, input: str, input_type: InputType,
ai_connection_name: str | None = '', embeddings_model: str | OpenAIModels | None =
OpenAIModels.DefaultEmbeddingsModel, max_chunk_length: int | None = 8 * 1024,
max_overlap: int | None = 128, data_type: DataType | str | None = None, **kwargs)
-> Callable[[...], Any]

```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the binding parameter in the function code.
input Required*	<code>str</code> The input source containing the data to generate embeddings for.
input_type Required*	<code>InputType</code> The type of the input (e.g., string, list, file reference).
ai_connection_name	<code>str</code> <code>None</code> The name of the configuration section for AI service connectivity settings. Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this section. - If not specified or not found, falls back to the environment variables <code>AZURE_OPENAI_ENDPOINT</code> and <code>AZURE_OPENAI_KEY</code>. - For user-assigned managed identity authentication, this is required. OpenAI (non-Azure): Set the <code>OPENAI_API_KEY</code> environment variable. Default value: ""
model	

Name	Description
Required*	(Deprecated) Use <i>embeddings_model</i> instead. This parameter is unused and will be removed in future versions.
embeddings_model	str OpenAIModels None The deployment or model name for OpenAI Embeddings. Default is " <i>text-embedding-ada-002</i> ". Default value: <code>OpenAIModels.DefaultEmbeddingsModel</code>
max_chunk_length	int None The maximum number of characters into which the input should be chunked. Default is $8 * 1024$. Default value: $8 * 1024$
max_overlap	int None The maximum number of characters to overlap between input chunks. Default is <i>128</i> . Default value: 128
data_type	DataType str None Optional. Defines how the Functions runtime should interpret the parameter value. Default value: None
kwargs Required*	Additional keyword arguments to include in the <i>function.json</i> binding configuration.

Returns

[Expand table](#)

Type	Description
Callable [..., Any]	Decorator function.

embeddings_store_output

Add an embeddings input binding to the function.

The supported list of embeddings stores is extensible. Additional providers can be integrated by authoring a specially crafted NuGet package. Refer to the provider-specific folders for detailed usage instructions:

- Azure AI Search
- Azure Data Explorer

- Azure Cosmos DB using MongoDB

Python

```
embeddings_store_output(arg_name: str, input: str, input_type: InputType,
store_connection_name: str, collection: str, ai_connection_name: str | None = '',
embeddings_model: str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel,
max_chunk_length: int | None = 8 * 1024, max_overlap: int | None = 128, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
input Required*	str The input data for which embeddings should be generated.
input_type Required*	InputType The type of the input (e.g., string, list).
store_connection_name Required*	str The name of an app setting or environment variable containing the vector store connection value.
collection Required*	str The collection or table to search within.
ai_connection_name	str None The name of the configuration section for AI service connectivity settings. • For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section. • If not specified, or if the section is missing, falls back to environment variables: <code>AZURE_OPENAI_ENDPOINT</code> and <code>AZURE_OPENAI_KEY</code>. • For user-assigned managed identity, this property is required. • For OpenAI (non-Azure), set the <code>OPENAI_API_KEY</code> environment variable. Default value: ""
model Required*	Deprecated. Use <i>embeddings_model</i> instead. This parameter is unused and will be removed in future versions.
embeddings_model	str OpenAIModels None

Name	Description
	The deployment or model name for OpenAI Embeddings. The default is "text-embedding-ada-002". Default value: OpenAIModels.DefaultEmbeddingsModel
max_chunk_length	int None The maximum number of characters to chunk the input into. Default value: 8 * 1024
max_overlap	int None The maximum number of characters to overlap between chunks. Default value: 128
data_type	DataType str None Optional. Defines how the Functions runtime should treat the parameter value. Defaults to None. Default value: None
kwargs Required*	Additional keyword arguments to include in the <i>function.json</i> binding.

Returns

[Expand table](#)

Type	Description
Callable [..., Any]	Decorator function.

entity_trigger

Register an Entity Function.

Python
<code>entity_trigger(context_name: str, entity_name: str None = None)</code>

Parameters

[Expand table](#)

Name	Description
context_name Required*	str Parameter name of the Entity input.

Name	Description
entity_name	<xref:Optional>[str] Name of Entity Function. The value is None by default, in which case the name of the method is used. Default value: None

event_grid_output

The *event_grid_output* decorator adds <xref:agent_framework.azure.EventGridOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This is equivalent to defining the output binding in the *function.json*, which enables the function to write events to a custom topic.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/eventgridtrigger> ↗

Python

```
event_grid_output(arg_name: str, topic_endpoint_uri: str | None = None,
topic_key_setting: str | None = None, connection: str | None = None, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The variable name used in the function code that represents the event.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
topic_endpoint_uri	str None The name of an app setting that contains the URI for the custom topic. Default value: None
topic_key_setting	str None The name of an app setting that contains an access key for the custom topic. Default value: None

Name	Description
connection	str None The value of the common prefix for the setting that contains the topic endpoint URI. Default value: None

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

event_grid_trigger

The *event_grid_trigger* decorator adds `<xref:agent_framework.azure.EventGridTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining an Event Grid trigger in *function.json*, which enables the function to be triggered in response to an event sent to an Event Grid topic.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/eventgridtrigger> [↗](#)

Python

```
event_grid_trigger(arg_name: str, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The variable name used in the function code for the parameter that receives the event data.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value.

Name	Description
	Default value: None

Returns

[Expand table](#)

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

event_hub_message_trigger

The *event_hub_message_trigger* decorator adds

<xref:agent_framework.azure.EventHubTrigger> to the

<xref:agent_framework.azure.FunctionBuilder> object for building a

<xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining an *EventHubTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Event Hub.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs>

Python

```
event_hub_message_trigger(arg_name: str, connection: str, event_hub_name: str,
data_type: DataType | str | None = None, cardinality: Cardinality | str | None =
None, consumer_group: str | None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the <xref:agent_framework.azure.EventHubEvent> object in the function code.
connection Required*	<code>str</code> The name of an app setting or setting collection that specifies how to connect to Event Hubs.

Name	Description
event_hub_name Required*	str The name of the Event Hub.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
cardinality	Cardinality str None Set to "many" to enable batching. Default value: None
consumer_group	str None Optional. The consumer group used to subscribe to events in the hub. Default value: None
kwargs Required*	Any Additional keyword arguments to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

event_hub_output

The *event_hub_output* decorator adds [<xref:agent_framework.azure.EventHubOutput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining *EventHubOutput* in the *function.json*, which enables the function to write messages to an Azure Event Hub.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-event-hubs> [↗](#)

Python

```
event_hub_output(arg_name: str, connection: str, event_hub_name: str, data_type:
DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable representing the Event Hub output object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Event Hub.
event_hub_name Required*	str The name of the Event Hub to send messages to.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [... , Any]	Decorator function.

function_name

Optional: Sets name of the <xref:agent_framework.azure.Function> object. If not set, it will default to the name of the method name.

additional setting fields :return: Decorator function.

Python

```
function_name(name: str, setting_extra_fields: Dict[str, Any] | None = None) -> Callable[..., Any]
```

Parameters

[Expand table](#)

Name	Description
name Required*	str Name of the function.
setting_extra_fields	Dict[str, Any] None Keyword arguments for specifying Default value: None

Returns

[Expand table](#)

Type	Description
Callable[[...], Any]	

generic_input_binding

The *generic_input_binding* decorator adds `<xref:agent_framework.azure.GenericInputBinding>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a generic input binding in the *function.json*, which enables the function to read data from a custom-defined input source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> [↗](#)

Python

```
generic_input_binding(arg_name: str, type: str, data_type: DataType | str | None =  
None, **kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the input parameter in the function code.

Name	Description
type Required*	str The type of the binding.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

 Expand table

Type	Description
Callable [... , Any]	Decorator function.

generic_output_binding

The *generic_output_binding* decorator adds `<xref:agent_framework.azure.GenericOutputBinding>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining a generic output binding in the *function.json*, which enables the function to write data to a custom-defined output source.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-custom> 

Python
<pre>generic_output_binding(arg_name: str, type: str, data_type: DataType str None = None, **kwargs) -> Callable[..., Any]</pre>

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the output parameter in the function code.
type Required*	str The type of the binding.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields in the <i>function.json</i> .

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

generic_trigger

The *generic_trigger* decorator adds [<xref:agent_framework.azure.GenericTrigger>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) used in the worker function indexing model.

This is equivalent to defining a generic trigger in the *function.json*, which triggers the function to execute when generic trigger events are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-custom> 

Python
<pre>generic_trigger(arg_name: str, type: str, data_type: DataType str None = None, **kwargs) -> Callable[[...], Any]</pre>

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
type Required*	str The type of the binding.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

get_agent

Return a DurableAI Agent proxy for a registered agent.

Python
<pre>get_agent(context: Any, agent_name: str) -> DurableAIAgent</pre>

Parameters

 Expand table

Name	Description
context Required*	Any Durable Functions orchestration context invoking the agent.
agent_name Required*	str Name of the agent registered on this app.

Returns

[Expand table](#)

Type	Description
DurableAIAgent	DurableAIAgent wrapper bound to the orchestration context.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the requested agent has not been registered.

get_functions

Get the function objects in the function app.

Python
<code>get_functions() -> List[Function]</code>

Returns

[Expand table](#)

Type	Description
List[Function]	A list of <xref:agent_framework.azure.Function> objects defined in the app.

http_type

Set http type of the [<xref:agent_framework.azure.Function>](#) object.

Python
<code>http_type(http_type: str) -> Callable[[...], Any]</code>

Parameters

[Expand table](#)

Name	Description
http_type Required*	str Http type of the function.

Returns

[Expand table](#)

Type	Description
Callable [... , Any]	Decorator function.

kafka_output

The *kafka_output* decorator adds [<xref:agent_framework.azure.KafkaOutput>](#) to the [<xref:agent_framework.azure.FunctionBuilder>](#) object for building a [<xref:agent_framework.azure.Function>](#) object used in the worker function indexing model.

This is equivalent to defining a Kafka output binding in *function.json*, which enables the function to write events to a Kafka topic. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/kafkaoutput> [↗](#)

Python

```
kafka_output(arg_name: str, topic: str, broker_list: str, avro_schema: str | None
= None, username: str | None = None, password: str | None = None,
ssl_key_location: str | None = None, ssl_ca_location: str | None = None,
ssl_certificate_location: str | None = None, ssl_key_password: str | None = None,
schema_registry_url: str | None = None, schema_registry_username: str | None =
None, schema_registry_password: str | None = None, o_auth_bearer_method:
OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None,
o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None =
None, o_auth_bearer_token_endpoint_url: str | None = None,
o_auth_bearer_extensions: str | None = None, max_message_bytes: int = 1_000_000,
batch_size: int = 10_000, enable_idempotence: bool = False, message_timeout_ms:
int = 300_000, request_timeout_ms: int = 5_000, max_retries: int = 2_147_483_647,
authentication_mode: BrokerAuthenticationMode | str | None = 'NOTSET', protocol:
BrokerProtocol | str | None = 'NOTSET', linger_ms: int = 5, data_type: DataType |
str | None = None, **kwargs) -> Callable[..., Any]
```

Parameters

Name	Description
arg_name Required*	str The variable name used in the function code that represents the output event.
topic Required*	str The Kafka topic to which messages are published.
broker_list Required*	str The list of Kafka brokers to which the producer connects.
avro_schema	str None Optional. Avro schema to generate a generic record. Default value: None
username	str None SASL username for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to ' <i>sasl.username</i> ' in librdkafka. Default value: None
password	str None SASL password for use with the PLAIN and SASL-SCRAM mechanisms. Equivalent to ' <i>sasl.password</i> ' in librdkafka. Default value: None
ssl_key_location	str None Path to the client's private key (PEM) for authentication. Equivalent to ' <i>ssl.key.location</i> ' in librdkafka. Default value: None
ssl_ca_location	str None Path to the CA certificate for verifying the broker's certificate. Equivalent to ' <i>ssl.ca.location</i> ' in librdkafka. Default value: None
ssl_certificate_location	str None Path to the client's certificate file. Equivalent to ' <i>ssl.certificate.location</i> ' in librdkafka. Default value: None
ssl_key_password	str None Password for the client's SSL key. Equivalent to ' <i>ssl.key.password</i> ' in librdkafka. Default value: None
schema_registry_url	str None URL of the Avro Schema Registry. Default value: None

Name	Description
<code>schema_registry_username</code>	str None Username for accessing the Schema Registry. Default value: None
<code>schema_registry_password</code>	str None Password for accessing the Schema Registry. Default value: None
<code>o_auth_bearer_method</code>	OAuthBearerMethod str None OAuth bearer method to use, e.g., <code>'default'</code> or <code>'oidc'</code> . Equivalent to <code>'sasl.oauthbearer.method'</code> in librdkafka. Default value: None
<code>o_auth_bearer_client_id</code>	str None Used with <code>oidc</code> method. Equivalent to <code>'sasl.oauthbearer.client.id'</code> in librdkafka. Default value: None
<code>o_auth_bearer_client_secret</code>	str None Used with <code>oidc</code> method. Equivalent to <code>'sasl.oauthbearer.client.secret'</code> in librdkafka. Default value: None
<code>o_auth_bearer_scope</code>	str None Scope of the access request sent to the broker. Equivalent to <code>'sasl.oauthbearer.scope'</code> in librdkafka. Default value: None
<code>o_auth_bearer_token_endpoint_url</code>	str None OAuth token endpoint. Equivalent to <code>'sasl.oauthbearer.token.endpoint.url'</code> in librdkafka. Default value: None
<code>o_auth_bearer_extensions</code>	str None Additional metadata sent to the broker. Comma-separated key=value pairs, e.g., <code>"supportFeatureX=true,organizationId=sales-emea"</code> . Equivalent to <code>'sasl.oauthbearer.extensions'</code> in librdkafka. Default value: None
<code>max_message_bytes</code>	int Maximum size (in bytes) of a transmitted message. Default is 1MB. Default value: 1_000_000
<code>batch_size</code>	int Maximum number of messages batched in one MessageSet. Default is 10,000. Default value: 10_000

Name	Description
<code>enable_idempotence</code>	bool If <i>True</i> , ensures messages are delivered exactly once and in order. Default is <i>False</i> . Default value: <code>False</code>
<code>message_timeout_ms</code>	int Local timeout for message delivery. Default is 300000 ms. Default value: <code>300_000</code>
<code>request_timeout_ms</code>	int Timeout for producer request acknowledgment. Default is 5000 ms. Default value: <code>5_000</code>
<code>max_retries</code>	int Maximum number of retry attempts for failed messages. Default is 2147483647. Retrying may cause reordering unless <i>enable_idempotence=True</i> . Default value: <code>2_147_483_647</code>
<code>authentication_mode</code>	BrokerAuthenticationMode str None SASL mechanism used for authentication. Allowed values: <i>Gssapi</i> , <i>Plain</i> , <i>ScramSha256</i> , <i>ScramSha512</i> . Default is <i>Plain</i> . Equivalent to ' <i>sasl.mechanism</i> ' in librdkafka. Default value: <code>"NOTSET"</code>
<code>protocol</code>	BrokerProtocol str None Security protocol used for broker communication. Default is plaintext. Equivalent to ' <i>security.protocol</i> ' in librdkafka. Default value: <code>"NOTSET"</code>
<code>linger_ms</code>	int Time to wait between sending batches of messages to allow for better throughput via batching. Default value: <code>5</code>
<code>data_type</code>	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: <code>None</code>
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

kafka_trigger

The *kafka_trigger* decorator adds `<xref:agent_framework.azure.KafkaTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a Kafka trigger in *function.json*, which enables the function to be triggered in response to an event sent to a Kafka topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/kafkatrigger> [↗](#)

Python

```
kafka_trigger(arg_name: str, topic: str, broker_list: str,
event_hub_connection_string: str | None = None, consumer_group: str | None = None,
avro_schema: str | None = None, username: str | None = None, password: str | None
= None, ssl_key_location: str | None = None, ssl_ca_location: str | None = None,
ssl_certificate_location: str | None = None, ssl_key_password: str | None = None,
schema_registry_url: str | None = None, schema_registry_username: str | None =
None, schema_registry_password: str | None = None, o_auth_bearer_method:
OAuthBearerMethod | str | None = None, o_auth_bearer_client_id: str | None = None,
o_auth_bearer_client_secret: str | None = None, o_auth_bearer_scope: str | None =
None, o_auth_bearer_token_endpoint_url: str | None = None,
o_auth_bearer_extensions: str | None = None, authentication_mode:
BrokerAuthenticationMode | str | None = 'NotSet', protocol: BrokerProtocol | str |
None = 'NotSet', cardinality: Cardinality | str | None = 'One', lag_threshold: int
= 1000, data_type: DataType | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

[↗](#) Expand table

Name	Description
arg_name Required*	<code>str</code> The variable name used in the function code for the parameter that receives the Kafka event data.
topic Required*	<code>str</code> The topic monitored by the trigger.

Name	Description
broker_list Required*	str The list of Kafka brokers monitored by the trigger.
event_hub_connection_string	str None The name of an app setting that contains the connection string for Event Hub (when using Kafka protocol headers with Azure Event Hubs). Default value: None
consumer_group	str None Kafka consumer group used by the trigger. Default value: None
avro_schema	str None Used only if a generic Avro record should be generated. Default value: None
username	str None SASL username for use with the PLAIN or SASL-SCRAM mechanisms. Equivalent to 'sasl.username' in librdkafka. Default is empty string. Default value: None
password	str None SASL password for use with the PLAIN or SASL-SCRAM mechanisms. Equivalent to 'sasl.password' in librdkafka. Default is empty string. Default value: None
ssl_key_location	str None Path to the client's private key (PEM) used for authentication. Equivalent to 'ssl.key.location' in librdkafka. Default value: None
ssl_ca_location	str None Path to the CA certificate file for verifying the broker's certificate. Equivalent to 'ssl.ca.location' in librdkafka. Default value: None
ssl_certificate_location	str None Path to the client's certificate. Equivalent to 'ssl.certificate.location' in librdkafka. Default value: None
ssl_key_password	str None Password for the client's certificate. Equivalent to 'ssl.key.password' in librdkafka. Default value: None

Name	Description
<code>schema_registry_url</code>	str None URL of the Avro Schema Registry. Default value: None
<code>schema_registry_username</code>	str None Username for the Schema Registry. Default value: None
<code>schema_registry_password</code>	str None Password for the Schema Registry. Default value: None
<code>o_auth_bearer_method</code>	OAuthBearerMethod str None Either 'default' or 'oidc'. Equivalent to 'sasloauthbearer.method' in librdkafka. Default value: None
<code>o_auth_bearer_client_id</code>	str None Specify only if <code>o_auth_bearer_method</code> is 'oidc'. Equivalent to 'sasloauthbearer.client.id' in librdkafka. Default value: None
<code>o_auth_bearer_client_secret</code>	str None Specify only if <code>o_auth_bearer_method</code> is 'oidc'. Equivalent to 'sasloauthbearer.client.secret' in librdkafka. Default value: None
<code>o_auth_bearer_scope</code>	str None Specify only if <code>o_auth_bearer_method</code> is 'oidc'. Used to specify access scope. Equivalent to 'sasloauthbearer.scope'. Default value: None
<code>o_auth_bearer_token_endpoint_url</code>	str None Specify only if <code>o_auth_bearer_method</code> is 'oidc'. Equivalent to 'sasloauthbearer.token.endpoint.url' in librdkafka. Default value: None
<code>o_auth_bearer_extensions</code>	str None Additional information for the broker, in the form of a comma-separated list of key=value pairs (e.g., "orgId=abc,flag=true"). Equivalent to 'sasloauthbearer.extensions' in librdkafka. Default value: None
<code>authentication_mode</code>	BrokerAuthenticationMode str None SASL mechanism to use. Allowed: Gssapi, Plain, ScramSha256, ScramSha512. Default: Plain. Equivalent to 'sasl.mechanism'. Default value: "NotSet"

Name	Description
protocol	BrokerProtocol str None Security protocol used to communicate with brokers. Default: plaintext. Equivalent to 'security.protocol'. Default value: "NotSet"
lag_threshold	int Max number of unprocessed messages per worker instance. Used in scaling logic to estimate needed worker instances. Default is 1000. Default value: 1000
data_type	DataType str None Defines how Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for extra binding fields in the binding JSON.
cardinality	Cardinality str None Default value: "One"

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

mcp_tool_trigger

The *mcp_tool_trigger* decorator adds `<xref:agent_framework.azure.MCPToolTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining *MCPToolTrigger* in the *function.json*, which enables the function to be triggered when MCP tool requests are received by the host.

All optional fields will be given default values by the function host when they are parsed.

Ref: <https://aka.ms/remote-mcp-functions-python> 

Python

```
mcp_tool_trigger(arg_name: str, tool_name: str, description: str | None = None,
tool_properties: str | None = None, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the trigger parameter in the function code.
tool_name Required*	str The logical tool name exposed to the host.
description	str None Optional human-readable description of the tool. Default value: None
tool_properties	str None JSON-serialized tool properties/parameters list. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Keyword arguments for specifying additional binding fields to include in the binding JSON.

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

mysql_input

The *mysql_input* decorator adds <xref:agent_framework.azure.MySqlInput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySQLInput* in the *function.json*, which enables the function to read data from a MySQL database. All optional fields are automatically assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings>

Python

```
mysql_input(arg_name: str, command_text: str, connection_string_setting: str,
            command_type: str | None = 'Text', parameters: str | None = None, data_type:
            DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.MySqlRowList> input object in the function code.
command_text Required*	str The SQL query string or the name of the stored procedure to execute.
connection_string_setting Required*	str The name of the app setting that contains the connection string to the MySQL database.
command_type	str None The type of command being executed. Accepts "Text" for SQL queries or "StoredProcedure" for stored procedures. Default value: 'Text'
parameters	str None One or more parameter values passed to the SQL command. Should be provided as a comma-separated string in the format: " @param1=value1,@param2=value2 ". Default value: None
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments used to define fields in the <i>function.json</i> binding.

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

mysql_output

The *mysql_output* decorator adds `<xref:agent_framework.azure.MySqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator is supported only with extension bundle 4.x and above. It is equivalent to defining *MySqlOutput* in the *function.json*, enabling the function to write data to a MySQL database.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings> 

Python

```
mysql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the MySQL output object in the function code.
command_text Required*	<code>str</code> The SQL command or the name of the stored procedure executed by the binding.
connection_string_setting Required*	<code>str</code> The name of the app setting that contains the connection string for the target MySQL database.

Name	Description
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments used to define fields in the <i>function.json</i> binding.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

mysql_trigger

The *mysql_trigger* decorator adds `<xref:agent_framework.azure.MySqlTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *MySqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the MySQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/mysqlbindings>

Python
<pre>mysql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str None = None, data_type: DataType None = None, **kwargs) -> Callable[[...], Any]</pre>

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents a <code><xref:agent_framework.azure.MySqlRowList></code> object in the function

Name	Description
	code.
table_name Required*	str The name of the MySQL table monitored by the trigger.
connection_string_setting Required*	str The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.
leases_table_name	str None The name of the table used to store leases. If not specified, the default table name is <i>Leases_{FunctionId}_{TableId}</i> . Default value: None
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [..., Any]	Decorator function.

orchestration_trigger

Register an Orchestrator Function.

Python

```
orchestration_trigger(context_name: str, orchestration: str | None = None)
```

Parameters

[Expand table](#)

Name	Description
context_name Required*	str Parameter name of the DurableOrchestrationContext object.

Name	Description
orchestration	<xref:Optional> str Name of Orchestrator Function. The value is None by default, in which case the name of the method is used. Default value: None

queue_output

The *queue_output* decorator adds <xref:agent_framework.azure.QueueOutput> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining *QueueOutput* in the *function.json*, which enables the function to write messages to an Azure Storage Queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> ↗

Python

```
queue_output(arg_name: str, queue_name: str, connection: str, data_type: DataType
| None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the storage queue output object in the function code.
queue_name Required*	str The name of the queue to which messages will be written.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Azure Queues.
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None

Name	Description
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

queue_trigger

The *queue_trigger* decorator adds `<xref:agent_framework.azure.QueueTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *QueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to an Azure Storage queue.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-queue> [↗](#)

Python

```
queue_trigger(arg_name: str, queue_name: str, connection: str, data_type: DataType
| None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the <code><xref:agent_framework.azure.QueueMessage></code> object in the function code.
queue_name Required*	str The name of the queue to monitor.
connection	str

Name	Description
Required*	The name of an app setting or setting collection that specifies how to connect to Azure Queues.
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments to include in the binding JSON.

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

register_blueprint

Register a list of functions in the function app.

Python
<code>register_blueprint(function_container: DecoratorApi) -> None</code>

Parameters

[Expand table](#)

Name	Description
function_container Required*	DecoratorApi Instance extending <code><xref:agent_framework.azure.DecoratorApi></code> , which contains a list of functions to be registered.

Returns

[Expand table](#)

Type	Description
None	

Exceptions

 Expand table

Type	Description
TypeError	If the provided container is an instance of
agent_framework.azure.FunctionRegister	

register_functions

Register a list of functions in the function app.

Python
<code>register_functions(function_container: DecoratorApi) -> None</code>

Parameters

 Expand table

Name	Description
function_container Required*	DecoratorApi Instance extending <xref:agent_framework.azure.DecoratorApi> , which contains a list of functions to be registered.

Returns

 Expand table

Type	Description
None	

Exceptions

 Expand table

Type	Description
TypeError	If the provided container is an instance of
agent_framework.azure.FunctionRegister	

retry

The *retry* decorator adds a `<xref:agent_framework.azure.RetryPolicy>` to the function settings object.

Used for building a `<xref:agent_framework.azure.Function>` object in the worker function indexing model. This is equivalent to defining a `RetryPolicy` in *function.json*, which enables the function to automatically retry on failure.

All optional fields are assigned default values by the function host during parsing.

Ref: https://aka.ms/azure_functions_retries

Python

```
retry(strategy: str, max_retry_count: str, delay_interval: str | None = None,
minimum_interval: str | None = None, maximum_interval: str | None = None,
setting_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
strategy Required*	<code>str</code> The retry strategy to use.
max_retry_count Required*	<code>str</code> The maximum number of retry attempts.
delay_interval	<code>str</code> <code>None</code> The delay interval between retry attempts. Default value: None
minimum_interval	<code>str</code> <code>None</code> The minimum delay interval between retry attempts. Default value: None
maximum_interval	<code>str</code> <code>None</code> The maximum delay interval between retry attempts. Default value: None
setting_extra_fields	<code>Dict[str, Any]</code> <code>None</code> Keyword arguments for specifying additional setting fields. Default value: None

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

route

The *route* decorator adds `<xref:agent_framework.azure.HttpTrigger>` and `<xref:agent_framework.azure.HttpOutput>` bindings to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining both *HttpTrigger* and *HttpOutput* bindings in the *function.json*, which enables your function to be triggered when HTTP requests hit the specified route.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-http> 

Python

```
route(route: str | None = None, trigger_arg_name: str = 'req', binding_arg_name: str = '$return', methods: Iterable[str] | Iterable[HttpMethod] | None = None, auth_level: AuthLevel | str | None = None, trigger_extra_fields: Dict[str, Any] | None = None, binding_extra_fields: Dict[str, Any] | None = None) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
route	str None Route for the HTTP endpoint. If <code>None</code> , it defaults to the function name (if present) or the user-defined Python function name. Default value: <code>None</code>
trigger_arg_name	str Argument name for <code><xref:agent_framework.azure.HttpRequest></code> . Defaults to <code>'req'</code> . Default value: <code>'req'</code>

Name	Description
binding_arg_name	str Argument name for <xref:agent_framework.azure.HttpResponse>. Defaults to '\$return'. Default value: '\$return'
methods	Iterable[str] Iterable[HttpMethod] None A tuple of the HTTP methods to which the function responds. Default value: None
auth_level	AuthLevel str None Determines what keys, if any, need to be present on the request in order to invoke the function. Default value: None
trigger_extra_fields	Dict[str, Any] None Additional fields to include in the trigger JSON. For example: <pre> data_type='STRING' # results in 'dataType': 'STRING' in the trigger JSON</pre> Default value: None
binding_extra_fields	Dict[str, Any] None Additional fields to include in the binding JSON. For example: <pre> data_type='STRING' # results in 'dataType': 'STRING' in the binding JSON</pre> Default value: None

Returns

[Expand table](#)

Type	Description
Callable[[...], Any]	Decorator function.

schedule

The *schedule* or *timer* decorator adds <xref:agent_framework.azure.TimerTrigger> to the <xref:agent_framework.azure.FunctionBuilder> object for building a <xref:agent_framework.azure.Function> used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-timer> 

Python

```
schedule(arg_name: str, schedule: str, run_on_startup: bool | None = None,
use_monitor: bool | None = None, data_type: DataType | str | None = None,
**kwargs: Any) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <code><xref:agent_framework.azure.TimerRequest></code> object in function code.
schedule Required*	str A string representing a CRON expression used to schedule the function execution.
run_on_startup	bool None If True, the function is invoked when the runtime starts. Default value: None
use_monitor	bool None Set to True or False to indicate whether the schedule should be monitored. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

semantic_search_input

Enable semantic search capabilities using vector databases and OpenAI models.

This feature allows you to import documents into a vector database via an output binding and perform semantic queries against them via an input binding. For example, one function can import documents into the database, while another function issues queries to OpenAI using that data as context — a pattern known as Retrieval Augmented Generation (RAG).

Ref: <https://platform.openai.com/docs/guides/embeddings>

Python

```
semantic_search_input(arg_name: str, search_connection_name: str, collection: str,
query: str | None = None, ai_connection_name: str | None = '', embeddings_model:
str | OpenAIModels | None = OpenAIModels.DefaultEmbeddingsModel, chat_model: str |
OpenAIModels | None = OpenAIModels.DefaultChatModel, system_prompt: str | None =
semantic_search_system_prompt, max_knowledge_count: int | None = 1, temperature:
str | None = '0.5', top_p: str | None = None, max_tokens: str | None = '100',
is_reasoning_model: bool | None = False, data_type: DataType | str | None = None,
**kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the binding parameter in the function code.
search_connection_name Required*	<code>str</code> The name of the app setting or environment variable that contains the vector search connection value.
collection Required*	<code>str</code> The name of the collection or table to search or store.
query	<code>str None</code> The semantic query text used for searching the database. Default value: None
ai_connection_name	<code>str None</code>

Name	Description
	The name of the configuration section for AI service connectivity settings. - For Azure OpenAI: If specified, looks for "Endpoint" and/or "Key" in this configuration section. - If not specified or the section is missing, defaults to the environment variables <i>AZURE_OPENAI_ENDPOINT</i> and <i>AZURE_OPENAI_KEY</i>. - For user-assigned managed identity authentication, this field is required. - For OpenAI (non-Azure), set the <i>OPENAI_API_KEY</i> environment variable. Default value: ""
embeddings_model	str OpenAIModels None The deployment or model name for OpenAI Embeddings. Defaults to "text-embedding-ada-002". Default value: OpenAIModels.DefaultEmbeddingsModel
chat_model	str OpenAIModels None The deployment or model name for the OpenAI Chat Completion API. Defaults to "gpt-3.5-turbo". Default value: OpenAIModels.DefaultChatModel
system_prompt	str None Optional. The system prompt provided to the large language model. Default value: semantic_search_system_prompt
max_knowledge_count	int None Optional. The number of knowledge items to inject into the system prompt. Default is 1. Default value: 1
temperature	str None The sampling temperature to use (range: 0 to 2). Higher values like 0.8 yield more random output; lower values like 0.2 make output more focused. Default value: "0.5"
top_p	str None Alternative to temperature sampling (nucleus sampling). For example, 0.1 considers only the top 10% of tokens by probability mass. Use this or <i>temperature</i>. Default value: None
max_tokens	str None The maximum number of tokens to generate in the completion. The sum of prompt tokens and <i>max_tokens</i> must not exceed the model's context length (usually 2048 or 4096 for newer models).

Name	Description
	Default value: "100"
<code>is_reasoning_model</code>	<code>bool</code> <code>None</code> Indicates whether the chat model is a reasoning model. If True, <i>max_tokens</i> and <i>temperature</i> are not supported. Default value: False
<code>data_type</code>	<code>DataType</code> <code>str</code> <code>None</code> Optional. Defines how the Functions runtime should interpret the parameter value. Default is None. Default value: None
<code>kwargs</code> Required*	Additional keyword arguments for specifying extra fields in the <i>function.json</i> binding.

Returns

[Expand table](#)

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

service_bus_queue_output

The *service_bus_queue_output* decorator adds `<xref:agent_framework.azure.ServiceBusQueueOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining *ServiceBusQueueOutput* in the *function.json*, which enables the function to write messages to a Service Bus queue.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus>

Python

```
service_bus_queue_output(arg_name: str, connection: str, queue_name: str,
data_type: DataType | str | None = None, access_rights: AccessRights | str | None
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the Service Bus queue output object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
queue_name Required*	str The name of the queue to which messages will be sent.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
access_rights	AccessRights str None The access rights required for the connection string. Default value: None

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

service_bus_queue_trigger

The *on_service_bus_queue_change* decorator adds `<xref:agent_framework.azure.ServiceBusQueueTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *ServiceBusQueueTrigger* in the *function.json*, which enables the function to be triggered when new message(s) are sent to the Service Bus queue.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus> ↗

Python

```
service_bus_queue_trigger(arg_name: str, connection: str, queue_name: str,
data_type: DataType | str | None = None, access_rights: AccessRights | str | None
= None, is_sessions_enabled: bool | None = None, cardinality: Cardinality | str |
None = None, **kwargs: Any) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the <xref:agent_framework.azure.ServiceBusMessage> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
queue_name Required*	str The name of the queue to monitor.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
access_rights	AccessRights str None Access rights for the connection string. Default value: None
is_sessions_enabled	bool None Set to True if connecting to a session-aware queue or subscription. Default value: None
cardinality	Cardinality str None Set to "many" to enable batching. Default value: None
kwargs Required*	Any

Returns

 Expand table

Type	Description
<code>Callable[[...], Any]</code>	Decorator function.

service_bus_topic_output

The `service_bus_topic_output` decorator adds

`<xref:agent_framework.azure.ServiceBusTopicOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining `ServiceBusTopicOutput` in the `function.json`, which enables the function to write messages to a Service Bus topic.

All optional fields will be assigned default values by the function host when they are parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus>

Python

```
service_bus_topic_output(arg_name: str, connection: str, topic_name: str,
subscription_name: str | None = None, data_type: DataType | str | None = None,
access_rights: AccessRights | str | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the Service Bus topic output object in the function code.
connection Required*	<code>str</code> The name of an app setting or setting collection that specifies how to connect to Service Bus.
topic_name Required*	<code>str</code> The name of the topic to which messages will be sent.
subscription_name	<code>str None</code> The name of the subscription (optional for output). Default value: None
data_type	<code>DataType str None</code>

Name	Description
	Defines how the Functions runtime should treat the parameter value. Defaults to <code>DataType.UNDEFINED</code> . Default value: <code>None</code>
<code>access_rights</code>	AccessRights <code>str</code> <code>None</code> The access rights required for the connection string. Default value: <code>None</code>

Returns

[Expand table](#)

Type	Description
Callable [... , Any]	Decorator function.

service_bus_topic_trigger

The `on_service_bus_topic_change` decorator adds `<xref:agent_framework.azure.ServiceBusTopicTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a `ServiceBusTopicTrigger` in the `function.json`, which enables the function to be triggered when new message(s) are sent to a Service Bus topic.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-service-bus>

Python

```
service_bus_topic_trigger(arg_name: str, connection: str, topic_name: str,
subscription_name: str, data_type: DataType | str | None = None, access_rights:
AccessRights | str | None = None, is_sessions_enabled: bool | None = None,
cardinality: Cardinality | str | None = None, **kwargs: Any) -> Callable[[...],
Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the <code><xref:agent_framework.azure.ServiceBusMessage></code> object in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to Service Bus.
topic_name Required*	str The name of the topic to monitor.
subscription_name Required*	str The name of the subscription to monitor.
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: <code>None</code>
access_rights	AccessRights str None Access rights for the connection string. Default value: <code>None</code>
is_sessions_enabled	bool None Set to <code>True</code> if connecting to a session-aware queue or subscription. Default value: <code>None</code>
cardinality	Cardinality str None Set to <code>"many"</code> to enable batching. Default value: <code>None</code>
kwargs Required*	Any

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

sql_input

The `sql_input` decorator adds `<xref:agent_framework.azure.SqlInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a

<xref:agent_framework.azure.Function> object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlInput* in the *function.json*, which enables the function to read from a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> ↗

Python

```
sql_input(arg_name: str, command_text: str, connection_string_setting: str,
command_type: str | None = 'Text', parameters: str | None = None, data_type:
DataType | None = None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents a <xref:agent_framework.azure.SqlRowList> input object in the function code.
command_text Required*	str The Transact-SQL query or the name of the stored procedure executed by the binding.
connection_string_setting Required*	str The name of the app setting that contains the connection string to the target SQL database.
command_type	str None The command type. Use "Text" for a raw SQL query and "StoredProcedure" for a stored procedure. Default value: 'Text'
parameters	str None A comma-separated string of parameter assignments to pass to the SQL command. Format: *@param1=value1,@param2=value2* Default value: None
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs	

Name	Description
Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

sql_output

The *sql_output* decorator adds `<xref:agent_framework.azure.SqlOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlOutput* in the *function.json*, which enables the function to write to a SQL database. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> [↗](#)

Python
<pre>sql_output(arg_name: str, command_text: str, connection_string_setting: str, data_type: DataType None = None, **kwargs) -> Callable[[...], Any]</pre>

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the SQL output object in the function code.
command_text Required*	str The Transact-SQL query or the name of the stored procedure executed by the binding.
connection_string_setting	str

Name	Description
Required*	The name of the app setting that contains the connection string to the target SQL database.
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

sql_trigger

The *sql_trigger* decorator adds `<xref:agent_framework.azure.SqlTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This decorator works only with extension bundle 4.x and above. It is equivalent to defining *SqlTrigger* in the *function.json*, which enables the function to be triggered when there are changes in the SQL table. All optional fields are assigned default values by the function host when parsed.

Ref: <https://aka.ms/sqlbindings> [↗](#)

Python
<pre>sql_trigger(arg_name: str, table_name: str, connection_string_setting: str, leases_table_name: str None = None, data_type: DataType None = None, **kwargs) -> Callable[[...], Any]</pre>

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents a <code><xref:agent_framework.azure.SqlRowList></code> object in the function code.
table_name Required*	str The name of the SQL table monitored by the trigger.
connection_string_setting Required*	str The name of an app setting that contains the connection string for the database against which the query or stored procedure is executed.
leases_table_name	str None The name of the table used to store leases. If not specified, the default name is <code>Leases_{FunctionId}_{TableId}</code> . Default value: None
data_type	DataType None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <code>function.json</code> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

table_input

The `table_input` decorator adds `<xref:agent_framework.azure.TableInput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `TableInput` in the `function.json`, which enables the function to read from a table in Azure Storage or a Cosmos DB account.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings> [↗](#)

Python

```
table_input(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, take: int | None = None, filter: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the variable that represents the table or entity in the function code.
connection Required*	str The name of an app setting or setting collection that specifies how to connect to the table service.
table_name Required*	str The name of the table.
row_key	str None The row key of the table entity to read. Default value: None
partition_key	str None The partition key of the table entity to read. Default value: None
take	int None The maximum number of entities to return. Default value: None
filter	str None An OData filter expression to apply when retrieving entities. Default value: None
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

table_output

The `table_output` decorator adds `<xref:agent_framework.azure.TableOutput>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` object used in the worker function indexing model.

This is equivalent to defining `TableOutput` in the `function.json`, which enables the function to write entities to a table in Azure Storage.

All optional fields will be assigned default values by the function host when parsed.

Ref: <https://aka.ms/tablesbindings>

Python

```
table_output(arg_name: str, connection: str, table_name: str, row_key: str | None = None, partition_key: str | None = None, data_type: DataType | str | None = None) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the table or entity in the function code.
connection Required*	<code>str</code> The name of an app setting or setting collection that specifies how to connect to the table service.
table_name Required*	<code>str</code> The name of the table.
row_key	<code>str None</code> The row key of the table entity to read. Default value: None
partition_key	<code>str None</code> The partition key of the table entity to read. Default value: None
data_type	<code>DataType str None</code> Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

 Expand table

Type	Description
Callable [[...], Any]	Decorator function.

text_completion_input

The *textCompletion* input binding is used to invoke the OpenAI Chat Completions API and return the generated results to the function.

Ref: <https://platform.openai.com/docs/guides/text-generation/chat-completions-vs-completions> 

The examples typically define an HTTP-triggered function with a hardcoded prompt such as "who is {name}?", where {name} is dynamically substituted from the HTTP request path. The OpenAI input binding sends this prompt to the configured GPT model and returns the generated response to the function, which then returns the result as the HTTP response content.

Python

```
text_completion_input(arg_name: str, prompt: str, ai_connection_name: str | None =  
'', chat_model: str | OpenAIModels | None = OpenAIModels.DefaultChatModel,  
temperature: str | None = '0.5', top_p: str | None = None, max_tokens: str | None  
= '100', is_reasoning_model: bool | None = False, data_type: DataType | str | None  
= None, **kwargs) -> Callable[[...], Any]
```

Parameters

 Expand table

Name	Description
arg_name Required*	str The name of the binding parameter in the function code.
prompt Required*	str The text prompt to generate completions for.
ai_connection_name	str None The name of the configuration section that defines AI service connectivity settings.

Name	Description
	- For Azure OpenAI: If specified, it looks for "<i>Endpoint</i>" and/or "<i>Key</i>" in this section. If not specified or missing, it falls back to the environment variables <code>AZURE_OPENAI_ENDPOINT</code> and <code>AZURE_OPENAI_KEY</code>. - For user-assigned managed identity, this setting is required. - For non-Azure OpenAI, ensure the <code>OPENAI_API_KEY</code> environment variable is set. Default value: ""
model Required*	(Deprecated) Use <code>chat_model</code> instead. This parameter is unused and will be removed in future versions.
chat_model	str OpenAIModels None The deployment or model name to use for the Chat Completions API. Default is " <i>gpt-3.5-turbo</i> ". Default value: <code>OpenAIModels.DefaultChatModel</code>
temperature	str None Sampling temperature (0–2). Higher values (e.g., 0.8) produce more random output; lower values (e.g., 0.2) make output more focused and deterministic. Default value: "0.5"
top_p	str None Controls nucleus sampling. For example, <i>top_p=0.1</i> considers only tokens in the top 10% cumulative probability. Recommended to use either <i>temperature</i> or <i>top_p</i> , but not both. Default value: None
max_tokens	str None The maximum number of tokens to generate. The sum of prompt tokens and <i>max_tokens</i> must not exceed the model's context length (usually 2048 or 4096 for newer models). Default value: "100"
is_reasoning_model	bool None If <i>True</i> , indicates that the configured chat model is a reasoning model. In this case, <i>max_tokens</i> and <i>temperature</i> are not supported. Default value: False
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Additional keyword arguments for specifying binding fields to include in the <i>function.json</i> .

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

timer_trigger

The *schedule* or *timer* decorator adds `<xref:agent_framework.azure.TimerTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a *TimerTrigger* in the *function.json*, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-timer> [↗](#)

Python

```
timer_trigger(arg_name: str, schedule: str, run_on_startup: bool | None = None,
use_monitor: bool | None = None, data_type: DataType | str | None = None,
**kwargs: Any) -> Callable[[...], Any]
```

Parameters

[Expand table](#)

Name	Description
arg_name Required*	str The name of the variable that represents the <code><xref:agent_framework.azure.TimerRequest></code> object in function code.
schedule Required*	str A string representing a CRON expression used to schedule the function execution.
run_on_startup	bool None If True, the function is invoked when the runtime starts. Default value: None
use_monitor	bool None Set to True or False to indicate whether the schedule should be monitored. Default value: None

Name	Description
data_type	DataType str None Defines how the Functions runtime should treat the parameter value. Default value: None
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
Callable [[...], Any]	Decorator function.

validate_function_names

Validate that all functions have unique names within the app.

The *functions_bindings* dictionary maps function names to their bindings. If multiple functions share the same name, indexing will fail.

Python
<pre>validate_function_names(functions: List[Function])</pre>

Parameters

[Expand table](#)

Name	Description
functions Required*	List[Function] A list of <xref:agent_framework.azure.Function> objects to validate.

Exceptions

[Expand table](#)

Type	Description
ValueError	If any function name is duplicated.

warm_up_trigger

The `warm_up` decorator adds `<xref:agent_framework.azure.WarmUpTrigger>` to the `<xref:agent_framework.azure.FunctionBuilder>` object for building a `<xref:agent_framework.azure.Function>` used in the worker function indexing model.

This is equivalent to defining a `WarmUpTrigger` in the `function.json`, which enables your function to be triggered on the specified schedule.

All optional fields will be given default values by the function host when parsed.

Ref: <https://aka.ms/azure-function-binding-warmup>

Python

```
warm_up_trigger(arg_name: str, data_type: DataType | str | None = None, **kwargs)
-> Callable
```

Parameters

 Expand table

Name	Description
arg_name Required*	<code>str</code> The name of the variable that represents the <code><xref:agent_framework.azure.TimerRequest></code> object in the function code.
data_type	<code>DataType str None</code> Defines how the Functions runtime should treat the parameter value. Default value: None

Returns

 Expand table

Type	Description
<code>Callable</code>	Decorator function.

Attributes

agents

Returns dict of agent names to agent instances.

Returns

[Expand table](#)

Type	Description
	Dictionary mapping agent names to their AgentProtocol instances.

app_script_file

Name of function app script file in which all the functions are defined.

Script file defined here is for placeholder purpose, please refer to worker defined script file path as the single point of truth.

Returns

[Expand table](#)

Type	Description
	Script file name.

auth_level

Authorization level of the function app. Will be applied to the http trigger functions which do not have authorization level specified.

Returns

[Expand table](#)

Type	Description
	Authorization level of the function app.

enable_health_check

Whether health check endpoint is enabled

Python

```
enable_health_check: bool
```

enable_http_endpoints

Whether HTTP endpoints are created for agents

```
Python
```

```
enable_http_endpoints: bool
```

enable_mcp_tool_trigger

Whether MCP tool triggers are created for agents

```
Python
```

```
enable_mcp_tool_trigger: bool
```

max_poll_retries

Maximum polling attempts when waiting for responses

poll_interval_seconds

Delay (seconds) between polling attempts

AgentResponseCallbackProtocol Class

Protocol describing the callbacks invoked during agent execution.

Constructor

Python
<code>AgentResponseCallbackProtocol()</code>

Methods

 Expand table

<code>__init__</code>	
<code>__new__</code>	
<code>on_agent_response</code>	Handle the final agent response.
<code>on_streaming_response_update</code>	Handle a streaming response update emitted by the agent.

`__init__`

Python
<code>__init__(*args, **kwargs)</code>

`__new__`

Python
<code>__new__(**kwargs)</code>

`on_agent_response`

Handle the final agent response.

Python
<code>async on_agent_response(response: AgentRunResponse, context: AgentCallbackContext)</code>

-> `None`

Parameters

 Expand table

Name	Description
response Required*	AgentRunResponse
context Required*	AgentCallbackContext

Returns

 Expand table

Type	Description
None	

on_streaming_response_update

Handle a streaming response update emitted by the agent.

Python

```
async on_streaming_response_update(update: AgentRunResponseUpdate, context: AgentCallbackContext) -> None
```

Parameters

 Expand table

Name	Description
update Required*	AgentRunResponseUpdate
context Required*	AgentCallbackContext

Returns

 Expand table

Type	Description
None	

AzureAIAgentClient Class

Azure AI Agent Chat client.

Initialize an Azure AI Agent client.

Constructor

Python
<code>AzureAIAgentClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>agents_client</code>	An existing AgentsClient to use. If not provided, one will be created. Default value: None
<code>agent_id</code>	The ID of an existing agent to use. If not provided and agents_client is provided, a new agent will be created (and deleted after the request). If neither agents_client nor agent_id is provided, both will be created and managed automatically. Default value: None
<code>agent_name</code>	The name to use when creating new agents. Default value: None
<code>agent_description</code>	The description to use when creating new agents. Default value: None
<code>thread_id</code>	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
<code>project_endpoint</code>	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a agents_client is passed. Default value: None
<code>model_deployment_name</code>	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
<code>credential</code>	Azure async credential to use for authentication.

Name	Description
	Default value: None
should_cleanup_agent	Whether to cleanup (delete) agents created by this client when the client is closed or context is exited. Defaults to True. Only affects agents created by this client instance; existing agents passed via agent_id are never deleted. Default value: True
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Examples

Python

```
from agent_framework_azure_ai import AzureAIAgentClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIAgentClient(credential=credential)

# Or passing parameters directly
client = AzureAIAgentClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIAgentClient(credential=credential, env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an Azure AI Agent client.
<code>__new__</code>	
<code>close</code>	Close the agents_client and clean up any agents we created.

<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>from_settings</code>	Initialize a AzureAIAgentClient from a dictionary of settings.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the service URL for the chat client.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Azure AI Agent client.

Python

```
__init__(*, agents_client: AgentsClient | None = None, agent_id: str | None = None, agent_name: str | None = None, agent_description: str | None = None, thread_id: str | None = None, project_endpoint: str | None = None, model_deployment_name: str | None = None, credential: AsyncTokenCredential | None = None, should_cleanup_agent: bool = True, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
agents_client Required*	<xref:azure.ai.agents.aio._patch.AgentsClient> None
agent_id Required*	str None
agent_name Required*	str None
agent_description Required*	str None
thread_id Required*	str None
project_endpoint Required*	str None
model_deployment_name Required*	str None
credential Required*	AsyncTokenCredential None
should_cleanup_agent Required*	bool
env_file_path Required*	str None
env_file_encoding Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

Name	Description
agents_client	An existing AgentsClient to use. If not provided, one will be created. Default value: None
agent_id	The ID of an existing agent to use. If not provided and agents_client is provided, a new agent will be created (and deleted after the request). If neither agents_client nor agent_id is provided, both will be created and managed automatically. Default value: None
agent_name	The name to use when creating new agents. Default value: None
agent_description	The description to use when creating new agents. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a agents_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication. Default value: None
should_cleanup_agent	Whether to cleanup (delete) agents created by this client when the client is closed or context is exited. Defaults to True. Only affects agents created by this client instance; existing agents passed via agent_id are never deleted. Default value: True
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Returns

Type	Description
None	

Examples

Python

```
from agent_framework_azure_ai import AzureAIAgentClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIAgentClient(credential=credential)

# Or passing parameters directly
client = AzureAIAgentClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIAgentClient(credential=credential,
env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Close the agents_client and clean up any agents we created.

Python

```
async close() -> None
```

Returns

[Expand table](#)

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id	str None

Name	Description
Required*	
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent.

Name	Description
	Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request.

Name	Description
	Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

 Expand table

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent()
```

```
name="assistant", instructions="You are a helpful assistant.",
temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
from_dict()

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	MutableMapping[str, Any] The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
```

```

    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}

```



```

context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TCI ass>	New instance of the class with any specified dependencies injected.

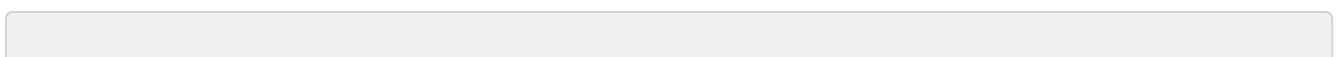
Exceptions

[Expand table](#)

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

from_settings

Initialize a `AzureAIAgentClient` from a dictionary of settings.



Python

```
from_settings(settings: dict[str, Any]) -> TAzureAIAgentClient
```

Parameters

[Expand table](#)

Name	Description
settings Required*	dict[str, Any] A dictionary of settings for the service.

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_azure_ai_chat_client.TAzureAIAgentClient>	

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async def get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

Name	Description
messages Required*	<code>str</code> <code>ChatMessage</code> <code>list[str]</code> <code>list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool</code> <code>None</code>
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias Required*	<code>dict[str int, float]</code> <code>None</code>
max_tokens Required*	<code>int</code> <code>None</code>
metadata Required*	<code>dict[str, Any]</code> <code>None</code>
model_id Required*	<code>str</code> <code>None</code>
presence_penalty Required*	<code>float</code> <code>None</code>
response_format Required*	<code>type[<xref:pydantic.main.BaseModel>]</code> <code>None</code>
seed Required*	<code>int</code> <code>None</code>
stop Required*	<code>str</code> <code>Sequence[str]</code> <code>None</code>
store Required*	<code>bool</code> <code>None</code>
temperature Required*	<code>float</code> <code>None</code>
tool_choice Required*	<code>ToolMode</code> <code>Literal['auto', 'required', 'none']</code> <code>dict[str, typing.Any]</code> <code>None</code>
tools Required*	<code>ToolProtocol</code> <code>Callable[[...], Any]</code> <code>MutableMapping[str, Any]</code> <code>Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]]</code> <code>None</code>
top_p Required*	<code>float</code> <code>None</code>

Name	Description
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None

Name	Description
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in `kwargs`) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |  
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,  
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None  
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,  
model_id: str | None = None, presence_penalty: float | None = None,  
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
```

```
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature	float None

Name	Description
Required*	
tool_choice Required*	ToolMode Literal ['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable [..., Any] MutableMapping [str, Any] Sequence [ToolProtocol Callable [..., Any] MutableMapping [str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict [str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response.

Name	Description
	Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

 Expand table

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the service URL for the chat client.

Python

```
service_url() -> str
```

Returns

 Expand table

Type	Description
<code>str</code>	The service URL for the chat client, or None if not set.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>exclude</code> Required*	<code>set[str] None</code>
<code>exclude_none</code> Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>exclude</code>	Set of field names to exclude from serialization. Default value: None
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None

Name	Description
<code>exclude_none</code>	Whether to exclude None values from the output. Defaults to True. Default value: True
<code>**kwargs</code>	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python
<code>DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}</code>

INJECTABLE

Python
<code>INJECTABLE: ClassVar[set[str]] = {}</code>

OTEL_PROVIDER_NAME

Python
<code>OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai'</code>

AzureAIClient Class

Azure AI Agent client.

Initialize an Azure AI Agent client.

Constructor

Python
<code>AzureAIClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>project_client</code>	An existing AIProjectClient to use. If not provided, one will be created. Default value: None
<code>agent_name</code>	The name to use when creating new agents or using existing agents. Default value: None
<code>agent_version</code>	The version of the agent to use. Default value: None
<code>agent_description</code>	The description to use when creating new agents. Default value: None
<code>conversation_id</code>	Default conversation ID to use for conversations. Can be overridden by <code>conversation_id</code> property when making a request. Default value: None
<code>project_endpoint</code>	The Azure AI Project endpoint URL. Can also be set via environment variable <code>AZURE_AI_PROJECT_ENDPOINT</code> . Ignored when a <code>project_client</code> is passed. Default value: None
<code>model_deployment_name</code>	The model deployment name to use for agent creation. Can also be set via environment variable <code>AZURE_AI_MODEL_DEPLOYMENT_NAME</code> . Default value: None
<code>credential</code>	Azure async credential to use for authentication. Default value: None
<code>use_latest_version</code>	

Name	Description
	Boolean flag that indicates whether to use latest agent version if it exists in the service. Default value: None
env_file_path	Path to environment file for loading settings. Default value: None
env_file_encoding	Encoding of the environment file. Default value: None
kwargs	Additional keyword arguments passed to the parent class.

Examples

Python

```
from agent_framework.azure import AzureAIClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIClient(credential=credential)

# Or passing parameters directly
client = AzureAIClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIClient(credential=credential, env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an Azure AI Agent client.
<code>__new__</code>	
<code>close</code>	Close the project_client.
<code>configure_azure_monitor</code>	Setup observability with Azure Monitor (Azure AI Foundry integration).

This method configures Azure Monitor for telemetry collection using the connection string from the Azure AI project client.

ⓘ **Note**

This method retrieves the Application Insights connection string from the

Azure AI project client automatically. You must have Application Insights

configured in your Azure AI project for this to work.

`create_agent`

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

`from_dict`

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

`from_json`

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

`get_response`

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

`get_streaming_response`

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

`service_url`

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

`to_dict`

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Azure AI Agent client.

Python

```
__init__(*, project_client: AIProjectClient | None = None, agent_name: str | None = None, agent_version: str | None = None, agent_description: str | None = None, conversation_id: str | None = None, project_endpoint: str | None = None, model_deployment_name: str | None = None, credential: AsyncTokenCredential | None = None, use_latest_version: bool | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
project_client Required*	<xref:azure.ai.projects.aio._patch.AIProjectClient> None
agent_name Required*	str None
agent_version Required*	str None
agent_description Required*	str None
conversation_id Required*	str None
project_endpoint Required*	str None
model_deployment_name Required*	str None

Name	Description
credential Required*	AsyncTokenCredential None
use_latest_version Required*	bool None
env_file_path Required*	str None
env_file_encoding Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
project_client	An existing AIProjectClient to use. If not provided, one will be created. Default value: None
agent_name	The name to use when creating new agents or using existing agents. Default value: None
agent_version	The version of the agent to use. Default value: None
agent_description	The description to use when creating new agents. Default value: None
conversation_id	Default conversation ID to use for conversations. Can be overridden by conversation_id property when making a request. Default value: None
project_endpoint	The Azure AI Project endpoint URL. Can also be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Ignored when a project_client is passed. Default value: None
model_deployment_name	The model deployment name to use for agent creation. Can also be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
credential	Azure async credential to use for authentication. Default value: None

Name	Description
<code>use_latest_version</code>	Boolean flag that indicates whether to use latest agent version if it exists in the service. Default value: None
<code>env_file_path</code>	Path to environment file for loading settings. Default value: None
<code>env_file_encoding</code>	Encoding of the environment file. Default value: None
<code>kwargs</code>	Additional keyword arguments passed to the parent class.

Returns

 Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureAIClient
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-
project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
credential = DefaultAzureCredential()
client = AzureAIClient(credential=credential)

# Or passing parameters directly
client = AzureAIClient(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4",
    credential=credential,
)

# Or loading from a .env file
client = AzureAIClient(credential=credential, env_file_path="path/to/.env")
```

`__new__`

Python

```
__new__(**kwargs)
```

close

Close the project_client.

Python

```
async close() -> None
```

Returns

[Expand table](#)

Type	Description
None	

configure_azure_monitor

Setup observability with Azure Monitor (Azure AI Foundry integration).

This method configures Azure Monitor for telemetry collection using the connection string from the Azure AI project client.

ⓘ Note

This method retrieves the Application Insights connection string from the Azure AI project client automatically. You must have Application Insights configured in your Azure AI project for this to work.

Python

```
async configure_azure_monitor(enable_sensitive_data: bool = False, **kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
<code>enable_sensitive_data</code>	<code>bool</code> Enable sensitive data logging (prompts, responses). Should only be enabled in development/test environments. Default is False. Default value: False
<code>**kwargs</code> Required [*]	<code>Any</code> Additional arguments passed to <code>configure_azure_monitor()</code> . Common options include: <code>enable_live_metrics</code> (<code>bool</code>): Enable Azure Monitor Live Metrics <code>credential</code> (<code>TokenCredential</code>): Azure credential for Entra ID auth <code>resource</code> (<code>Resource</code>): Custom OpenTelemetry resource See https://learn.microsoft.com/python/api/azure-monitor-opentelemetry/azure.monitor.opentelemetry.configure_azure_monitor for full list of options.

Returns

 Expand table

Type	Description
<code>None</code>	

Exceptions

 Expand table

Type	Description
<code>ImportError</code>	If <code>azure-monitor-opentelemetry-exporter</code> is not installed.

Examples

Python

```
from agent_framework.azure import AzureAIClient
from azure.ai.projects.aio import AIProjectClient
from azure.identity.aio import DefaultAzureCredential

async with (
    DefaultAzureCredential() as credential,
```

```

    AIProjectClient(
        endpoint="https://your-project.api.azureml.ms", credential=credential
    ) as project_client,
    AzureAIClient(project_client=project_client) as client,
):
    # Setup observability with defaults
    await client.configure_azure_monitor()

    # With live metrics enabled
    await client.configure_azure_monitor(enable_live_metrics=True)

    # With sensitive data logging (dev/test only)
    await client.configure_azure_monitor(enable_sensitive_data=True)

```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```

create_agent(*, id: str | None = None, name: str | None = None, description: str |
None = None, instructions: str | None = None, chat_message_store_factory:
Callable[[], ChatMessageStoreProtocol] | None = None, context_providers:
ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None,
middleware: AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent

```

Parameters

 Expand table

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None

Name	Description
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the <code>chat_client</code> <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

```
from_dict()
```

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_serialization.TClass>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```

client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext

```

```

from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id Required*	<code>str None</code>

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str |
Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None

Name	Description
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None
<code>tools</code>	The tools to use for the request. Default value: None
<code>top_p</code>	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Additional field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

 Expand table

Type	Description
str	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai'
```


AzureAISearchContextProvider Class

Azure AI Search Context Provider with hybrid search and semantic ranking.

This provider retrieves relevant documents from Azure AI Search to provide context to the AI agent. It supports two modes:

- **agentic**: Recommended for most scenarios. Uses Knowledge Bases for query planning and multi-hop reasoning. Slightly slower with more token consumption, but provides more accurate results (up to 36% improvement in response relevance).
- **semantic** (default): Fast hybrid search combining vector and keyword search with semantic reranking. Best for simple queries where speed is critical.

Initialize Azure AI Search Context Provider.

Constructor

Python
<pre>AzureAISearchContextProvider()</pre>

Parameters

 Expand table

Name	Description
endpoint	Azure AI Search endpoint URL. Can also be set via environment variable AZURE_SEARCH_ENDPOINT. Default value: None
index_name	Name of the search index to query. Can also be set via environment variable AZURE_SEARCH_INDEX_NAME. Default value: None
api_key	API key for authentication (string or AzureKeyCredential). Can also be set via environment variable AZURE_SEARCH_API_KEY. Default value: None
credential	AsyncTokenCredential for managed identity authentication. Use this for Entra ID authentication instead of api_key. Default value: None
mode	

Name	Description
Required*	Search mode - "semantic" for hybrid search with semantic ranking (fast) or "agentic" for multi-hop reasoning (slower). Default: "semantic".
top_k Required*	Maximum number of documents to retrieve. Only applies to semantic mode. In agentic mode, the server-side Knowledge Base determines retrieval based on query complexity and reasoning effort. Default: 5.
semantic_configuration_name Required*	Name of semantic configuration in the index. Required for semantic ranking. If None, uses index default.
vector_field_name Required*	Name of the vector field in the index for hybrid search. Required if using vector search. Default: None (keyword search only).
embedding_function Required*	Async function to generate embeddings for vector search. Signature: <code>async def embed(text: str) -> list[float]</code> Required if <code>vector_field_name</code> is specified and no server-side vectorization.
context_prompt Required*	Custom prompt to prepend to retrieved context. Default: "Use the following context to answer the question:"
azure_openai_resource_url Required*	Azure OpenAI resource URL for Knowledge Base model calls. Required when using agentic mode with <code>index_name</code> (to auto-create Knowledge Base). Not required when using an existing <code>knowledge_base_name</code> . Example: "https://myresource.openai.azure.com"
model_deployment_name Required*	Model deployment name in Azure OpenAI for Knowledge Base. Required when using agentic mode with <code>index_name</code> (to auto-create Knowledge Base). Not required when using an existing <code>knowledge_base_name</code> .
model_name Required*	The underlying model name (e.g., "gpt-4o", "gpt-4o-mini"). If not provided, defaults to <code>model_deployment_name</code> . Used for Knowledge Base configuration.
knowledge_base_name Required*	Name of an existing Knowledge Base to use. Required for agentic mode if not providing <code>index_name</code> . Supports KBs with any source type (web, blob, index, etc.).
retrieval_instructions Required*	Custom instructions for the Knowledge Base's retrieval planning. Only used in agentic mode.
azure_openai_api_key Required*	Azure OpenAI API key for Knowledge Base to call the model. Only needed when using API key authentication instead of managed identity.
knowledge_base_output_mode Required*	Output mode for Knowledge Base retrieval. Only used in agentic mode. "extractive_data": Returns raw chunks without synthesis (default, recommended for agent integration). "answer_synthesis":

Name	Description
	Returns synthesized answer from the LLM. Some knowledge sources require answer_synthesis mode. Default: "extractive_data".
retrieval_reasoning_effort Required*	Reasoning effort for Knowledge Base query planning. Only used in agentic mode. "minimal": Fastest, basic query planning. "medium": Moderate reasoning with some query decomposition. "low": Lower reasoning effort than medium. Default: "minimal".
agentic_message_history_count Required*	Number of recent messages from conversation history to send to the Knowledge Base. This context helps with query planning in agentic mode, allowing the Knowledge Base to understand the conversation flow and generate better retrieval queries. There is no technical limit - adjust based on your use case. Default: 10.
env_file_path Required*	Path to environment file for loading settings.
env_file_encoding Required*	Encoding of the environment file.

Keyword-Only Parameters

 Expand table

Name	Description
mode	Default value: semantic
top_k	Default value: 5
semantic_configuration_name	Default value: None
vector_field_name	Default value: None
embedding_function	Default value: None
context_prompt	Default value: None
azure_openai_resource_url	Default value: None
model_deployment_name	Default value: None
model_name	Default value: None
knowledge_base_name	Default value: None
retrieval_instructions	Default value: None
azure_openai_api_key	Default value: None

Name	Description
knowledge_base_output_mode	Default value: extractive_data
retrieval_reasoning_effort	Default value: minimal
agentic_message_history_count	Default value: 10
env_file_path	Default value: None
env_file_encoding	Default value: None

Examples

Python

```
from agent_framework_aisearch import AzureAISearchContextProvider
from azure.identity.aio import DefaultAzureCredential

# Using environment variables
# Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
# Set AZURE_SEARCH_INDEX_NAME=my-index
credential = DefaultAzureCredential()
provider = AzureAISearchContextProvider(credential=credential)

# Or passing parameters directly
provider = AzureAISearchContextProvider(
    endpoint="https://mysearch.search.windows.net",
    index_name="my-index",
    credential=credential,
)

# Or loading from a .env file
provider = AzureAISearchContextProvider(credential=credential,
env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize Azure AI Search Context Provider.
<code>__new__</code>	
<code>invoked</code>	Called after the agent has received a response from the underlying inference service. You can inspect the request and response messages, and update the state of the context provider.

<code>invoking</code>	Retrieve relevant context from Azure AI Search before model invocation.
<code>thread_created</code>	Called just after a new thread is created. Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

`__init__`

Initialize Azure AI Search Context Provider.

Python

```
__init__(endpoint: str | None = None, index_name: str | None = None, api_key: str | AzureKeyCredential | None = None, credential: AsyncTokenCredential | None = None, *, mode: Literal['semantic', 'agentic'] = 'semantic', top_k: int = 5, semantic_configuration_name: str | None = None, vector_field_name: str | None = None, embedding_function: Callable[[str], Awaitable[list[float]]] | None = None, context_prompt: str | None = None, azure_openai_resource_url: str | None = None, model_deployment_name: str | None = None, model_name: str | None = None, knowledge_base_name: str | None = None, retrieval_instructions: str | None = None, azure_openai_api_key: str | None = None, knowledge_base_output_mode: Literal['extractive_data', 'answer_synthesis'] = 'extractive_data', retrieval_reasoning_effort: Literal['minimal', 'medium', 'low'] = 'minimal', agentic_message_history_count: int = _DEFAULT_AGENTIC_MESSAGE_HISTORY_COUNT, env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
<code>endpoint</code>	<code>str</code> <code>None</code> Azure AI Search endpoint URL. Can also be set via environment variable <code>AZURE_SEARCH_ENDPOINT</code> . Default value: <code>None</code>
<code>index_name</code>	<code>str</code> <code>None</code> Name of the search index to query. Can also be set via environment variable <code>AZURE_SEARCH_INDEX_NAME</code> . Default value: <code>None</code>
<code>api_key</code>	<code>str</code> <code>AzureKeyCredential</code> <code>None</code> API key for authentication (string or <code>AzureKeyCredential</code>). Can also be set via environment variable <code>AZURE_SEARCH_API_KEY</code> . Default value: <code>None</code>

Name	Description
credential	AsyncTokenCredential None AsyncTokenCredential for managed identity authentication. Use this for Entra ID authentication instead of api_key. Default value: None
mode Required*	Literal ['semantic', 'agentic'] Search mode - "semantic" for hybrid search with semantic ranking (fast) or "agentic" for multi-hop reasoning (slower). Default: "semantic".
top_k Required*	int Maximum number of documents to retrieve. Only applies to semantic mode. In agentic mode, the server-side Knowledge Base determines retrieval based on query complexity and reasoning effort. Default: 5.
semantic_configuration_name Required*	str None Name of semantic configuration in the index. Required for semantic ranking. If None, uses index default.
vector_field_name Required*	str None Name of the vector field in the index for hybrid search. Required if using vector search. Default: None (keyword search only).
embedding_function Required*	Callable [[str], Awaitable [list [float]]] None Async function to generate embeddings for vector search. Signature: async def embed(text: str) -> list[float] Required if vector_field_name is specified and no server-side vectorization.
context_prompt Required*	str None Custom prompt to prepend to retrieved context. Default: "Use the following context to answer the question:"
azure_openai_resource_url Required*	str None Azure OpenAI resource URL for Knowledge Base model calls. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name. Example: "https://myresource.openai.azure.com ↗ "
model_deployment_name Required*	str None Model deployment name in Azure OpenAI for Knowledge Base. Required when using agentic mode with index_name (to auto-create Knowledge Base). Not required when using an existing knowledge_base_name.
model_name Required*	str None The underlying model name (e.g., "gpt-4o", "gpt-4o-mini"). If not provided, defaults to model_deployment_name. Used for

Name	Description
	Knowledge Base configuration.
knowledge_base_name Required*	str None Name of an existing Knowledge Base to use. Required for agentic mode if not providing index_name. Supports KBs with any source type (web, blob, index, etc.).
retrieval_instructions Required*	str None Custom instructions for the Knowledge Base's retrieval planning. Only used in agentic mode.
azure_openai_api_key Required*	str None Azure OpenAI API key for Knowledge Base to call the model. Only needed when using API key authentication instead of managed identity.
knowledge_base_output_mode Required*	Literal ['extractive_data', 'answer_synthesis'] Output mode for Knowledge Base retrieval. Only used in agentic mode. "extractive_data": Returns raw chunks without synthesis (default, recommended for agent integration). "answer_synthesis": Returns synthesized answer from the LLM. Some knowledge sources require answer_synthesis mode. Default: "extractive_data".
retrieval_reasoning_effort Required*	Literal ['minimal', 'medium', 'low'] Reasoning effort for Knowledge Base query planning. Only used in agentic mode. "minimal": Fastest, basic query planning. "medium": Moderate reasoning with some query decomposition. "low": Lower reasoning effort than medium. Default: "minimal".
agentic_message_history_count Required*	int Number of recent messages from conversation history to send to the Knowledge Base. This context helps with query planning in agentic mode, allowing the Knowledge Base to understand the conversation flow and generate better retrieval queries. There is no technical limit - adjust based on your use case. Default: 10.
env_file_path Required*	str None Path to environment file for loading settings.
env_file_encoding Required*	str None Encoding of the environment file.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: "semantic"
top_k	Default value: 5
semantic_configuration_name	Default value: None
vector_field_name	Default value: None
embedding_function	Default value: None
context_prompt	Default value: None
azure_openai_resource_url	Default value: None
model_deployment_name	Default value: None
model_name	Default value: None
knowledge_base_name	Default value: None
retrieval_instructions	Default value: None
azure_openai_api_key	Default value: None
knowledge_base_output_mode	Default value: "extractive_data"
retrieval_reasoning_effort	Default value: "minimal"
agentic_message_history_count	Default value: _DEFAULT_AGENTIC_MESSAGE_HISTORY_COUNT
env_file_path	Default value: None
env_file_encoding	Default value: None

Returns

 Expand table

Type	Description
None	

Examples

```
Python

from agent_framework_aisearch import AzureAISearchContextProvider
from azure.identity.aio import DefaultAzureCredential
```



```

# Using environment variables
# Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
# Set AZURE_SEARCH_INDEX_NAME=my-index
credential = DefaultAzureCredential()
provider = AzureAISearchContextProvider(credential=credential)

# Or passing parameters directly
provider = AzureAISearchContextProvider(
    endpoint="https://mysearch.search.windows.net",
    index_name="my-index",
    credential=credential,
)

# Or loading from a .env file
provider = AzureAISearchContextProvider(credential=credential,
env_file_path="path/to/.env")

```

__new__

Python

`__new__(**kwargs)`

invoked

Called after the agent has received a response from the underlying inference service.

You can inspect the request and response messages, and update the state of the context provider.

Python

```

async invoked(request_messages: ChatMessage | Sequence[ChatMessage],
response_messages: ChatMessage | Sequence[ChatMessage] | None = None,
invoke_exception: Exception | None = None, **kwargs: Any) -> None

```

Parameters

[Expand table](#)

Name	Description
request_messages Required*	ChatMessage Sequence[ChatMessage] The messages that were sent to the model/agent.
response_messages	ChatMessage Sequence[ChatMessage] None

Name	Description
	The messages that were returned by the model/agent. Default value: None
invoke_exception	Exception None The exception that was thrown, if any. Default value: None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
kwargs	Additional keyword arguments (not used at present).

Returns

[Expand table](#)

Type	Description
None	

invoking

Retrieve relevant context from Azure AI Search before model invocation.

Python

```
async invoking(messages: ChatMessage | MutableSequence[ChatMessage], **kwargs: Any) -> Context
```

Parameters

[Expand table](#)

Name	Description
messages Required*	ChatMessage MutableSequence[ChatMessage] User messages to use for context retrieval.

Name	Description
**kwargs Required*	Any Additional arguments (unused).

Returns

[Expand table](#)

Type	Description
Context	Context object with retrieved documents as messages.

thread_created

Called just after a new thread is created.

Implementers can use this method to perform any operations required at the creation of a new thread. For example, checking long-term storage for any data that is relevant to the current session.

Python

```
async thread_created(thread_id: str | None) -> None
```

Parameters

[Expand table](#)

Name	Description
thread_id Required*	str None The ID of the new thread.

Returns

[Expand table](#)

Type	Description
None	

Attributes

DEFAULT_CONTEXT_PROMPT

Python

```
DEFAULT_CONTEXT_PROMPT: Final[str] = '## Memories\nConsider the following memories  
when answering user questions:'
```

AzureAISearchSettings Class

Settings for Azure AI Search Context Provider with auto-loading from environment.

The settings are first loaded from environment variables with the prefix '>>AZURE_SEARCH_<<'.
If the environment variables are not found, the settings can be loaded from a .env file.

Initialize the settings class.

JSON

```
{
  "title": "AzureAISearchSettings",
  "description": "Settings for Azure AI Search Context Provider with auto-loading from environment.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_SEARCH_'. \nIf the environment variables are not found, the settings can be loaded from a .env file.\n\nKeyword Args:\n    endpoint: Azure AI Search endpoint URL.\n    Can be set via environment variable AZURE_SEARCH_ENDPOINT.\n    index_name: Name of the search index.\n    Can be set via environment variable AZURE_SEARCH_INDEX_NAME.\n    knowledge_base_name: Name of an existing Knowledge Base (for agentic mode).\n    Can be set via environment variable AZURE_SEARCH_KNOWLEDGE_BASE_NAME.\n    api_key: API key for authentication (optional, use managed identity if not provided).\n    Can be set via environment variable AZURE_SEARCH_API_KEY.\n    env_file_path: If provided, the .env settings are read from this file path location.\n    env_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n    .. code-block:: python\n    from agent_framework_aisearch import AzureAISearchSettings\n    # Using environment variables\n    # Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net\n    # Set AZURE_SEARCH_INDEX_NAME=my-index\n    settings = AzureAISearchSettings()\n    # Or passing parameters directly\n    settings = AzureAISearchSettings(\n        endpoint=\"https://mysearch.search.windows.net\", \n        index_name=\"my-index\", \n    )\n    # Or loading from a .env file\n    settings = AzureAISearchSettings(env_file_path=\"path/to/.env\"),
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Env File Path"
    },
    "env_file_encoding": {
      "anyOf": [
```

```
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": "utf-8",
    "title": "Env File Encoding"
},
"endpoint": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Endpoint"
},
"index_name": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Index Name"
},
"knowledge_base_name": {
    "anyOf": [
        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Knowledge Base Name"
},
"api_key": {
    "anyOf": [
        {
            "format": "password",
            "type": "string",
            "writeOnly": true
        },
        {
            "type": "null"
        }
    ]
}
```

```
        }
    ],
    "default": null,
    "title": "Api Key"
}
}
```

Constructor

Python
pydantic settings AzureAISearchSettings

Keyword-Only Parameters

 Expand table

Name	Description
endpoint	Azure AI Search endpoint URL. Can be set via environment variable AZURE_SEARCH_ENDPOINT. Default value: None
index_name	Name of the search index. Can be set via environment variable AZURE_SEARCH_INDEX_NAME. Default value: None
knowledge_base_name	Name of an existing Knowledge Base (for agentic mode). Can be set via environment variable AZURE_SEARCH_KNOWLEDGE_BASE_NAME. Default value: None
api_key	API key for authentication (optional, use managed identity if not provided). Can be set via environment variable AZURE_SEARCH_API_KEY. Default value: None
env_file_path	If provided, the .env settings are read from this file path location. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python
<pre>from agent_framework_aisearch import AzureAISearchSettings</pre>

```

# Using environment variables
# Set AZURE_SEARCH_ENDPOINT=https://mysearch.search.windows.net
# Set AZURE_SEARCH_INDEX_NAME=my-index
settings = AzureAISearchSettings()

# Or passing parameters directly
settings = AzureAISearchSettings(
    endpoint="https://mysearch.search.windows.net",
    index_name="my-index",
)

# Or loading from a .env file
settings = AzureAISearchSettings(env_file_path="path/to/.env")

```

Methods

 Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the new method to set the <code>env_prefix</code> .
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
<code>dict</code>	
<code>from_orm</code>	
<code>json</code>	
<code>model_construct</code>	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed

	values are ignored. Because no validation is performed with a call to <code>model_construct()</code>, having <code>model_config.extra == 'forbid'</code> does not result in an error if extra values are passed, but they will be ignored.
<code>model_copy</code>	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's <code>[dict][object.dict]</code> attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of <code>[cached properties][functools.cached_property]</code>).
<code>model_dump</code>	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
<code>model_dump_json</code>	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <code>to_json</code> method.
<code>model_json_schema</code>	Generates a JSON schema for a model class.
<code>model_parametrized_name</code>	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModel.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <code>model_construct</code>. This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	

schema
schema_json
update_forward_refs
validate

`__init__`

Initialize the settings class.

Python
<code>__init__(**kwargs: Any) -> None</code>

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Override the `new` method to set the `env_prefix`.

Python
<code>static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings</code>

Parameters

[Expand table](#)

Name	Description
cls Required*	type [<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

[Expand table](#)

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
_fields_set	set[str] None Default value: None
values Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use `model_copy` instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python
<pre>dict(*, include: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, exclude: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]</pre>

Parameters

[Expand table](#)

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_orm

Python
from_orm(obj: Any) -> Self

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

 Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

 Expand table

Name	Description
_fields_set	set[str] None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

 Expand table

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int</code> <code>None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool</code> <code>None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <code>round_trip</code> parameter instead.
round_trip Required*	<code>bool</code> If <code>True</code> , dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. <code>False</code> /"none" ignores them, <code>True</code> /"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .

Name	Description
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
                  schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
                  Literal['validation', 'serialization'] = 'validation', *, union_format:
                  Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>by_alias</code>	<code>bool</code> Whether to use attribute aliases or not. Default value: True
<code>ref_template</code>	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
<code>union_format</code> Required*	<code>Literal['any_of', 'primitive_type_array']</code> The format to use when combining schemas from unions together. Can be one of: <code>'any_of'</code>: Use the anyOf keyword to combine schemas (the default). <code>'primitive_type_array'</code>: Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
<code>schema_generator</code>	<code>type[<xref:pydantic.json_schema.GenerateJsonSchema>]</code> To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
<code>mode</code>	<code>Literal['validation', 'serialization']</code> The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>union_format</code>	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
<code>params</code> Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model</i> [<i>str</i> , <i>int</i>], the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<code>model_post_init(context: Any, /) -> None</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	

Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python
<pre>model_rebuild(*, force: bool = False, raise_errors: bool = True, _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace None = None) -> bool None</pre>

Parameters

 Expand table

Name	Description
force Required*	<code>bool</code> Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	<code>bool</code> Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<code><xref:MappingNamespace> None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

 Expand table

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
<code>_types_namespace</code>	Default value: <code>None</code>

Returns

[↗](#) Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[↗](#) Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.
by_alias	<code>bool</code> <code>None</code>

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj	Any

Name	Description
Required*	

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]
```

Parameters

[↗](#) Expand table

Name	Description
<code>by_alias</code>	bool Default value: True
<code>ref_template</code>	str Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
by_alias Required*	bool
ref_template Required*	str
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
localns Required*	Any

Returns

 Expand table

Type	Description
None	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

 Expand table

Name	Description
value Required*	Any

Returns

 Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

 Expand table

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

 Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | None = None
```

endpoint

Python

```
field endpoint: str | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_SEARCH_'
```

index_name

Python

```
field index_name: str | None = None
```

knowledge_base_name

Python

```
field knowledge_base_name: str | None = None
```

AzureAISettings Class

Azure AI Project settings.

The settings are first loaded from environment variables with the prefix '>>AZURE_AI_<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON

```
{
  "title": "AzureAISettings",
  "description": "Azure AI Project settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_AI_'. \nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the settings are missing.\n\nKeyword Args:\nproject_endpoint: The Azure AI Project endpoint URL.\nCan be set via environment variable AZURE_AI_PROJECT_ENDPOINT.\nmodel_deployment_name: The name of the model deployment to use.\nCan be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME.\nenv_file_path: If provided, the .env settings are read from this file path location.\nenv_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\nExamples:\n.. code-block:: python\nfrom agent_framework.azure import AzureAISettings\n# Using environment variables\n# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com\n# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4\nsettings = AzureAISettings()\n# Or passing parameters directly\nsettings = AzureAISettings(\n    project_endpoint=\"https://your-project.cognitiveservices.azure.com\", model_deployment_name=\"gpt-4\"\n)\n# Or loading from a .env file\nsettings = AzureAISettings(env_file_path=\"path/to/.env\"),
  "type": "object",
  "properties": {
    "env_file_path": {
      "anyOf": [
        {
          "type": "string"
        },
        {
          "type": "null"
        }
      ],
      "default": null,
      "title": "Env File Path"
    },
  },
}
```



```

"env_file_encoding": {
  "anyOf": [
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": "utf-8",
  "title": "Env File Encoding"
},
"project_endpoint": {
  "anyOf": [
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Project Endpoint"
},
"model_deployment_name": {
  "anyOf": [
    {
      "type": "string"
    },
    {
      "type": "null"
    }
  ],
  "default": null,
  "title": "Model Deployment Name"
}
}
}

```

Constructor

Python

pydantic settings AzureAISettings

Keyword-Only Parameters

[Expand table](#)

Name	Description
project_endpoint	The Azure AI Project endpoint URL. Can be set via environment variable AZURE_AI_PROJECT_ENDPOINT. Default value: None
model_deployment_name	The name of the model deployment to use. Can be set via environment variable AZURE_AI_MODEL_DEPLOYMENT_NAME. Default value: None
env_file_path	If provided, the .env settings are read from this file path location. Default value: None
env_file_encoding	The encoding of the .env file, defaults to 'utf-8'. Default value: utf-8

Examples

Python

```
from agent_framework.azure import AzureAISettings

# Using environment variables
# Set AZURE_AI_PROJECT_ENDPOINT=https://your-project.cognitiveservices.azure.com
# Set AZURE_AI_MODEL_DEPLOYMENT_NAME=gpt-4
settings = AzureAISettings()

# Or passing parameters directly
settings = AzureAISettings(
    project_endpoint="https://your-project.cognitiveservices.azure.com",
    model_deployment_name="gpt-4"
)

# Or loading from a .env file
settings = AzureAISettings(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the new method to set the env_prefix.
<code>construct</code>	
<code>copy</code>	Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

dict

from_orm

json

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [*dict*][*object.dict*] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [*cached properties*][*functools.cached_property*]).

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

model_json_schema

Generates a JSON schema for a model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

model_post_init	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

`__init__`

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Returns

 Expand table

Type	Description
None	

__new__

Override the **new** method to set the env_prefix.

Python
<pre>static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings</pre>

Parameters

 Expand table

Name	Description
cls Required*	type [<xref:TSettings>]
args Required*	Any
kwargs Required*	Any

Returns

 Expand table

Type	Description
<xref:agent_framework._pydantic.TSettings>	

construct

Python
<pre>construct(_fields_set: set[str] None = None, **values: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

 Expand table

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
<code>include</code>	<code><xref:AbstractSetIntStr> <xref:MappingIntStrAny> None</code>

Name	Description
Required*	Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[🔗](#) Expand table

Name	Description
obj Required*	<code>Any</code>

Returns

[🔗](#) Expand table

Type	Description
<code>Self</code>	

json

Python
<code>json(*, include: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, exclude: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] bool] None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str</code>

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
exclude Required*	set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx>] bool Mapping[str, <xref:IncEx>] bool bool None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Name	Description
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

[Expand table](#)

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None

Name	Description
values Required*	Any Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.***dict***] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
mode Required*	Literal ['json', 'python'] str The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults	<code>bool</code>

Name	Description
Required*	Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
<code>by_alias</code>	<code>bool</code> Whether to use attribute aliases or not. Default value: True
<code>ref_template</code>	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
<code>union_format</code>	<code>Literal['any_of', 'primitive_type_array']</code>

Name	Description
Required*	The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type[<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: <i>GenerateJsonSchema</i>
mode	Literal['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

[Expand table](#)

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python
<pre>model_rebuild(*, force: bool = False, raise_errors: bool = True, _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace None = None) -> bool None</pre>

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors	bool

Name	Description
Required*	Whether to raise errors, defaults to <i>True</i> .
<code>_parent_namespace_depth</code> Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
<code>_types_namespace</code> Required*	<code><xref:MappingNamespace></code> <code>None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

 Expand table

Name	Description
<code>force</code>	Default value: <i>False</i>
<code>raise_errors</code>	Default value: <i>True</i>
<code>_parent_namespace_depth</code>	Default value: 2
<code>_types_namespace</code>	Default value: <i>None</i>

Returns

 Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding was required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python
<pre>model_validate_json(json_data: str bytes bytearray, *, strict: bool None = None, extra: Literal['allow', 'ignore', 'forbid'] None = None, context: Any None = None, by_alias: bool None = None, by_name: bool None = None) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
path Required*	<code>str</code> <code><xref:Path></code>
content_type Required*	<code>str</code> <code>None</code>
encoding Required*	<code>str</code>
proto Required*	<code><xref:DeprecatedParseProtocol></code> <code>None</code>
allow_pickle Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: <code>None</code>
encoding	Default value: <code>'utf8'</code>
proto	Default value: <code>None</code>
allow_pickle	Default value: <code>False</code>

Returns

 Expand table

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python
<code>parse_raw(b: str bytes, *, content_type: str None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol None = None, allow_pickle: bool = False) -> Self</code>

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes

Name	Description
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

schema

Python
<pre>schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Default value: True
ref_template	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python
<pre>schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE, **kwargs: Any) -> str</pre>

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True

Name	Description
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python
update_forward_refs(**localns: Any) -> None

Parameters

[Expand table](#)

Name	Description
localns Required*	Any

Returns

[Expand table](#)

Type	Description
None	

validate

Python
validate(value: Any) -> Self

Parameters

[Expand table](#)

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

env_file_encoding

--

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_AI_'
```

model_deployment_name

Python

```
field model_deployment_name: str | None = None
```

project_endpoint

Python

```
field project_endpoint: str | None = None
```

AzureOpenAIAssistantsClient Class

Azure OpenAI Assistants client.

Initialize an Azure OpenAI Assistants client.

Constructor

Python
<code>AzureOpenAIAssistantsClient()</code>

Keyword-Only Parameters

 Expand table

Name	Description
<code>deployment_name</code>	The Azure OpenAI deployment name for the model to use. Can also be set via environment variable <code>AZURE_OPENAI_CHAT_DEPLOYMENT_NAME</code> . Default value: None
<code>assistant_id</code>	The ID of an Azure OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None
<code>assistant_name</code>	The name to use when creating new assistants. Default value: None
<code>assistant_description</code>	The description to use when creating new assistants. Default value: None
<code>thread_id</code>	Default thread ID to use for conversations. Can be overridden by <code>conversation_id</code> property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
<code>api_key</code>	The API key to use. If provided will override the env vars or <code>.env</code> file value. Can also be set via environment variable <code>AZURE_OPENAI_API_KEY</code> . Default value: None
<code>endpoint</code>	The deployment endpoint. If provided will override the value in the env vars or <code>.env</code> file. Can also be set via environment variable <code>AZURE_OPENAI_ENDPOINT</code> . Default value: None
<code>base_url</code>	

Name	Description
	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential to use for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to environment variables. Default value: None
env_file_encoding	The encoding of the environment settings file. Default value: None

Examples

Python

```
from agent_framework.azure import AzureOpenAIAssistantsClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIAssistantsClient()

# Or passing parameters directly
client = AzureOpenAIAssistantsClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
```

```
api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIAssistantsClient(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize an Azure OpenAI Assistants client.
<code>__new__</code>	
<code>close</code>	Clean up any assistants we created.
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code> . All the same serialization rules apply as in <code>to_dict()</code> , including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Azure OpenAI Assistants client.

Python

```
__init__(*, deployment_name: str | None = None, assistant_id: str | None = None, assistant_name: str | None = None, assistant_description: str | None = None, thread_id: str | None = None, api_key: str | None = None, endpoint: str | None = None, base_url: str | None = None, api_version: str | None = None, ad_token: str | None = None, ad_token_provider: Callable[[], str | Awaitable[str]] | None = None, token_endpoint: str | None = None, credential: TokenCredential | None = None, default_headers: Mapping[str, str] | None = None, async_client: AsyncAzureOpenAI | None = None, env_file_path: str | None = None, env_file_encoding: str | None = None) -> None
```

Parameters

 Expand table

Name	Description
deployment_name Required*	<code>str</code> <code>None</code>
assistant_id Required*	<code>str</code> <code>None</code>
assistant_name Required*	<code>str</code> <code>None</code>
assistant_description Required*	<code>str</code> <code>None</code>
thread_id Required*	<code>str</code> <code>None</code>
api_key	<code>str</code> <code>None</code>

Name	Description
Required*	
endpoint Required*	str None
base_url Required*	str None
api_version Required*	str None
ad_token Required*	str None
ad_token_provider Required*	Callable[[], str Awaitable[str]] None
token_endpoint Required*	str None
credential Required*	<xref:TokenCredential> None
default_headers Required*	Mapping[str, str] None
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> None
env_file_path Required*	str None
env_file_encoding Required*	str None

Keyword-Only Parameters

[Expand table](#)

Name	Description
deployment_name	The Azure OpenAI deployment name for the model to use. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
assistant_id	The ID of an Azure OpenAI assistant to use. If not provided, a new assistant will be created (and deleted after the request). Default value: None

Name	Description
assistant_name	The name to use when creating new assistants. Default value: None
assistant_description	The description to use when creating new assistants. Default value: None
thread_id	Default thread ID to use for conversations. Can be overridden by conversation_id property when making a request. If not provided, a new thread will be created (and deleted after the request). Default value: None
api_key	The API key to use. If provided will override the env vars or .env file value. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential to use for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None

Name	Description
<code>env_file_path</code>	Use the environment settings file as a fallback to environment variables. Default value: None
<code>env_file_encoding</code>	The encoding of the environment settings file. Default value: None

Returns

 Expand table

Type	Description
<code>None</code>	

Examples

Python

```
from agent_framework.azure import AzureOpenAIAssistantsClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIAssistantsClient()

# Or passing parameters directly
client = AzureOpenAIAssistantsClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
    api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIAssistantsClient(env_file_path="path/to/.env")
```

__new__

Python

```
__new__(**kwargs)
```

close

Clean up any assistants we created.

Python

```
async close() -> None
```

Returns

[Expand table](#)

Type	Description
None	

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```
create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str | None = None, additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

 Expand table

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None

Name	Description
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None

Name	Description
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the <code>chat_client</code> <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient
```

```
# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python

from_dict()

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	<code>MutableMapping[str, Any]</code> The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	<code>MutableMapping[str, Any]</code> <code>None</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
<code>ValueError</code>	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

Python

```
from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
```

```

client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use

```

Middleware Context Injection - Agent execution context:

Python

```

from agent_framework._middleware import AgentRunContext

```



```

from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution

```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

`from_json()`

Positional-Only Parameters

[Expand table](#)

Name	Description
value	

Name	Description
Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<code><xref:agent_framework_serialization.TCI ass></code>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
None, top_p: float | None = None, user: str | None = None, additional_properties:
dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	<code>str ChatMessage list[str] list[ChatMessage]</code> The message or messages to send to the model.
allow_multiple_tool_calls Required*	<code>bool None</code>
frequency_penalty Required*	<code>float None</code>
logit_bias Required*	<code>dict[str int, float] None</code>
max_tokens Required*	<code>int None</code>
metadata Required*	<code>dict[str, Any] None</code>
model_id Required*	<code>str None</code>

Name	Description
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None

Name	Description
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async def get_streaming_response(messages: str | ChatMessage | list[str] |
    list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
    frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
    = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
    model_id: str | None = None, presence_penalty: float | None = None,
    response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str |
    Sequence[str] | None = None, store: bool | None = None, temperature: float |
    None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
    dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
    MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
    MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
    None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
    > AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None

Name	Description
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None
<code>tools</code>	The tools to use for the request. Default value: None
<code>top_p</code>	The nucleus sampling probability to use. Default value: None

Name	Description
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts `additional_properties` fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

 Expand table

Type	Description
<code>dict[str, Any]</code>	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str]</code> <code>None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_AZURE_API_VERSION

Python

```
DEFAULT_AZURE_API_VERSION: ClassVar[str] = '2024-05-01-preview'
```

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'openai'
```

AzureOpenAIChatClient Class

Azure OpenAI Chat completion class.

Initialize an Azure OpenAI Chat completion client.

Constructor

Python

```
AzureOpenAIChatClient()
```

Keyword-Only Parameters

[Expand table](#)

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (chat_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	

Name	Description
	The token endpoint to request an Azure token. Can also be set via environment variable <code>AZURE_OPENAI_TOKEN_ENDPOINT</code> . Default value: None
<code>credential</code>	The Azure credential for authentication. Default value: None
<code>default_headers</code>	The default headers mapping of string keys to string values for HTTP requests. Default value: None
<code>async_client</code>	An existing client to use. Default value: None
<code>env_file_path</code>	Use the environment settings file as a fallback to using env vars. Default value: None
<code>env_file_encoding</code>	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
<code>instruction_role</code>	The role to use for 'instruction' messages, for example, summarization prompts could use <i>developer</i> or <i>system</i> . Default value: None
<code>kwargs</code>	Other keyword parameters.

Examples

Python

```

from agent_framework.azure import AzureOpenAIChatClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIChatClient()

# Or passing parameters directly
client = AzureOpenAIChatClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
    api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIChatClient(env_file_path="path/to/.env")

```

Methods

<code>__init__</code>	Initialize an Azure OpenAI Chat completion client.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Azure OpenAI Chat completion client.

Python

```
__init__(*, api_key: str | None = None, deployment_name: str | None = None,
endpoint: str | None = None, base_url: str | None = None, api_version: str | None
= None, ad_token: str | None = None, ad_token_provider: Callable[[], str |
Awaitable[str]] | None = None, token_endpoint: str | None = None, credential:
TokenCredential | None = None, default_headers: Mapping[str, str] | None = None,
async_client: AsyncAzureOpenAI | None = None, env_file_path: str | None = None,
env_file_encoding: str | None = None, instruction_role: str | None = None,
**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
api_key Required*	<code>str</code> <code>None</code>
deployment_name Required*	<code>str</code> <code>None</code>
endpoint Required*	<code>str</code> <code>None</code>
base_url Required*	<code>str</code> <code>None</code>
api_version Required*	<code>str</code> <code>None</code>
ad_token Required*	<code>str</code> <code>None</code>
ad_token_provider Required*	<code>Callable[[], str Awaitable[str]]</code> <code>None</code>
token_endpoint Required*	<code>str</code> <code>None</code>
credential Required*	<code>TokenCredential</code> <code>None</code>
default_headers	<code>Mapping[str, str]</code> <code>None</code>

Name	Description
Required*	
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> None
env_file_path Required*	str None
env_file_encoding Required*	str None
instruction_role Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (chat_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider.

Name	Description
	Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable <code>AZURE_OPENAI_TOKEN_ENDPOINT</code> . Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use <i>developer</i> or <i>system</i> . Default value: None
kwargs	Other keyword parameters.

Returns

[Expand table](#)

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureOpenAIChatClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIChatClient()
```

```

# Or passing parameters directly
client = AzureOpenAIChatClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4",
    api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIChatClient(env_file_path="path/to/.env")

```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```

create_agent(*, id: str | None = None, name: str | None = None, description: str |
None = None, instructions: str | None = None, chat_message_store_factory:
Callable[[], ChatMessageStoreProtocol] | None = None, context_providers:
ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None,
middleware: AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext,
Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] |
FunctionMiddleware | Callable[[FunctionInvocationContext,
Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] |
ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]],
Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None,
conversation_id: str | None = None, frequency_penalty: float | None = None,
logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None,
metadata: dict[str, Any] | None = None, model_id: str | None = None,
presence_penalty: float | None = None, response_format: type[BaseModel] | None =
None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools:
ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =

```

```
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

[Expand table](#)

Name	Description
id Required*	<code>str</code> <code>None</code>
name Required*	<code>str</code> <code>None</code>
description Required*	<code>str</code> <code>None</code>
instructions Required*	<code>str</code> <code>None</code>
chat_message_store_factory Required*	<code>Callable[[], ChatMessageStoreProtocol]</code> <code>None</code>
context_providers Required*	<code>ContextProvider</code> <code>list[ContextProvider]</code> <code>AggregateContextProvider</code> <code>None</code>
middleware Required*	<code>AgentMiddleware</code> <code>Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]]</code> <code>FunctionMiddleware</code> <code>Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]]</code> <code>ChatMiddleware</code> <code>Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]</code> <code>list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]]</code> <code>None</code>
allow_multiple_tool_calls Required*	<code>bool</code> <code>None</code>
conversation_id Required*	<code>str</code> <code>None</code>
frequency_penalty Required*	<code>float</code> <code>None</code>
logit_bias	<code>dict[str int, float]</code> <code>None</code>

Name	Description
Required*	
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None

Name	Description
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the <code>chat_client</code> <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
from_dict()

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code>

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python
```

```

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected

```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async def get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
    allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
    None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
    None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
    presence_penalty: float | None = None, response_format: type[BaseModel] | None =
    None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
    bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
    Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
    ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
    Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
    None, top_p: float | None = None, user: str | None = None, additional_properties:
    dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens	int None

Name	Description
Required*	
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.

Name	Description
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties	dict[str, Any] None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None

Name	Description
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

 Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai.openai'
```

AzureOpenAIResponsesClient Class

Azure Responses completion class.

Initialize an Azure OpenAI Responses client.

Constructor

Python

```
AzureOpenAIResponsesClient()
```

Keyword-Only Parameters

[Expand table](#)

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (responses_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Currently, the base_url must end with "/openai/v1/". Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Currently, the api_version must be "preview". Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None
ad_token_provider	The Azure Active Directory token provider. Default value: None

Name	Description
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable <code>AZURE_OPENAI_TOKEN_ENDPOINT</code> . Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use <i>developer</i> or <i>system</i> . Default value: None
kwargs	Additional keyword arguments.

Examples

Python

```
from agent_framework.azure import AzureOpenAIResponsesClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME=gpt-4o
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIResponsesClient()

# Or passing parameters directly
client = AzureOpenAIResponsesClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4o",
    api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIResponsesClient(env_file_path="path/to/.env")
```

Methods

<code>__init__</code>	Initialize an Azure OpenAI Responses client.
<code>__new__</code>	
<code>create_agent</code>	Create a ChatAgent with this client. This is a convenience method that creates a ChatAgent instance with this chat client already configured.
<code>from_dict</code>	Create an instance from a dictionary with optional dependency injection. This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.
<code>from_json</code>	Create an instance from a JSON string. This is a convenience method that parses the JSON string using <code>json.loads()</code> and then calls <code>from_dict()</code> to reconstruct the object. All dependency injection capabilities are available through the <code>dependencies</code> parameter.
<code>get_response</code>	Get a response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>get_streaming_response</code>	Get a streaming response from a chat client. When both <code>chat_options</code> (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in <code>chat_options</code>. Tools from both sources are combined into a single list.
<code>service_url</code>	Get the URL of the service. Override this in the subclass to return the proper URL. If the service does not have a URL, return None.
<code>to_dict</code>	Convert the instance to a dictionary. Extracts <code>additional_properties</code> fields to the root level.
<code>to_json</code>	Convert the instance to a JSON string. This is a convenience method that calls <code>to_dict()</code> and then serializes the result using <code>json.dumps()</code>. All the same serialization rules apply as in <code>to_dict()</code>, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

`__init__`

Initialize an Azure OpenAI Responses client.

Python

```
__init__(*, api_key: str | None = None, deployment_name: str | None = None,
endpoint: str | None = None, base_url: str | None = None, api_version: str | None
= None, ad_token: str | None = None, ad_token_provider: Callable[[], str |
Awaitable[str]] | None = None, token_endpoint: str | None = None, credential:
TokenCredential | None = None, default_headers: Mapping[str, str] | None = None,
async_client: AsyncAzureOpenAI | None = None, env_file_path: str | None = None,
env_file_encoding: str | None = None, instruction_role: str | None = None,
**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
api_key Required*	<code>str</code> <code>None</code>
deployment_name Required*	<code>str</code> <code>None</code>
endpoint Required*	<code>str</code> <code>None</code>
base_url Required*	<code>str</code> <code>None</code>
api_version Required*	<code>str</code> <code>None</code>
ad_token Required*	<code>str</code> <code>None</code>
ad_token_provider Required*	<code>Callable[[], str Awaitable[str]]</code> <code>None</code>
token_endpoint Required*	<code>str</code> <code>None</code>
credential Required*	<code>TokenCredential</code> <code>None</code>
default_headers	<code>Mapping[str, str]</code> <code>None</code>

Name	Description
Required*	
async_client Required*	<xref:openai.lib.azure.AsyncAzureOpenAI> None
env_file_path Required*	str None
env_file_encoding Required*	str None
instruction_role Required*	str None
kwargs Required*	Any

Keyword-Only Parameters

 [Expand table](#)

Name	Description
api_key	The API key. If provided, will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
deployment_name	The deployment name. If provided, will override the value (responses_deployment_name) in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
endpoint	The deployment endpoint. If provided will override the value in the env vars or .env file. Can also be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
base_url	The deployment base URL. If provided will override the value in the env vars or .env file. Currently, the base_url must end with "/openai/v1/". Can also be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
api_version	The deployment API version. If provided will override the value in the env vars or .env file. Currently, the api_version must be "preview". Can also be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
ad_token	The Azure Active Directory token. Default value: None

Name	Description
ad_token_provider	The Azure Active Directory token provider. Default value: None
token_endpoint	The token endpoint to request an Azure token. Can also be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
credential	The Azure credential for authentication. Default value: None
default_headers	The default headers mapping of string keys to string values for HTTP requests. Default value: None
async_client	An existing client to use. Default value: None
env_file_path	Use the environment settings file as a fallback to using env vars. Default value: None
env_file_encoding	The encoding of the environment settings file, defaults to 'utf-8'. Default value: None
instruction_role	The role to use for 'instruction' messages, for example, summarization prompts could use <i>developer</i> or <i>system</i> . Default value: None
kwargs	Additional keyword arguments.

Returns

 Expand table

Type	Description
None	

Examples

Python

```
from agent_framework.azure import AzureOpenAIResponsesClient

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME=gpt-4o
# Set AZURE_OPENAI_API_KEY=your-key
client = AzureOpenAIResponsesClient()
```

```

# Or passing parameters directly
client = AzureOpenAIResponsesClient(
    endpoint="https://your-endpoint.openai.azure.com", deployment_name="gpt-4o", api_key="your-key"
)

# Or loading from a .env file
client = AzureOpenAIResponsesClient(env_file_path="path/to/.env")

```

__new__

Python

```
__new__(**kwargs)
```

create_agent

Create a ChatAgent with this client.

This is a convenience method that creates a ChatAgent instance with this chat client already configured.

Python

```

create_agent(*, id: str | None = None, name: str | None = None, description: str | None = None, instructions: str | None = None, chat_message_store_factory: Callable[[], ChatMessageStoreProtocol] | None = None, context_providers: ContextProvider | list[ContextProvider] | AggregateContextProvider | None = None, middleware: AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] | list[AgentMiddleware | Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] | FunctionMiddleware | Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] | ChatMiddleware | Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] | None = None, allow_multiple_tool_calls: bool | None = None, conversation_id: str | None = None, frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None, model_id: str | None = None, presence_penalty: float | None = None, response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str | Sequence[str] | None = None, store: bool | None = None, temperature: float | None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] | dict[str, Any] | None = 'auto', tools: ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =

```

```
None, top_p: float | None = None, user: str | None = None,
additional_chat_options: dict[str, Any] | None = None, **kwargs: Any) -> ChatAgent
```

Parameters

 Expand table

Name	Description
id Required*	str None
name Required*	str None
description Required*	str None
instructions Required*	str None
chat_message_store_factory Required*	Callable[[], ChatMessageStoreProtocol] None
context_providers Required*	ContextProvider list[ContextProvider] AggregateContextProvider None
middleware Required*	AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]] list[AgentMiddleware Callable[[AgentRunContext, Callable[[AgentRunContext], Awaitable[None]]], Awaitable[None]] FunctionMiddleware Callable[[FunctionInvocationContext, Callable[[FunctionInvocationContext], Awaitable[None]]], Awaitable[None]] ChatMiddleware Callable[[ChatContext, Callable[[ChatContext], Awaitable[None]]], Awaitable[None]]] None
allow_multiple_tool_calls Required*	bool None
conversation_id Required*	str None
frequency_penalty Required*	float None
logit_bias	dict[str int, float] None

Name	Description
Required*	
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_chat_options Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

Name	Description
id	The unique identifier for the agent. Will be created automatically if not provided. Default value: None
name	The name of the agent. Default value: None
description	A brief description of the agent's purpose. Default value: None
instructions	Optional instructions for the agent. These will be put into the messages sent to the chat client service as a system message. Default value: None
chat_message_store_factory	Factory function to create an instance of ChatMessageStoreProtocol. If not provided, the default in-memory store will be used. Default value: None
context_providers	Context providers to include during agent invocation. Default value: None
middleware	List of middleware to intercept agent and function invocations. Default value: None
allow_multiple_tool_calls	Whether to allow multiple tool calls per agent turn. Default value: None
conversation_id	The conversation ID to associate with the agent's messages. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None

Name	Description
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default value: "auto"
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_chat_options	A dictionary of other values that will be passed through to the chat_client <code>get_response</code> and <code>get_streaming_response</code> methods. This can be used to pass provider specific parameters. Default value: None
kwargs	Any additional keyword arguments. Will be stored as <code>additional_properties</code> .

Returns

[Expand table](#)

Type	Description
ChatAgent	A ChatAgent instance configured with this chat client.

Examples

Python

```
from agent_framework.clients import OpenAIChatClient

# Create a client
client = OpenAIChatClient(model_id="gpt-4")

# Create an agent using the convenience method
agent = client.create_agent(
    name="assistant", instructions="You are a helpful assistant.",
    temperature=0.7
)

# Run the agent
response = await agent.run("Hello!")
```

from_dict

Create an instance from a dictionary with optional dependency injection.

This method reconstructs an object from its dictionary representation, automatically handling type conversion and dependency injection. It supports three patterns of dependency injection to handle different scenarios where external dependencies need to be provided at deserialization time.

Python
from_dict()

Positional-Only Parameters

[Expand table](#)

Name	Description
value Required*	

Parameters

[Expand table](#)

Name	Description
value Required*	<code>MutableMapping[str, Any]</code>

Name	Description
	The dictionary containing the instance data (positional-only). Must include a 'type' field matching the class type identifier.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

[Expand table](#)

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. The structure varies based on injection pattern: • Simple injection: <code>{"<type>": {"<parameter>": value}}</code> • Dict parameter injection: <code>{"<type>": {"<dict-parameter>": {"<key>": value}}}</code> • Instance-specific injection: <code>{"<type>": {"<field>:<value>": {"<parameter>": value}}}</code> Default value: None

Returns

[Expand table](#)

Type	Description
<code><xref:agent_framework_serialization.TClass></code>	New instance of the class with injected dependencies.

Exceptions

[Expand table](#)

Type	Description
ValueError	If the 'type' field in the data doesn't match the class type identifier.

Examples

Simple Client Injection - OpenAI client dependency injection:

```
Python
```

```

from agent_framework.openai import OpenAIChatClient
from openai import AsyncOpenAI

# OpenAI chat client requires an AsyncOpenAI client instance
# The client is marked as INJECTABLE = {"client"} in OpenAIBase

# Serialized data contains only the model configuration
client_data = {
    "type": "open_ai_chat_client",
    "model_id": "gpt-4o-mini",
    # client is excluded from serialization
}

# Provide the OpenAI client during deserialization
openai_client = AsyncOpenAI(api_key="your-api-key")
dependencies = {"open_ai_chat_client": {"client": openai_client}}

# The chat client is reconstructed with the OpenAI client injected
chat_client = OpenAIChatClient.from_dict(client_data,
dependencies=dependencies)
# Now ready to make API calls with the injected client

```

Function Injection for Tools - AIFunction runtime dependency:

Python

```

from agent_framework import AIFunction
from typing import Annotated

# Define a function to be wrapped
async def get_current_weather(location: Annotated[str, "The city name"]) ->
str:
    # In real implementation, this would call a weather API
    return f"Current weather in {location}: 72°F and sunny"

# AIFunction has INJECTABLE = {"func"}
function_data = {
    "type": "ai_function",
    "name": "get_weather",
    "description": "Get current weather for a location",
    # func is excluded from serialization
}

# Inject the actual function implementation during deserialization
dependencies = {"ai_function": {"func": get_current_weather}}

# Reconstruct the AIFunction with the callable injected

```

```
weather_func = AIFunction.from_dict(function_data, dependencies=dependencies)
# The function is now callable and ready for agent use
```

Middleware Context Injection - Agent execution context:

Python

```
from agent_framework._middleware import AgentRunContext
from agent_framework import BaseAgent

# AgentRunContext has INJECTABLE = {"agent", "result"}
context_data = {
    "type": "agent_run_context",
    "messages": [{"role": "user", "text": "Hello"}],
    "is_streaming": False,
    "metadata": {"session_id": "abc123"},
    # agent and result are excluded from serialization
}

# Inject agent and result during middleware processing
my_agent = BaseAgent(name="test-agent")
dependencies = {
    "agent_run_context": {
        "agent": my_agent,
        "result": None, # Will be populated during execution
    }
}

# Reconstruct context with agent dependency for middleware chain
context = AgentRunContext.from_dict(context_data, dependencies=dependencies)
# Middleware can now access context.agent and process the execution
```

This injection system allows the agent framework to maintain clean separation between serializable configuration and runtime dependencies like API clients, functions, and execution contexts that cannot or should not be persisted.

from_json

Create an instance from a JSON string.

This is a convenience method that parses the JSON string using `json.loads()` and then calls `from_dict()` to reconstruct the object. All dependency injection capabilities are available through the `dependencies` parameter.

Python

```
from_json()
```

Positional-Only Parameters

 Expand table

Name	Description
value Required*	

Parameters

 Expand table

Name	Description
value Required*	str The JSON string containing the instance data (positional-only). Must be valid JSON that deserializes to a dictionary with a 'type' field.
dependencies Required*	MutableMapping[str, Any] None

Keyword-Only Parameters

 Expand table

Name	Description
dependencies	A nested dictionary mapping type identifiers to their injectable dependencies. See from_dict for detailed structure and examples of the three injection patterns (simple, dict parameter, and instance-specific). Default value: None

Returns

 Expand table

Type	Description
<xref:agent_framework._serialization.TCI ass>	New instance of the class with any specified dependencies injected.

Exceptions

 Expand table

Type	Description
JSONDecodeError	If the JSON string is malformed.
ValueError	If the parsed data doesn't contain a valid 'type' field.

get_response

Get a response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async def get_response(messages: str | ChatMessage | list[str] | list[ChatMessage], *,
    allow_multiple_tool_calls: bool | None = None, frequency_penalty: float | None =
    None, logit_bias: dict[str | int, float] | None = None, max_tokens: int | None =
    None, metadata: dict[str, Any] | None = None, model_id: str | None = None,
    presence_penalty: float | None = None, response_format: type[BaseModel] | None =
    None, seed: int | None = None, stop: str | Sequence[str] | None = None, store:
    bool | None = None, temperature: float | None = None, tool_choice: ToolMode |
    Literal['auto', 'required', 'none'] | dict[str, Any] | None = None, tools:
    ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any] |
    Sequence[ToolProtocol | Callable[[...], Any] | MutableMapping[str, Any]] | None =
    None, top_p: float | None = None, user: str | None = None, additional_properties:
    dict[str, Any] | None = None, **kwargs: Any) -> ChatResponse
```

Parameters

 Expand table

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens	int None

Name	Description
Required*	
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties Required*	dict[str, Any] None
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
allow_multiple_tool_calls	Whether to allow multiple tool calls in a single response. Default value: None
frequency_penalty	The frequency penalty to use. Default value: None
logit_bias	The logit bias to use. Default value: None
max_tokens	The maximum number of tokens to generate. Default value: None
metadata	Additional metadata to include in the request. Default value: None
model_id	The model_id to use for the agent. Default value: None
presence_penalty	The presence penalty to use. Default value: None
response_format	The format of the response. Default value: None
seed	The random seed to use. Default value: None
stop	The stop sequence(s) for the request. Default value: None
store	Whether to store the response. Default value: None
temperature	The sampling temperature to use. Default value: None
tool_choice	The tool choice for the request. Default is <i>auto</i> . Default value: None
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters.

Name	Description
	Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
ChatResponse	A chat response from the model_id.

get_streaming_response

Get a streaming response from a chat client.

When both `chat_options` (in kwargs) and individual parameters are provided, the individual parameters take precedence and override the corresponding values in `chat_options`. Tools from both sources are combined into a single list.

Python

```
async get_streaming_response(messages: str | ChatMessage | list[str] |
list[ChatMessage], *, allow_multiple_tool_calls: bool | None = None,
frequency_penalty: float | None = None, logit_bias: dict[str | int, float] | None
= None, max_tokens: int | None = None, metadata: dict[str, Any] | None = None,
model_id: str | None = None, presence_penalty: float | None = None,
response_format: type[BaseModel] | None = None, seed: int | None = None, stop: str
| Sequence[str] | None = None, store: bool | None = None, temperature: float |
None = None, tool_choice: ToolMode | Literal['auto', 'required', 'none'] |
dict[str, Any] | None = None, tools: ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any] | Sequence[ToolProtocol | Callable[[...], Any] |
MutableMapping[str, Any]] | None = None, top_p: float | None = None, user: str |
None = None, additional_properties: dict[str, Any] | None = None, **kwargs: Any) -
> AsyncIterable[ChatResponseUpdate]
```

Parameters

[Expand table](#)

Name	Description
messages Required*	str ChatMessage list[str] list[ChatMessage] The message or messages to send to the model.

Name	Description
allow_multiple_tool_calls Required*	bool None
frequency_penalty Required*	float None
logit_bias Required*	dict[str int, float] None
max_tokens Required*	int None
metadata Required*	dict[str, Any] None
model_id Required*	str None
presence_penalty Required*	float None
response_format Required*	type[<xref:pydantic.main.BaseModel>] None
seed Required*	int None
stop Required*	str Sequence[str] None
store Required*	bool None
temperature Required*	float None
tool_choice Required*	ToolMode Literal['auto', 'required', 'none'] dict[str, typing.Any] None
tools Required*	ToolProtocol Callable[[...], Any] MutableMapping[str, Any] Sequence[ToolProtocol Callable[[...], Any] MutableMapping[str, Any]] None
top_p Required*	float None
user Required*	str None
additional_properties	dict[str, Any] None

Name	Description
Required*	
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>allow_multiple_tool_calls</code>	Whether to allow multiple tool calls in a single response. Default value: None
<code>frequency_penalty</code>	The frequency penalty to use. Default value: None
<code>logit_bias</code>	The logit bias to use. Default value: None
<code>max_tokens</code>	The maximum number of tokens to generate. Default value: None
<code>metadata</code>	Additional metadata to include in the request. Default value: None
<code>model_id</code>	The <code>model_id</code> to use for the agent. Default value: None
<code>presence_penalty</code>	The presence penalty to use. Default value: None
<code>response_format</code>	The format of the response. Default value: None
<code>seed</code>	The random seed to use. Default value: None
<code>stop</code>	The stop sequence(s) for the request. Default value: None
<code>store</code>	Whether to store the response. Default value: None
<code>temperature</code>	The sampling temperature to use. Default value: None
<code>tool_choice</code>	The tool choice for the request. Default is <i>auto</i> . Default value: None

Name	Description
tools	The tools to use for the request. Default value: None
top_p	The nucleus sampling probability to use. Default value: None
user	The user to associate with the request. Default value: None
additional_properties	Additional properties to include in the request. Can be used for provider-specific parameters. Default value: None
kwargs	Any additional keyword arguments. May include <code>chat_options</code> which provides base values that can be overridden by direct parameters.

Returns

[Expand table](#)

Type	Description
<code>AsyncIterable[ChatResponseUpdate]</code>	

service_url

Get the URL of the service.

Override this in the subclass to return the proper URL. If the service does not have a URL, return None.

Python
<code>service_url() -> str</code>

Returns

[Expand table](#)

Type	Description
<code>str</code>	The service URL or 'Unknown' if not implemented.

to_dict

Convert the instance to a dictionary.

Extracts additional_properties fields to the root level.

Python

```
to_dict(*, exclude: set[str] | None = None, exclude_none: bool = True) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
exclude Required*	set[str] None
exclude_none Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
exclude	Set of field names to exclude from serialization. Default value: None
exclude_none	Whether to exclude None values from the output. Defaults to True. Default value: True

Returns

 Expand table

Type	Description
dict[str, Any]	Dictionary representation of the instance.

to_json

Convert the instance to a JSON string.

This is a convenience method that calls `to_dict()` and then serializes the result using `json.dumps()`. All the same serialization rules apply as in `to_dict()`, including automatic exclusion of injectable dependencies and deep serialization of nested objects.

Python

```
to_json(*, exclude: set[str] | None = None, exclude_none: bool = True, **kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
exclude Required*	<code>set[str] None</code>
exclude_none Required*	<code>bool</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
exclude	Additional field names to exclude from serialization. Default value: <code>None</code>
exclude_none	Whether to exclude <code>None</code> values from the output. Defaults to <code>True</code> . Default value: <code>True</code>
**kwargs	Additional keyword arguments passed through to <code>json.dumps()</code> . Common options include <code>indent</code> for pretty-printing and <code>ensure_ascii</code> for Unicode handling.

Returns

[Expand table](#)

Type	Description
<code>str</code>	JSON string representation of the instance.

Attributes

DEFAULT_EXCLUDE

Python

```
DEFAULT_EXCLUDE: ClassVar[set[str]] = {'additional_properties'}
```

FILE_SEARCH_MAX_RESULTS

Python

```
FILE_SEARCH_MAX_RESULTS: int = 50
```

INJECTABLE

Python

```
INJECTABLE: ClassVar[set[str]] = {'client'}
```

OTEL_PROVIDER_NAME

Python

```
OTEL_PROVIDER_NAME: ClassVar[str] = 'azure.ai.openai'
```


AzureOpenAISettings Class

AzureOpenAI model settings.

The settings are first loaded from environment variables with the prefix '>>AZURE_OPENAI_<<'. If the environment variables are not found, the settings can be loaded from a .env file with the encoding 'utf-8'. If the settings are not found in the .env file, the settings are ignored; however, validation will fail alerting that the settings are missing.

Initialize the settings class.

JSON

```
{
  "title": "AzureOpenAISettings",
  "description": "AzureOpenAI model settings.\n\nThe settings are first loaded from environment variables with the prefix 'AZURE_OPENAI_'.\nIf the environment variables are not found, the settings can be loaded from a .env file\nwith the encoding 'utf-8'. If the settings are not found in the .env file, the settings\nare ignored; however, validation will fail alerting that the settings are missing.\n\nKeyword Args:\n    endpoint: The endpoint of the Azure deployment. This value\n              can be found in the Keys & Endpoint section when examining\nyour resource from the Azure portal, the endpoint should end in openai.azure.com.\nIf both base_url and endpoint are supplied, base_url will be used.\n              Can be set via environment variable AZURE_OPENAI_ENDPOINT.\n    chat_deployment_name: The name of the Azure Chat deployment. This value\n                          will correspond to the custom name you chose for your deployment\n                          when you deployed a model. This value can be found under\n                          Resource Management > Deployments in the Azure portal or,\nalternatively,\n                          under Management > Deployments in Azure AI Foundry.\nCan be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME.\n    responses_deployment_name: The name of the Azure Responses deployment. This value\n                              will correspond to the custom name you chose for your deployment\n                              when you deployed a model. This value can be found under\n                              Resource Management > Deployments in the Azure portal or, alternatively,\n                              under Management > Deployments in Azure AI Foundry.\n                              Can be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME.\n    api_key: The API key for the Azure deployment. This value can be\n            found in the Keys & Endpoint section when examining your resource in\n            the Azure portal. You can use either KEY1 or KEY2.\n            Can be set via environment variable AZURE_OPENAI_API_KEY.\n    api_version: The API version to use. The default value is `default_api_version`.\n                Can be set via environment variable AZURE_OPENAI_API_VERSION.\n    base_url: The url of the Azure deployment. This value\n             can be found in the Keys & Endpoint section when examining\n             your resource from the Azure portal, the base_url consists of the endpoint,\n             followed by /openai/deployments/{deployment_name}/,\n             use endpoint if you only want to supply the endpoint.\n             Can be set via environment variable AZURE_OPENAI_BASE_URL.\n    token_endpoint: The token endpoint to use to retrieve the authentication token.\n                   The default value is `default_token_endpoint`.\n}
```

Can be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT.\n

default_api_version: The default API version to use if not specified.\n The default value is \"2024-10-21\".\n

default_token_endpoint: The default token endpoint to use if not specified.\n The default value is \"https://cognitiveservices.azure.com/.default\".\n

env_file_path: The path to the .env file to load settings from.\n

env_file_encoding: The encoding of the .env file, defaults to 'utf-8'.\n\n

Examples:\n .. code-block:: python\n\n from agent_framework.azure import AzureOpenAISettings\n\n # Using environment variables\n # Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com\n # Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4\n # Set AZURE_OPENAI_API_KEY=your-key\n settings = AzureOpenAISettings()\n\n # Or passing parameters directly\n settings = AzureOpenAISettings(\n endpoint=\"https://your-endpoint.openai.azure.com\", chat_deployment_name=\"gpt-4\", api_key=\"your-key\")\n\n # Or loading from a .env file\n settings = AzureOpenAISettings(env_file_path=\"path/to/.env\"),

```

    "type": "object",
    "properties": {
      "env_file_path": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "title": "Env File Path"
      },
      "env_file_encoding": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": "utf-8",
        "title": "Env File Encoding"
      },
      "chat_deployment_name": {
        "anyOf": [
          {
            "type": "string"
          },
          {
            "type": "null"
          }
        ],
        "default": null,
        "title": "Chat Deployment Name"
      },
      "responses_deployment_name": {
        "anyOf": [

```

```

        {
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Responses Deployment Name"
},
"endpoint": {
    "anyOf": [
        {
            "format": "uri",
            "maxLength": 2083,
            "minLength": 1,
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Endpoint"
},
"base_url": {
    "anyOf": [
        {
            "format": "uri",
            "maxLength": 2083,
            "minLength": 1,
            "type": "string"
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Base Url"
},
"api_key": {
    "anyOf": [
        {
            "format": "password",
            "type": "string",
            "writeOnly": true
        },
        {
            "type": "null"
        }
    ],
    "default": null,
    "title": "Api Key"
},
"api_version": {

```

```

        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Api Version"
    },
    "token_endpoint": {
        "anyOf": [
            {
                "type": "string"
            },
            {
                "type": "null"
            }
        ],
        "default": null,
        "title": "Token Endpoint"
    },
    "default_api_version": {
        "default": "2024-10-21",
        "title": "Default Api Version",
        "type": "string"
    },
    "default_token_endpoint": {
        "default": "https://cognitiveservices.azure.com/.default",
        "title": "Default Token Endpoint",
        "type": "string"
    }
}

```

Constructor

Python

pydantic settings AzureOpenAISettings

Keyword-Only Parameters

[Expand table](#)

Name	Description
endpoint	The endpoint of the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource from the Azure

Name	Description
	portal, the endpoint should end in openai.azure.com. If both base_url and endpoint are supplied, base_url will be used. Can be set via environment variable AZURE_OPENAI_ENDPOINT. Default value: None
chat_deployment_name	The name of the Azure Chat deployment. This value will correspond to the custom name you chose for your deployment when you deployed a model. This value can be found under Resource Management > Deployments in the Azure portal or, alternatively, under Management > Deployments in Azure AI Foundry. Can be set via environment variable AZURE_OPENAI_CHAT_DEPLOYMENT_NAME. Default value: None
responses_deployment_name	The name of the Azure Responses deployment. This value will correspond to the custom name you chose for your deployment when you deployed a model. This value can be found under Resource Management > Deployments in the Azure portal or, alternatively, under Management > Deployments in Azure AI Foundry. Can be set via environment variable AZURE_OPENAI_RESPONSES_DEPLOYMENT_NAME. Default value: None
api_key	The API key for the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource in the Azure portal. You can use either KEY1 or KEY2. Can be set via environment variable AZURE_OPENAI_API_KEY. Default value: None
api_version	The API version to use. The default value is <i>default_api_version</i> . Can be set via environment variable AZURE_OPENAI_API_VERSION. Default value: None
base_url	The url of the Azure deployment. This value can be found in the Keys & Endpoint section when examining your resource from the Azure portal, the base_url consists of the endpoint, followed by /openai/deployments/{deployment_name}/, use endpoint if you only want to supply the endpoint. Can be set via environment variable AZURE_OPENAI_BASE_URL. Default value: None
token_endpoint	The token endpoint to use to retrieve the authentication token. The default value is <i>default_token_endpoint</i> . Can be set via environment variable AZURE_OPENAI_TOKEN_ENDPOINT. Default value: None
default_api_version	The default API version to use if not specified. The default value is "2024-10-21". Default value: 2024-10-21
default_token_endpoint	

Name	Description
	The default token endpoint to use if not specified. The default value is " https://cognitiveservices.azure.com/.default ". Default value: <code>https://cognitiveservices.azure.com/.default</code>
<code>env_file_path</code>	The path to the <code>.env</code> file to load settings from. Default value: <code>None</code>
<code>env_file_encoding</code>	The encoding of the <code>.env</code> file, defaults to <code>'utf-8'</code> . Default value: <code>utf-8</code>

Examples

Python

```
from agent_framework.azure import AzureOpenAISettings

# Using environment variables
# Set AZURE_OPENAI_ENDPOINT=https://your-endpoint.openai.azure.com
# Set AZURE_OPENAI_CHAT_DEPLOYMENT_NAME=gpt-4
# Set AZURE_OPENAI_API_KEY=your-key
settings = AzureOpenAISettings()

# Or passing parameters directly
settings = AzureOpenAISettings(
    endpoint="https://your-endpoint.openai.azure.com", chat_deployment_name="gpt-4", api_key="your-key"
)

# Or loading from a .env file
settings = AzureOpenAISettings(env_file_path="path/to/.env")
```

Methods

 Expand table

<code>__init__</code>	Initialize the settings class.
<code>__new__</code>	Override the <code>new</code> method to set the <code>env_prefix</code> .
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <code>model_copy</code> instead. If you need <i>include</i> or <i>exclude</i> , use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

[dict](#)

[from_orm](#)

[get_azure_auth_token](#)

Retrieve a Microsoft Entra Auth Token for a given token endpoint for the use with Azure OpenAI.

The required role for the token is *Cognitive Services OpenAI Contributor*. The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None. The *token_endpoint* argument takes precedence over the *token_endpoint* attribute.

[json](#)

[model_construct](#)

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

[model_copy](#)

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.***dict***] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

[model_dump](#)

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

[model_dump_json](#)

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

[model_json_schema](#)

Generates a JSON schema for a model class.

<code>model_parametrized_name</code>	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModels.
<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Initialize the settings class.

Python

```
__init__(**kwargs: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
kwargs Required*	Any

Returns

 Expand table

Type	Description
None	

`__new__`

Override the `new` method to set the `env_prefix`.

Python
<pre>static __new__(cls: type[TSettings], *args: Any, **kwargs: Any) -> TSettings</pre>

Parameters

 Expand table

Name	Description
cls Required*	<code>type[<xref:TSettings>]</code>
args Required*	Any
kwargs Required*	Any

Returns

 Expand table

Type	Description
<code><xref:agent_framework_pydantic.TSettings></code>	

`construct`

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

 Expand table

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] |  
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str,  
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |  
bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] |  
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool |  
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
```

```
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>None</code>
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx>]</code> <code>bool</code> <code>Mapping[str, <xref:IncEx>]</code> <code>bool</code> <code>bool</code> <code>None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>
exclude_defaults Required*	<code>bool</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<pre>from_orm(obj: Any) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
obj Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

get_azure_auth_token

Retrieve a Microsoft Entra Auth Token for a given token endpoint for the use with Azure OpenAI.

The required role for the token is *Cognitive Services OpenAI Contributor*. The token endpoint may be specified as an environment variable, via the .env file or as an argument. If the token endpoint is not provided, the default is None. The *token_endpoint* argument takes precedence over the *token_endpoint* attribute.

Python
<pre>get_azure_auth_token(credential: TokenCredential, token_endpoint: str None = None, **kwargs: Any) -> str None</pre>

Parameters

 Expand table

Name	Description
credential Required*	TokenCredential The Azure AD credential to use.
token_endpoint	str None The token endpoint to use. Defaults to https://cognitiveservices.azure.com/.default  . Default value: None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
**kwargs	Additional keyword arguments to pass to the token retrieval method.

Returns

 Expand table

Type	Description
str None	The Azure token or None if the token could not be retrieved.

Exceptions

 Expand table

Type	Description
ServiceInitializationError	If the token endpoint is not provided.

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] |  
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str,
```

```
set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] |
bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] |
Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx
| bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool =
False, exclude_defaults: bool = False, exclude_none: bool = False, encoder:
Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool =
PydanticUndefined, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>by_alias</code>	Default value: False
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>encoder</code>	Default value: PydanticUndefined
<code>models_as_dict</code>	Default value: PydanticUndefined

Returns

[🔗](#) Expand table

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: <code>None</code>
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
<code>Self</code>	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's `[dict][object.dict]` attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of `[cached properties][functools.cached_property]`).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>update</code> Required*	<code>Mapping[str, Any] None</code> Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
<code>deep</code>	<code>bool</code>

Name	Description
Required*	Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

[Expand table](#)

Name	Description
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback	<code>Callable[[Any], Any] None</code>

Name	Description
Required*	A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
<code>serialize_as_any</code> Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>mode</code>	Default value: 'python'
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include:
set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx |
bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] |
Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None,
exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int,
IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] |
set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None
= None, context: Any | None = None, by_alias: bool | None = None, exclude_unset:
bool = False, exclude_defaults: bool = False, exclude_none: bool = False,
exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None =
None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to exclude from the JSON output.
context	<code>Any None</code>

Name	Description
Required*	Additional context to pass to the serializer.
by_alias Required*	<code>bool</code> <code>None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None

Name	Description
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code>

Name	Description
	Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

Python

model_parametrized_name(params: tuple[type[Any], ...]) -> str

Parameters

 Expand table

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

 Expand table

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

 Expand table

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

model_post_init(context: Any, /) -> None

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,
    _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =
    None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .

Name	Description
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
bool None	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated Pydantic model.

Exceptions

 Expand table

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
path Required*	<code>str</code> <code><xref:Path></code>
content_type Required*	<code>str</code> <code>None</code>
encoding Required*	<code>str</code>
proto Required*	<code><xref:DeprecatedParseProtocol></code> <code>None</code>
allow_pickle Required*	<code>bool</code>

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: <code>None</code>
encoding	Default value: <code>'utf8'</code>
proto	Default value: <code>None</code>
allow_pickle	Default value: <code>False</code>

Returns

 Expand table

Type	Description
Self	

parse_obj

Python
<pre>parse_obj(obj: Any) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python
<pre>parse_raw(b: str bytes, *, content_type: str None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol None = None, allow_pickle: bool = False) -> Self</pre>

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes

Name	Description
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

 Expand table

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

 Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]
```

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Default value: True
ref_template	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True

Name	Description
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[⌵](#) Expand table

Type	Description
str	

update_forward_refs

Python
<code>update_forward_refs(**localns: Any) -> None</code>

Parameters

[⌵](#) Expand table

Name	Description
localns Required*	Any

Returns

[⌵](#) Expand table

Type	Description
None	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

api_key

Python

```
field api_key: SecretStr | None = None
```

api_version

Python

```
field api_version: str | None = None
```

base_url

Python

```
field base_url: AnyUrl | None = None
```

chat_deployment_name

Python

```
field chat_deployment_name: str | None = None
```

default_api_version

Python

```
field default_api_version: str = '2024-10-21'
```

default_token_endpoint

Python

```
field default_token_endpoint: str = 'https://cognitiveservices.azure.com/.default'
```

endpoint

Python

```
field endpoint: AnyUrl | None = None
```

env_file_encoding

Python

```
field env_file_encoding: str | None = 'utf-8'
```

env_file_path

Python

```
field env_file_path: str | None = None
```

env_prefix

Python

```
env_prefix: ClassVar[str] = 'AZURE_OPENAI_'
```

responses_deployment_name

Python

```
field responses_deployment_name: str | None = None
```

token_endpoint

Python

```
field token_endpoint: str | None = None
```

DurableAI Agent Class

A durable agent implementation that uses entity methods to interact with agent entities.

This class implements AgentProtocol and provides methods to work with Azure Durable Functions orchestrations, which use generators and yield instead of async/await.

Key methods:

- `get_new_thread()`: Create a new conversation thread
- `run()`: Execute the agent and return a Task for yielding in orchestrations

Note: The `run()` method is NOT async. It returns a Task directly that must be yielded in orchestrations to wait for the entity call to complete.

Example usage in orchestration: `writer = app.get_agent(context, "WriterAgent") thread = writer.get_new_thread() # NOT yielded - returns immediately`

```
response = yield writer.run( # Yielded - waits for entity call message="Write a haiku about coding", thread=thread
```

```
)
```

Initialize the DurableAI Agent.

Constructor

Python

```
DurableAI Agent()
```

Parameters

[Expand table](#)

Name	Description
context Required*	The orchestration context
agent_name Required*	Name of the agent (used to construct entity ID)

Methods

 Expand table

<code>__init__</code>	Initialize the DurableAIAgent.
<code>__new__</code>	
<code>get_new_thread</code>	Create a new agent thread for this orchestration instance. Each call creates a unique thread with its own conversation context. The session ID is deterministic (uses <code>context.new_uuid()</code>) to ensure orchestration replay works correctly.
<code>run</code>	Execute the agent with messages and return an AgentTask for orchestrations. This method implements AgentProtocol and returns an AgentTask (subclass of TaskBase) that can be yielded in Durable Functions orchestrations. The task's result will be a typed AgentRunResponse.
<code>run_stream</code>	Run the agent with streaming (not supported for durable agents).

`__init__`

Initialize the DurableAIAgent.

Python

```
__init__(context: Any, agent_name: str)
```

Parameters

 Expand table

Name	Description
<code>context</code> Required*	<code>Any</code> The orchestration context
<code>agent_name</code> Required*	<code>str</code> Name of the agent (used to construct entity ID)

`__new__`

Python

```
__new__(**kwargs)
```

get_new_thread

Create a new agent thread for this orchestration instance.

Each call creates a unique thread with its own conversation context. The session ID is deterministic (uses context.new_uuid()) to ensure orchestration replay works correctly.

Python

```
get_new_thread(**kwargs: Any) -> AgentThread
```

Parameters

 Expand table

Name	Description
kwargs Required*	Any

Returns

 Expand table

Type	Description
AgentThread	A new AgentThread instance with a unique session ID

run

Execute the agent with messages and return an AgentTask for orchestrations.

This method implements AgentProtocol and returns an AgentTask (subclass of TaskBase) that can be yielded in Durable Functions orchestrations. The task's result will be a typed AgentRunResponse.

Python

```
run(messages: str | ChatMessage | list[str] | list[ChatMessage] | None = None, *,
thread: AgentThread | None = None, response_format: type[BaseModel] | None = None,
**kwargs: Any) -> AgentTask
```

Parameters

[Expand table](#)

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None The message(s) to send to the agent Default value: None
thread Required*	AgentThread None Optional agent thread for conversation context
response_format Required*	type[<xref:pydantic.main.BaseModel>] None Optional Pydantic model for response parsing
**kwargs Required*	Any Additional arguments (enable_tool_calls)

Keyword-Only Parameters

[Expand table](#)

Name	Description
thread	Default value: None
response_format	Default value: None

Returns

[Expand table](#)

Type	Description
<xref:agent_framework_azurefunctions_orchestration.AgentTask>	An AgentTask that resolves to an AgentRunResponse when yielded

Examples

```
@app.orchestration_trigger(context_name="context") def my_orchestration(context):
```

```
    agent = app.get_agent(context, "MyAgent") thread = agent.get_new_thread() response =  
    yield agent.run("Hello", thread=thread)
```

response is typed as AgentRunResponse

run_stream

Run the agent with streaming (not supported for durable agents).

Python

```
run_stream(messages: str | ChatMessage | list[str] | list[ChatMessage] | None =  
None, *, thread: AgentThread | None = None, **kwargs: Any) ->  
AsyncIterator[AgentRunResponseUpdate]
```

Parameters

 Expand table

Name	Description
messages	str ChatMessage list[str] list[ChatMessage] None Default value: None
thread Required*	AgentThread None
kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
thread	Default value: None

Returns

 Expand table

Type	Description
AsyncIterator [AgentRunResponseUpdate]	

Exceptions

 Expand table

Type	Description
<code>NotImplementedError</code>	Streaming is not supported for durable agents.

Attributes

description

Get the description of the agent.

display_name

Get the display name of the agent.

id

Get the unique identifier for this agent.

name

Get the name of the agent.

chatkit Package

Classes

 Expand table

ThreadItemConverter	Helper class to convert ChatKit thread items to Agent Framework ChatMessage objects. This class provides a base implementation for converting ChatKit thread items to Agent Framework messages. It can be extended to handle attachments, @-mentions, hidden context items, and custom thread item formats. Initialize the converter.
----------------------------	--

Functions

simple_to_agent_input

Helper function that uses the default ThreadItemConverter.

This function provides a quick way to get started with ChatKit integration without needing to create a custom ThreadItemConverter instance.

Python
<pre>async simple_to_agent_input(thread_items: Sequence[UserMessageItem AssistantMessageItem ClientToolCallItem WidgetItem GeneratedImageItem WorkflowItem TaskItem HiddenContextItem SDKHiddenContextItem EndOfTurnItem] UserMessageItem AssistantMessageItem ClientToolCallItem WidgetItem GeneratedImageItem WorkflowItem TaskItem HiddenContextItem SDKHiddenContextItem EndOfTurnItem) -> list[ChatMessage]</pre>

Parameters

 Expand table

Name	Description
thread_items Required*	Sequence[<xref:chatkit.types.UserMessageItem> <xref:chatkit.types.AssistantMessageItem> <xref:chatkit.types.ClientToolCallItem> <xref:chatkit.types.WidgetItem> <xref:chatkit.types.GeneratedImageItem> <xref:chatkit.types.WorkflowItem> <xref:chatkit.types.TaskItem> <xref:chatkit.types.HiddenContextItem> <xref:chatkit.types.SDKHiddenContextItem>]

Name	Description
	<xref:chatkit.types.EndOfTurnItem>] <xref:chatkit.types.UserMessageItem> <xref:chatkit.types.AssistantMessageItem> <xref:chatkit.types.ClientToolCallItem> <xref:chatkit.types.WidgetItem> <xref:chatkit.types.GeneratedImageItem> <xref:chatkit.types.WorkflowItem> <xref:chatkit.types.TaskItem> <xref:chatkit.types.HiddenContextItem> <xref:chatkit.types.SDKHiddenContextItem> <xref:chatkit.types.EndOfTurnItem> A single ThreadItem or a sequence of ThreadItems to convert.

Returns

 Expand table

Type	Description
<code>list[ChatMessage]</code>	A list of ChatMessage objects that can be sent to an Agent Framework agent.

Examples

Python

```
from agent_framework_chatkit import simple_to_agent_input

# Convert a single item
messages = await simple_to_agent_input(user_message_item)

# Convert multiple items
messages = await simple_to_agent_input([user_message_item,
assistant_message_item, task_item])
```

stream_agent_response

Convert a streamed AgentRunResponseUpdate from Agent Framework to ChatKit events.

This helper function takes a stream of AgentRunResponseUpdate objects from a Microsoft Agent Framework agent and converts them to ChatKit ThreadStreamEvent objects that can be consumed by the ChatKit UI.

The function supports real-time token-by-token streaming by emitting ThreadItemUpdated events with AssistantMessageContentPartTextDelta for each text chunk as it arrives from the agent.

Python

```
async stream_agent_response(response_stream:
AsyncIterable[AgentRunResponseUpdate], thread_id: str, generate_id:
Callable[[str], str] | None = None) -> AsyncIterator[ThreadCreatedEvent |
ThreadUpdatedEvent | ThreadItemDoneEvent | ThreadItemAddedEvent |
ThreadItemUpdatedEvent | ThreadItemRemovedEvent | ThreadItemReplacedEvent |
StreamOptionsEvent | ProgressUpdateEvent | ClientEffectEvent | ErrorEvent |
NoticeEvent]
```

Parameters

 Expand table

Name	Description
response_stream Required*	AsyncIterable[AgentRunResponseUpdate] An async iterable of AgentRunResponseUpdate objects from an Agent Framework agent.
thread_id Required*	str The ChatKit thread ID for the conversation.
generate_id	Callable[[str], str] None Optional function to generate IDs for ChatKit items. If not provided, simple incremental IDs will be used. Default value: None

Returns

 Expand table

Type	Description
AsyncIterator [<xref:chatkit.types.ThreadCreatedEvent> <xref:chatkit.types.ThreadUpdatedEvent> <xref:chatkit.types.ThreadItemDoneEvent> <xref:chatkit.types.ThreadItemAddedEvent> <xref:chatkit.types.ThreadItemUpdatedEvent> <xref:chatkit.types.ThreadItemRemovedEvent> <xref:chatkit.types.ThreadItemReplacedEvent> <xref:chatkit.types.StreamOptionsEvent> <xref:chatkit.types.ProgressUpdateEvent> <xref:chatkit.types.ClientEffectEvent> <xref:chatkit.types.ErrorEvent> <xref:chatkit.types.NoticeEvent>]	

ThreadItemConverter Class

Helper class to convert ChatKit thread items to Agent Framework ChatMessage objects.

This class provides a base implementation for converting ChatKit thread items to Agent Framework messages. It can be extended to handle attachments, @-mentions, hidden context items, and custom thread item formats.

Initialize the converter.

Constructor

Python
<code>ThreadItemConverter()</code>

Parameters

 Expand table

Name	Description
<code>attachment_data_fetcher</code>	Optional async function to fetch attachment binary data. If provided, it should take an attachment ID and return the binary data as bytes. If not provided, attachments will be converted to UriContent using available URLs. Default value: None
<code>attachment_data_fetcher</code> Required*	Optional async function to fetch attachment data by ID.

Methods

 Expand table

<code>__init__</code>	Initialize the converter.
<code>__new__</code>	
<code>assistant_message_to_input</code>	Convert a ChatKit AssistantMessageItem to Agent Framework ChatMessage(s). The default implementation extracts text from all content parts and creates an assistant message.

ⓘ **Note**

Instead of calling this method directly, use `to_agent_input()` which handles

all `ThreadItem` types and provides proper message ordering.

`attachment_to_message_content`

Convert a ChatKit attachment to Agent Framework content.

This method is called internally by `user_message_to_input()` to handle attachments. Override this method to customize attachment handling for your storage backend.

The default implementation provides two strategies:

1. If an `attachment_data_fetcher` was provided, it fetches the binary data
and creates a `DataContent` object
2. Otherwise, for `ImageAttachment` with `preview_url`, it creates a `UriContent` object

For `FileAttachment` without a data fetcher, returns `None` (attachment is skipped).

ⓘ **Note**

Instead of calling this method directly, use `to_agent_input()` which handles

all `ThreadItem` types including attachments within user messages.

`client_tool_call_to_input`

Convert a ChatKit `ClientToolCallItem` to Agent Framework `ChatMessage(s)`.

The default implementation converts completed tool calls into function call and result content.

ⓘ **Note**

Instead of calling this method directly, use `to_agent_input()` which handles

all ThreadItem types and provides proper message ordering.

[end_of_turn_to_input](#)

Convert a ChatKit EndOfTurnItem to Agent Framework ChatMessage(s).

The default implementation skips end-of-turn markers as they are only UI hints.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles

all ThreadItem types and provides proper message ordering.

[hidden_context_to_input](#)

Convert a ChatKit HiddenContextItem or SDKHiddenContextItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how hidden context is converted.

The default implementation wraps the hidden context in XML tags and returns a system message. This allows the model to distinguish hidden context from regular conversation.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles

all ThreadItem types and provides proper message ordering.

[tag_to_message_content](#)

Convert a ChatKit tag (@-mention) to Agent Framework content.

This method is called internally by `user_message_to_input()` to handle tags. Override this method to customize tag conversion for your application.

The default implementation extracts the tag's display name and wraps it in XML tags to provide context to the model about the @-mention.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles
all `ThreadItem` types including tags within user messages.

`task_to_input`

Convert a ChatKit TaskItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how tasks are converted.

The default implementation converts custom tasks with title/content into a user message explaining what task was displayed to the user.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles
all `ThreadItem` types and provides proper message ordering.

`to_agent_input`

Convert ChatKit thread items to Agent Framework ChatMessages.

This is the main entry point for converting ChatKit thread items. It handles all `ThreadItem` types (`UserMessageItem`, `AssistantMessageItem`, `TaskItem`, etc.) and calls the appropriate conversion method for each.

`user_message_to_input`

Convert a ChatKit UserMessageItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how user messages are converted.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles
all `ThreadItem` types and provides proper message ordering.

`widget_to_input`

Convert a ChatKit WidgetItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how widgets are converted.

The default implementation converts the widget to a JSON representation and includes it in a user message, allowing the model to understand what UI element was displayed to the user.

 **Note**

Instead of calling this method directly, use `to_agent_input()` which handles

all `ThreadItem` types and provides proper message ordering.

`workflow_to_input`

Convert a ChatKit WorkflowItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how workflows are converted.

The default implementation converts each custom task in the workflow into a separate user message explaining what tasks were performed.

 **Note**

Instead of calling this method directly, use `to_agent_input()` which handles

all `ThreadItem` types and provides proper message ordering.

`__init__`

Initialize the converter.

Python

```
__init__(attachment_data_fetcher: Callable[[str], Awaitable[bytes]] | None = None)
-> None
```

Parameters

 Expand table

Name	Description
<code>attachment_data_fetcher</code>	<code>Callable[[str], Awaitable[bytes]] None</code> Optional async function to fetch attachment data by ID.

Name	Description
	Default value: None

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

assistant_message_to_input

Convert a ChatKit AssistantMessageItem to Agent Framework ChatMessage(s).

The default implementation extracts text from all content parts and creates an assistant message.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles all ThreadItem types and provides proper message ordering.

Python
<code>async assistant_message_to_input(item: AssistantMessageItem) -> ChatMessage list[ChatMessage] None</code>

Parameters

[Expand table](#)

Name	Description
item	<xref:chatkit.types.AssistantMessageItem>

Name	Description
Required*	The ChatKit assistant message item to convert.

Returns

[Expand table](#)

Type	Description
ChatMessage list[ChatMessage] None	A ChatMessage with assistant role, or None to skip.

attachment_to_message_content

Convert a ChatKit attachment to Agent Framework content.

This method is called internally by *user_message_to_input()* to handle attachments. Override this method to customize attachment handling for your storage backend.

The default implementation provides two strategies:

1. If an attachment_data_fetcher was provided, it fetches the binary data
and creates a DataContent object
2. Otherwise, for ImageAttachment with preview_url, it creates a UriContent object

For FileAttachment without a data fetcher, returns None (attachment is skipped).

ⓘ Note

Instead of calling this method directly, use *to_agent_input()* which handles all ThreadItem types including attachments within user messages.

Python

```
async attachment_to_message_content(attachment: FileAttachment | ImageAttachment)
-> DataContent | UriContent | None
```

Parameters

[Expand table](#)

Name	Description
attachment Required*	<xref:chatkit.types.FileAttachment> <xref:chatkit.types.ImageAttachment> The ChatKit attachment to convert (FileAttachment or ImageAttachment).

Returns

 Expand table

Type	Description
DataContent UriContent None	DataContent if binary data is available, UriContent if only URL is available, or None if the attachment cannot be converted.

Examples

Python

```
# With data fetcher
async def fetch_data(attachment_id: str) -> bytes:
    return await my_storage.get_file(attachment_id)

converter = ThreadItemConverter(attachment_data_fetcher=fetch_data)
messages = await converter.to_agent_input(thread_items)

# Without data fetcher (uses URLs for images)
converter = ThreadItemConverter()
messages = await converter.to_agent_input(thread_items)
```

client_tool_call_to_input

Convert a ChatKit ClientToolCallItem to Agent Framework ChatMessage(s).

The default implementation converts completed tool calls into function call and result content.

 Note

Instead of calling this method directly, use `to_agent_input()` which handles all ThreadItem types and provides proper message ordering.

Python

```
async client_tool_call_to_input(item: ClientToolCallItem) -> ChatMessage |  
list[ChatMessage] | None
```

Parameters

[Expand table](#)

Name	Description
item Required*	<xref:chatkit.types.ClientToolCallItem> The ChatKit client tool call item to convert.

Returns

[Expand table](#)

Type	Description
ChatMessage list[ChatMessage] None	A list containing function call and result messages, or None for pending calls.

end_of_turn_to_input

Convert a ChatKit EndOfTurnItem to Agent Framework ChatMessage(s).

The default implementation skips end-of-turn markers as they are only UI hints.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles all ThreadItem types and provides proper message ordering.

Python

```
async end_of_turn_to_input(item: EndOfTurnItem) -> ChatMessage | list[ChatMessage]  
| None
```

Parameters

[Expand table](#)

Name	Description
item Required*	<xref:chatkit.types.EndOfTurnItem> The ChatKit end-of-turn item to convert.

Returns

 Expand table

Type	Description
ChatMessage list[ChatMessage] None	None (end-of-turn items are not converted).

hidden_context_to_input

Convert a ChatKit HiddenContextItem or SDKHiddenContextItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how hidden context is converted.

The default implementation wraps the hidden context in XML tags and returns a system message. This allows the model to distinguish hidden context from regular conversation.

Note

Instead of calling this method directly, use *to_agent_input()* which handles all ThreadItem types and provides proper message ordering.

Python

```
hidden_context_to_input(item: HiddenContextItem | SDKHiddenContextItem) ->  
ChatMessage | list[ChatMessage] | None
```

Parameters

 Expand table

Name	Description
item Required*	<xref:chatkit.types.HiddenContextItem> <xref:chatkit.types.SDKHiddenContextItem> The ChatKit hidden context item to convert.

Returns

 Expand table

Type	Description
ChatMessage list[ChatMessage] None	A ChatMessage with system role, a list of messages, or None to skip.

Examples

Python

```
# Default behavior
converter = ThreadItemConverter()
hidden_item = HiddenContextItem(
    id="ctx_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    content="User's email: user@example.com",
)
message = converter.hidden_context_to_input(hidden_item)
# Returns: ChatMessage(role=SYSTEM, text="<HIDDEN_CONTEXT>User's email: ...
</HIDDEN_CONTEXT>")
```

tag_to_message_content

Convert a ChatKit tag (@-mention) to Agent Framework content.

This method is called internally by *user_message_to_input()* to handle tags. Override this method to customize tag conversion for your application.

The default implementation extracts the tag's display name and wraps it in XML tags to provide context to the model about the @-mention.

Note

Instead of calling this method directly, use *to_agent_input()* which handles all ThreadItem types including tags within user messages.

Python

```
tag_to_message_content(tag: UserMessageTagContent) -> TextContent
```

Parameters

[Expand table](#)

Name	Description
tag Required*	<xref:chatkit.types.UserMessageTagContent> The ChatKit tag content to convert.

Returns

[Expand table](#)

Type	Description
TextContent	TextContent with the tag information.

Examples

Python

```
# Default behavior
converter = ThreadItemConverter()
tag = UserMessageTagContent(
    type="input_tag", id="tag_1", text="john", data={"name": "John Doe"},
    interactive=False
)
content = converter.tag_to_message_content(tag)
# Returns: TextContent(text="<TAG>Name:John Doe</TAG>")
```

task_to_input

Convert a ChatKit TaskItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how tasks are converted.

The default implementation converts custom tasks with title/content into a user message explaining what task was displayed to the user.

 **Note**

Instead of calling this method directly, use `to_agent_input()` which handles all `ThreadItem` types and provides proper message ordering.

Python

```
task_to_input(item: TaskItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

[Expand table](#)

Name	Description
item Required*	<xref:chatkit.types.TaskItem> The ChatKit task item to convert.

Returns

[Expand table](#)

Type	Description
ChatMessage list[ChatMessage] None	A ChatMessage, a list of messages, or None to skip the task.

Examples

Python

```
# Task with both title and content
from chatkit.types import Task

task_item = TaskItem(
    id="task_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    task=Task(type="custom", title="Data Analysis", content="Analyzed sales
data"),
)
message = converter.task_to_input(task_item)
# Returns message explaining the task was performed
```

to_agent_input

Convert ChatKit thread items to Agent Framework ChatMessages.

This is the main entry point for converting ChatKit thread items. It handles all ThreadItem types (UserMessageItem, AssistantMessageItem, TaskItem, etc.) and calls the appropriate conversion method for each.

Python

```
async to_agent_input(thread_items: Sequence[UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem] | UserMessageItem | AssistantMessageItem | ClientToolCallItem | WidgetItem | GeneratedImageItem | WorkflowItem | TaskItem | HiddenContextItem | SDKHiddenContextItem | EndOfTurnItem) -> list[ChatMessage]
```

Parameters

 Expand table

Name	Description
thread_items Required*	Sequence [<xref:chatkit.types.UserMessageItem> <xref:chatkit.types.AssistantMessageItem> <xref:chatkit.types.ClientToolCallItem> <xref:chatkit.types.WidgetItem> <xref:chatkit.types.GeneratedImageItem> <xref:chatkit.types.WorkflowItem> <xref:chatkit.types.TaskItem> <xref:chatkit.types.HiddenContextItem> <xref:chatkit.types.SDKHiddenContextItem> <xref:chatkit.types.EndOfTurnItem>] <xref:chatkit.types.UserMessageItem> <xref:chatkit.types.AssistantMessageItem> <xref:chatkit.types.ClientToolCallItem> <xref:chatkit.types.WidgetItem> <xref:chatkit.types.GeneratedImageItem> <xref:chatkit.types.WorkflowItem> <xref:chatkit.types.TaskItem> <xref:chatkit.types.HiddenContextItem> <xref:chatkit.types.SDKHiddenContextItem> <xref:chatkit.types.EndOfTurnItem> A single ThreadItem or a sequence of ThreadItems to convert.

Returns

 Expand table

Type	Description
list [ChatMessage]	A list of ChatMessage objects that can be sent to an Agent Framework agent.

Examples

Python

```

from agent_framework_chatkit import ThreadItemConverter

converter = ThreadItemConverter()

# Convert a single thread item
messages = await converter.to_agent_input(user_message_item)

# Convert multiple thread items
messages = await converter.to_agent_input([user_message_item,
assistant_message_item, task_item])

# Use with agent
from agent_framework import ChatAgent

agent = ChatAgent(...)
response = await agent.run_stream(messages)

```

user_message_to_input

Convert a ChatKit UserMessageItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how user messages are converted.

ⓘ Note

Instead of calling this method directly, use *to_agent_input()* which handles all ThreadItem types and provides proper message ordering.

Python

```

async def user_message_to_input(item: UserMessageItem, is_last_message: bool = True) -
    > ChatMessage | list[ChatMessage] | None

```

Parameters

[🔗](#) Expand table

Name	Description
item Required*	<xref:chatkit.types.UserMessageItem> The ChatKit user message item to convert.
is_last_message	bool

Name	Description
	Whether this is the last message in the thread (used for quoted_text handling). Default value: True

Returns

[Expand table](#)

Type	Description
ChatMessage list[ChatMessage] None	A ChatMessage, list of messages, or None to skip.

widget_to_input

Convert a ChatKit WidgetItem to Agent Framework ChatMessage(s).

This method is called internally by *to_agent_input()*. Override this method to customize how widgets are converted.

The default implementation converts the widget to a JSON representation and includes it in a user message, allowing the model to understand what UI element was displayed to the user.

ⓘ Note

Instead of calling this method directly, use *to_agent_input()* which handles all ThreadItem types and provides proper message ordering.

Python

```
widget_to_input(item: WidgetItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

[Expand table](#)

Name	Description
item Required*	<xref:chatkit.types.WidgetItem> The ChatKit widget item to convert.

Returns

Type	Description
ChatMessage list[ChatMessage] None	A ChatMessage describing the widget, or None to skip.

Examples

Python

```
# Widget item
from chatkit.widgets import Card, Text

widget_item = WidgetItem(
    id="widget_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    widget=Card(children=[Text(value="Hello")]),
)
message = converter.widget_to_input(widget_item)
# Returns message with JSON representation of the widget
```

workflow_to_input

Convert a ChatKit WorkflowItem to Agent Framework ChatMessage(s).

This method is called internally by `to_agent_input()`. Override this method to customize how workflows are converted.

The default implementation converts each custom task in the workflow into a separate user message explaining what tasks were performed.

ⓘ Note

Instead of calling this method directly, use `to_agent_input()` which handles all ThreadItem types and provides proper message ordering.

Python

```
workflow_to_input(item: WorkflowItem) -> ChatMessage | list[ChatMessage] | None
```

Parameters

[Expand table](#)

Name	Description
item Required*	<xref:chatkit.types.WorkflowItem> The ChatKit workflow item to convert.

Returns

[Expand table](#)

Type	Description
ChatMessage list[ChatMessage] None	A list of ChatMessages (one per task), a single message, or None to skip.

Examples

Python

```
# Workflow with multiple tasks
from chatkit.types import Workflow, Task

workflow_item = WorkflowItem(
    id="wf_1",
    thread_id="thread_1",
    created_at=datetime.now(),
    workflow=Workflow(
        type="custom",
        tasks=[
            Task(type="custom", title="Step 1", content="Gathered data"),
            Task(type="custom", title="Step 2", content="Analyzed results"),
        ],
    ),
)
messages = converter.workflow_to_input(workflow_item)
# Returns list of messages for each task
```

declarative Package

devui Package

Classes

 Expand table

AgentFrameworkRequest	OpenAI ResponseCreateParams with Agent Framework routing. This properly extends the real OpenAI API request format. • Uses 'model' field as entity_id (agent/workflow name) • Uses 'conversation' field for conversation context (OpenAI standard) Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
DevServer	Development Server - OpenAI compatible API server for debugging agents. Initialize the development server.
DiscoveryResponse	Response model for entity discovery. Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
EntityInfo	Entity information for discovery and detailed views. Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
OpenAIError	OpenAI standard error response model. Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Functions

main

CLI entry point for devui command.

Python

```
main() -> None
```

Returns

[Expand table](#)

Type	Description
None	

serve

Launch Agent Framework DevUI with simple API.

Python

```
serve(entities: list[Any] | None = None, entities_dir: str | None = None, port: int = 8080, host: str = '127.0.0.1', auto_open: bool = False, cors_origins: list[str] | None = None, ui_enabled: bool = True, instrumentation_enabled: bool = False, mode: str = 'developer', auth_enabled: bool = False, auth_token: str | None = None) -> None
```

Parameters

[Expand table](#)

Name	Description
entities	<code>list[Any] None</code> List of entities for in-memory registration (IDs auto-generated) Default value: None

Name	Description
entities_dir	str None Directory to scan for entities Default value: None
port	int Port to run server on Default value: 8080
host	str Host to bind server to Default value: "127.0.0.1"
auto_open	bool Whether to automatically open browser Default value: False
cors_origins	list[str] None List of allowed CORS origins Default value: None
ui_enabled	bool Whether to enable the UI Default value: True
instrumentation_enabled	bool Whether to enable OpenTelemetry instrumentation Default value: False
mode	str Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: "developer"
auth_enabled	bool Whether to enable Bearer token authentication Default value: False
auth_token	str None Custom authentication token (auto-generated if not provided with auth_enabled=True) Default value: None

Returns

[Expand table](#)

Type	Description
None	

AgentFrameworkRequest Class

OpenAI ResponseCreateParams with Agent Framework routing.

This properly extends the real OpenAI API request format.

- Uses 'model' field as entity_id (agent/workflow name)
- Uses 'conversation' field for conversation context (OpenAI standard)

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model AgentFrameworkRequest

Keyword-Only Parameters

 Expand table

Name	Description
model	Default value: None
input Required*	
stream	Default value: False
conversation	Default value: None
instructions	Default value: None
metadata	Default value: None
temperature	Default value: None
max_output_tokens	Default value: None

Name	Description
<code>top_p</code>	Default value: None
<code>tools</code>	Default value: None
<code>reasoning</code>	Default value: None
<code>extra_body</code>	Default value: None

Methods

 Expand table

<code>__init__</code>	Create a new model by parsing and validating input data from keyword arguments. Raises <code>[ValidationError][pydantic_core.ValidationError]</code> if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
<code>__new__</code>	
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>
<code>dict</code>	
<code>from_orm</code>	
<code>get_entity_id</code>	Get <code>entity_id</code> from <code>metadata.entity_id</code>. In DevUI, <code>entity_id</code> is specified in metadata for routing.
<code>json</code>	
<code>model_construct</code>	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is

	performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][object.<i>dict</i>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModel.
model_post_init	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i>. This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.

parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
to_openai_params	Convert to dict for OpenAI client compatibility.
update_forward_refs	
validate	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises `[ValidationError][pydantic_core.ValidationError]` if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

[Expand table](#)

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Python

```
__new__(**kwargs)
```

construct

Python

```
construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> Default value: None
<code>values</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code>
by_alias Required*	<code>bool</code>
exclude_unset Required*	<code>bool</code>
exclude_defaults Required*	<code>bool</code>
exclude_none Required*	<code>bool</code>

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False

Name	Description
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>obj</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

get_entity_id

Get `entity_id` from `metadata.entity_id`.

In DevUI, `entity_id` is specified in `metadata` for routing.

Python

get_entity_id() -> str | None

Returns

[Expand table](#)

Type	Description
str None	

json

Python

json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults	bool

Name	Description
Required*	
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

[Expand table](#)

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
_fields_set	set[str] None A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
values Required*	Any Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
Self	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [***dict***][object.**dict**] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int</code> <code>None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool</code> <code>None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <code>round_trip</code> parameter instead.
round_trip Required*	<code>bool</code> If <code>True</code> , dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. <code>False</code> /"none" ignores them, <code>True</code> /"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .

Name	Description
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
                  schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
                  Literal['validation', 'serialization'] = 'validation', *, union_format:
                  Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Whether to use attribute aliases or not. Default value: True
ref_template	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	<code>Literal['any_of', 'primitive_type_array']</code> The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	<code>type[<xref:pydantic.json_schema.GenerateJsonSchema>]</code> To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
mode	<code>Literal['validation', 'serialization']</code> The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>union_format</code>	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
<code>params</code> Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model</i> [<i>str</i> , <i>int</i>], the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

[Expand table](#)

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<code>model_post_init(context: Any, /) -> None</code>

Positional-Only Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	

Parameters

[Expand table](#)

Name	Description
<code>context</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python
<pre>model_rebuild(*, force: bool = False, raise_errors: bool = True, _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace None = None) -> bool None</pre>

Parameters

[Expand table](#)

Name	Description
force Required*	<code>bool</code> Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	<code>bool</code> Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<code><xref:MappingNamespace> None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
<code>_types_namespace</code>	Default value: <code>None</code>

Returns

[↗](#) Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[↗](#) Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.
by_alias	<code>bool</code> <code>None</code>

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj	Any

Name	Description
Required*	

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) -> Dict[str, Any]
```

Parameters

[↗](#) Expand table

Name	Description
<code>by_alias</code>	bool Default value: True
<code>ref_template</code>	str Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
by_alias Required*	bool
ref_template Required*	str
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

to_openai_params

Convert to dict for OpenAI client compatibility.

Python

```
to_openai_params() -> dict[str, Any]
```

Returns

[\[\]](#) Expand table

Type	Description
<code>dict[str, Any]</code>	

update_forward_refs

Python

```
update_forward_refs(**locals: Any) -> None
```

Parameters

[\[\]](#) Expand table

Name	Description
<code>locals</code> Required*	<code>Any</code>

Returns

[\[\]](#) Expand table

Type	Description
<code>None</code>	

validate

Python

```
validate(value: Any) -> Self
```

Parameters

[\[\]](#) Expand table

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

conversation

Python

```
field conversation: str | dict[str, Any] | None = None
```

extra_body

Python

```
field extra_body: dict[str, Any] | None = None
```

input

Python

```
field input: str | list[Any] | dict[str, Any] [Required]
```

instructions

Python

```
field instructions: str | None = None
```

max_output_tokens

Python

```
field max_output_tokens: int | None = None
```

metadata

Python

```
field metadata: dict[str, Any] | None = None
```

model

Python

```
field model: str | None = None
```

reasoning

Python

```
field reasoning: dict[str, Any] | None = None
```

stream

Python

```
field stream: bool | None = False
```

temperature

Python

```
field temperature: float | None = None
```

tools

Python

```
field tools: list[dict[str, Any]] | None = None
```

top_p

Python

```
field top_p: float | None = None
```

DevServer Class

Development Server - OpenAI compatible API server for debugging agents.

Initialize the development server.

Constructor

Python
<code>DevServer()</code>

Parameters

 Expand table

Name	Description
<code>entities_dir</code>	Directory to scan for entities Default value: None
<code>port</code>	Port to run server on Default value: 8080
<code>host</code>	Host to bind server to Default value: 127.0.0.1
<code>cors_origins</code>	List of allowed CORS origins Default value: None
<code>ui_enabled</code>	Whether to enable the UI Default value: True
<code>mode</code>	Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: developer

Methods

 Expand table

<code>__init__</code>	Initialize the development server.
<code>__new__</code>	

create_app	Create the FastAPI application.
get_app	Get the FastAPI application instance.
register_entities	Register entities to be discovered when server starts.

`__init__`

Initialize the development server.

Python

```
__init__(entities_dir: str | None = None, port: int = 8080, host: str = '127.0.0.1', cors_origins: list[str] | None = None, ui_enabled: bool = True, mode: str = 'developer') -> None
```

Parameters

 Expand table

Name	Description
<code>entities_dir</code>	<code>str</code> <code>None</code> Directory to scan for entities Default value: <code>None</code>
<code>port</code>	<code>int</code> Port to run server on Default value: <code>8080</code>
<code>host</code>	<code>str</code> Host to bind server to Default value: <code>"127.0.0.1"</code>
<code>cors_origins</code>	<code>list[str]</code> <code>None</code> List of allowed CORS origins Default value: <code>None</code>
<code>ui_enabled</code>	<code>bool</code> Whether to enable the UI Default value: <code>True</code>
<code>mode</code>	<code>str</code> Server mode - 'developer' (full access, verbose errors) or 'user' (restricted APIs, generic errors) Default value: <code>"developer"</code>

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`create_app`

Create the FastAPI application.

Python
<code>create_app() -> FastAPI</code>

Returns

[Expand table](#)

Type	Description
<xref:fastapi.applications.FastAPI>	

`get_app`

Get the FastAPI application instance.

Python
<code>get_app() -> FastAPI</code>

Returns

[Expand table](#)

Type	Description
<xref:fastapi.applications.FastAPI>	

register_entities

Register entities to be discovered when server starts.

Python

```
register_entities(entities: list[Any]) -> None
```

Parameters

 Expand table

Name	Description
entities Required*	<code>list[Any]</code> List of entity objects to register

Returns

 Expand table

Type	Description
<code>None</code>	

DiscoveryResponse Class

Response model for entity discovery.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model DiscoveryResponse

Keyword-Only Parameters

 Expand table

Name	Description
entities	Default value: <factory>

Methods

 Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
__new__	
construct	
copy	Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

dict

from_orm

json

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [*dict*][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

model_dump

!!! abstract "Usage Documentation" *model_dump*

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

model_dump_json

!!! abstract "Usage Documentation" *model_dump_json*

Generates a JSON representation of the model using Pydantic's *to_json* method.

model_json_schema

Generates a JSON schema for a model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModels.

<code>model_post_init</code>	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
<code>model_rebuild</code>	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.
<code>model_validate</code>	Validate a pydantic model instance.
<code>model_validate_json</code>	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
<code>model_validate_strings</code>	Validate the given object with string data against the Pydantic model.
<code>parse_file</code>	
<code>parse_obj</code>	
<code>parse_raw</code>	
<code>schema</code>	
<code>schema_json</code>	
<code>update_forward_refs</code>	
<code>validate</code>	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises `[ValidationError][pydantic_core.ValidationError]` if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

 Expand table

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
None	

__new__

Python
<code>__new__(**kwargs)</code>

construct

Python
<code>construct(_fields_set: set[str] None = None, **values: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
_fields_set	set[str] None Default value: None
values Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

copy

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use `model_copy` instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

```
copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude:
AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None
= None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python
<pre>dict(*, include: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] None = None, exclude: set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] Mapping[str, set[int] set[str] Mapping[int, IncEx bool] Mapping[str, IncEx bool] bool] bool] None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]</pre>

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False

Returns

[Expand table](#)

Type	Description
Dict[str, Any]	

from_orm

Python
from_orm(obj: Any) -> Self

Parameters

[Expand table](#)

Name	Description
obj Required*	Any

Returns

 Expand table

Type	Description
Self	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **dumps_kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool

Name	Description
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
by_alias	Default value: False
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
encoder	Default value: PydanticUndefined
models_as_dict	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
str	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

 Expand table

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: None
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

 Expand table

Type	Description
<code>Self</code>	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" *model_copy*

Returns a copy of the model.

!!! note The underlying instance's [*dict*][object.dict] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

 Expand table

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.

Name	Description
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any]</code> <code>None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False

Name	Description
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

[Expand table](#)

Name	Description
indent Required*	<code>int</code> <code>None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int]</code> <code>set[str]</code> <code>Mapping[int, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>Mapping[str, set[int]]</code> <code>set[str]</code> <code>Mapping[int, <xref:IncEx> bool]</code> <code>Mapping[str, <xref:IncEx> bool]</code> <code>bool</code> <code>None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool</code> <code>None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <code>None</code> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <code>round_trip</code> parameter instead.
round_trip Required*	<code>bool</code> If <code>True</code> , dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool</code> <code>Literal['none', 'warn', 'error']</code> How to handle serialization errors. <code>False</code> /"none" ignores them, <code>True</code> /"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .

Name	Description
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

 Expand table

Name	Description
indent	Default value: None
ensure_ascii	Default value: False
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

 Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
                  schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
                  Literal['validation', 'serialization'] = 'validation', *, union_format:
                  Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

 Expand table

Name	Description
<code>by_alias</code>	<code>bool</code> Whether to use attribute aliases or not. Default value: True
<code>ref_template</code>	<code>str</code> The reference template. Default value: DEFAULT_REF_TEMPLATE
<code>union_format</code> Required*	<code>Literal['any_of', 'primitive_type_array']</code> The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string, boolean, null, integer or number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
<code>schema_generator</code>	<code>type[<xref:pydantic.json_schema.GenerateJsonSchema>]</code> To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
<code>mode</code>	<code>Literal['validation', 'serialization']</code> The mode in which to generate the schema.

Name	Description
	Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>union_format</code>	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python
<code>model_parametrized_name(params: tuple[type[Any], ...]) -> str</code>

Parameters

[Expand table](#)

Name	Description
<code>params</code> Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model</i> [<i>str</i> , <i>int</i>], the value (<i>str</i> , <i>int</i>) would be passed to <i>params</i> .

Returns

[Expand table](#)

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

 Expand table

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python
<code>model_post_init(context: Any, /) -> None</code>

Positional-Only Parameters

 Expand table

Name	Description
<code>context</code> Required*	

Parameters

 Expand table

Name	Description
<code>context</code> Required*	<code>Any</code>

Returns

 Expand table

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python
<pre>model_rebuild(*, force: bool = False, raise_errors: bool = True, _parent_namespace_depth: int = 2, _types_namespace: MappingNamespace None = None) -> bool None</pre>

Parameters

[Expand table](#)

Name	Description
force Required*	<code>bool</code> Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	<code>bool</code> Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	<code>int</code> The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<code><xref:MappingNamespace> None</code> The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2

Name	Description
<code>_types_namespace</code>	Default value: <code>None</code>

Returns

[↗](#) Expand table

Type	Description
<code>bool</code> <code>None</code>	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[↗](#) Expand table

Name	Description
obj Required*	<code>Any</code> The object to validate.
strict Required*	<code>bool</code> <code>None</code> Whether to enforce types strictly.
extra Required*	<code>Literal</code> ['allow', 'ignore', 'forbid'] <code>None</code> Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	<code>bool</code> <code>None</code> Whether to extract data from object attributes.
context Required*	<code>Any</code> <code>None</code> Additional context to pass to the validator.
by_alias	<code>bool</code> <code>None</code>

Name	Description
Required*	Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated model instance.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [<i>extra</i> configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name	bool None

Name	Description
Required*	Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>strict</code>	Default value: None
<code>extra</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>by_name</code>	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False)
-> Self
```

Parameters

[Expand table](#)

Name	Description
<code>path</code> Required*	str <xref:Path>
<code>content_type</code> Required*	str None

Name	Description
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
obj	Any

Name	Description
Required*	

Returns

[Expand table](#)

Type	Description
Self	

parse_raw

Python

```
parse_raw(b: str | bytes, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>content_type</code>	Default value: None
<code>encoding</code>	Default value: 'utf8'
<code>proto</code>	Default value: None
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE) ->
Dict[str, Any]
```

Parameters

[↗](#) Expand table

Name	Description
<code>by_alias</code>	bool Default value: True
<code>ref_template</code>	str Default value: DEFAULT_REF_TEMPLATE

Returns

[↗](#) Expand table

Type	Description
Dict[str, Any]	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

[Expand table](#)

Name	Description
by_alias Required*	bool
ref_template Required*	str
kwargs Required*	Any

Keyword-Only Parameters

[Expand table](#)

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

[Expand table](#)

Type	Description
str	

update_forward_refs

Python

```
update_forward_refs(**kwargs: Any) -> None
```

Parameters

 Expand table

Name	Description
localns Required*	Any

Returns

 Expand table

Type	Description
None	

validate

Python
<pre>validate(value: Any) -> Self</pre>

Parameters

 Expand table

Name	Description
value Required*	Any

Returns

 Expand table

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

 Expand table

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

 Expand table

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

entities

Python
<code>field entities: list[EntityInfo] [Optional]</code>

EntityInfo Class

Entity information for discovery and detailed views.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model EntityInfo

Keyword-Only Parameters

 Expand table

Name	Description
id Required*	
type Required*	
name Required*	
description	Default value: None
framework Required*	
tools	Default value: None
metadata	Default value: <factory>
source	Default value: directory
required_env_vars	Default value: None
deployment_supported	Default value: False

Name	Description
deployment_reason	Default value: None
instructions	Default value: None
model_id	Default value: None
chat_client_type	Default value: None
context_providers	Default value: None
middleware	Default value: None
executors	Default value: None
workflow_dump	Default value: None
input_schema	Default value: None
input_type_name	Default value: None
start_executor_id	Default value: None

Methods

 Expand table

<code>__init__</code>	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
<code>__new__</code>	
<code>construct</code>	
<code>copy</code>	Returns a copy of the model. !!! warning "Deprecated" This method is now deprecated; use <i>model_copy</i> instead. If you need <i>include</i> or <i>exclude</i>, use: <pre>python {test="skip" lint="skip"} data = self.model_dump(include=include, exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied = self.model_validate(data)</pre>

dict	
from_orm	
json	
model_construct	Creates a new instance of the <i>Model</i> class with validated data. Creates a new model setting <i>dict</i> and <i>pydantic_fields_set</i> from trusted or pre-validated data. Default values are respected, but no other validation is performed. !!! note <i>model_construct()</i> generally respects the <i>model_config.extra</i> setting on the provided model. That is, if <i>model_config.extra</i> == 'allow', then all extra passed values are added to the model instance's <i>dict</i> and <i>pydantic_extra</i> fields. If <i>model_config.extra</i> == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to <i>model_construct()</i>, having <i>model_config.extra</i> == 'forbid' does not result in an error if extra values are passed, but they will be ignored.
model_copy	!!! abstract "Usage Documentation" model_copy Returns a copy of the model. !!! note The underlying instance's [<i>dict</i>][object.<i>dict</i>] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [cached properties][functools.cached_property]).
model_dump	!!! abstract "Usage Documentation" model_dump Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.
model_dump_json	!!! abstract "Usage Documentation" model_dump_json Generates a JSON representation of the model using Pydantic's <i>to_json</i> method.
model_json_schema	Generates a JSON schema for a model class.
model_parametrized_name	Compute the class name for parametrizations of generic classes. This method can be overridden to achieve a custom naming scheme for generic BaseModel.
model_post_init	Override this method to perform additional initialization after <i>init</i> and <i>model_construct</i> . This is useful if you want to do some validation that requires the entire model to be initialized.
model_rebuild	Try to rebuild the pydantic-core schema for the model. This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and

	automatic rebuilding fails.
model_validate	Validate a pydantic model instance.
model_validate_json	!!! abstract "Usage Documentation" JSON Parsing Validate the given JSON data against the Pydantic model.
model_validate_strings	Validate the given object with string data against the Pydantic model.
parse_file	
parse_obj	
parse_raw	
schema	
schema_json	
update_forward_refs	
validate	

`__init__`

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Python

```
__init__(**data: Any) -> None
```

Parameters

 Expand table

Name	Description
data Required*	Any

Returns

[Expand table](#)

Type	Description
None	

`__new__`

Python
<code>__new__(**kwargs)</code>

`construct`

Python
<code>construct(_fields_set: set[str] None = None, **values: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	set[str] None Default value: None
<code>values</code> Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

`copy`

Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use `model_copy` instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied =
self.model_validate(data)
```

Python

copy(*, include: AbstractSetIntStr | MappingIntStrAny | None = None, exclude: AbstractSetIntStr | MappingIntStrAny | None = None, update: Dict[str, Any] | None = None, deep: bool = False) -> Self

Parameters

 Expand table

Name	Description
include Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to include in the copied model.
exclude Required*	<xref:AbstractSetIntStr> <xref:MappingIntStrAny> None Optional set or mapping specifying which fields to exclude in the copied model.
update Required*	Dict[str, Any] None Optional dictionary of field-value pairs to override field values in the copied model.
deep Required*	bool If True, the values of fields that are Pydantic models will be deep-copied.

Keyword-Only Parameters

 Expand table

Name	Description
include	Default value: None
exclude	Default value: None
update	Default value: None
deep	Default value: False

Returns

[Expand table](#)

Type	Description
Self	A copy of the model with included, excluded and updated fields as specified.

dict

Python

```
dict(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False) -> Dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>by_alias</code>	Default value: False
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False

Returns

[Expand table](#)

Type	Description
<code>Dict[str, Any]</code>	

from_orm

Python
<code>from_orm(obj: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
<code>obj</code> Required*	<code>Any</code>

Returns

[Expand table](#)

Type	Description
<code>Self</code>	

json

Python

```
json(*, include: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, exclude: set[int] | set[str] | Mapping[int, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | Mapping[str, set[int] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool] | None = None, by_alias: bool = False, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, encoder: Callable[[Any], Any] | None = PydanticUndefined, models_as_dict: bool = PydanticUndefined, **kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
include Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
exclude Required*	set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None
by_alias Required*	bool
exclude_unset Required*	bool
exclude_defaults Required*	bool
exclude_none Required*	bool
encoder Required*	Callable[[Any], Any] None
models_as_dict Required*	bool
dumps_kwargs Required*	Any

Keyword-Only Parameters

 Expand table

Name	Description
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>by_alias</code>	Default value: False
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>encoder</code>	Default value: PydanticUndefined
<code>models_as_dict</code>	Default value: PydanticUndefined

Returns

 Expand table

Type	Description
<code>str</code>	

model_construct

Creates a new instance of the *Model* class with validated data.

Creates a new model setting ***dict*** and ***pydantic_fields_set*** from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra* == 'allow', then all extra passed values are added to the model instance's ***dict*** and ***pydantic_extra*** fields. If *model_config.extra* == 'ignore' (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra* == 'forbid' does not result in an error if extra values are passed, but they will be ignored.

Python

```
model_construct(_fields_set: set[str] | None = None, **values: Any) -> Self
```

Parameters

[Expand table](#)

Name	Description
<code>_fields_set</code>	<code>set[str] None</code> A set of field names that were originally explicitly set during instantiation. If provided, this is directly used for the <code>[model_fields_set][pydantic.BaseModel.model_fields_set]</code> attribute. Otherwise, the field names from the <i>values</i> argument will be used. Default value: <code>None</code>
<code>values</code> Required*	<code>Any</code> Trusted or pre-validated data dictionary.

Returns

[Expand table](#)

Type	Description
<code>Self</code>	A new instance of the <i>Model</i> class with validated data.

model_copy

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's `[dict][object.dict]` attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of `[cached properties][functools.cached_property]`).

Python

```
model_copy(*, update: Mapping[str, Any] | None = None, deep: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
update Required*	Mapping[str, Any] None Values to change/add in the new model. Note: the data is not validated before creating the new model. You should trust this data.
deep Required*	bool Set to <i>True</i> to make a deep copy of the model.

Keyword-Only Parameters

 Expand table

Name	Description
update	Default value: None
deep	Default value: False

Returns

 Expand table

Type	Description
Self	New model instance.

model_dump

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

Python

```
model_dump(*, mode: Literal['json', 'python'] | str = 'python', include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool |
```



```
Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
mode Required*	<code>Literal['json', 'python'] str</code> The mode in which <i>to_python</i> should run. If mode is 'json', the output will only contain JSON serializable types. If mode is 'python', the output may contain non-JSON-serializable Python objects.
include Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to include in the output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] Mapping[str, set[int] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool] None</code> A set of fields to exclude from the output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to use the field's alias in the dictionary key if defined.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings	<code>bool Literal['none', 'warn', 'error']</code>

Name	Description
Required*	How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
mode	Default value: 'python'
include	Default value: None
exclude	Default value: None
context	Default value: None
by_alias	Default value: None
exclude_unset	Default value: False
exclude_defaults	Default value: False
exclude_none	Default value: False
exclude_computed_fields	Default value: False
round_trip	Default value: False
warnings	Default value: True
fallback	Default value: None
serialize_as_any	Default value: False

Returns

[Expand table](#)

Type	Description
<code>dict[str, Any]</code>	A dictionary representation of the model.

model_dump_json

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's `to_json` method.

Python

```
model_dump_json(*, indent: int | None = None, ensure_ascii: bool = False, include: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, exclude: set[int] | set[str] | Mapping[int, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | Mapping[str, set[int]] | set[str] | Mapping[int, IncEx | bool] | Mapping[str, IncEx | bool] | bool | None = None, context: Any | None = None, by_alias: bool | None = None, exclude_unset: bool = False, exclude_defaults: bool = False, exclude_none: bool = False, exclude_computed_fields: bool = False, round_trip: bool = False, warnings: bool | Literal['none', 'warn', 'error'] = True, fallback: Callable[[Any], Any] | None = None, serialize_as_any: bool = False) -> str
```

Parameters

 Expand table

Name	Description
indent Required*	<code>int None</code> Indentation to use in the JSON output. If <code>None</code> is passed, the output will be compact.
ensure_ascii Required*	<code>bool</code> If <code>True</code> , the output is guaranteed to have all incoming non-ASCII characters escaped. If <code>False</code> (the default), these characters will be output as-is.
include Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to include in the JSON output.
exclude Required*	<code>set[int] set[str] Mapping[int, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool </code>

Name	Description
	<code>Mapping[str, set[int]] set[str] Mapping[int, <xref:IncEx> bool] Mapping[str, <xref:IncEx> bool] bool None</code> Field(s) to exclude from the JSON output.
context Required*	<code>Any None</code> Additional context to pass to the serializer.
by_alias Required*	<code>bool None</code> Whether to serialize using field aliases.
exclude_unset Required*	<code>bool</code> Whether to exclude fields that have not been explicitly set.
exclude_defaults Required*	<code>bool</code> Whether to exclude fields that are set to their default value.
exclude_none Required*	<code>bool</code> Whether to exclude fields that have a value of <i>None</i> .
exclude_computed_fields Required*	<code>bool</code> Whether to exclude computed fields. While this can be useful for round-tripping, it is usually recommended to use the dedicated <i>round_trip</i> parameter instead.
round_trip Required*	<code>bool</code> If True, dumped values should be valid as input for non-idempotent types such as <code>Json[T]</code> .
warnings Required*	<code>bool Literal['none', 'warn', 'error']</code> How to handle serialization errors. False/"none" ignores them, True/"warn" logs errors, "error" raises a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> .
fallback Required*	<code>Callable[[Any], Any] None</code> A function to call when an unknown value is encountered. If not provided, a <code>[PydanticSerializationError]</code> <code>[pydantic_core.PydanticSerializationError]</code> error is raised.
serialize_as_any Required*	<code>bool</code> Whether to serialize fields with duck-typing serialization behavior.

Keyword-Only Parameters

[Expand table](#)

Name	Description
indent	Default value: None

Name	Description
<code>ensure_ascii</code>	Default value: False
<code>include</code>	Default value: None
<code>exclude</code>	Default value: None
<code>context</code>	Default value: None
<code>by_alias</code>	Default value: None
<code>exclude_unset</code>	Default value: False
<code>exclude_defaults</code>	Default value: False
<code>exclude_none</code>	Default value: False
<code>exclude_computed_fields</code>	Default value: False
<code>round_trip</code>	Default value: False
<code>warnings</code>	Default value: True
<code>fallback</code>	Default value: None
<code>serialize_as_any</code>	Default value: False

Returns

[🔗](#) Expand table

Type	Description
<code>str</code>	A JSON string representation of the model.

model_json_schema

Generates a JSON schema for a model class.

Python

```
model_json_schema(by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
schema_generator: type[GenerateJsonSchema] = GenerateJsonSchema, mode:
Literal['validation', 'serialization'] = 'validation', *, union_format:
Literal['any_of', 'primitive_type_array'] = 'any_of') -> dict[str, Any]
```

Parameters

[Expand table](#)

Name	Description
by_alias	bool Whether to use attribute aliases or not. Default value: True
ref_template	str The reference template. Default value: DEFAULT_REF_TEMPLATE
union_format Required*	Literal ['any_of', 'primitive_type_array'] The format to use when combining schemas from unions together. Can be one of: 'any_of': Use the anyOf keyword to combine schemas (the default). 'primitive_type_array': Use the type keyword as an array of strings, containing each type of the combination. If any of the schemas is not a primitive type (<i>string</i>, <i>boolean</i>, <i>null</i>, <i>integer</i> or <i>number</i>) or contains constraints/metadata, falls back to <i>any_of</i>.
schema_generator	type [<xref:pydantic.json_schema.GenerateJsonSchema>] To override the logic used to generate the JSON schema, as a subclass of <i>GenerateJsonSchema</i> with your desired modifications Default value: GenerateJsonSchema
mode	Literal ['validation', 'serialization'] The mode in which to generate the schema. Default value: 'validation'

Keyword-Only Parameters

[Expand table](#)

Name	Description
union_format	Default value: 'any_of'

Returns

[Expand table](#)

Type	Description
dict [str , Any]	The JSON schema for the given model class.

model_parametrized_name

Compute the class name for parametrizations of generic classes.

This method can be overridden to achieve a custom naming scheme for generic BaseModel.

Python

```
model_parametrized_name(params: tuple[type[Any], ...]) -> str
```

Parameters

 Expand table

Name	Description
params Required*	<code>tuple[type[Any], ...]</code> Tuple of types of the class. Given a generic class <i>Model</i> with 2 type variables and a concrete model <i>Model[str, int]</i> , the value <i>(str, int)</i> would be passed to <i>params</i> .

Returns

 Expand table

Type	Description
<code>str</code>	String representing the new class where <i>params</i> are passed to <i>cls</i> as type variables.

Exceptions

 Expand table

Type	Description
<code>TypeError</code>	Raised when trying to generate concrete names for non-generic models.

model_post_init

Override this method to perform additional initialization after *init* and *model_construct*. This is useful if you want to do some validation that requires the entire model to be initialized.

Python

```
model_post_init(context: Any, /) -> None
```

Positional-Only Parameters

[Expand table](#)

Name	Description
context Required*	

Parameters

[Expand table](#)

Name	Description
context Required*	Any

Returns

[Expand table](#)

Type	Description
None	

model_rebuild

Try to rebuild the pydantic-core schema for the model.

This may be necessary when one of the annotations is a ForwardRef which could not be resolved during the initial attempt to build the schema, and automatic rebuilding fails.

Python

```
model_rebuild(*, force: bool = False, raise_errors: bool = True,  
_parent_namespace_depth: int = 2, _types_namespace: MappingNamespace | None =  
None) -> bool | None
```

Parameters

[Expand table](#)

Name	Description
force Required*	bool Whether to force the rebuilding of the model schema, defaults to <i>False</i> .
raise_errors Required*	bool Whether to raise errors, defaults to <i>True</i> .
_parent_namespace_depth Required*	int The depth level of the parent namespace, defaults to 2.
_types_namespace Required*	<xref:MappingNamespace> None The types namespace, defaults to <i>None</i> .

Keyword-Only Parameters

[Expand table](#)

Name	Description
force	Default value: False
raise_errors	Default value: True
_parent_namespace_depth	Default value: 2
_types_namespace	Default value: None

Returns

[Expand table](#)

Type	Description
bool None	Returns <i>None</i> if the schema is already "complete" and rebuilding was not required. If rebuilding <i>was</i> required, returns <i>True</i> if rebuilding was successful, otherwise <i>False</i> .

model_validate

Validate a pydantic model instance.

Python

```
model_validate(obj: Any, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, from_attributes: bool | None = None, context:
```

```
Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

[Expand table](#)

Name	Description
obj Required*	Any The object to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
from_attributes Required*	bool None Whether to extract data from object attributes.
context Required*	Any None Additional context to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
from_attributes	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

 Expand table

Type	Description
Self	The validated model instance.

Exceptions

 Expand table

Type	Description
ValidationError	If the object could not be validated.

model_validate_json

!!! abstract "Usage Documentation" [JSON Parsing](#)

Validate the given JSON data against the Pydantic model.

Python

```
model_validate_json(json_data: str | bytes | bytearray, *, strict: bool | None = None, extra: Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None, by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
json_data Required*	str bytes bytearray The JSON data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.

Name	Description
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

[Expand table](#)

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

Exceptions

[Expand table](#)

Type	Description
ValidationError	If <i>json_data</i> is not a JSON string or the object could not be validated.

model_validate_strings

Validate the given object with string data against the Pydantic model.

Python

```
model_validate_strings(obj: Any, *, strict: bool | None = None, extra:
Literal['allow', 'ignore', 'forbid'] | None = None, context: Any | None = None,
by_alias: bool | None = None, by_name: bool | None = None) -> Self
```

Parameters

 Expand table

Name	Description
obj Required*	Any The object containing string data to validate.
strict Required*	bool None Whether to enforce types strictly.
extra Required*	Literal ['allow', 'ignore', 'forbid'] None Whether to ignore, allow, or forbid extra data during model validation. See the [extra configuration value][pydantic.ConfigDict.extra] for details.
context Required*	Any None Extra variables to pass to the validator.
by_alias Required*	bool None Whether to use the field's alias when validating against the provided input data.
by_name Required*	bool None Whether to use the field's name when validating against the provided input data.

Keyword-Only Parameters

 Expand table

Name	Description
strict	Default value: None
extra	Default value: None
context	Default value: None
by_alias	Default value: None
by_name	Default value: None

Returns

[Expand table](#)

Type	Description
Self	The validated Pydantic model.

parse_file

Python

```
parse_file(path: str | Path, *, content_type: str | None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol | None = None, allow_pickle: bool = False) -> Self
```

Parameters

[Expand table](#)

Name	Description
path Required*	str <xref:Path>
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None

Name	Description
<code>allow_pickle</code>	Default value: False

Returns

[↗](#) Expand table

Type	Description
Self	

parse_obj

Python
<code>parse_obj(obj: Any) -> Self</code>

Parameters

[↗](#) Expand table

Name	Description
<code>obj</code> Required*	Any

Returns

[↗](#) Expand table

Type	Description
Self	

parse_raw

Python
<code>parse_raw(b: str bytes, *, content_type: str None = None, encoding: str = 'utf8', proto: DeprecatedParseProtocol None = None, allow_pickle: bool = False) -> Self</code>

Parameters

[Expand table](#)

Name	Description
b Required*	str bytes
content_type Required*	str None
encoding Required*	str
proto Required*	<xref:DeprecatedParseProtocol> None
allow_pickle Required*	bool

Keyword-Only Parameters

[Expand table](#)

Name	Description
content_type	Default value: None
encoding	Default value: 'utf8'
proto	Default value: None
allow_pickle	Default value: False

Returns

[Expand table](#)

Type	Description
Self	

schema

Python

```
schema(by_alias: bool = True, ref_template: str = DEFAULT\_REF\_TEMPLATE) ->  
Dict[str, Any]
```


Parameters

 Expand table

Name	Description
by_alias	<code>bool</code> Default value: True
ref_template	<code>str</code> Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>Dict[str, Any]</code>	

schema_json

Python

```
schema_json(*, by_alias: bool = True, ref_template: str = DEFAULT_REF_TEMPLATE,
**kwargs: Any) -> str
```

Parameters

 Expand table

Name	Description
by_alias Required*	<code>bool</code>
ref_template Required*	<code>str</code>
kwargs Required*	<code>Any</code>

Keyword-Only Parameters

 Expand table

Name	Description
by_alias	Default value: True
ref_template	Default value: DEFAULT_REF_TEMPLATE

Returns

 Expand table

Type	Description
<code>str</code>	

update_forward_refs

Python
<code>update_forward_refs(**locals: Any) -> <code>None</code></code>

Parameters

 Expand table

Name	Description
<code>locals</code> Required*	<code>Any</code>

Returns

 Expand table

Type	Description
<code>None</code>	

validate

Python
<code>validate(value: Any) -> Self</code>

Parameters

[Expand table](#)

Name	Description
value Required*	Any

Returns

[Expand table](#)

Type	Description
Self	

Attributes

model_extra

Get extra fields set during validation.

Returns

[Expand table](#)

Type	Description
	A dictionary of extra fields, or <i>None</i> if <i>config.extra</i> is not set to "allow".

model_fields_set

Returns the set of fields that have been explicitly set on this model instance.

Returns

[Expand table](#)

Type	Description
	A set of strings representing the fields that have been set, i.e. that were not filled from defaults.

chat_client_type

--

Python

```
field chat_client_type: str | None = None
```

context_providers

Python

```
field context_providers: list[str] | None = None
```

deployment_reason

Python

```
field deployment_reason: str | None = None
```

deployment_supported

Python

```
field deployment_supported: bool = False
```

description

Python

```
field description: str | None = None
```

executors

Python

```
field executors: list[str] | None = None
```

framework

Python

```
field framework: str [Required]
```

id

Python

```
field id: str [Required]
```

input_schema

Python

```
field input_schema: dict[str, Any] | None = None
```

input_type_name

Python

```
field input_type_name: str | None = None
```

instructions

Python

```
field instructions: str | None = None
```

metadata

Python

```
field metadata: dict[str, Any] [Optional]
```

middleware

Python

```
field middleware: list[str] | None = None
```

model_id

Python

```
field model_id: str | None = None
```

name

Python

field name: str [Required]

required_env_vars

Python

field required_env_vars: list[EnvVarRequirement] | None = None

source

Python

field source: str = 'directory'

start_executor_id

Python

field start_executor_id: str | None = None

tools

Python

field tools: list[str | dict[str, Any]] | None = None

type

Python

field type: str [Required]

workflow_dump

Python

field workflow_dump: dict[str, Any] | None = None

OpenAIError Class

OpenAI standard error response model.

Create a new model by parsing and validating input data from keyword arguments.

Raises [*ValidationError*][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model.

self is explicitly positional-only to allow *self* as a field name.

Constructor

Python
pydantic model OpenAIError

Keyword-Only Parameters

 Expand table

Name	Description
error Required*	

Methods

 Expand table

__init__	Create a new model by parsing and validating input data from keyword arguments. Raises [<i>ValidationError</i>][pydantic_core.ValidationError] if the input data cannot be validated to form a valid model. <i>self</i> is explicitly positional-only to allow <i>self</i> as a field name.
__new__	
construct	
copy	Returns a copy of the model.

!!! warning "Deprecated" This method is now deprecated; use *model_copy* instead.

If you need *include* or *exclude*, use:

```
python {test="skip" lint="skip"} data = self.model_dump(include=include,
exclude=exclude, round_trip=True) data = {**data, **(update or {})} copied
= self.model_validate(data)
```

[create](#)

Create a standard OpenAI error response.

[dict](#)

[from_orm](#)

[json](#)

[model_construct](#)

Creates a new instance of the *Model* class with validated data.

Creates a new model setting *dict* and *pydantic_fields_set* from trusted or pre-validated data. Default values are respected, but no other validation is performed.

!!! note *model_construct()* generally respects the *model_config.extra* setting on the provided model. That is, if *model_config.extra == 'allow'*, then all extra passed values are added to the model instance's *dict* and *pydantic_extra* fields. If *model_config.extra == 'ignore'* (the default), then all extra passed values are ignored. Because no validation is performed with a call to *model_construct()*, having *model_config.extra == 'forbid'* does not result in an error if extra values are passed, but they will be ignored.

[model_copy](#)

!!! abstract "Usage Documentation" [model_copy](#)

Returns a copy of the model.

!!! note The underlying instance's [*dict*][*object.dict*] attribute is copied. This might have unexpected side effects if you store anything in it, on top of the model fields (e.g. the value of [*cached properties*][*functools.cached_property*]).

[model_dump](#)

!!! abstract "Usage Documentation" [model_dump](#)

Generate a dictionary representation of the model, optionally specifying which fields to include or exclude.

[model_dump_json](#)

!!! abstract "Usage Documentation" [model_dump_json](#)

Generates a JSON representation of the model using Pydantic's *to_json* method.

[model_json_schema](#)

Generates a JSON schema for a model class.

[model_parametrized_name](#)

Compute the class name for parametrizations of generic classes.